

1.处理器问题

题目描述

某公司研发了一款高性能AI处理器。每台物理设备具备8颗AI处理器，编号分别为0、1、2、3、4、5、6、7。

编号0-3的处理器处于同一个链路中，编号4-7的处理器处于另外一个链路中，不通链路中的处理器不能通信。

如下图所示。现给定服务器可用的处理器编号数组array，以及任务申请的处理器数量num，找出符合下列亲和性调度原则的芯片组合。

如果不存在符合要求的组合，则返回空列表。

亲和性调度原则：

-如果申请处理器个数为1，则选择同一链路，剩余可用的处理器数量为1个的最佳，其次是剩余3个的为次佳，然后是剩余2个，最后是剩余4个。

-如果申请处理器个数为2，则选择同一链路剩余可用的处理器数量2个的为最佳，其次是剩余4个，最后是剩余3个。

-如果申请处理器个数为4，则必须选择同一链路剩余可用的处理器数量为4个。

-如果申请处理器个数为8，则申请节点所有8个处理器。

提示：

1. 任务申请的处理器数量只能是1、2、4、8。
2. 编号0-3的处理器处于一个链路，编号4-7的处理器处于另外一个链路。
3. 处理器编号唯一，且不存在相同编号处理器。

输入描述

输入包含可用的处理器编号数组array，以及任务申请的处理器数量num两个部分。

第一行为array，第二行为num。例如：

```
[0, 1, 4, 5, 6, 7]
```

```
1
```

表示当前编号为0、1、4、5、6、7的处理器可用。任务申请1个处理器。

- `0 <= array.length <= 8`
- `0 <= array[i] <= 7`
- `num in [1, 2, 4, 8]`

输出描述

输出为组合列表，当array=[0, 1, 4, 5, 6, 7]，num=1 时，输出为[[0], [1]]。

用例

输入	[0, 1, 4, 5, 6, 7] 1
输出	[[0], [1]]
说明	根据第一条亲和性调度原则，在剩余两个处理器的链路（0, 1, 2, 3）中选择处理器。由于只有0和1可用，则返回任意一颗处理器即可。

输入	[0, 1, 4, 5, 6, 7] 4
输出	[[4, 5, 6, 7]]
说明	根据第三条亲和性调度原则，必须选择同一链路剩余可用的处理器数量为4个的环

题目解析

用例中，链路link1=[0,1]，链路link2=[4,5,6,7]

现在要选1个处理器，则需要按照亲和性调度原则

如果申请处理器个数为1，则选择同一链路，剩余可用的处理器数量为1个的最佳，其次是剩余3个的为次佳，然后是剩余2个，最后是剩余4个。

最佳的是，找剩余可用1个处理器的链路，发现没有，link1剩余可用2，link2剩余可用4

其次的是，找剩余可用3个处理器的链路，发现没有

再次的是，找剩余可用2个处理器的链路，link1符合要求，即从0和1处理器中任选一个，有两种选择，可以使用dfs找对应组合。

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Integer[] arr =
            Arrays.stream(sc.nextLine().split("[\\[\\]\\s\\,\\s]"))
                .filter(str -> !"".equals(str))
```

```

        .map(Integer::parseInt)
        .toArray(Integer[]::new);

String num = sc.next();

System.out.println(getResult(arr, num));
}

public static String getResult(Integer[] arr, String num) {
    ArrayList<Integer> link1 = new ArrayList<>();
    ArrayList<Integer> link2 = new ArrayList<>();

    Arrays.sort(arr, (a, b) -> a - b);
    for (Integer e : arr) {
        if (e < 4) {
            link1.add(e);
        } else {
            link2.add(e);
        }
    }

    ArrayList<ArrayList<Integer>> ans = new ArrayList<>();
    int len1 = link1.size();
    int len2 = link2.size();

    switch (num) {
        case "1":
            if (len1 == 1 || len2 == 1) {
                if (len1 == 1) dfs(link1, 0, 1, new ArrayList<>(), ans);
                if (len2 == 1) dfs(link2, 0, 1, new ArrayList<>(), ans);
            } else if (len1 == 3 || len2 == 3) {
                if (len1 == 3) dfs(link1, 0, 1, new ArrayList<>(), ans);
                if (len2 == 3) dfs(link2, 0, 1, new ArrayList<>(), ans);
            } else if (len1 == 2 || len2 == 2) {
                if (len1 == 2) dfs(link1, 0, 1, new ArrayList<>(), ans);
                if (len2 == 2) dfs(link2, 0, 1, new ArrayList<>(), ans);
            } else if (len1 == 4 || len2 == 4) {
                if (len1 == 4) dfs(link1, 0, 1, new ArrayList<>(), ans);
                if (len2 == 4) dfs(link2, 0, 1, new ArrayList<>(), ans);
            }
            break;
        case "2":
            if (len1 == 2 || len2 == 2) {
                if (len1 == 2) dfs(link1, 0, 2, new ArrayList<>(), ans);
                if (len2 == 2) dfs(link2, 0, 2, new ArrayList<>(), ans);
            } else if (len1 == 4 || len2 == 4) {
                if (len1 == 4) dfs(link1, 0, 2, new ArrayList<>(), ans);
                if (len2 == 4) dfs(link2, 0, 2, new ArrayList<>(), ans);
            } else if (len1 == 3 || len2 == 3) {
                if (len1 == 3) dfs(link1, 0, 2, new ArrayList<>(), ans);
                if (len2 == 3) dfs(link2, 0, 2, new ArrayList<>(), ans);
            }
            break;
        case "4":
            if (len1 == 4 || len2 == 4) {

```

```

        if (len1 == 4) ans.add(link1);
        if (len2 == 4) ans.add(link2);
    }
    break;
case "8":
    if (len1 == 4 && len2 == 4) {
        ans.add(
            Stream.concat(link1.stream(), link2.stream())
                .collect(Collectors.toCollection(ArrayList<Integer>::new)));
    }
    break;
}

return ans.toString();
}

public static void dfs(
    ArrayList<Integer> arr,
    int index,
    int level,
    ArrayList<Integer> path,
    ArrayList<ArrayList<Integer>> res) {
    if (path.size() == level) {
        res.add((ArrayList<Integer>) path.clone());
    }

    for (int i = index; i < arr.size(); i++) {
        path.add(arr.get(i));
        dfs(arr, i + 1, level, path, res);
        path.remove(path.size() - 1);
    }
}
}

```

2. 真正的密码

在一行中输入一个[字符串数组](#)，如果其中一个字符串的所有以索引0开头的子串在数组中都有，那么这个字符串就是潜在密码，

在所有潜在密码中最长的是真正的密码，如果有多个长度相同的真正的密码，那么取[字典序](#)最大的为唯一的真正的密码，求唯一的真正的密码。

输入描述

无

输出描述

无

用例

输入	h he hel hell hello o ok n ni nin ninj ninja
输出	ninja
说明	按要求，hello、ok、 ninja 都是潜在密码。检查长度，hello、ninja是真正的密码。检查字典序，ninja是唯一真正密码。

输入	a b c d f
输出	f
说明	按要求，a b c d f都是潜在密码。检查长度，a b c d f是真正的密码。检查字典序，f是唯一真正密码。

题目解析

我的解题思路如下：

将输入的所有字符串先按照长度升序，如果长度相同，则按照字典序升序。

这样最后一个字符串必然就是长度最长，字典序最大的，我们从最后一个字符串str开始：

接下来，我们不停地截取str的子串，即如下闭区间的子串：

0~str.length-2

0~str.length-3

0~str.length-4

....

0~2

0~1

0~0

然后将在输入的所有字符串中查找每次截取的子串是否存在，如果多存在则返回当str作为题解，如果一个不存在，则直接中断查找，开始下一个str

Java算法源码

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String[] arr = sc.nextLine().split(" ");
        System.out.println(getResult(arr));
    }

    public static String getResult(String[] arr) {
        HashSet<String> set = new HashSet<>(Arrays.asList(arr));

        Arrays.sort(arr, (a, b) -> a.length() != b.length() ? a.length() - b.length()
        : a.compareTo(b));

        outer:
        for (int i = arr.length - 1; i >= 0; i--) {
            String str = arr[i];

            for (int j = str.length() - 1; j >= 1; j--) {
                if (!set.contains(str.substring(0, j))) continue outer;
            }

            return str;
        }

        return "";
    }
}
```

3. 端口合并

题目描述

有M个端口组($1 \leq M \leq 10$),
每个端口组是长度为N的整数数组($1 \leq N \leq 100$),
如果端口组间存在2个及以上不同端口相同,则认为这2个端口组互相关联,可以合并。

输入描述

第一行输入端口组个数M,再输入M行,每行逗号分割,代表端口组。

备注:端口组内数字可以重复。

输出描述

输出合并后的端口组，用二维数组表示。

- 组内相同端口仅保留一个，从小到大排序。
- 组外顺序保持输入顺序

备注：M,N不在限定范围内，统一输出一组空数组 [[]]

用例

输入	4 4 2,3,2 1,2 5
输出	[[4],[2,3],[1,2],[5]]
说明	仅有一个端口2相同，不可以合并。

输入	3 2,3,1 4,3,2 5
输出	[[1,2,3,4],[5]]
说明	无

输入	6 10 4,2,1 9 3,6,9,2 6,3,4 8
输出	[[10],[1,2,3,4,6,9],[9],[8]]
说明	无

输入	11
输出	[[]]
说明	无

题目解析

现在本题的题目意思就已经很明确了。

主要难度在于：组外顺序保持输入顺序

这里我利用反序遍历的方式，将后面的端口组并入前面的端口组，这样就可以避免组外顺序发生改变了。

Java算法源码

```
import java.util.*;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

int m = Integer.parseInt(sc.nextLine());

// M,N不在限定范围内，统一输出一组空数组[][]
if (m > 10 || m < 1) {
    System.out.println("[][]");
    return;
}

ArrayList<TreeSet<Integer>> ports = new ArrayList<>();
for (int i = 0; i < m; i++) {
    List<Integer> tmp =
        Arrays.stream(sc.nextLine().split(","))
            .map(Integer::parseInt)
            .collect(Collectors.toList());

    // M,N不在限定范围内，统一输出一组空数组[][]
    int n = tmp.size();
    if (n < 1 || n > 100) {
        System.out.println("[][]");
        return;
    }

    ports.add(new TreeSet<>(tmp));
}

System.out.println(getResult(ports, m));
}

public static String getResult(ArrayList<TreeSet<Integer>> ports, int m) {

    outer:
    while (true) {
        for (int i = m - 1; i >= 0; i--) {
            TreeSet<Integer> port1 = ports.get(i);
            if (port1.size() == 0) continue;

            for (int j = i - 1; j >= 0; j--) {
                TreeSet<Integer> port2 = ports.get(j);
                if (port2.size() == 0) continue;

                if (hasTwoSamePort(port1, port2)) {
                    port2.addAll(port1);
                    port1.clear();
                    continue outer;
                }
            }
        }
        break;
    }

    return ports.stream().filter(port -> port.size() >
0).collect(Collectors.toList()).toString();
}

```



```
// 判断两个区间是否可以合并
public static boolean hasTwoSamePort(TreeSet<Integer> port1, TreeSet<Integer>
port2) {
    int count = 0;
    for (Integer val : port1) {
        if (port2.contains(val)) {
            if (++count >= 2) return true;
        }
    }

    return false;
}
}
```

4. 密室逃生游戏

题目描述

小强在参加《密室逃生》游戏，当前关卡要求找到符合给定 密码K（升序的不重复小写字母组成）的箱子，并给出箱子编号，箱子编号为 1~N。

每个箱子中都有一个字符串s，字符串由大写字母、小写字母、数字、标点符号、空格组成，需要在这些字符串中找到所有的字母，忽略大小写后排列出对应的密码串，并返回匹配密码的箱子序号。

提示：满足条件的箱子不超过1个。

输入描述

第一行为 key 的字符串，

第二行为箱子 boxes，为数组样式，以逗号分隔

- 箱子 N 数量满足 $1 \leq N \leq 10000$,
- s 长度满足 $0 \leq s.length \leq 50$,
- 密码为仅包含小写字母的升序字符串，且不存在重复字母，
- 密码 K 长度 $1 \leq K.length \leq 26$

输出描述

返回对应箱子编号

如不存在符合要求的密码箱，则返回 -1。

用例

输入	abc s,sdf134 A2c4b
输出	2
说明	第 2 个箱子中的 Abc，符合密码 abc。

题目解析

题目不难，但是感觉本题的输入样例是有问题的：

题目描述中说：

每个箱子中都有一个字符串s，字符串由大写字母、小写字母、数字、标点符号、空格组成

输入描述中说：

第二行为箱子 boxes，为数组样式，以逗号分隔

因此，这里我理解每个箱子s字符串中应该不含有逗号。当然，考试的时候随机应变处理这个问题。我这里解题不考虑字符串s中有逗号的情况。

我的解题思路如下：

密码K是一个（升序的不重复小写字母组成）

首先，由于密码K都是小写字母，因此首先需要将箱子内的字符串s做一个转小写字母的操作。

其次，由于密码K不包含重复字母，因此这里可以将字符串s转为字符数组后，再变为[Set集合](#)，然后遍历密码K，如果K中每个字符都在Set中存在，则对应的字符串s就是一个符合要求的。

Java算法源码

```
import java.util.HashSet;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String key = sc.nextLine();
        String[] boxes = sc.nextLine().split(",");

        System.out.println(getResult(key, boxes));
    }

    public static int getResult(String key, String[] boxes) {
        outer:
        for (int i = 0; i < boxes.length; i++) {
            String box = boxes[i];
            if (box.length() < key.length()) continue;

            HashSet<Character> set = new HashSet<>();
            for (char c : box.toLowerCase().toCharArray()) set.add(c);

            for (int j = 0; j < key.length(); j++) {
                if (!set.contains(key.charAt(j))) continue outer;
            }

            return i + 1;
        }

        return -1;
    }
}
```

```
}  
}
```

5. 投篮大赛

题目描述

你现在是一场采用特殊赛制投篮大赛的记录员。这场比赛由若干回合组成，过去几回合的得分可能会影响以后几回合的得分。

比赛开始时，记录是空白的。

你会得到一个记录操作的字符串列表 `ops`，其中`ops[i]`是你需要记录的第*i*项操作，`ops`遵循下述规则：

- 整数`x` - 表示本回合新获得分数`x`
- `“+”` - 表示本回合新获得的得分是前两次得分的总和。
- `“D”` - 表示本回合新获得的得分是前一次得分的两倍。
- `“C”` - 表示本回合没有分数，并且前一次得分无效，将其从记录中移除。

请你返回记录中所有得分的总和。

输入描述

输入为一个字符串数组

输出描述

输出为一个整形数字

提示

1. $1 \leq ops.length \leq 1000$
2. `ops[i]` 为 `“C”`、`“D”`、`“+”`，或者一个表示整数的字符串。整数范围是 $[-3 * 10^4, 3 * 10^4]$
3. 需要考虑异常的存在，如有异常情况，请返回-1
4. 对于`“+”`操作，题目数据不保证记录此操作时前面总是存在两个有效的分数
5. 对于`“C”`和`“D”`操作，题目数据不保证记录此操作时前面存在一个有效的分数
6. 题目输出范围不会超过整型的最大范围，不超过 $2^{63} - 1$

用例

输入	5 2 C D +
输出	30

输入	5 2 C D +
说明	“5”-记录加5，记录现在是[5] “2”-记录加2，记录现在是[5,2] “C”-使前一次得分的记录无效并将其移除，记录现在是[5]. “D”-记录加2*5=10，记录现在是[5, 10]. “+”-记录加5+10=15，记录现在是[5, 10, 15]. 所有得分的总和5+10+15=30

输入	5 -2 4 C D 9 + +
输出	27
说明	“5”-记录加5，记录现在是[5] “-2”-记录加-2，记录现在是[5,-2] “4”-记录加4，记录现在是[5,-2,4] “C”-使前一次得分的记录无效并将其移除，记录现在是[5,-2]. “D”-记录加2*-2=4，记录现在是[5, -2, -4]. “9”-记录加9，记录现在是[5, -2, -4, 9]. “+”-记录加-4+9=5，记录现在是[5, -2, -4, 9, 5]. “+”-记录加-9+5=14，记录现在是[5, -2, -4, 9, 5, 14]. 所以得分的总和 $5 - 2 - 4 + 9 + 5 + 14 = 27$

输入	1
输出	1
说明	无

输入	+
输出	-1
说明	无

题目解析

简单的[逻辑题](#)，按照题目意思写就行。

Java算法源码

```
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        String[] ops = str.split(" ");

        System.out.println(getResult(ops));
    }

    public static int getResult(String[] ops) {
```

```

// ans用于保存每轮的得分
LinkedList<Integer> ans = new LinkedList<>();
String reg = "^\\-?\\d+$";

for (String op : ops) {
    // 如果op是整数，则表示本轮得分，直接加入ans
    if (op.matches(reg)) {
        ans.addLast(Integer.parseInt(op));
    } else {
        switch (op) {
            // 如果op是+，则表示本轮得分是前两轮得分之和，注意越界处理
            case "+":
                if (ans.size() < 2) return -1;
                ans.addLast(ans.getLast() + ans.get(ans.size() - 2));
                break;
            // 如果op是D，表示本轮得分是前一轮得分的双倍，注意越界处理
            case "D":
                if (ans.size() < 1) return -1;
                ans.addLast(ans.getLast() * 2);
                break;
            // 如果op是C，则表示本轮无得分，且上一轮得分无效，需要去除
            case "C":
                // 感谢网友m0_71826536的提示，由于题目说：对于“C”和“D”操作，题目数据不保证记录
                // 此操作时前面存在一个有效的分数，因此这里C操作，不能直接removeLast，需要先判断ans是否有数据
                if (ans.size() < 1) return -1;
                ans.removeLast();
                break;
        }
    }
}

int sum = 0;
for (Integer an : ans) {
    sum += an;
}
return sum;
}
}

```

6.货币单位换算

题目描述

记账本上记录了若干条多国货币金额，需要转换成人民币分（fen），汇总后输出。

每行记录一条金额，金额带有货币单位，格式为数字+单位，可能是单独元，或者单独分，或者元与分的组合。

要求将这些货币全部换算成人民币分（fen）后进行汇总，汇总结果仅保留整数，小数部分舍弃。

元和分的换算关系都是1:100，如下：

1CNY=100fen（1元=100分）

1HKD=100cents（1港元=100港分）

1JPY=100sen（1日元=100仙）

1EUR=100eurocents（1欧元=100欧分）

1GBP=100pence（1英镑=100便士）

汇率表如下：

CNY	JPY	HKD	EUR	GBP
100	1825	123	14	12

即：100CNY = 1825JPY = 123HKD = 14EUR = 12GBP

输入描述

第一行输入为N，N表示记录数。0<N<100

之后N行，每行表示一条货币记录，且该行只会是一种货币。

输出描述

将每行货币转换成人民币分（fen）后汇总求和，只保留整数部分。
输出格式只有整数数字，不带小数，不带单位。

用例

输入	1 100CNY
输出	10000
说明	100CNY转换后是10000fen，所以输出结果为10000

输入	1 3000fen
输出	3000
说明	3000fen，结果就是3000

输入	1 123HKD
输出	10000
说明	HKD与CNY的汇率关系是123:100，所以换算后，输出结果为10000

输入	2 20CNY53fen 53HKD87cents
输出	6432
说明	20元53分 + 53港元87港分，换算成人民币分后汇总，为6432

题目解析

纯逻辑题。

本题唯一的难度应该在于如何解析输入字符串中的金额和单位，这里最好是使用正则表达式：

```
> const regExp =  
    /(\d+)((CNY)|(JPY)|(HKD)|(EUR)|(GBP)|(fen)|(cents)|(sen)|(eurocents)|(pence))/g;  
{ undefined  
> regExp.exec('20CNY53fen')  
{ (13) ['20CNY', '20', 'CNY', 'CNY', undefined, undefined, undefined, undefined, undefined, undefi  
  ▶ ned, undefined, undefined, undefined, undefined, index: 0, input: '20CNY53fen', groups: undefined]  
> regExp.exec('20CNY53fen')  
{ (13) ['53fen', '53', 'fen', undefined, undefined, undefined, undefined, undefined, undefined, 'fen', undefi  
  ▶ ned, undefined, undefined, undefined, undefined, index: 5, input: '20CNY53fen', groups: undefined]  
> |
```

CSDN @伏城之外

剩下的就是将不同单位转换成人民币分的汇率计算工作了，如下源码中exchange记录的不同单位转成人民币分的汇率。

Java算法源码

正则解法

```
import java.util.*;  
import java.util.regex.Matcher;  
import java.util.regex.Pattern;  
  
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int n = sc.nextInt();  
  
        String[] arr = new String[n];  
        for (int i = 0; i < n; i++) {  
            arr[i] = sc.next();  
        }  
  
        System.out.println(getResult(arr));  
    }  
  
    public static int getResult(String[] arr) {  
        String reg = "(\\d+)((CNY)|(JPY)|(HKD)|(EUR)|(GBP)|(fen)|(cents)|(sen)|  
(eurocents)|(pence))";  
  
        HashMap<String, Double> exchange = new HashMap<>();  
        exchange.put("CNY", 100.0);  
        exchange.put("JPY", 100.0 / 1825 * 100);  
        exchange.put("HKD", 100.0 / 123 * 100);  
        exchange.put("EUR", 100.0 / 14 * 100);  
        exchange.put("GBP", 100.0 / 12 * 100);  
        exchange.put("fen", 1.0);  
        exchange.put("cents", 100.0 / 123);  
        exchange.put("sen", 100.0 / 1825);  
        exchange.put("eurocents", 100.0 / 14);  
        exchange.put("pence", 100.0 / 12);  
    }  
}
```

```

        double ans = 0.0;
        Pattern p = Pattern.compile(reg);

        for (String s : arr) {
            Matcher m = p.matcher(s);
            while (true) {
                if (m.find()) {
                    Double amount = Double.parseDouble(m.group(1));
                    String unit = m.group(2);
                    ans += amount * exchange.get(unit);
                } else {
                    break;
                }
            }
        }

        return (int) ans;
    }
}

```

非正则解法

```

import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        String[] arr = new String[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.next();
        }

        System.out.println(getResult(arr));
    }

    public static int getResult(String[] arr) {
        HashMap<String, Double> exchange = new HashMap<>();
        exchange.put("CNY", 100.0);
        exchange.put("JPY", 100.0 / 1825 * 100);
        exchange.put("HKD", 100.0 / 123 * 100);
        exchange.put("EUR", 100.0 / 14 * 100);
        exchange.put("GBP", 100.0 / 12 * 100);
        exchange.put("fen", 1.0);
        exchange.put("cents", 100.0 / 123);
        exchange.put("sen", 100.0 / 1825);
        exchange.put("eurocents", 100.0 / 14);
        exchange.put("pence", 100.0 / 12);

        double ans = 0.0;

        StringBuilder sb = new StringBuilder();
    }
}

```



```

for (String s : arr) sb.append(s);
String str = sb.toString() + "0";

StringBuilder num = new StringBuilder();
StringBuilder unit = new StringBuilder();
for (int i = 0; i < str.length(); i++) {
    char c = str.charAt(i);

    if (c >= '0' && c <= '9') {
        if (unit.length() != 0) {
            ans += Integer.parseInt(num.toString()) *
exchange.get(unit.toString());
            num = new StringBuilder();
            unit = new StringBuilder();
        }
        num.append(c);
    } else if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')) {
        unit.append(c);
    }
}

return (int) ans;
}
}

```

7.新员工座位

题目描述

工位由序列F1,F2...Fn组成，Fi值为0、1或2。其中0代表空置，1代表有人，2代表障碍物。

- 1、某一空位的友好度为左右连续老员工数之和，
- 2、为方便新员工学习求助，优先安排友好度高的空位，

给出工位序列，求所有空位中**友好度的最大值**。

输入描述

第一行为工位序列：F1，F2...Fn组成，
1<=n<=10000，Fi值为0、1或2。其中0代表空置，1代表有人，2代表障碍物。

输出描述

所有空位中友好度的最大值。如果没有空位，返回0。

用例

输入	0 1 0
输出	1
说明	第1个位置和第3个位置，友好度均为1。

输入	1 1 0 1 2 1 0
输出	3
说明	第3个位置友好度为3。因障碍物隔断，左边得2分，右边只能得1分。

题目解析

本题最优解题思路如下：

定义一个变量friendShip = 0,

首先对[输入数组](#)进行正向遍历（从左到右）：

- 遇到“1”，则friendShip++
- 遇到“0”，则说明该空位左边的友好度为friendShip，记录下来，然后将friendShip重置为0
- 遇到“2”，则直接将friendShip重置为0

这样每个空位的左边的友好度就统计出来了。

接着，将friendShip重置为0,

然后对输入数组进行反向遍历（从右到左）：

- 遇到“1”，则friendShip++
- 遇到“0”，则说明该空位右边的友好度为friendShip，累加下来，然后将friendShip重置为0
- 遇到“2”，则直接将friendShip重置为0

这样每个空位的整体友好度就统计出来了，取最大值返回。

Java算法源码

```
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String[] arr = sc.nextLine().split(" ");

        System.out.println(getResult(arr));
    }

    public static int getResult(String[] arr) {
        // 记录空位的友好度
```

```

HashMap<Integer, Integer> ep = new HashMap<>();

int friendship = 0;
// 从左向右遍历，记录每个空位左边的友好度
for (int i = 0; i < arr.length; i++) {
    switch (arr[i]) {
        case "0":
            ep.put(i, friendship);
            friendship = 0;
            break;
        case "1":
            friendship++;
            break;
        case "2":
            friendship = 0;
            break;
    }
}

friendship = 0;
int ans = 0;
// 从右向左遍历，累加每个空位右边的友好度，这样就得到了每个空位的友好度，取最大值即可
for (int i = arr.length - 1; i >= 0; i--) {
    switch (arr[i]) {
        case "0":
            ans = Math.max(ans, ep.get(i) + friendship);
            friendship = 0;
            break;
        case "1":
            friendship++;
            break;
        case "2":
            friendship = 0;
            break;
    }
}

return ans;
}
}

```

8. 日志限流

题目描述

某软件系统会在运行过程中持续产生日志，系统每天运行N单位时间，运行期间每单位时间产生的日志条数保行在数组records中。records[i]表示第i单位时间内产生日志条数。

由于系统磁盘空间限制，每天可记录保存的日志总数上限为total条。

如果一天产生的日志总条数大于total，则需要对当天内每单位时间产生的日志条数进行限流后保存，请计算每单位时间最大可保存日志条数limit，以确保当天保存的总日志条数不超过total。

对于单位时间内产生日志条数不超过limit的日志全部记录保存；

对于单位时间内产生日志条数超过limit的日志，则只记录保存limit条日志；

如果一天产生的日志条数总和小于等于total，则不需要启动限流机制。result为-1。
请返回result的最大值或者-1。

输入描述

第一行为系统某一天运行的单位时间数N, $1 \leq N \leq 10^5$
第二行为表示这一天每单位时间产生的日志数量的数组records[], $0 \leq records[i] \leq 10^5$
第三行为系统一天可以保存的总日志条数total。 $1 \leq total \leq 10^9$

输出描述

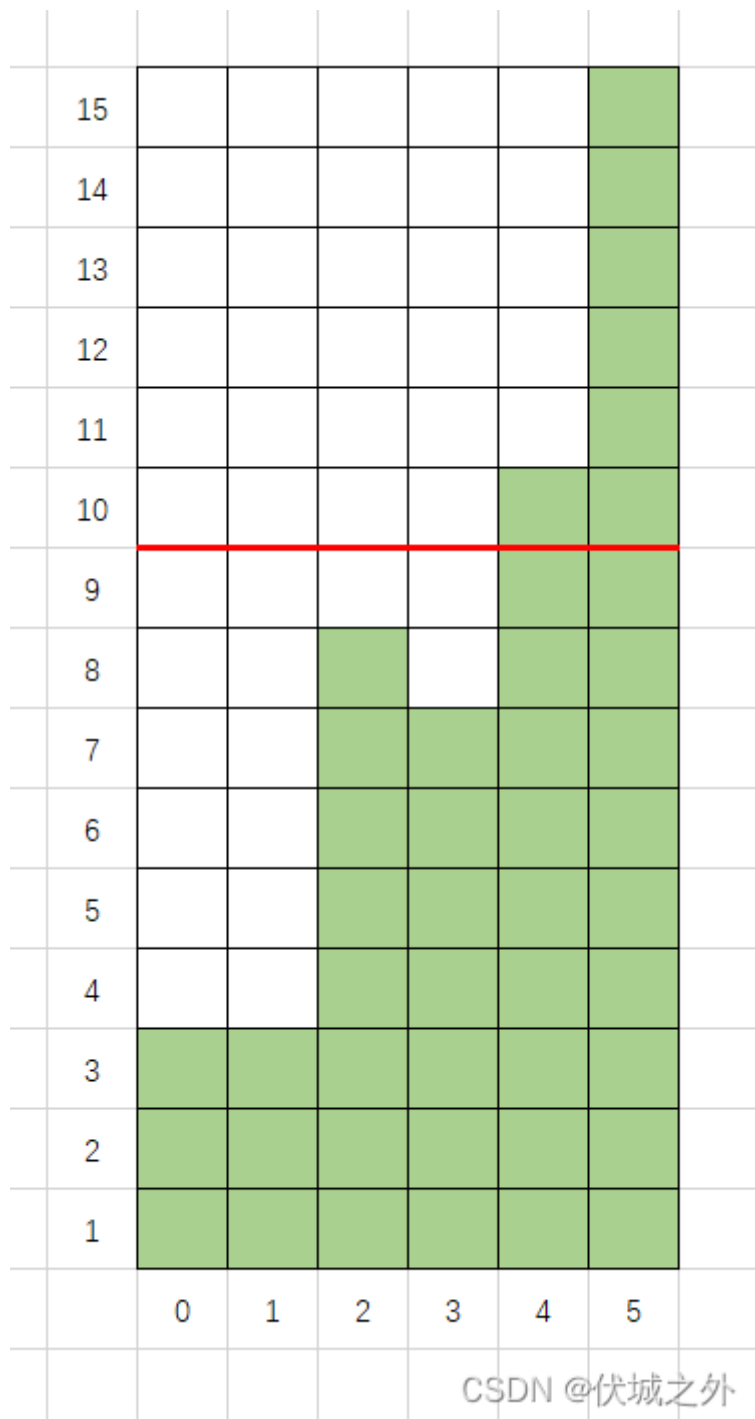
每单位时间内最大可保存的日志条数limit，如果不需要启动限流机制，返回-1。

用例

输入	6 3 3 8 7 10 15 40
输出	9
说明	无

题目解析

[用例图](#)示如下



CSDN @伏城之外

我们可以知道，本题有一个理想的limit，值为 total / n ，取这个limit，得到限流后总日志数只可能小于等于total，原因很简单，大家可以自己想想。

因此，我们可以将 total / n 当成最小limit取值，而最大limit取值其实就是 $\text{max}(\text{records})$ ，即初始时：

- $\text{max_limit} = \text{max}(\text{records})$
- $\text{min_limit} = \text{total} / n$

我们每次都取 min_limit 和 max_limit 的二分值作为测试limit，测试逻辑如下：

- 遍历records每一个record，如果record数量小于等于limit，则不做限流，如果record大于limit，则限流为limit条，然后求限流后日志总条数tmp

得到日志总条数后，如果

- $\text{tmp} < \text{total}$ ，则说明limit取小了，则应该提高limit值，即让 $\text{min_limit} = \text{limit}$
- $\text{tmp} > \text{total}$ ，则说明limit取大了，则应该降低limit值，即让 $\text{max_limit} = \text{limit}$

- $tmp == total$, 则说明limit取得刚刚好, 此时的limit就是最佳limit

当然我们可能无法遇到 $tmp == total$ 的情况, 此时我们应该定义一个ans变量, 保存 $tmp < total$ 时的limit值, 如果最终没有 $tmp == total$ 的情况, 则最后应该返回ans作为题解。

另外, 在这些逻辑之前, 我们可以先对records求和sum, 如果 $sum \leq total$, 则不需要做上面逻辑。

上面算法的时间复杂度为 $O(\log(total) * N)$

- $1 \leq N \leq 10^5$
- $1 \leq total \leq 10^9$

因此性能还算比较优异。

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] records = new int[n];
        for (int i = 0; i < n; i++) {
            records[i] = sc.nextInt();
        }

        int total = sc.nextInt();

        System.out.println(getResult(n, records, total));
    }

    /**
     * @param n 系统某一天运行的单位时间数N
     * @param records 这一天每单位时间产生的日志数量的数组
     * @param total 系统一天可以保存的总日志条数total
     * @return 每单位时间内最大可保存的日志条数limit, 如果不需要启动限流机制, 返回-1
     */
    public static int getResult(int n, int[] records, int total) {
        int sum = Arrays.stream(records).reduce(Integer::sum).getAsInt();

        // 如果一天产生的日志条数总和小于等于total, 则不需要启动限流机制. result为-1
        if (sum <= total) return -1;

        // Arrays.sort(records);

        // int max_limit = records[records.length - 1];
        int max_limit = Arrays.stream(records).max().getAsInt();
        int min_limit = total / n;
```

```

int ans = min_limit;
while (max_limit - min_limit > 1) {
    int limit = (max_limit + min_limit) / 2;

    int tmp = 0;
    for (int record : records) {
        tmp += Math.min(record, limit);
    }

    if (tmp > total) {
        max_limit = limit;
    } else if (tmp < total) {
        min_limit = limit;
        ans = limit;
    } else {
        return limit;
    }
}

return ans;
}
}

```

9. 称砝码

题目描述

现有 n 种砝码，重量互不相等，分别为 $m_1, m_2, m_3 \dots m_n$ ；

每种砝码对应的数量为 $x_1, x_2, x_3 \dots x_n$ 。现在要用这些砝码去称物体的重量(放在同一侧)，问能称出多少种不同的重量。

输入描述

对于每组测试数据：

第一行： n --- 砝码的种数(范围 $[1, 10]$)

第二行： $m_1 \ m_2 \ m_3 \ \dots \ m_n$ --- 每种砝码的重量(范围 $[1, 2000]$)

第三行： $x_1 \ x_2 \ x_3 \ \dots \ x_n$ --- 每种砝码对应的数量(范围 $[1, 10]$)

输出描述

利用给定的砝码可以称出的不同的重量数

备注

数据范围：每组输入数据满足：

- $1 \leq n \leq 10$
- $1 \leq m_i \leq 2000$
- $1 \leq x_i \leq 10$

用例

输入	2 1 2 2 1
输出	5
说明	可以表示出0, 1, 2, 3, 4五种重量。

题目解析

本题可以使用[多重背包](#)求解。关于多重背包问题，其实就是01背包的变种问题。

[01背包](#)问题：

有N种物品，每种物品只有一个，第i个物品的重量为 w_i ，价值为 p_i ，另外还有一个承重为W的背包，问该背包在不超载的情况下，装入物品的最大价值是多少？

[多重背包](#)问题

有N种物品，第i个物品的重量为 w_i ，价值为 p_i ，数量为 c_i ，另外还有一个承重为W的背包，问该背包在不超载的情况下，装入物品的最大价值是多少？

学会01背包后，多重背包问题的求解就非常简单了，其实我们可以将多重背包问题，转化为01背包问题，怎么转化呢？

举个例子：你有红富士苹果10个，那么是不是等价于红富士苹果A有1个，红富士苹果B有1个，...，红富士苹果J有1个？

其实这就是多重背包转化为01背包的方法。即将1种物品N个，转化为N种物品1个。

关于多重背包的求解有两种解法：

- 1、朴素解法
- 2、二进制优化解法

本题中

- N种砝码 → N种物品
- 每种砝码的重量 → 每种物品的重量
- 每种砝码的重量 → 每种物品的价值
- 每种砝码的数量 → 每种物品的数量
- 所有砝码的重量之和 → 背包承重

本题要求的是：用给的砝码能组合出多少种重量

而这其实刚好和求解背包问题时，遍历所有可能的背包承重相符，因此本题非常适合当成背包问题求解。

朴素解法

Java算法源码

```
import java.util.Scanner;
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        int[] m = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            m[i] = sc.nextInt();
        }

        int[] x = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            x[i] = sc.nextInt();
        }

        System.out.println(getResult(n, m, x));
    }

    public static int getResult(int n, int[] m, int[] x) {
        int bag = 0;
        for (int i = 1; i <= n; i++) bag += m[i] * x[i];

        boolean[] dp = new boolean[bag + 1];
        dp[0] = true;

        for (int i = 1; i <= n; i++) {
            for (int j = bag; j >= m[i]; j--) {
                for (int k = 1; k <= x[i]; k++) {
                    if (j >= m[i] * k) {
                        if (dp[j - m[i] * k]) dp[j] = true;
                    }
                }
            }
        }

        int count = 0;
        for (boolean flag : dp) {
            if (flag) count++;
        }

        return count;
    }
}
```

二进制优化解法

Java算法源码

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        int[] m = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            m[i] = sc.nextInt();
        }

        int[] x = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            x[i] = sc.nextInt();
        }

        System.out.println(getResult(n, m, x));
    }

    public static int getResult(int n, int[] m, int[] x) {
        int bag = 0;
        ArrayList<Integer> newM = new ArrayList<>();

        for (int i = 1; i <= n; i++) {
            // 背包承重等于所有砝码重量之和
            bag += m[i] * x[i];

            // 将1种物品N个，进行二进制优化
            for (int j = 1; j <= x[i]; j <= 1) {
                newM.add(m[i] * j);
                x[i] -= j;
            }

            if (x[i] != 0) {
                newM.add(m[i] * x[i]);
            }
        }

        // 01背包滚动数组优化求解
        boolean[] dp = new boolean[bag + 1];
        // 0重量组合肯定存在
        dp[0] = true;

        for (Integer w : newM) {
            for (int j = bag; j >= w; j--) {
                // 如果j-w组合重量存在，那么j重量组合也一定存在
                if (dp[j - w]) dp[j] = true;
            }
        }
    }
}
```

```
}

int count = 0;
for (boolean flag : dp) {
    if (flag) count++;
}

return count;
}
}
```

10. 最优资源分配

题目描述

某块业务芯片最小容量单位为1.25G，总容量为 $M \times 1.25G$ ，对该芯片资源编号为1, 2, ..., M。该芯片支持3种不同的配置，分别为A、B、C。

- 配置A：占用容量为 $1.25 \times 1 = 1.25G$
- 配置B：占用容量为 $1.25 \times 2 = 2.5G$
- 配置C：占用容量为 $1.25 \times 8 = 10G$

某块板卡上集成了N块上述芯片，对芯片编号为1, 2, ..., N，各个芯片之间彼此独立，不能跨芯片占用资源。

给定板卡上芯片数量N、每块芯片容量M、用户按次序配置后，请输出芯片资源占用情况，保证消耗的芯片数量最少。

资源分配规则：按照芯片编号从小到大分配所需资源，芯片上资源如果被占用标记为1，没有被占用标记为0。

用户配置序列：用户配置是按次序依次配置到芯片中，如果用户配置序列中某个配置超过了芯片总容量，丢弃该配置，继续遍历用户后续配置。

输入描述

M：每块芯片容量为 $M \times 1.25G$ ，取值范围为：1~256

N：每块板卡包含芯片数量，取值范围为1~32

用户配置序列：例如ACABA，长度不超过1000

输出描述

板卡上每块芯片的占用情况

备注

用户配置是按次序依次配置到芯片中，如果用户配置序列种某个配置超过了芯片总容量，丢弃该配置，继续遍历用户后续配置。

用例

输入	8 2 ACABA
输出	11111000 11111111
说明	用户第1个配置A：占用第1块芯片第1个资源，芯片占用情况为：1000000000000000用户第2个配置C：第1块芯片剩余8.75G，配置C容量不够，只能占用第2块芯片，芯片占用情况为：1000000011111111用户第3个配置A：第1块芯片剩余8.75G，还能继续配置，占用第1块芯片第2个资源，芯片占用情况为：1100000011111111用户第4个配置B：第1块芯片剩余7.5G，还能继续配置，占用第1块芯片第3、4个资源，芯片占用情况为：1111000011111111用户第5个配置A：第1块芯片剩余5G，还能继续配置，占用第1块芯片第5个资源，芯片占用情况为：1111100011111111

输入	8 2 ACBCB
输出	11111000 11111111
说明	用户第1个配置A：占用第1块芯片第1个资源，芯片占用情况为：1000000000000000用户第2个配置C：第1块芯片剩余8.75G，配置C容量不够，只能占用第2块芯片，芯片占用情况为：1000000011111111用户第3个配置B：第1块芯片剩余8.75G，还能继续配置，占用第1块芯片第2、3个资源，芯片占用情况为：1110000011111111用户第4个配置C：芯片资源不够，丢弃配置，继续下一个配置，本次配置后芯片占用情况保持不变：1110000011111111用户第5个配置B：第1块芯片剩余6.25G,还能继续配置，占用第1块芯片第4、5个资源，芯片占用情况为：1111100011111111

题目解析

本题输出比较难以理解，我这里以用例1解释一下：

用例1的前两行输入表示：

板卡上有N=2个芯片，而每个芯片有8个单位容量，因此对应如下：

```
00000000
00000000
```

其中每个0代表一个单位容量，而一个芯片有8单位容量，因此第一排8个0代表一个芯片的总容量，第二排8个0代表另一个芯片的总容量。

理解了这个，本题就不难了。

Java算法源码

```
import java.util.Arrays;
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
        int n = sc.nextInt();
        String sequence = sc.next();

        getResult(m, n, sequence);
    }

    public static void getResult(int m, int n, String sequence) {
        double[] boardCard = new double[n];
        Arrays.fill(boardCard, m * 1.25);

        HashMap<Character, Integer> dict = new HashMap<>();
        dict.put('A', 1);
        dict.put('B', 2);
        dict.put('C', 8);

        for (int i = 0; i < sequence.length(); i++) {
            double need = 1.25 * dict.get(sequence.charAt(i));
            for (int j = 0; j < n; j++) {
                if (boardCard[j] >= need) {
                    boardCard[j] -= need;
                    break;
                }
            }
        }

        for (int i = 0; i < n; i++) {
            int unused = (int) (boardCard[i] / 1.25);
            int used = m - unused;

            StringBuilder sb = new StringBuilder();
            for (int j = 0; j < used; j++) {
                sb.append(1);
            }
            for (int k = 0; k < unused; k++) {
                sb.append(0);
            }
            System.out.println(sb);
        }
    }
}
```

11. 优选核酸检测点

题目描述

张三要去外地出差，需要做核酸，需要在指定时间点前做完核酸，请帮他找到满足条件的[核酸检测点](#)。

- 给出一组核酸检测点的距离和每个核酸检测点当前的人数
- 给出张三去做核酸的出发时间 出发时间是10分钟的倍数，同时给出张三做核酸的最晚结束时间
- 题目中给出的距离是整数，单位是公里，时间1分钟为一基本单位

去找核酸点时，有如下的限制：

- 去往核酸点的路上，每公里距离花费时间10分钟，费用是10元
- 核酸点每检测一个人的时间花费是1分钟
- 每个核酸点工作时间都是8点到20点中间不休息，核酸点准时工作，早到晚到都不检测
- 核酸检测结果可立刻知道
- 在张三去某个核酸点的路上花费的时间内，此核酸检测点的人数是动态变化的，变化规则是
 1. 在非核酸检测时间内，没有人排队
 2. 8点-10点每分钟增加3人
 3. 12点-14点每分钟增加10人

要求将所有满足条件的核酸检测点按照优选规则排序列出：

优选规则：

1. 花费时间最少的核酸检测点排在前面。
2. 花费时间一样,花费费用最少的核酸检测点排在前面。
3. 时间和费用一样，则ID值最小的排在前面

输入描述

```
H1 M1
H2 M2
N
ID1 D1 C1
ID2 D2 C2
...
IDn Dn Cn
```

H1: 当前时间的小时数。

M1: 当前时间的分钟数，

H2: 指定完成核算时间的小时数。

M2: 指定完成核算时间的分钟数。

N: 所有核酸检测点个数。

ID1: 核酸点的ID值。

D1: 核酸检测点距离张三的距离。

C1: 核酸检测点当前检测的人数。

输出描述

N

I2 T2 M2

I3 T3 M3

N: 满足要求的核酸检测点个数

I2: 选择后的核酸检测点ID

T2: 做完核酸花费的总时间(分钟)

M3: 去该核算点花费的费用

用例

输入	10 30 14 50 3 1 10 19 2 8 20 3 21 3
输出	2 2 80 80 1 190 100
说明	无

题目解析

用例意思是：

张三在10:30出门，要在14:50之前做完核酸。

现在张三可选三个核酸检测点：

- 检测点1：距离张三10公里，10:30的时候有19个人排队
- 检测点2：距离张三8公里，10:30的时候有20个人排队
- 检测点3：距离张三21公里，10:30的时候有3个人排队

张三赶到检测点1，需要 $10 \times 10 = 100$ 元， $10 \times 10 = 100$ 分钟，而在张三到达检测点时12:10，此时排队的人数是为：



通过上图，我们可以看出：在10:30~12:00期间不会有人加入，只会有人离开，每分钟离开1人，因此到12:00时，最多离开 $1260 - (1060 + 30) = 90$ 人，而10:30时只有19人排队，因此到12:00时，检测点1只有0人排队。

然后12:00到12:10阶段，每分钟离开1人，增加10人，因此相当于每分钟净增9人，因此到12:10，即张三到达时，检测点共有： $10 * 9 = 90$ 人。

因此张三还需排90分钟，才能做完核酸。

因此张三到检测点1的代价是：路上100分钟，到达后等待90分钟，共需190分钟，花费100元。

同理，可得张三去其他检测点的代价。

然后，过滤掉花费时间超出限制的代价，剩下的按照花费时间、花费金额排序即可。

我们可以通过求区间交集的方式，来获取张三【出发时间，到达时间】和【8:00, 10:00】以及【10:00, 12:00】，以及【12:00, 14:00】以及【14:00, 20:00】的交集。

其中，

- 和【8:00, 10:00】的交集，每分钟净增2人
- 和【10:00, 12:00】的交集，每分钟净减1人
- 和【12:00, 14:00】的交集，每分钟净增9人
- 和【14:00, 20:00】的交集，每分钟净减1人

2023.1.17补充说明：

根据网友m0_71826536的提示，如果张三在8:00前就赶到了核酸监测点，但是8:00前是不给排队的，因此张三还要等待到8:00，因此张三花费的时间其实是：路上时间 + 等待时间 + 排队时间

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int h1 = sc.nextInt();
        int m1 = sc.nextInt();

        int h2 = sc.nextInt();
        int m2 = sc.nextInt();

        int n = sc.nextInt();

        int[][] idcs = new int[n][3];
        for (int i = 0; i < n; i++) {
            idcs[i][0] = sc.nextInt();
            idcs[i][1] = sc.nextInt();
            idcs[i][2] = sc.nextInt();
        }

        getResult(h1, m1, h2, m2, idcs);
    }
}
```



```

/**
 * @param h1 当前时间的小时数
 * @param m1 当前时间的分钟数
 * @param h2 指定完成核算时间的小时数
 * @param m2 指定完成核算时间的分钟数
 * @param idcs [[核酸点的ID值, 核酸检测点距离张三的距离, 核酸检测点当前检测的人数]]
 */
public static void getResult(int h1, int m1, int h2, int m2, int[][] idcs) {
    int start = h1 * 60 + m1;
    int end = h2 * 60 + m2;

    int[][] ans =
        Arrays.stream(idcs)
            .map(
                idc -> {
                    int id = idc[0];
                    int distance = idc[1];
                    int count = idc[2];

                    int money = distance * 10;
                    int road = distance * 10; // 花在路上的时间
                    int arrived = start + road; // 到达核酸检测点的时间

                    // 如果在8:00之前就赶到了, 那么其实要等待到8:00才能排队, 这里其实花费的
                    // 时间应该包括等待的时间
                    if (arrived < 8 * 60) {
                        arrived = 8 * 60;
                        road = arrived - start;
                    }

                    // 出发时间, 结束时间
                    int[] ran1 = {start, arrived};

                    // 和[8:00, 10:00]的交集, 每分钟净增2人
                    int[] ran2 = {8 * 60, 10 * 60};
                    int add1 = getIntersection(ran1, ran2);
                    if (add1 != -1) {
                        count += 2 * add1;
                    }

                    // 和[10:00, 12:00]的交集, 每分钟净减1人
                    int[] ran3 = {10 * 60, 12 * 60};
                    int min1 = getIntersection(ran1, ran3);
                    if (min1 != -1) {
                        count -= min1;
                        count = Math.max(0, count); // 注意至多减到0
                    }

                    // 和[12:00, 14:00]的交集, 每分钟净增9人
                    int[] ran4 = {12 * 60, 14 * 60};
                    int add2 = getIntersection(ran1, ran4);
                    if (add2 != -1) {
                        count += 9 * add2;
                    }
                }
            )
            .toArray(int[][]::new);
}

```

```

        // 和[14:00, 20:00]的交集，每分钟净减1人
        int[] ran5 = {14 * 60, 20 * 60};
        int min2 = getIntersection(ran1, ran5);
        if (min2 != -1) {
            count -= min2;
            count = Math.max(0, count); // 注意至多减到0
        }

        return new int[] {id, count + road, money};
    })

    // arr[1] = count + road，因此实际到达时间为start + arr[1]，此处需要过滤出
    规定结束时间end之前可达的核酸点
    .filter(arr -> start + arr[1] <= end)
    .sorted((a, b) -> a[1] != b[1] ? a[1] - b[1] : a[2] != b[2] ? a[2] -
b[2] : a[0] - b[0])
    .toArray(int[][]::new);

    System.out.println(ans.length);
    for (int[] arr : ans) {
        System.out.println(arr[0] + " " + arr[1] + " " + arr[2]);
    }
}

// 获取交集长度，如果没有交集返回-1
public static int getIntersection(int[] ran1, int[] ran2) {
    int s1 = ran1[0], e1 = ran1[1];
    int s2 = ran2[0], e2 = ran2[1];

    if (s1 < s2) {
        if (s2 >= e1) return -1;
        else return Math.min(e1, e2) - s2;
    } else if (s1 > s2) {
        if (s1 >= e2) return -1;
        else return Math.min(e1, e2) - s1;
    } else {
        return Math.min(e1, e2) - s1;
    }
}
}
}

```

12. 日志首次上报最多积分

题目描述

日志采集是运维系统的的核心组件。日志是按行生成，每行记做一条，由采集系统分批上报。

- 如果上报太频繁，会对服务端造成压力；
- 如果上报太晚，会降低用户的体验；
- 如果一次上报的条数太多，会导致超时失败。

为此，项目组设计了如下的上报策略：

1. 每成功上报一条日志，奖励1分
2. 每条日志每延迟上报1秒，扣1分

3. 积累日志达到100条，必须立即上报

给出日志序列，根据该规则，计算首次上报能获得的最多积分数。

输入描述

按时序产生的日志条数 T1,T2...Tn，其中 1<=n<=1000，0<=Ti<=100

输出描述

首次上报最多能获得的积分数

用例

输入	1 98 1
输出	98
说明	T1 时刻上报得 1 分 T2 时刻上报得98分，最大 T3 时刻上报得 0 分

输入	50 60 1
输出	50
说明	如果第1个时刻上报，获得积分50。如果第2个时刻上报，最多上报100条，前50条延迟上报1s，每条扣除1分，共获得积分为 100-50=50

输入	3 7 40 10 60
输出	37
说明	T1时刻上报得3分T2时刻上报得7分T3时刻上报得37分，最大T4时刻上报得-3分T5时刻上报，因为已经超了100条限制，所以只能上报100条，得-23分

题目解析

用例1意思是：

如果在T1时刻上报日志，由于只有1条日志，因此可得1分。

如果在T2时刻上报日志，由于T1时刻产生的1条日志延迟了1秒上报，因此要扣1分，到了T2时刻可以上报1+98条日志，可得99分，因此99-1=98分。

如果在T3时刻上报日志，则T1日志延迟了2s，要扣12分，T2日志延迟了1s，要扣98分，T3时刻可以上报100条日志，可得100分，因此 $100 - 2 - 98 = 0$ 分。

我的解题思路如下：

这种前后数据具有依赖关系，一般都是用[动态规划](#)DP解决。

首先，我用前缀和思路，将每个时刻，可得的正向分计算出来缓存进dp数组中。所谓正向分，比如T2时刻积累了99条日志，那么就应该得到99分。这是正向得分。但是最终得分还需要扣除延迟上报的负向得分。比如T2时刻上报日志的话，则T1时刻产生的每条日志都需要扣除1分，这是负向得分。因此T2时刻的最终得分是：正向得分 - 负向得分 = $99 - 1 = 98$ 分。

每个时刻都有两种选择，上报、不上报。如果某时刻选择不上报，则会正向得分、负向得分不会清零，即会累加给下一个时刻，如果某时刻选择上报了，则对应的正向得分和负向得分都会清零。

需要注意的是，当日志累计到100条或以上时，则必须要上报，这意味着会产生一次得分清零。

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Integer[] arr =
            Arrays.stream(sc.nextLine().split("
"))
                .map(Integer::parseInt)
                .toArray(Integer[]::new);

        System.out.println(getResult(arr));
    }

    public static int getResult(Integer[] arr) {
        int n = arr.length;

        // dp[i]表示：第i时刻可得的正向分
        int[] dp = new int[n];
        dp[0] = arr[0];

        // delay[i]表示：第i时刻被扣除的负向分
        int[] delay = new int[n];
        delay[0] = 0;

        // score[i]表示：第i时刻最终得分
        int[] score = new int[n];
        score[0] = arr[0];

        for (int i = 1; i < n; i++) {
            dp[i] = Math.min(100, dp[i - 1] + arr[i]); // 最多上报100条
            delay[i] = delay[i - 1] + dp[i - 1];
            score[i] = dp[i] - delay[i];
        }
    }
}
```

```
// 达到100条时必须上报，此时完成首次上报，结束循环
if (dp[i] >= 100) break;
}

return Arrays.stream(score).max().getAsInt();
}
}
```

13. 冗余覆盖

题目描述

给定两个字符串s1和s2和正整数K，其中s1长度为n1，s2长度为n2，在s2中选一个子串，满足：

- 该子串长度为n1+k
- 该子串中包含s1中全部字母，
- 该子串每个字母出现次数不小于s1中对应的字母，

我们称s2以长度k冗余覆盖s1，给定s1，s2，k，求最左侧的s2以长度k冗余覆盖s1的子串的**首个元素的下标**，如果没有返回-1。

输入描述

输入三行，第一行为s1，第二行为s2，第三行为k，s1和s2只包含小写字母

输出描述

最左侧的s2以长度k冗余覆盖s1的子串首个元素下标，如果没有返回-1。

用例

输入	ab aabcd 1
输出	0
说明	无

题目解析

本题的难点在于如何计算s2选中子串是否能够覆盖住s1，即s2选中子串中的对应字符个数都大于s1中每个字符个数。

本题可以参考最小覆盖子串中统计覆盖子串字符个数的求解思路。

Java算法源码

下面统计s1中各字符数量时，容器没有使用HashMap，因为后期获取和处理HashMap中数据时比较麻烦，而是利用s1中所有字符都是小写字母的特点，使用128长度的int数组，因为小写字母的ASCII码范围是97~122，因此可以对应到0~127的int数组的索引上。

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String s1 = sc.next();
        String s2 = sc.next();
        int k = sc.nextInt();

        System.out.println(getResult(s1, s2, k));
    }

    public static int getResult(String s1, String s2, int k) {
        // 在s2中选一个子串，满足：该子串长度为 n1+k
        int n1 = s1.length();
        int n2 = s2.length();
        if (n2 < n1 + k) return -1;

        // 由于字符串s1中都是小写字母，因此每个字母的ASCII码范围是97~122，因此这里初始化128长度
        // 数组来作为统计容器
        int[] count = new int[128];

        // 统计s1中所有每个字符出现的次数到count中
        for (int i = 0; i < n1; i++) {
            int c = s1.charAt(i);
            count[c]++;
        }

        // s1字符总数
        int total = n1;

        // 滑动窗口的左边界从0开始，最大maxI：滑动窗口长度len
        int maxI = n2 - n1 - k;
        // s2子串长度
        int len = n1 + k;

        // 统计s2的0~len范围内出现的s1中字符的次数
        for (int j = 0; j < len; j++) {
            int c = s2.charAt(j);

            // 如果s2的0~len范围内的字符c，在count[c]存在，则说明c是s1内有的字符，
            // 此时我们需要count[c]--，如果自减之前，count[c] > 0，则自减时，total也应该--，否则
            // total不--
            if (count[c]-- > 0) {
                total--;
            }

            // 如果total为0了，则说明在s2的0~len范围内找到了所有s1中字符
        }
    }
}
```

```

        if (total == 0) {
            // 此时可以直接返回起始索引0
            return 0;
        }
    }

    // 下面是从左边界1开始的滑动窗口，利用差异思想，避免重复部分求解
    for (int i = 1; i <= maxI; i++) {
        // 滑动窗口右移一格后，失去了s2[i - 1]，得到了s2[i - 1 + len]，其余部分不变
        int remove = s2.charAt(i - 1);
        int add = s2.charAt(i - 1 + len);

        if (count[remove]++ >= 0) {
            total++;
        }

        if (count[add]-- > 0) {
            total--;
        }

        if (total == 0) {
            return i;
        }
    }
    return -1;
}
}

```

14. 最多颜色的车辆

题目描述

在一个狭小的路口，每秒只能通过一辆车，假设车辆的颜色只有 3 种，找出 N 秒内经过的最多颜色的车辆数量。

三种颜色编号为0，1，2

输入描述

第一行输入的是通过的车辆颜色信息

[0,1,1,2] 代表4秒钟通过的车辆颜色分别是 0, 1, 1, 2

第二行输入的是统计时间窗，整型，单位为秒

输出描述

输出指定时间窗内经过的最多颜色的车辆数量。

用例

输入	0 1 2 1 3
输出	2
说明	在 3 秒时间窗内，每个颜色最多出现 2 次。例如：[1,2,1]

输入	0 1 2 1 2
输出	1
说明	在 2 秒时间窗内，每个颜色最多出现1 次。

题目解析

简单的[滑动窗口](#)应用。我们可以利用相邻两个滑窗的差异比较，来避免重复的计算。

比如：下图是用例1的滑窗（黄色部分）运动过程



第二个滑窗相较于第一个滑窗而言，失去了0，新增了1，因此我们不需要重新统计第二个滑窗内部各种颜色的数量，只需要在第一个滑窗的统计结果基础上，减少0颜色数量1个，增加1颜色数量1个即可。

Java算法源码

```
import java.util.Arrays;
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Integer[] arr =
            Arrays.stream(sc.nextLine().split("
"))
                .map(Integer::parseInt)
                .toArray(Integer[]::new);
        int n = sc.nextInt();

        System.out.println(getResult(arr, n));
    }

    public static int getResult(Integer[] arr, int n) {
        // count用于统计滑动窗口内各种颜色的数目
        HashMap<Integer, Integer> count = new HashMap<>();
        count.put(0, 0);
        count.put(1, 0);
```



```

count.put(2, 0);

// 初始滑动窗口的左右边界，注意这里的右边界r是不包含的
int l = 0;
int r = l + n;

// 记录滑窗内部最多颜色数量
int max = 0;

// 统计初始滑动窗口中各种颜色的数量
for (int i = l; i < Math.min(r, arr.length); i++) {
    Integer c = arr[i];
    count.put(c, count.get(c) + 1);
    max = Math.max(max, count.get(c));
}

// 如果滑动窗口右边界未达到数组尾巴，就继续右移
// 注意，初始滑窗的右边界r是不包含的，因此r可以直接当成下一个滑窗的右边界使用
while (r < arr.length) {
    // 当滑动窗口右移后，新的滑动窗口相比移动前来看，新增了arr[r]，失去了arr[l]，注意此时左
    // 边界l还是指向上一个滑窗的左边界，而不是新滑窗的左边界，因此可以直接通过arr[l]取得失去的
    Integer add = arr[r++];
    Integer remove = arr[l++];

    count.put(add, count.get(add) + 1);
    count.put(remove, count.get(remove) - 1);

    // 只有新增数量的颜色可能突破最大值
    max = Math.max(max, count.get(add));
}

return max;
}
}

```

15. 等和子数组最小和

题目描述

给定一个数组nums，将元素分为若干个组，使得每组和相等，求出满足条件的所有分组中，组内元素和的最小值。

输入描述

第一行输入 m

接着输入m个数，表示此数组nums

数据范围：1<=m<=50, 1<=nums[i]<=50

输出描述

最小拆分数组和

用例

输入	7 4 3 2 3 5 2 1
输出	5
说明	可以等分的情况有：4 个子集 (5) , (1,4) , (2,3) , (2,3) 2 个子集 (5, 1, 4) , (2,3, 2,3) 但最小的为5。

题目解析

Java算法源码

感谢m0_71826536指正41行错误，41行在用例

```
5
5 5 5 5 5
```

时会出现越界异常

```
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();

        LinkedList<Integer> link = new LinkedList<>();
        for (int i = 0; i < m; i++) {
            link.add(sc.nextInt());
        }

        System.out.println(getResult(link, m));
    }

    public static int getResult(LinkedList<Integer> link, int m) {
        link.sort((a, b) -> b - a);

        int sum = 0;
        for (Integer ele : link) {
            sum += ele;
        }
    }
}
```

```

while (m > 0) {
    LinkedList<Integer> link_cp = new LinkedList<>(link);
    if (canPartitionMSubsets(link_cp, sum, m)) return sum / m;
    m--;
}

return sum;
}

public static boolean canPartitionMSubsets(LinkedList<Integer> link, int sum,
int m) {
    if (sum % m != 0) return false;

    int subSum = sum / m;

    if (subSum < link.get(0)) return false;

    // while (link.get(0) == subSum) { // 此段代码可能越界
    while (link.size() > 0 && link.get(0) == subSum) {
        link.removeFirst();
        m--;
    }

    int[] buckets = new int[m];
    return partition(link, 0, buckets, subSum);
}

public static boolean partition(LinkedList<Integer> link, int index, int[]
buckets, int subSum) {
    if (index == link.size()) return true;

    int select = link.get(index);

    for (int i = 0; i < buckets.length; i++) {
        if (i > 0 && buckets[i] == buckets[i - 1]) continue;
        if (select + buckets[i] <= subSum) {
            buckets[i] += select;
            if (partition(link, index + 1, buckets, subSum)) return true;
            buckets[i] -= select;
        }
    }

    return false;
}
}

```

16. 任务调度

题目描述

现有一个CPU和一些任务需要处理，已提前获知每个任务的任务ID、优先级、所需执行时间和到达时间。

CPU同时只能运行一个任务，请编写一个任务调度程序，采用“可抢占优先权调度”调度算法进行任务调度，规则如下：

- 如果一个任务到来时，CPU是空闲的，则CPU可以运行该任务直到任务执行完毕。
- 但是如果运行中有一个更高优先级的任务到来，则CPU必须暂停当前任务去运行这个优先级更高的任务；
- 如果一个任务到来时，CPU正在运行一个比他优先级更高的任务时，信道大的任务必须等待；
- 当CPU空闲时，如果还有任务在等待，CPU会从这些任务中选择一个优先级最高的任务执行，相同优先级的任务选择到达时间最早的任务。

输入描述

输入有若干行，每一行有四个数字（均小于10^8）,分别为任务ID，任务优先级，执行时间和到达时间。

每个任务的任务ID不同，优先级数字越大优先级越高，并且相同优先级的任务不会同时到达。

输入的任务已按照到达时间从小到大排序，并且保证在任何时间，处于等待的任务不超过10000个。

输出描述

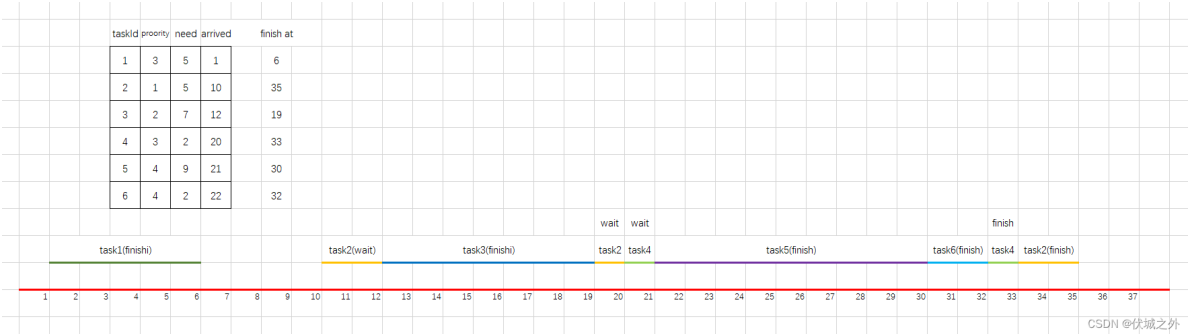
按照任务执行结束的顺序

用例

输入	1 3 5 1 2 1 5 10 3 2 7 12 4 3 2 20 5 4 9 21 6 4 2 22
输出	1 6 3 19 5 30 6 32 4 33 2 35
说明	无

题目解析

用例图示如下



task1在1时刻到达，此时CPU空闲，因此执行task1，task1需要执行5个时间，而执行期间没有新任务加入，因此task1首先执行完成，结束时刻是6。

task2在10时刻达到，此时CPU空闲，因此执行task2，task2需要执行5个时间，但是在task2执行到12时刻时，有新任务task3加入，且优先级更高，因此task2让出执行权，task2还需要 $5 - (12 - 10) = 3$ 个时间才能执行完，task2进入等待队列。

task3在12时刻到达，此时CPU正在执行task2，但是由于task3的优先级高于task2，因此task3获得执行权开始执行，task3需要7个时间，而在下一个任务task4会在20时刻到达，因此task3可以执行完，结束时刻是19。

task3执行结束时刻是19，而task4到达时间是20，因此中间有一段CPU空闲期，而等待队列中还有一个task2等待执行，因此此时CPU会调出task2继续执行，但是只能执行1时间，因此task2还需要 $3 - 1 = 2$ 个时间才能执行完。

task4在20时刻到达，此时CPU正在执行task2，但是由于task4的优先级更高，因此task4获得执行权开始执行，task2重回等待队列，task4需要2个时间，但是执行到时刻21时，task5到达了，且优先级更高，因此task4还需 $2 - (21 - 20) = 1$ 个时间才能执行完，task4进入等待队列。

此时等待队列有两个任务task2，task4，因此需要按照优先级排序，优先级高的在队头，因此`queue = [task4, task2]`

task5在21时刻到达，此时CPU正在执行task4，但是task5的优先级更高，因此task5获得执行权开始执行，task4进入等待队列，task5需要9个时间，但是执行到时刻22时，task6到达了，但是task6的优先级和task5相同，因此task5执行不受影响，task5会在 $21 + 9 = 30$ 时刻执行完成。

而task6则进入等待队列，有新任务进入队列后，就要按优先级重新排序，优先级高的在队头，因此`queue = [task6, task4, task2]`。

此时所有任务已经遍历完，我们检查等待队列是否有任务，若有，则此时任务队列中的任务必然是按优先级降序的，因此我们依次取出队头任务，在上一次结束时刻基础上添加需要的时间，就是新的结束时刻，比如

task6出队，上一次结束时刻是30，因此task6的结束时刻 = $30 + 2 = 32$ ，新的结束时刻变为32

task4出队，上一次结束时刻是32，因此task4的结束时刻 = $32 + 1 = 33$ ，新的结束时刻变为33

task2出队，上一次结束时刻是33，因此task2的结束时刻 = $33 + 2 = 35$ ，新的结束时刻变为35。

本题实现的难点在于：

1、等待队列的实现

这里的等待队列其实就是优先队列，优先队列我们可以基于有序数组实现，但是有序数组实现最优先队列的时间复杂度至少 $O(n)$ ，算是比较高的。优先队列其实只要每次保证最高优先级的任务处于队头即可，无需实现整体有序，因此基于最大堆实现优先队列是更好的选择，最大堆每次实现优先队列，只需要 $O(\log N)$ 的时间复杂度，因此在处理大数量级是更具有优势，但是JS语言并没有实现基于堆结构的优先队列，因此我们需要手动实现，相较于有序数组而言，难度较大。关于基于堆的优先队列实现，请看：[LeetCode - 1705 吃苹果的最大数目](#) [伏城之外的博客-CSDN博客](#)

2、CPU的任务执行逻辑

CPU执行某个任务时，如果有新任务加入，则我们应该比较正在执行的任务和新任务的优先级，

- 如果新任务优先级较高，则应该将正在执行的任务撤出，加入到等待队列中，然后执行新任务。

- 如果新任务优先级不高于正在执行的任务，则新任务进入等待队列，继续执行当前任务。

CPU空转期间，应该检查等待队列是否有任务，并取出最高优先级任务执行。

2023.02.19 根据网友反馈上面逻辑的通过率为20%，我重新看了一下，发现上面遗漏一个逻辑：

题目说

当CPU空闲时，如果还有任务在等待，CPU会从这些任务中选择一个优先级最高的任务执行，相同优先级的任务选择到达时间最早的任务。

而上面逻辑中遗漏考虑了相同优先级时，按照到达时间为第二优先级来安排任务执行的场景。已修复。

Java算法源码

Java已有专门的优先队列实现类PriorityQueue，因此我们可以直接使用它，而不需要自己实现。

```
import java.util.Arrays;
import java.util.LinkedList;
import java.util.PriorityQueue;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        LinkedList<Task> list = new LinkedList<>();

        while (sc.hasNextLine()) {
            String s = sc.nextLine();
            if ("".equals(s)) break;
            Integer[] arr = Arrays.stream(s.split("
")).map(Integer::parseInt).toArray(Integer[]::new);
            Task task = new Task(arr[0], arr[1], arr[2], arr[3]);
            list.add(task);
        }

        getResult(list);
    }

    /**
     * @param tasks 任务列表
     */
    public static void getResult(LinkedList<Task> tasks) {
        PriorityQueue<Task> pq =
            new PriorityQueue<>(
                (a, b) -> a.priority != b.priority ? b.priority - a.priority :
                a.arrived - b.arrived);

        pq.offer(tasks.removeFirst());
        int curTime = pq.peek().arrived; // curTime记录当前时刻

        while (tasks.size() > 0) {
            Task curTask = pq.peek(); // 当前正在运行的任务curTask
            Task nextTask = tasks.removeFirst(); // 下一个任务nextTask
```

```

        int curtTask_endTime = curTime + curtTask.need; // 当前正在运行任务的“理想”结束
        时间

        if (curtTask_endTime > nextTask.arrived) { // 如果当前正在运行任务的理想结束时间
        超过了 下一个任务的开始时间
            curtTask.need -= nextTask.arrived - curTime; // 先看优先级，先将当前任务可以
            运行的时间减去
            curTime = nextTask.arrived;
        } else { // 如果当前正在运行任务的理想结束时间 没有超过 下一个任务的开始时间，则当前
        任务可以执行完
            pq.poll(); // 当前任务出队
            System.out.println(curtTask.id + " " + curtTask_endTime); // 打印执行完的任
            务的id和结束时间
            curTime = curtTask_endTime;

            if (nextTask.arrived > curtTask_endTime) { // 如果当前任务结束时，下一次任务还
            没有达到，那么存在CPU空转(即idle)
                while (pq.size() > 0) { // 此时，我们应该从优先队列中取出最优先的任务出来执行，
                逻辑同上

                    Task idleTask = pq.peek();
                    int idleTask_endTime = curTime + idleTask.need;

                    if (idleTask_endTime > nextTask.arrived) {
                        idleTask.need -= nextTask.arrived - curTime;
                        break;
                    } else {
                        pq.poll();
                        System.out.println(idleTask.id + " " + idleTask_endTime);
                        curTime = idleTask_endTime;
                    }
                }
                curTime = nextTask.arrived;
            }
        }

        pq.offer(nextTask);
    }

    // 所有任务都加入优先队列后，我们就可以按照优先队列的安排，顺序取出任务来执行了
    while (pq.size() > 0) {
        Task pollTask = pq.poll();
        int pollTask_endTime = curTime + pollTask.need;
        System.out.println(pollTask.id + " " + pollTask_endTime);
        curTime = pollTask_endTime;
    }
}

class Task {
    int id; // 任务id
    int priority; // 任务优先级
    int need; // 任务执行时长
    int arrived; // 任务到达时刻
}

```

```
public Task(int id, int priority, int need, int arrived) {  
    this.id = id;  
    this.priority = priority;  
    this.need = need;  
    this.arrived = arrived;  
}  
}
```

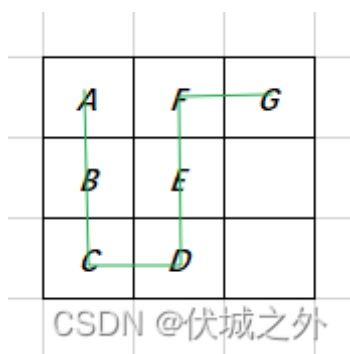
17. 箱子之字形摆放

题目描述

有一批箱子（形式为字符串，设为str），

要求将这批箱子按从上到下以之字形的顺序摆放在宽度为 n 的空地，请输出箱子的摆放位置。

例如：箱子ABCDEFGG，空地宽度为3，摆放结果如图：



则输出结果为：

AFG

BE

CD

输入描述

输入一行字符串，通过空格分隔，前面部分为字母或数字组成的字符串str，表示箱子；

后面部分为数字n，表示空地的宽度。例如：

ABCDEFGG 3

输出描述

箱子摆放结果，如题目示例所示

```
AFG`  
`BE`  
`CD`
```


备注

- 1. 请不要在最后一行输出额外的空行
- 2. str只包含字母和数字, $1 \leq \text{len}(\text{str}) \leq 1000$
- 3. $1 \leq n \leq 1000$

用例

输入	ABCDEFGF 3
输出	AFG BE CD
说明	

题目解析

我的解题思路如下：

首先定义一个不定宽的二维矩阵，JS的话很简单，Java的话需要定义为List结构，这个二维矩阵的高度就是给定的空地的n大小，即用例对应的初始二维矩阵应该如下

```
{
  {},
  {},
  {}
}
```

然后，遍历字符串的每一个字符，比如用例中ABCDEFGF，首先A被插入二维矩阵的第0行，即字符A在字符串中的索引 $i \% n$ 的值，比如A字符索引为 $i=0$, $n=3$, 因此插入行为 $i \% n = 0$

```
{
  {A},
  {},
  {}
}
```

遍历B，插入二维矩阵第1行，即 $i=1$, $n=3$, $i \% n = 1$

```
{
  {A},
  {B},
  {}
}
```

遍历C，插入二维矩阵的第2行，即 $i=2$, $n=3$, $i \% n=2$

```
{  
    {A},  
    {B},  
    {C}  
}
```

下面到关键点了，遍历D，应该插入二维矩阵的第几行呢？按照前面公式 $i=3$ ， $n=3$ ，插入行应该是第 $i \% n = 0$ 行。

但是实际上，应该要是第2行，如下面所示

```
{  
    {A},  
    {B},  
    {C, D}  
}
```

因此，此时插入行的公式应该变为 $\text{插入行} = n - 1 - (i \% n) = 3 - 1 - 0 = 2$

后面E、F的插入都遵循该公式

```
{  
    {A, F},  
    {B, E},  
    {C, D}  
}
```

但是遍历到G时，G的索引 $i = 6$ ， $n=3$ ， $i \% n = 0$ ，那么此时G应该插入哪一行呢？按照前面的公式，应该插入 $n - 1 - (i \% n) = 3 - 1 - 0 = 2$ ，但是实际上应该插入第0行，如下所示

```
{  
    {A, F, G},  
    {B, E},  
    {C, D}  
}
```

此时，我们可以总结出，遍历的字符插入到二维矩阵的行数和其索引有关，索引和行数的转换方程有两个：

- 行数 = $n - 1 - (i \% n)$
- 行数 = $i \% n$

初始时，用公式：行数 = $i \% n$ ，

当遇到 $i \% n == 0$ 时，变换公式为：行数 = $n - 1 - (i \% n)$ ，

当遇到 $i \% n == 0$ 时，再变换公式为：行数 = $i \% n$ ，

当遇到 $i \% n == 0$ 时，变换公式为：行数 = $n - 1 - (i \% n)$ ，

当遇到 $i \% n == 0$ 时，再变换公式为：行数 $= i \% n$ ，

.....

因此我们可以用一个reverse来标记，

当reverse=true，用公式：行数 $= i \% n$

当reverse=false，用公式：行数 $= n - 1 - (i \% n)$

而reverse变化的条件就是 $i \% n == 0$ ，每遇到 $i \% n == 0$ ， $reverse = !reverse$

Java算法源码

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.next();
        int n = sc.nextInt();

        getResult(str, n);
    }

    public static void getResult(String str, int n) {
        ArrayList<ArrayList<Character>> matrix = new ArrayList<>();
        for (int i = 0; i < n; i++) matrix.add(new ArrayList<>());

        boolean reverse = true;
        for (int i = 0; i < str.length(); i++) {
            int k = i % n;
            if (k == 0) reverse = !reverse;
            if (reverse) k = n - 1 - k;
            matrix.get(k).add(str.charAt(i));
        }

        for (ArrayList<Character> list : matrix) {
            StringBuilder sb = new StringBuilder();
            for (Character character : list) {
                sb.append(character);
            }
            System.out.println(sb);
        }
    }
}
```

18.开心消消乐

题目描述

给定一个N行M列的二维矩阵，矩阵中每个位置的数字取值为0或1。矩阵示例如：

1100
0001
0011
1111

现需要将矩阵中所有的1进行反转为0，规则如下：

- 1) 当点击一个1时，该1便被反转为0，同时相邻的上、下、左、右，以及左上、左下、右上、右下8个方向的1（如果存在1）均会自动反转为0；
- 2) 进一步地，一个位置上的1被反转为0时，与其相邻的8个方向的1（如果存在1）均会自动反转为0；

按照上述规则示例中的矩阵只最少需要点击2次后，所有值均为0。请问，给定一个矩阵，最少需要点击几次后，所有数字均为0？

输入描述

第一行输入两个数字N，M，分别表示二维矩阵的行数、列数，并用空格隔开

之后输入N行，每行M个数字，并用空格隔开

输出描述

最少需要点击几次后，矩阵中所有数字均为0

用例

输入	4 4 1 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1
输出	2
说明	无

题目解析

[用例图](#)示如下


```

        continue;
    }

    // 不要紧张，这里固定循环8次，不会造成O(n^3)
    for (Integer[] offset : offsets) {
        int newI = i + offset[0];
        int newJ = j + offset[1];

        if (newI >= 0 && newI < n && newJ >= 0 && newJ < m &&
matrix[newI][newJ] == 1) {
            ufs.union(i * m + j, newI * m + newJ);
        }
    }
}

return ufs.count;
}
}

// 并查集
class UnionFindSet {
    int[] fa;
    int count;

    public UnionFindSet(int n) {
        this.fa = new int[n];
        this.count = n;
        for (int i = 0; i < n; i++) this.fa[i] = i;
    }

    public int find(int x) {
        if (x != this.fa[x]) {
            return (this.fa[x] = this.find(this.fa[x]));
        }
        return x;
    }

    public void union(int x, int y) {
        int x_fa = this.find(x);
        int y_fa = this.find(y);

        if (x_fa != y_fa) {
            this.fa[y_fa] = x_fa;
            this.count--;
        }
    }
}
}

```

19. 预订酒店

题目描述

放暑假了，小明决定到某旅游景点游玩，他在网上搜索到了各种价位的酒店（长度为n的数组A），他的心理价位是x元，请帮他筛选出k个最接近x元的酒店（ $n \geq k > 0$ ），并由低到高打印酒店的价格。

输入描述

第一行：n, k, x
第二行：A[0] A[1] A[2]...A[n-1]

输出描述

从低到高打印筛选出的酒店价格

用例

输入	10 5 6 1 2 3 4 5 6 7 8 9 10
输出	4 5 6 7 8
说明	无

输入	10 4 6 10 9 8 7 6 5 4 3 2 1
输出	4 5 6 7
说明	无

输入	6 3 1000 30 30 200 500 70 300
输出	200 300 500
说明	无

题目解析

我的解题思路如下：

首先，我们需要将给定的酒店价格列表price升序。

然后，找到最接近心理价位x的值price[i]，将i作为中心位置。然后，分别遍历中心位置左边位值left=i-1，右边位置right=i+1，比较price[left]和price[right]谁更接近x（即与x的差的绝对值更小），如果price[left]接近，则left--，如果price[right]接近，则right++，如果price[left]和price[right]一样近，则优先选取price[left]

（PS：根据用例2得出的结论，因为用例输出是4 5 6 7，而不是5 6 7 8，其中4和8距离6相同，但是用例2选择了4）

如果left--或者right++之后，发生了越界，则对应的price值置为[Infinity](#)，表示和中心值距离无穷大，这样就可以只会产生一边延申的效果了。

注意本题不会发生两边同时越界的情况，因为 $n \geq k > 0$ 。

以上就是解题的大致逻辑。

但是上面逻辑还有一个关键点没有实现，那就是一开始如何找到最接近心理价位x的price[i]值。

- 方案1：顺序遍历price数组，找 $\text{price}[i] == x$ 或者 $\text{price}[i] < x < \text{price}[i+1]$ ，
 1. 对于 $\text{price}[i] == x$ 情况，则i位置就是中心位置
 2. 对于 $\text{price}[i] < x < \text{price}[i+1]$ ，则取 $\text{price}[i]$ ， $\text{price}[i+1]$ 更接近x值得那个位置，如果一样近，则取靠左得i。

方案1得时间复杂度是 $O(n)$

- 方案2：由于我们已经将price升序了，那么可以借助二分查找，这里得二分查找我们可以参考Java得Arrays.binarySearch来实现

关于Java得Arrays.binarySearch，如果要找得值在数组中，则直接返回值在数组中得索引，如果要找的值不在数组中，则返回 $-\text{idx}-1$ ，其中idx是要查找的值在数组中的有序插入位置。

比如[有序数组](#) [1,3,5,7,9]，现在我们要在该数组中二分查找4的位置，可以发现该数组中根本没有4，但是Java得Arrays.binarySearch方法就会返回 $-2-1$ ，即-3，其中2就是查找值4理应在数组中的插入位置，即如果将4插入到有序数组中，则其位置应该是2，如 [1,3,4,5,7,9]

方案2的时间复杂度是 $O(\log N)$

由于本题没有给出数量级，因此可以选择简单的方案1先试试。下面算法源码使用了方案2。

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;
import java.util.StringJoiner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int k = sc.nextInt();
        int x = sc.nextInt();

        int[] prices = new int[n];
        for (int i = 0; i < n; i++) {
            prices[i] = sc.nextInt();
        }

        System.out.println(getResult(n, k, x, prices));
    }

    public static String getResult(int n, int k, int x, int[] prices) {
```



```

// 数组升序
Arrays.sort(prices);

// 二分查找数组中最接近心理价位x的元素的位置idx
int idx = Arrays.binarySearch(prices, x);

// 如果idx<0, 说明在数组中没有找到对应值, 因此此时idx是有序插入位置
if (idx < 0) {
    idx = -idx - 1; // 将idx转换为有序插入位置

    // 如果idx的插入位置越界, 则只会是右边界越界, 或者插入的位置为0
    if (idx == prices.length || idx == 0) {
        int[] ans;
        if (idx == prices.length) ans = Arrays.copyOfRange(prices, idx - k,
idx);
        else ans = Arrays.copyOfRange(prices, 0, k);

        StringJoiner sj = new StringJoiner(" ");
        for (int an : ans) {
            sj.add(an + "");
        }
        return sj.toString();
    }

    int diff1 = Math.abs(x - prices[idx]);
    int diff2 = Math.abs(x - prices[idx - 1]);
    if (diff1 > diff2) {
        // 选择最接近的值的位置作为中心位置
        idx -= 1;
    }
}

// 中心位置的值得1个接近心里价位的值, 因此已经找到1个, 所以k--
k--;

int left = idx;
int right = idx;
while (k > 0) {
    int leftVal = left == 0 ? Integer.MAX_VALUE : prices[left - 1];
    int rightVal = right == n - 1 ? Integer.MAX_VALUE : prices[right + 1];

    // 从中心位置向两边发散~
    int diff1 = Math.abs(x - leftVal);
    int diff2 = Math.abs(x - rightVal);

    // 那边值更接近, 则范围向哪边扩大
    if (diff1 <= diff2) {
        left--;
    } else {
        right++;
    }

    // 每次都能找到一个最接近的心里价位
    k--;
}

```

```

// left,right范围就是题解
int[] ans = Arrays.copyOfRange(prices, left, right + 1);
StringJoiner sj = new StringJoiner(" ");
for (int an : ans) {
    sj.add(an + "");
}
return sj.toString();
}
}

```

20.字符串重新排列、字符串重新排序

题目描述

给定一个字符串s，s包括以空格分隔的若干个单词，请对s进行如下处理后输出：

- 1、单词内部调整：对每个单词字母重新按[字典序排序](#)
- 2、单词间顺序调整：
 - 1) 统计每个单词出现的次数，并按次数[降序排列](#)
 - 2) 次数相同，按[单词长度](#)升序排列
 - 3) 次数和单词长度均相同，按字典升序排列

请输出处理后的字符串，每个单词以一个空格分隔。

输入描述

一行字符串，每个字符取值范围：[a-zA-z0-9]以及空格，字符串长度范围：[1, 1000]

输出描述

输出处理后的字符串，每个单词以一个空格分隔。

用例

输入	This is an apple
输出	an is This aelpp
说明	无

输入	My sister is in the house not in the yard
输出	in in eht eht My is not adry ehosu eirsst
说明	无

题目解析

本题需要注意的是，先进行单词内部调整，然后再进行单词间顺序。

考察的是排序

Java算法源码

```
import java.util.Arrays;
import java.util.HashMap;
import java.util.Scanner;
import java.util.StringJoiner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String[] arr = sc.nextLine().split(" ");
        System.out.println(getResult(arr));
    }

    public static String getResult(String[] arr) {
        arr =
            Arrays.stream(arr)
                .map(
                    str -> {
                        char[] cArr = str.toCharArray();
                        Arrays.sort(cArr);
                        return new String(cArr);
                    }
                )
                .toArray(String[]::new);

        HashMap<String, Integer> count = new HashMap<>();
        for (String s : arr) {
            count.put(s, count.getOrDefault(s, 0) + 1);
        }

        Arrays.sort(
            arr,
            (a, b) ->
                !count.get(a).equals(count.get(b))
                ? count.get(b) - count.get(a)
                : a.length() != b.length() ? a.length() - b.length() :
a.compareTo(b));

        StringJoiner sj = new StringJoiner(" ", "", "");
        for (String s : arr) {
            sj.add(s);
        }
        return sj.toString();
    }
}
```

21.星际篮球争霸赛

题目描述

在星球争霸篮球赛对抗赛中，最大的宇宙战队希望每个人都能拿到MVP，MVP的条件是单场最高分得分获得者。

可以并列所以宇宙战队决定在比赛中尽可能让更多队员上场，并且让所有得分的选手得分都相同，然而比赛过程中的每1分钟的得分都只能由某一个人包揽。

输入描述

输入第一行为一个数字 t ，表示为有得分的分钟数 $1 \leq t \leq 50$

第二行为 t 个数字，代表每一分钟的得分 p ， $1 \leq p \leq 50$

输出描述

输出有得分的队员都是MVP时，最少得MVP得分。

用例

输入	9 5 2 1 5 2 1 5 2 1
输出	6
说明	样例解释 一共 4 人得分，分别都是 6 分 $5 + 1$ ， $5 + 1$ ， $5 + 1$ ， $2 + 2 + 2$

题目解析

Java算法源码

感谢m0_71826536指正41行错误，41行在用例

```
5
5 5 5 5 5
```

时会出现越界异常

```
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();

        LinkedList<Integer> link = new LinkedList<>();
        for (int i = 0; i < m; i++) {
            link.add(sc.nextInt());
        }
    }
}
```

```

    }

    System.out.println(getResult(link, m));
}

public static int getResult(LinkedList<Integer> link, int m) {
    link.sort((a, b) -> b - a);

    int sum = 0;
    for (Integer ele : link) {
        sum += ele;
    }

    while (m >= 1) {
        // 根据网友指正，由于canPartition方法中会删除link元素，因此我们不能直接传递link过去，
        // 需要传递link备份，否则会影响下一次count判断
        // if (canPartitionMSubsets(link, sum, m)) return sum / m;
        LinkedList<Integer> link_cp = new LinkedList<>(link);
        if (canPartitionMSubsets(link_cp, sum, m)) return sum / m;
        m--;
    }

    return sum;
}

public static boolean canPartitionMSubsets(LinkedList<Integer> link, int sum,
int m) {
    if (sum % m != 0) return false;

    int subSum = sum / m;

    if (subSum < link.get(0)) return false;

    // while (link.get(0) == subSum) { // 此段代码可能会出现越界
    while (link.size() > 0 && link.get(0) == subSum) {
        link.removeFirst();
        m--;
    }

    int[] buckets = new int[m];
    return partition(link, 0, buckets, subSum);
}

public static boolean partition(LinkedList<Integer> link, int index, int[]
buckets, int subSum) {
    if (index == link.size()) return true;

    int select = link.get(index);

    for (int i = 0; i < buckets.length; i++) {
        if (i > 0 && buckets[i] == buckets[i - 1]) continue;
        if (select + buckets[i] <= subSum) {
            buckets[i] += select;
            if (partition(link, index + 1, buckets, subSum)) return true;
            buckets[i] -= select;
        }
    }
}

```

```
    }  
  }  
  
  return false;  
}  
}
```

22. 最大报酬

题目描述

小明每周上班都会拿到自己的工作清单，工作清单内包含 n 项工作，每项工作都有对应的耗时时间（单位 h ）和报酬，工作的总报酬为所有已完成工作的报酬之和，那么请你帮小明安排一下工作，保证小明在指定的工作时间内工作收入最大化。

输入描述

输入的第一行为两个正整数 T, n 。

T 代表工作时长（单位 h ， $0 < T < 1000000$ ），

n 代表工作数量（ $1 < n \leq 3000$ ）。

接下来是 n 行，每行包含两个整数 t, w 。

t 代表该工作消耗的时长（单位 h ， $t > 0$ ）， w 代表该项工作的报酬。

输出描述

输出小明指定工作时长内工作可获得的最大报酬。

用例

输入	40 3 20 10 20 20 20 5
输出	30
说明	无

题目解析

本题是[01背包问题](#)。可以使用动态规划求解。关于01背包问题请先看下面这个博客

[算法设计 - 01背包问题 伏城之外的博客-CSDN博客](#)

本题中

- 工作时长 T 相当于背包承重
- 每一项工作相当于每件物品

1. 工作消耗的时长相当于物品重量
2. 工作的报酬相当于物品的价值

01背包二维数组解法（100%）

Java算法源码

```
import java.util.*;

public class Main {
    static ArrayList<Integer[]> nodes;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int T = sc.nextInt();
        int n = sc.nextInt();

        Integer[][] tws = new Integer[n][2];
        for (int i = 0; i < n; i++) {
            tws[i][0] = sc.nextInt();
            tws[i][1] = sc.nextInt();
        }

        System.out.println(getResult(T, tws));
    }

    /**
     *
     * @param T 工作时长
     * @param tws 数组，元素是tw，也是数组，含义为[该工作消耗的时长，该项工作的报酬]
     * @return 最大报酬
     */
    public static int getResult(int T, Integer[][] tws) {
        int maxI = tws.length + 1;
        int maxJ = T + 1;

        int[][] dp = new int[maxI][maxJ];

        for (int i = 0; i < maxI; i++) {
            for (int j = 0; j < maxJ; j++) {
                if (i == 0 || j == 0) continue; // 第0行或第0列最大报酬保持0

                int t = tws[i - 1][0]; // 要选择的工作的[权重]
                int w = tws[i - 1][1]; // 要选择的工作的[价值]

                if (t > j) {
                    // 如果要选择的工作的权重 > 当前背包权重，则无法放入背包，最大价值
                    // 继承自上一行该列值
                    dp[i][j] = dp[i - 1][j];
                } else {
                    // 如果要选择的工作的权重 <= 当前背包权重
                    // 则我们有两种选择
                    // 1、不进行该工作，则最大价值继承自上一行该列值
                }
            }
        }
    }
}
```

```

        // 2、进行该工作，则纳入该工作的价值w，加上+ 剩余权重，在不进行该
        工作的范围内，可得的最大价值dp[i - 1][j - t]
        // 比较两种选择下的最大价值，取最大的
        dp[i][j] = Math.max(dp[i - 1][j], w + dp[i - 1][j - t]);
    }
}

return dp[maxI - 1][maxJ - 1];
}
}

```

01背包滚动数组解法

关于01背包的滚动数组优化，请看[算法设计 - 01背包问题的状态转移方程优化](#)，以及[完全背包问题01背包问题状态转移方程伏城之外的博客-CSDN博客](#)

Java算法源码

```

import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    static ArrayList<Integer[]> nodes;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int T = sc.nextInt();
        int n = sc.nextInt();

        Integer[][] tws = new Integer[n + 1][2];
        tws[0] = new Integer[] {0, 0};
        for (int i = 1; i <= n; i++) {
            tws[i][0] = sc.nextInt();
            tws[i][1] = sc.nextInt();
        }

        System.out.println(getResult(T, tws, n));
    }

    /**
     * @param T 工作时长
     * @param tws 数组，元素是tw，也是数组，含义为[该工作消耗的时长，该项工作的报酬]
     * @param n 工作数量
     * @return 最大报酬
     */
    public static int getResult(int T, Integer[][] tws, int n) {
        int[] dp = new int[T + 1];

        // 01背包问题滚动数组优化
        for (int i = 1; i <= n; i++) {
            int t = tws[i][0];
            int w = tws[i][1];

```



```
// 注意背包承重遍历必须倒序，正序的话就变成完全背包了
for (int j = T; j >= t; j--) {
    dp[j] = Math.max(dp[j], dp[j - t] + w);
}

return dp[T];
}
```

23. 二元组个数

题目描述

给定两个数组a, b, 若 $a[i] == b[j]$ 则称 $[i, j]$ 为一个二元组, 求在给定的两个数组中, 二元组的个数。

输入描述

第一行输入 m

第二行输入m个数, 表示第一个数组

第三行输入 n

第四行输入n个数, 表示第二个数组

输出描述

二元组个数。

用例

输入	4 1 2 3 4 1 1
输出	1
说明	二元组个数为 1 个

输入	6 1 1 2 2 4 5 3 2 2 4
输出	5
说明	二元组个数为 5 个。

题目解析

很简单的双重for,

```
/**
 *
```

```

* @param {Array} arrM 第二行输入的数组
* @param {Number} m 第一行输入的数字m
* @param {Array} arrN 第四行输入的数组
* @param {Number} n 第二行输入的数字n
* @returns
*/
function getResult(arrM, m, arrN, n) {
    let count = 0;

    for (let i = 0; i < m; i++) {
        for (let j = 0; j < n; j++) {
            if (arrM[i] === arrN[j]) {
                count++;
            }
        }
    }

    return count;
}

```

但是不知道数量级多少，如果数量级比较大的话，则 $O(n*m)$ 可能罩不住。

因此，我们还需要考虑下大数量级的情况。

我的思路如下：

先找出m数组中，在n数组中出现的数及个数，在找出n数组中，在m数组中出现的数及个数，比如：

用例2中

countM = {2:2, 4:1}

countN = {2:2, 4:1}

然后将相同数的个数相乘，最后求和即为题解，比如 $2*2 + 1*1 = 5$

Java算法源码

```

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
        ArrayList<Integer> listM = new ArrayList<>();
        for (int i = 0; i < m; i++) {
            listM.add(sc.nextInt());
        }

        int n = sc.nextInt();
        ArrayList<Integer> listN = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            listN.add(sc.nextInt());
        }
    }
}

```

```

    }

    System.out.println(getResult(listM, listN));
}

public static int getResult(ArrayList<Integer> listM, ArrayList<Integer>
listN) {
    HashSet<Integer> setM = new HashSet<Integer>(listM);
    HashSet<Integer> setN = new HashSet<Integer>(listN);

    HashMap<Integer, Integer> countM = new HashMap<>();
    for (Integer m : listM) {
        if (setN.contains(m)) {
            countM.put(m, countM.getOrDefault(m, 0) + 1);
        }
    }

    HashMap<Integer, Integer> countN = new HashMap<>();
    for (Integer n : listN) {
        if (setM.contains(n)) {
            countN.put(n, countN.getOrDefault(n, 0) + 1);
        }
    }

    int count = 0;
    for (Integer k : countM.keySet()) {
        count += countM.get(k) * countN.get(k);
    }

    return count;
}
}

```

24. 计算数组中心位置

题目描述

给你一个整数数组nums，请计算数组的中心位置，数组的中心位置是数组的一个下标，其左侧所有元素相乘的积等于右侧所有元素相乘的积。数组第一个元素的左侧积为1，最后一个元素的右侧积为1。

如果数组有多个中心位置，应该返回最靠近左边的那一个，如果数组不存在中心位置，返回-1。

输入描述

输入只有一行，给出N个正整数用空格分隔：nums = 2 5 3 6 5 6

$1 \leq \text{nums.length} \leq 1024$

$1 \leq \text{nums}[i] \leq 10$

输出描述

输出：3

解释：中心位置是3

用例

输入	2 5 3 6 5 6
输出	3
说明	无

题目解析

很简单的单指针操作。

应该是考察优化操作，即我们不应该每次都重新计算中心位置左边所有值和右边所有值的各自乘积，应该利用差异剔除的方式。

比如中心位置右移一格到*i*位置后，左边所有值的乘积*left* 应该变为了 *left* *= *arr*[*i*-1]，右边所有值得乘积*right* 应该变为了 *right* /= *arr*[*i*]

Java算法源码

感谢网友m0_71826536提出改进意见，本题中

1 <= nums.length <= 1024

1 <= nums[i] <= 10

也就是说单边乘积最大可以是10^1024，这个值已经远远超过了int,long范围，因此这里我们应该使用大数来处理，Java可以使用BigInteger来处理。

```
import java.math.BigInteger;
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Integer[] arr =
            Arrays.stream(sc.nextLine().split("
"))
                .map(Integer::parseInt)
                .toArray(Integer[]::new);

        System.out.println(getResult(arr));
    }

    public static int getResult(Integer[] nums) {
        BigInteger fact = BigInteger.valueOf(1);
```

```

    for (Integer num : nums) {
        fact = fact.multiply(BigInteger.valueOf(num));
    }

    BigInteger left = BigInteger.valueOf(1);
    BigInteger right = fact.divide(BigInteger.valueOf(nums[0]));

    if (left.compareTo(right) == 0) {
        return 0;
    }

    for (int i = 1; i < nums.length; i++) {
        left = left.multiply(BigInteger.valueOf(nums[i - 1]));
        right = right.divide(BigInteger.valueOf(nums[i]));

        if (left.compareTo(right) == 0) return i;
    }

    return -1;
}

// public static int getResult(Integer[] nums) {
//     int fact = 1;
//     for (Integer num : nums) {
//         fact *= num;
//     }
//
//     int left = 1;
//     int right = fact / nums[0];
//
//     if (left == right) {
//         return 0;
//     }
//
//     for (int i = 1; i < nums.length; i++) {
//         left *= nums[i - 1];
//         right /= nums[i];
//
//         if (left == right) return i;
//     }
//
//     return -1;
// }
}

```

25. 机器人、机器人活动区域

题目描述

现有一个机器人，可放置于 $M \times N$ 的网格中任意位置，每个网格包含一个非负整数编号，当相邻网格的数字编号差值的绝对值小于等于 1 时，机器人可以在网格间移动。

问题： 求机器人可活动的最大范围对应的网格点数目。

说明： 网格左上角坐标为 (0,0) ,右下角坐标为(m-1,n-1)， 机器人只能在相邻网格间上下左右移动

输入描述

第 1 行输入为 M 和 N ， M 表示网格的行数 N 表示网格的列数
之后 M 行表示网格数值， 每行 N 个数值（数值大小用 k 表示），
数值间用单个空格分隔， 行首行尾无多余空格。

M、 N、 k 均为整数， 且 $1 \leq M, N \leq 150, 0 \leq k \leq 50$

输出描述

输出 1 行， 包含 1 个数字， 表示最大活动区域的网格点数目，
行首行尾无多余空格。

用例

输入	4 4 1 2 5 2 2 4 4 5 3 5 7 1 4 6 2 4																																																																																																																					
输出	6																																																																																																																					
说明	<table><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>1</td><td>2</td><td>5</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>2</td><td>4</td><td>4</td><td>5</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>3</td><td>5</td><td>7</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>4</td><td>6</td><td>2</td><td>4</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table> 图中绿色区域，相邻网格差值绝对值都小于等于 1，且 为最大区域，对应网格点数目为 6。																																			1	2	5	2														2	4	4	5														3	5	7	1														4	6	2	4																												
		1	2	5	2																																																																																																																	
		2	4	4	5																																																																																																																	
		3	5	7	1																																																																																																																	
		4	6	2	4																																																																																																																	

题目解析

本题其实就是求最大的[连通分量](#)， 相邻网格是否连通的规则如下

当相邻网格的数字编号差值的绝对值小于等于 1 时

因此， 求解连通分量

Java算法源码

```
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
        int n = sc.nextInt();

        int[][] matrix = new int[m][n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        System.out.println(getResult(matrix, m, n));
    }

    public static int getResult(int[][] matrix, int m, int n) {
        UnionFindSet ufs = new UnionFindSet(m * n);

        // 上下左右的偏移量
        int[][] offsets = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                // 注意下面这层for是常量级的，就四次循环，因此，这里的时间复杂度还是O(n*m)，即
                for (int[] offset : offsets) {
                    int newI = i + offset[0];
                    int newJ = j + offset[1];

                    if (newI < 0 || newI >= m || newJ < 0 || newJ >= n) continue;
                    // 当相邻网格的数字编号差值的绝对值小于等于 1 时，机器人可以在网格间移动，即表示网格
                    // 连通，可以合并
                    if (Math.abs(matrix[i][j] - matrix[newI][newJ]) <= 1) {
                        ufs.union(i * n + j, newI * n + newJ);
                    }
                }
            }
        }

        int total = m * n;

        // ufs.count是连通分量的个数，如果只有一个连通分量，那么代表所有的点都可达
        if (ufs.count == 1) return total;

        // 这个for循环是有必要的，确保每个点都指向祖先
        for (int i = 0; i < total; i++) {
            ufs.find(i);
        }
    }
}
```

```

// 统计指向同一个祖先下点的个数，即每个连通分量下的点个数
HashMap<Integer, Integer> count = new HashMap<>();
for (int i : ufs.fa) {
    count.put(i, count.getDefault(i, 0) + 1);
}

// 取最大个数，这里必然有值，因此不需要在get前判空
return count.values().stream().max((a, b) -> a - b).get();
}
}

// 并查集
class UnionFindSet {
    int[] fa;
    int count;

    public UnionFindSet(int n) {
        this.fa = new int[n];
        this.count = n;
        for (int i = 0; i < n; i++) this.fa[i] = i;
    }

    public int find(int x) {
        if (x != this.fa[x]) {
            return (this.fa[x] = this.find(this.fa[x]));
        }
        return x;
    }

    public void union(int x, int y) {
        int x_fa = this.find(x);
        int y_fa = this.find(y);

        if (x_fa != y_fa) {
            this.fa[y_fa] = x_fa;
            this.count--;
        }
    }
}

```

26. 异常的打卡记录

题目描述

考勤记录是分析和考核职工工作时间利用情况的原始依据，也是计算职工工资的原始依据，为了正确地计算职工工资和监督工资基金使用情况，公司决定对员工的手机打卡记录进行异常排查。

如果出现以下两种情况，则认为打卡异常：

1. 实际设备号与注册设备号不一样
2. 或者，同一个员工的两个打卡记录的时间小于60分钟并且打卡距离超过5km。

给定打卡记录的字符串数组clockRecords（每个打卡记录组成为：工号;时间（分钟）;打卡距离（km）；实际设备号;注册设备号），返回其中异常的打卡记录（按输入顺序输出）。

输入描述

第一行输入为N，表示打卡记录数；
之后的N行为打卡记录，每一行为一条打卡记录。

输出描述

输出异常的打卡记录。

备注

- clockRecords长度 ≤ 1000
- clockRecords[i] 格式：{id},{time},{distance},{actualDeviceNumber},{registeredDeviceNumber}
- id由6位数字组成
- time由整数组成，范围为0~1000
- distance由整数组成，范围为0~100
- actualDeviceNumber与registeredDeviceNumber由思维大写字母组成

用例

输入	2 100000,10,1,ABCD,ABCD 100000,50,10,ABCD,ABCD
输出	100000,10,1,ABCD,ABCD;100000,50,10,ABCD,ABCD
说明	第一条记录是异常得，因为第二题记录与它得间隔不超过60分钟，但是打卡距离超过了5km，同理第二条记录也是异常得。

输入	2 100000,10,1,ABCD,ABCD 100001,80,10,ABCE,ABCE
输出	null
说明	无异常打卡记录，所以返回null

输入	2 100000,10,1,ABCD,ABCD 100000,80,10,ABCE,ABCD
输出	100000,80,10,ABCE,ABCD
说明	第二条记录得注册设备号与打卡设备号不一致，所以是异常记录

题目解析

我的解题思路如下：

首先，打卡记录异常，有两种类型：

- 1、单条打卡记录自身比较，即单条打卡记录的实际设备号和注册设备号不同，则视为异常。
- 2、同一员工的两条打卡记录对比，如果打卡时间间隔小于60min，但是打卡距离却超过了5km，则视为异常

因此，我们应该先对每条打卡记录进行自身比较，即比较实际设备号，和注册设备号，如果不同，则直接判定异常。

另外，我理解，同一员工的（非设备号异常）打卡记录不需要再和（设备号异常）的打卡记录对比。

根据考友反馈：

同一员工的（非设备号异常）打卡记录需要再和（设备号异常）的打卡记录对比。

对于同一员工下设备号一致的打卡记录，需要两两对比，如果两个打卡记录的时间小于60分钟并且打卡距离超过5km，则视为异常。

但是这里有一个疑点，那就是一旦有两条打卡记录对比异常了，那么其他打卡记录是否还需要和这两条异常记录对比吗？

这点，题目描述没说，给的用例也无法验证。我理解是需要的，需要的。

另外，本题还有一个关键点就是：异常的打卡记录需要（按输入顺序输出）。

因此，我们在保存异常打卡记录时，还需要附加其输入时的索引，已方便按照输入索引输出。

Java算法源码

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        String[][] clockRecords = new String[n][];
        for (int i = 0; i < n; i++) {
            clockRecords[i] = sc.next().split(",");
        }

        System.out.println(getResult(clockRecords));
    }

    /**
     * @param clockRecords 打卡记录的字符串数组，[工号，时间，打卡距离，实际设备号，注册设备号]
     * @return 异常打卡记录列表（按输入顺序排序）
     */
}
```

```

*/
public static String getResult(String[][] clockRecords) {
    HashMap<String, ArrayList<String[]>> employees = new HashMap<>();
    TreeSet<Integer> ans = new TreeSet<>((a, b) -> a - b);

    for (int i = 0; i < clockRecords.length; i++) {
        // 由于异常打卡记录需要按输入顺序输出，因此这里追加一个输入索引到打卡记录中
        String[] clockRecord = Arrays.copyOf(clockRecords[i],
clockRecords[i].length + 1);
        clockRecord[clockRecord.length - 1] = i + "";

        String id = clockRecord[0];
        String act_device = clockRecord[3];
        String reg_device = clockRecord[4];

        // 实际设备号与注册设备号不一样，则认为打卡异常
        if (!act_device.equals(reg_device)) {
            ans.add(i);
        }

        // 统计该员工的打卡
        employees.putIfAbsent(id, new ArrayList<>());
        employees.get(id).add(clockRecord);
    }

    for (String id : employees.keySet()) {
        // 某id员工的所有打卡记录records
        ArrayList<String[]> records = employees.get(id);

        // 将该员工打卡记录按照打卡时间升序
        records.sort((a, b) -> Integer.parseInt(a[1]) - Integer.parseInt(b[1]));

        for (int i = 0; i < records.size(); i++) {
            int time1 = Integer.parseInt(records.get(i)[1]);
            int dist1 = Integer.parseInt(records.get(i)[2]);

            for (int j = i + 1; j < records.size(); j++) {
                int time2 = Integer.parseInt(records.get(j)[1]);
                int dist2 = Integer.parseInt(records.get(j)[2]);

                // 如果两次打卡时间超过60分治，则不计入异常，由于已按打卡时间升序，因此后面的都不用
                // 检查了
                if (time2 - time1 >= 60) break;
                else {
                    // 如果两次打开时间小于60MIN，且打卡距离超过5KM，则这两次打卡记录算作异常
                    if (Math.abs(dist2 - dist1) > 5) {
                        // 如果打卡记录已经加入异常列表ans，则无需再次加入，否则需要加入
                        ans.add(Integer.parseInt(records.get(i)[5]));
                        ans.add(Integer.parseInt(records.get(j)[5]));
                    }
                }
            }
        }
    }
}

```

```
// 如果没有异常打卡记录，则返回null
if (ans.size() == 0) return "null";

StringJoiner sj = new StringJoiner(";");
ans.stream()
    .map(i -> clockRecords[i])
    .forEach(
        sArr -> {
            StringJoiner sj1 = new StringJoiner(",");
            for (String s : sArr) {
                sj1.add(s);
            }
            sj.add(sj1.toString());
        });

return sj.toString();
}
}
```

27. 积木最远距离、相同数字的积木游戏

题目描述

小华和小薇一起通过玩积木游戏学习数学。

他们有很多积木，每个积木块上都有一个数字，积木块上的数字可能相同。

小华随机拿一些积木挨着排成一排，请小薇找到这排积木中数字相同且所处位置最远的2块积木块，计算他们的距离，小薇请你帮忙替她解决这个问题。

输入描述

第一行输入为N，表示小华排成一排的积木总数。

接下来N行每行一个数字，表示小华排成一排的积木上数字。

输出描述

相同数字的积木的位置最远距离；如果所有积木数字都不相同，请返回-1。

备注

- $0 \leq \text{积木上的数字} < 10^9$
- $1 \leq \text{积木长度} \leq 10^5$

用例

输入	5 1 2 3 1 4
输出	3
说明	共有5个积木，第1个积木和第4个积木数字相同，其距离为3。

输入	2 1 2
输出	-1
说明	一共有两个积木，没有积木数字相同，返回-1。

题目解析

这题第一眼看上去好像是要用[双指针](#)做，但是实操起来却不行。

这题的数组长度会达到 10^5 ，因此时间复杂度要至少控制在 $O(n)$ 。

我的解题思路如下：

定义一个idx对象，用于存放每个num出现的索引位置，num作为idx对象属性，num出现的索引位置作为idx[num]的数组的元素。

然后遍历nums数组（即从第二行开始输入的数的集合），开始录入num的索引位置到idx对象中。

统计完后，开始遍历idx对象属性，即每个num，然后先判断idx[num]的数组长度是否大于1，若不大于，则不考虑，若大于，则用idx[num]数组的最后一个索引位置 减去 idx[num]数组的第一个索引位置。按照上面逻辑，计算出最大的索引差作为题解。

若没有符合要求的，则返回-1。

Java算法源码

```
import java.util.HashMap;
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.println(getResult(arr));
    }
}
```

```

public static int getResult(int[] arr) {
    HashMap<Integer, LinkedList<Integer>> idx = new HashMap<>();

    for (int i = 0; i < arr.length; i++) {
        int num = arr[i];
        idx.putIfAbsent(num, new LinkedList<>());
        idx.get(num).add(i);
    }

    int ans = -1;

    for (Integer k : idx.keySet()) {
        LinkedList<Integer> link = idx.get(k);
        if (link.size() > 1) {
            ans = Math.max(ans, link.getLast() - link.getFirst());
        }
    }

    return ans;
}

```

28. 日志首次上报最多积分

题目描述

日志采集是运维系统的的核心组件。日志是按行生成，每行记做一条，由采集系统分批上报。

- 如果上报太频繁，会对服务端造成压力；
- 如果上报太晚，会降低用户的体验；
- 如果一次上报的条数太多，会导致超时失败。

为此，项目组设计了如下的上报策略：

1. 每成功上报一条日志，奖励1分
2. 每条日志每延迟上报1秒，扣1分
3. 积累日志达到100条，必须立即上报

给出日志序列，根据该规则，计算首次上报能获得的最多积分数。

输入描述

按时序产生的日志条数 $T_1, T_2, \dots, T_n$ ，其中 $1 \leq n \leq 1000$ ， $0 \leq T_i \leq 100$

输出描述

首次上报最多能获得的积分数

用例

输入	1 98 1
输出	98
说明	T1 时刻上报得 1 分 T2 时刻上报得98分，最大 T3 时刻上报得 0 分

输入	50 60 1
输出	50
说明	如果第1个时刻上报，获得积分50。如果第2个时刻上报，最多上报100条，前50条延迟上报1s，每条扣除1分，共获得积分为 100-50=50

输入	3 7 40 10 60
输出	37
说明	T1时刻上报得3分T2时刻上报得7分T3时刻上报得37分，最大T4时刻上报得-3分T5时刻上报，因为已经超了100条限制，所以只能上报100条，得-23分

题目解析

用例1意思是：

如果在T1时刻上报日志，由于只有1条日志，因此可得1分。

如果在T2时刻上报日志，由于T1时刻产生的1条日志延迟了1秒上报，因此要扣1分，到了T2时刻可以上报1+98条日志，可得99分，因此99-1=98分。

如果在T3时刻上报日志，则T1日志延迟了2s，要扣12分，T2日志延迟了1s，要扣98分，T3时刻可以上报100条日志，可得100分，因此100-2-98=0分。

我的解题思路如下：

这种前后数据具有依赖关系，一般都是用[动态规划](#)DP解决。

首先，我用前缀和思路，将每个时刻，可得的正向分计算出来缓存进dp数组中。所谓正向分，比如T2时刻积累了99条日志，那么就应该得到99分。这是正向得分。但是最终得分还需要扣除延迟上报的负向得分。比如T2时刻上报日志的话，则T1时刻产生的每条日志都需要扣除1分，这是负向得分。因此T2时刻的最终得分是：正向得分 - 负向得分 = 99 - 1 = 98分。

每个时刻都有两种选择，上报、不上报。如果某时刻选择不上报，则会正向得分、负向得分不会清零，即会累加给下一个时刻，如果某时刻选择上报了，则对应的正向得分和负向得分都会清零。

需要注意的是，当日志累计到100条或以上时，则必须要上报，这意味着会产生一次得分清零。

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Integer[] arr =
            Arrays.stream(sc.nextLine().split("
")).map(Integer::parseInt).toArray(Integer[]::new);

        System.out.println(getResult(arr));
    }

    public static int getResult(Integer[] arr) {
        int n = arr.length;

        // dp[i]表示：第i时刻可得的正向分
        int[] dp = new int[n];
        dp[0] = arr[0];

        // delay[i]表示：第i时刻被扣除的负向分
        int[] delay = new int[n];
        delay[0] = 0;

        // score[i]表示：第i时刻最终得分
        int[] score = new int[n];
        score[0] = arr[0];

        for (int i = 1; i < n; i++) {
            dp[i] = Math.min(100, dp[i - 1] + arr[i]); // 最多上报100条
            delay[i] = delay[i - 1] + dp[i - 1];
            score[i] = dp[i] - delay[i];

            // 达到100条时必须上报，此时完成首次上报，结束循环
            if (dp[i] >= 100) break;
        }

        return Arrays.stream(score).max().getAsInt();
    }
}
```

29. 挑选字符串

题目描述

给定a-z，26个英文字母小写字符串组成的字符串A和B，其中A可能存在重复字母，B不会存在重复字母，现从字符串A中按规则挑选一些字母可以组成字符串B。

挑选规则如下：

- 同一个位置的字母只能挑选一次，
- 被挑选字母的相对先后顺序不能被改变，
- 求最多可以同时从A中挑选多少组能组成B的字符串。

输入描述

输入为2行，第一行输入字符串a,第二行输入字符串b，行首行尾没有多余空格

输出描述

输出一行，包含一个数字，表示最多可以同时从a中挑选多少组能组成b的字符串，行末没有多余空格

用例

输入	badc bac
输出	1
说明	无

题目解析

本题求解可以参考

[LeetCode - 1419 数青蛙 伏城之外的博客-CSDN博客](#)

[华为机试 - 数大雁 伏城之外的博客-CSDN博客](#)

题目的用例不能说明问题，我们可以通过下面用例

bbadcbacdaccccbac

bac

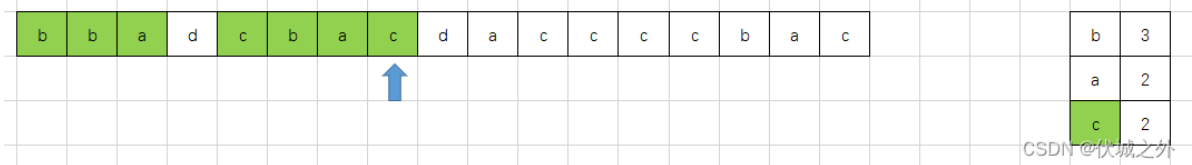
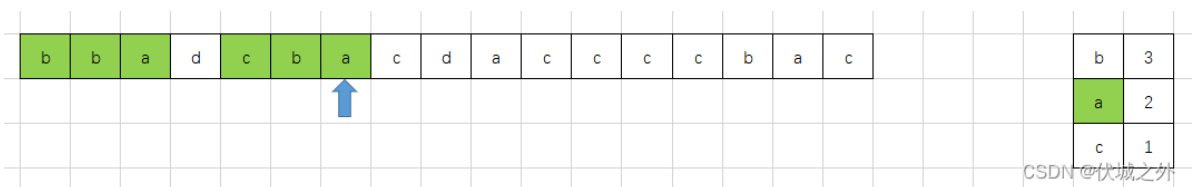
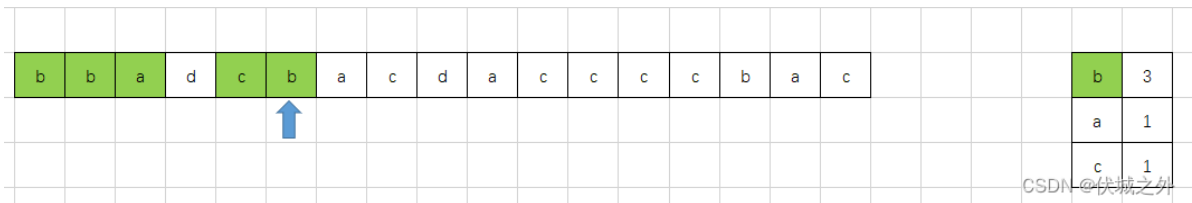
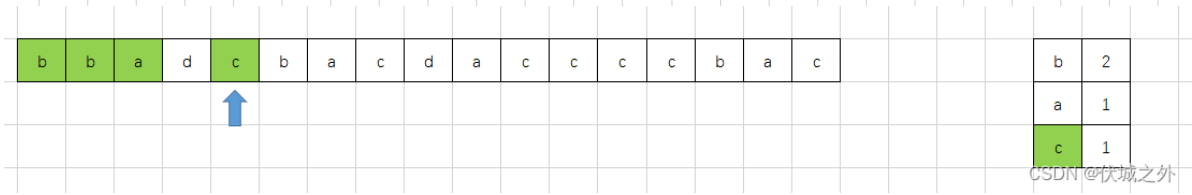
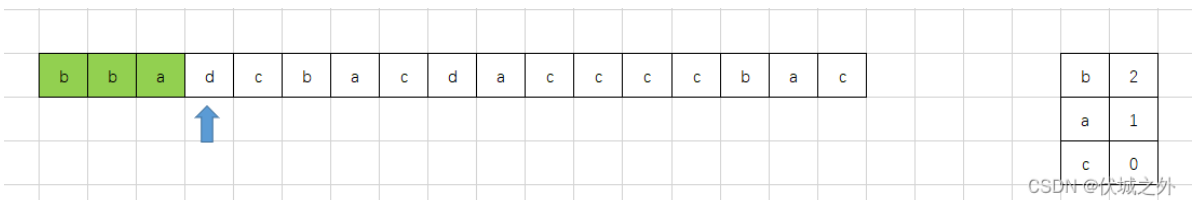
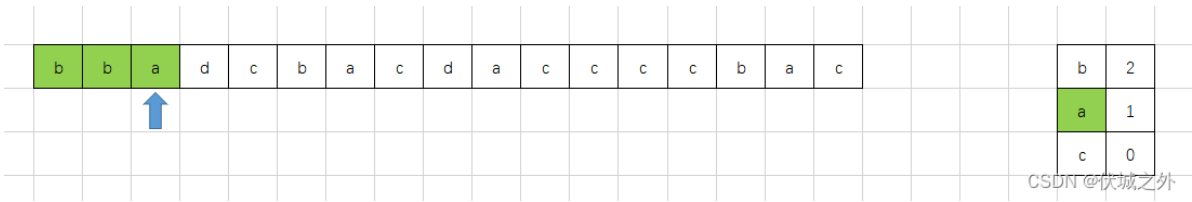
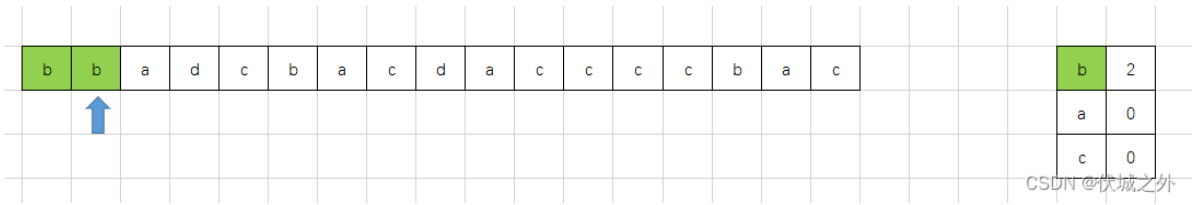
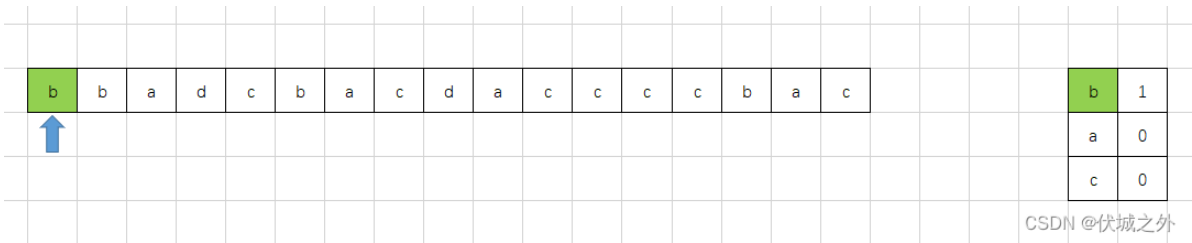
a字符串

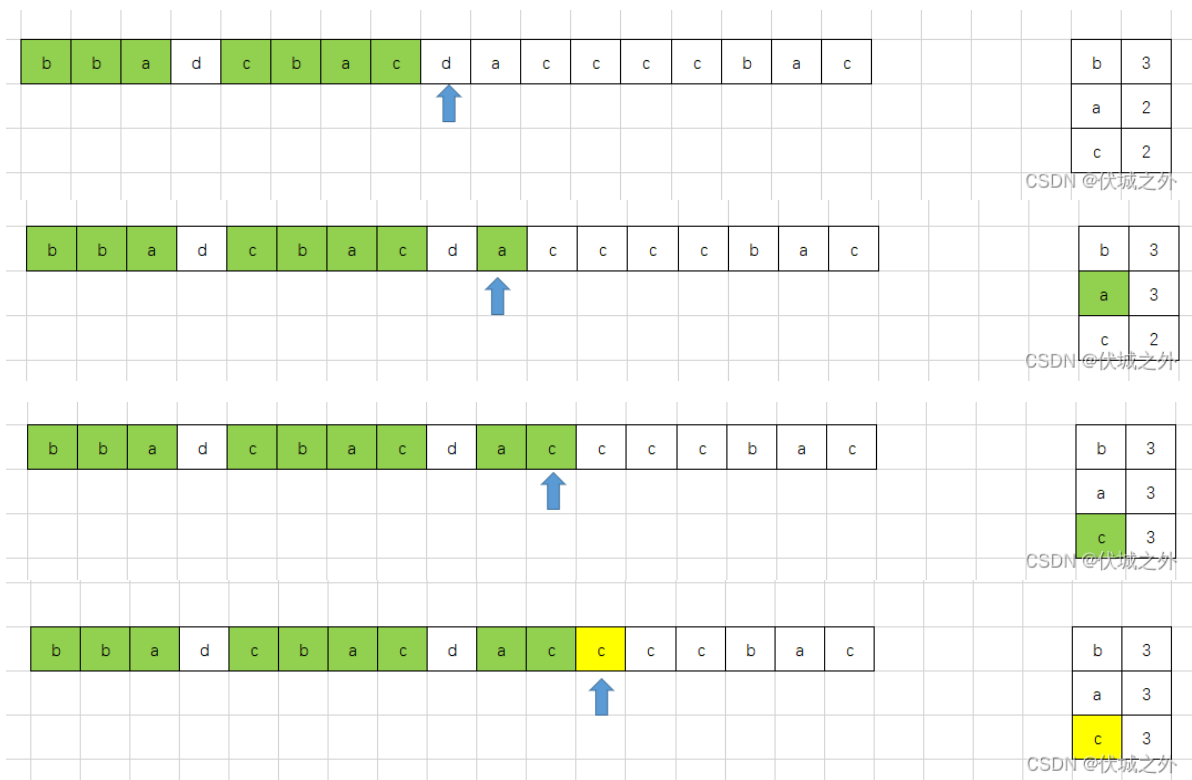
b	b	a	d	c	b	a	c	d	a	c	c	c	c	b	a	c
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

b字符串

b	0
a	0
c	0

CSDN @伏城之外

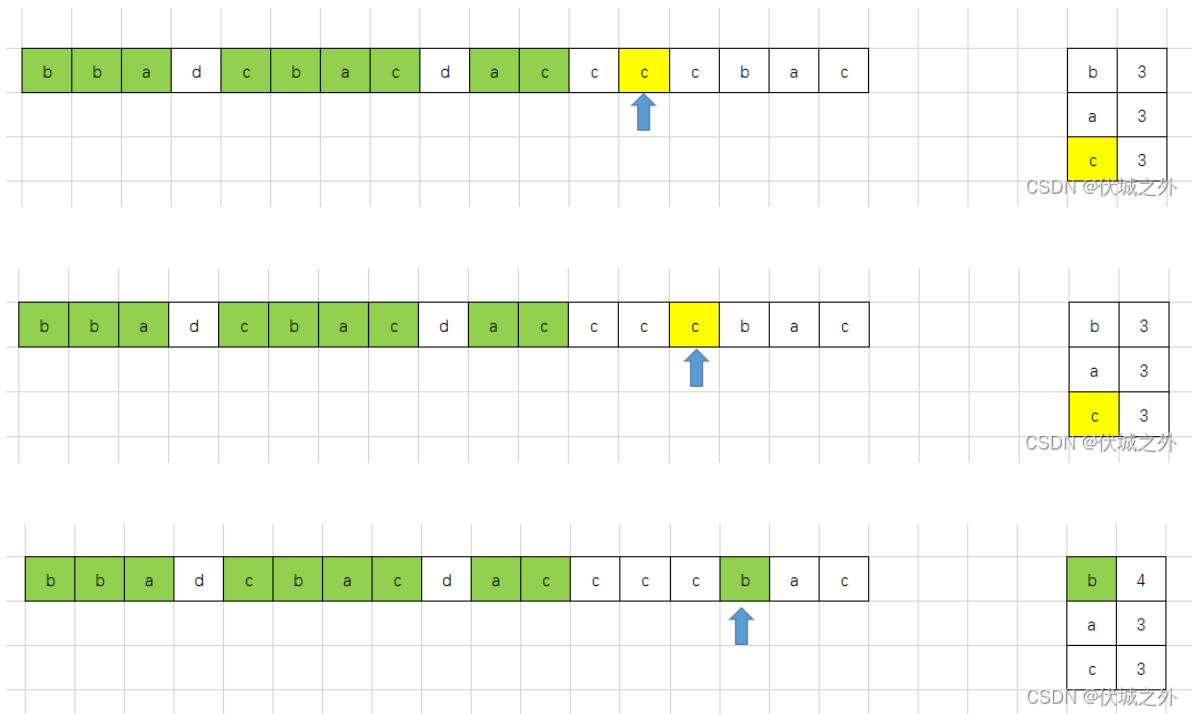


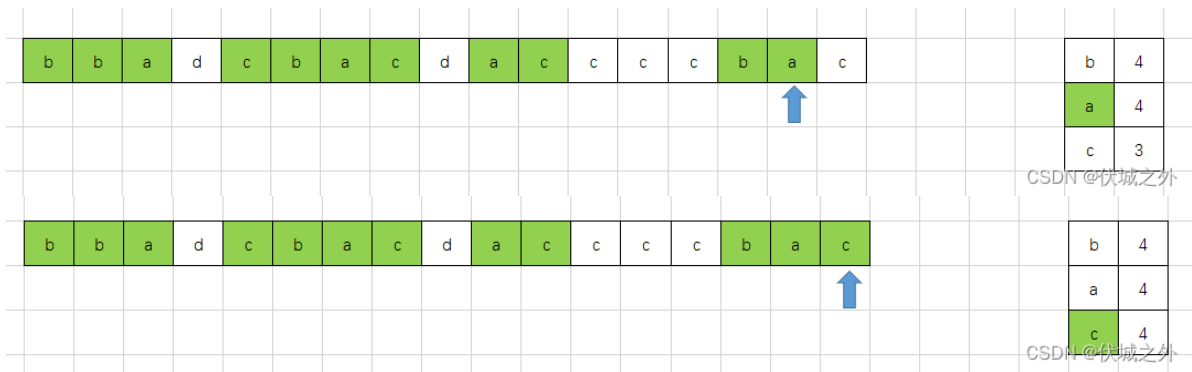


注意：统计时，a的数量应该小于等于b的数量，c的数量应该小于等于a的数量，这样才能满足顺序要求：

被挑选字母的相对先后顺序不能被改变

因此上面这步统计到的c不应该被计入。





由于 $c \leq b \leq a$ ，因此有几个 c ，字符串 a 中就能挑选出几个字符串 b 。

上面算法只需要一次遍历，即可完成题解，时间复杂度 $O(n)$

Java算法源码

```
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String a = sc.next();
        String b = sc.next();

        System.out.println(getResult(a, b));
    }

    public static int getResult(String a, String b) {
        // idxs对象记录字符串b中每个字符的索引
        HashMap<Character, Integer> idxs = new HashMap<>();
        for (int i = 0; i < b.length(); i++) {
            Character c = b.charAt(i);
            idxs.put(c, i); // B不会存在重复字母
        }

        // count对象用于记录遍历字符串a每个字符串过程中，统计到的符合顺序要求的字符串b中字符出现次数
        int[] count = new int[b.length()];
        for (int i = 0; i < a.length(); i++) {
            Character c = a.charAt(i);

            if (idxs.containsKey(c)) {
                int idx = idxs.get(c);
                // 下面判断逻辑请看图解
                if (idx == 0 || count[idx] < count[idx - 1]) {
                    count[idx]++;
                }
            }
        }

        return count[count.length - 1];
    }
}
```

```
}  
}
```

30. 查找重复代码

题目描述

小明负责维护项目下的代码，需要查找出重复代码，用以支撑后续的代码优化，请你帮助小明找出重复的代码。

重复代码查找方法：以字符串形式给定两行代码（字符串长度 $1 < \text{length} \leq 100$ ，由英文字母、数字和空格组成），找出两行代码中的[最长公共子串](#)。

注：如果不存在公共子串，返回空字符串

输入描述

输入的参数text1, text2分别表示两行代码

输出描述

输出任一最长公共子串

用例

输入	hello123world hello123abc4
输出	hello123
说明	无

输入	private_void_method public_void_method
输出	_void_method
说明	无

输入	hiworld hiweb
输出	hiw
说明	无

题目解析

本题可以使用[动态规划](#)求解，下面解释下本题动态规划解题思路。

动态规划，一般是将大问题分解为规模更小的相同的子问题，如果子问题依旧很难求解，则继续分解，直到规模小到子问题可以被轻松求解。

比如上面求两个字符串的最长公共子串，如果两个字符串很长，那么我们将很难一下子发现最长公共子串，那么我们可以只看两个字符串的部分范围，比如字符串str1，只看0~i范围，字符串str2，只看0~j范围，然后求解str1的0~i范围和str2的0~j范围的最长公共子串，最简单就是str1的0~0范围，即空串，和任意str2范围的最长公共子串都是空串，接着我们可以慢慢扩大str1的范围。

可能上面说的还比较抽象，我们可以用一个二维数组dp来描述上面逻辑。

dp[i][j] 表示的是，str1的0~i范围和str2的0~j范围 的公共子串的长度

初始时，我们可以得出如下二维矩阵

str2		""	h	e	l	l	o	1	2	3	a	b	c	4
str1	""	0	0	0	0	0	0	0	0	0	0	0	0	0
h	0													
e	0													
l	0													
l	0													
o	0													
1	0													
2	0													
3	0													
w	0													
o	0													
r	0													
l	0													
d	0													

CSDN @伏城之外

即str1取0~0，即空串时，和任意str2范围的公共子串的长度都是0。

同理，str2取0~0，即空串时，和任意str1范围的公共子串的长度都是0。

接下来，str1取0~1范围（即h），和str2取0~1范围（即h），的公共子串其实就是h，因此长度为1。
如下图所示

str1 \ str2	""	h	e	l	l	o	1	2	3	a	b	c	4
""	0	0	0	0	0	0	0	0	0	0	0	0	0
h	0	1											
e	0												
l	0												
l	0												
o	0												
1	0												
2	0												
3	0												
w	0												
o	0												
r	0												
l	0												
d	0												

CSDN @伏城之外

但是上面公共子串长度求解，也可以看成是，str1扩大范围新增的h，和str2扩大范围新增的h相同，因此出现了公共子串，那么就相当于在str1未扩大范围前（即空串时），和str2未扩大范围前（即空串时），的公共子串长度的基础上+1。

即：dp[1][1] = dp[0][0] + 1

进一步分析，其实可得出[状态转移方程](#)：

if(str1[i] == str2[j]) dp[i][j] = dp[i-1][j-1] + 1

再接下来，str1还是取0~1范围（即h），和str2取0~2范围（即he），

str2 \ str1	""	h	e	l	l	o	1	2	3	a	b	c	4
""	0	0	0	0	0	0	0	0	0	0	0	0	0
h	0	1	0										
e	0												
l	0												
l	0												
o	0												
1	0												
2	0												
3	0												
w	0												
o	0												
r	0												
l	0												
d	0												

CSDN @伏城之外

此时str1还是相当于新增h，而str2相当于新增e，新增部分不相同，所以此处没有增长前面基础上得到的公共子串的长度。

那么此时相当于 $\text{if}(\text{str1}[i] \neq \text{str2}[j]) \text{ dp}[i][j] = \text{dp}[i-1][j-1] + 0$,

但是这种思路是错误的，因为当str1,str2新增范围字符不同时，意味着公共子串的中断，我们应该将此时的dp[i][j]置为0，这样才能防止dp[i+1][j+1]对应的字符相同时，继承前面的公共子串长度。

请看下面错误例子

		a	b	f	c
	0	0	0	0	0
a	0	1			
b	0		2		
e	0			2	
c	0				3

CSDN @伏城之外

正确例子应该是

		a	b	f	c
	0	0	0	0	0
a	0	1			
b	0		2		
e	0			0	
c	0				1

CSDN@伏城之外

因此正确的动态规划状态转移方程是：

- $dp[i][0] = 0$
- $dp[0][j] = 0$
- $\text{if}(\text{str1}[i] == \text{str2}[j]) \text{ dp}[i][j] = \text{dp}[i-1][j-1] + 1 \text{ else } \text{dp}[i][j] = 0$

但是此题并不是让我们求解最长的公共子串的长度，而是求出任一最长公共子串。因此我们可以在遍历求解上面二维矩阵每一个元素时，记录下公共子串最大的长度值，比如下面例子中，求解到黄色元素 $dp[2][2]$ 时，得到了目前最大的公共子串长度2，

		a	b	f	c
	0	0	0	0	0
a	0	1	0	0	0
b	0	0	2		
e	0				
c	0				

CSDN@伏城之外

因此我们可以从任一维度来获取这个公共子串，比如从列维度，相当于 $\text{str2}.\text{slice}(j - \text{maxLen}, j)$ ，其中 j 对应 $dp[i][j]$ 中的 j 。 $\text{maxLen} = dp[i][j]$ 。

		a	b	f	c
	0	0	0	0	0
a	0	1	0	0	0
b	0	0	2		
e	0				
c	0				

CSDN@伏城之外

Java算法源码

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str1 = sc.next();
        String str2 = sc.next();

        System.out.println(getResult(str1, str2));
    }

    public static String getResult(String str1, String str2) {
        int n = str1.length();
        int m = str2.length();

        int[][] dp = new int[n + 1][m + 1];

        int max = 0;
        String ans = "";

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (str1.charAt(i - 1) == str2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;

                    if (dp[i][j] > max) {
                        max = dp[i][j];
                        ans = str1.substring(i - max, i);
                    }
                } else {
                    dp[i][j] = 0;
                }
            }
        }

        return ans;
    }
}
```

```
}  
}
```

31. 创建二叉树

请按下列描述构建一颗二叉树，并返回该树的根节点：

- 1、先创建值为-1的根结点，根节点在第0层；
- 2、然后根据operations依次添加节点： operations[i] = [height, index] 表示对第 height 层的第index 个节点node， 添加值为 i 的子节点：
 - 若node 无「左子节点」， 则添加左子节点；
 - 若node 有「左子节点」， 但无「右子节点」， 则添加右子节点；
 - 否则不作任何处理。

height、index 均从0开始计数；

index 指所在层的创建顺序。

注意：

- 输入用例保证每次操作对应的节点已存在；
- 控制台输出的内容是根据返回的树根节点，按照层序遍历二叉树打印的结果。

输入描述

operations

输出描述

根据返回的树根节点，按照[层序遍历](#)二叉树打印的结果

备注

- 1 <= operations.length <= 100
- operations[i].length == 2
- 0 <= operations[i][0] < 100
- 0 <= operations[i][1] < 100

用例

输入	[[0, 0], [0, 0], [1, 1], [1, 0], [0, 0]]
输出	[-1, 0, 1, 3, null, 2]

输入	[[0, 0], [0, 0], [1, 1], [1, 0], [0, 0]]
说明	首个值是根节点的值，也是返回值；null 表示是空节点，此特殊层序遍历会遍历有值节点的 null 子节点

输入	[[0, 0], [1, 0], [1, 0], [2, 1], [2, 1], [2, 1], [2, 0], [3, 1], [2, 0]]
输出	[-1, 0, null, 1, 2, 6, 8, 3, 4, null, null, null, null, null, null, 7]
说明	首个值是根节点的值，也是返回值；null 表示是空节点，此特殊层序遍历会遍历有值节点的 null 子节点

题目解析

首先，解释下上面两个用例的输入，输出

用例1含义

	0 1 2 3 4
输入	[[0, 0], [0, 0], [1, 1], [1, 0], [0, 0]]

operations[0] = [0, 0]

即：第0行第0个创建的节点插入子节点值0



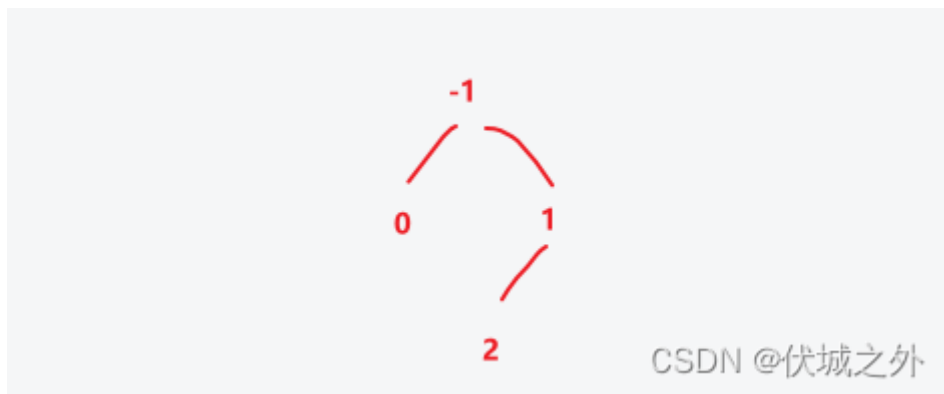
operations[1] = [0, 0]

即：第0行第0个创建的节点插入子节点值1



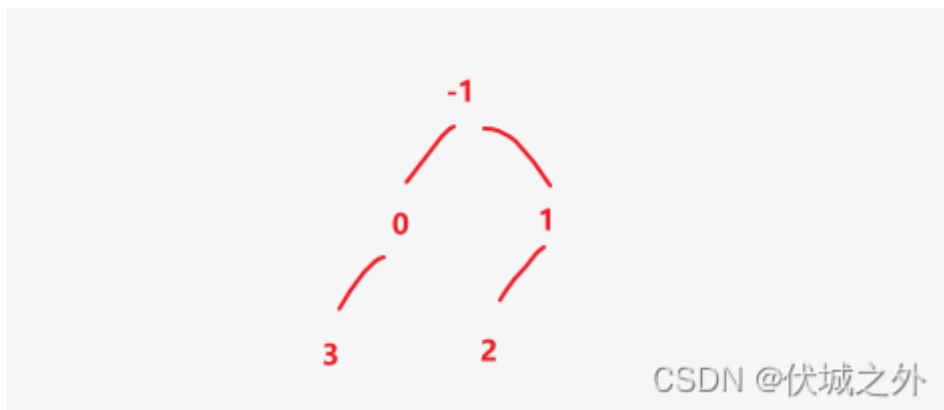
operations[2] = [1, 1]

即：第1行第1个创建的节点插入子节点值2



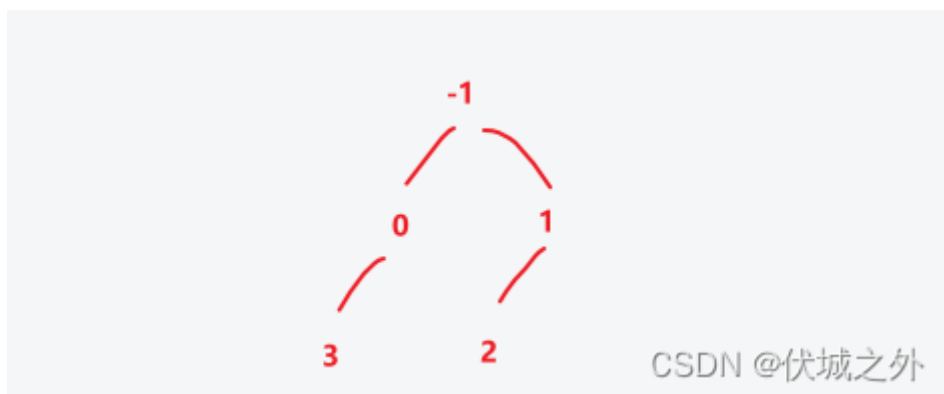
operations[3] = [1, 0]

即：第1行第0个创建的节点插入子节点值3

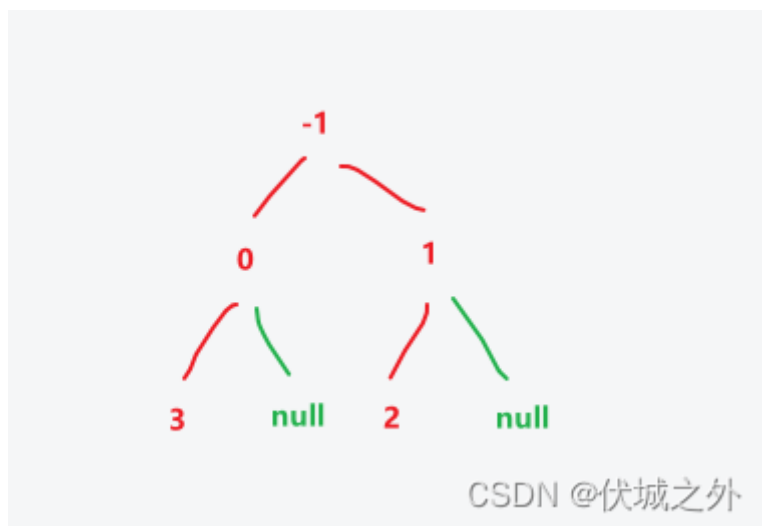


operations[4] = [0, 0]

即：第0行第0个创建的节点插入子节点值4，但是由于第0行第0个创建的节点左右子节点都已有了，因此不插入



上图就是用例1的二叉树，输出时，由于题目说：层序遍历会遍历有值节点的 null 子节点，因此为上面二叉树的有值节点加入null子节点，如下图所示



层序遍历结果为：-1, 0, 1, 3, null, 2, null

但是用例输出是：[-1, 0, 1, 3, null, 2]，因此最后一个null子节点可以不用输出。

用例2含义

	0	1	2	3	4	5	6	7	8
输入	[[0, 0], [1, 0], [1, 0], [2, 1], [2, 1], [2, 1], [2, 0], [3, 1], [2, 0]]								

CSDN @伏城之外

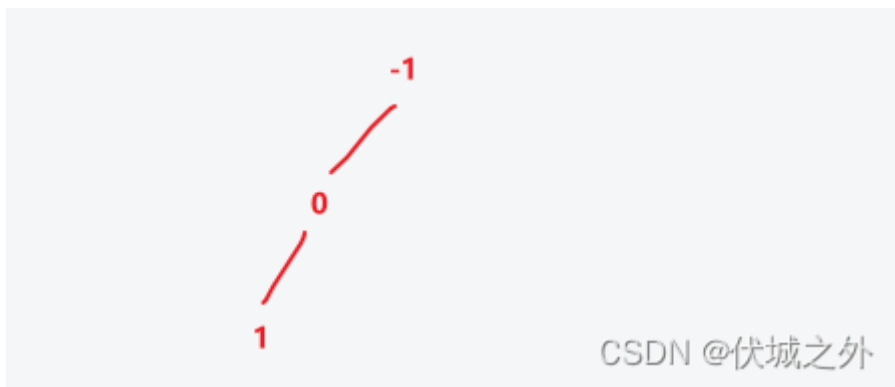
operations[0] = [0, 0]

即：第0行第0个创建的节点插入子节点值0



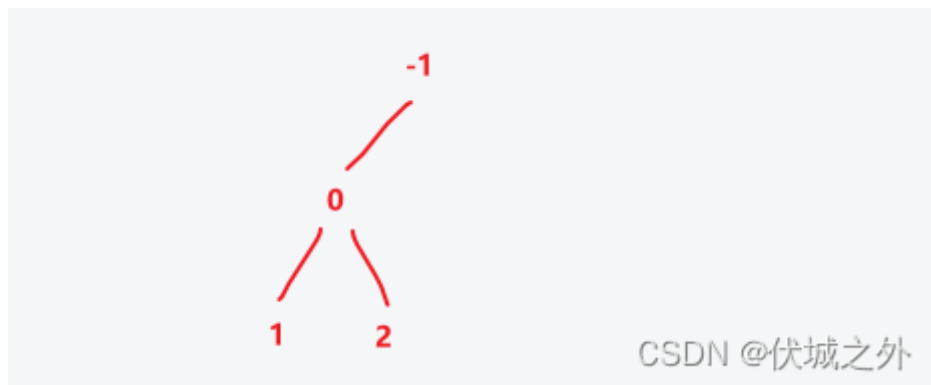
operations[1] = [1, 0]

即：第1行第0个创建的节点插入子节点值1



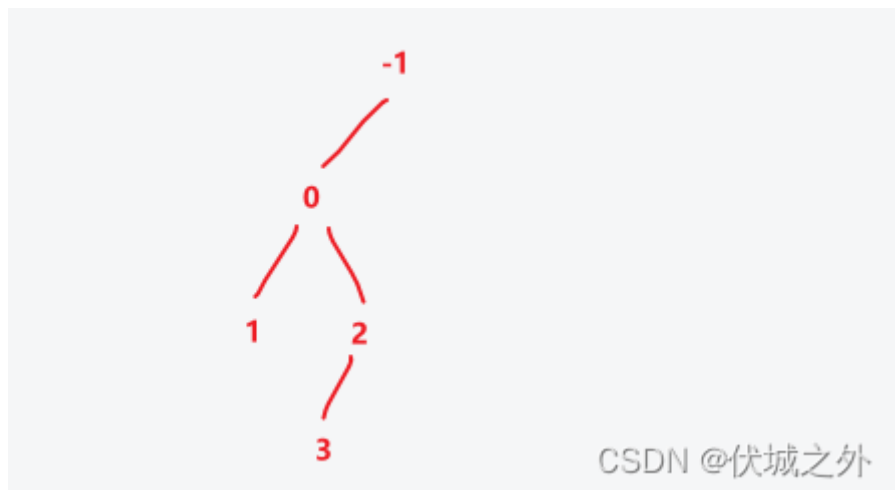
`operations[2] = [1, 0]`

即：第1行第0个创建的节点插入子节点值2



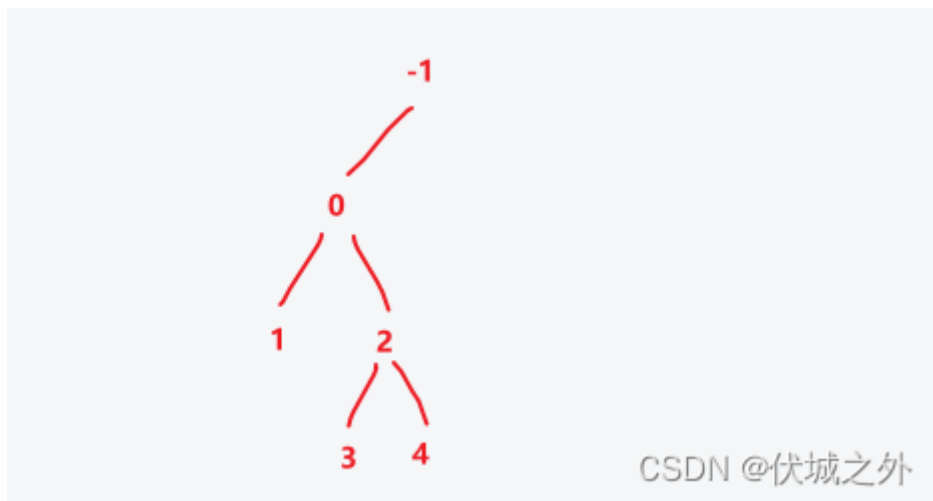
`operations[3] = [2, 1]`

即：第2行第1个创建的节点插入子节点值3



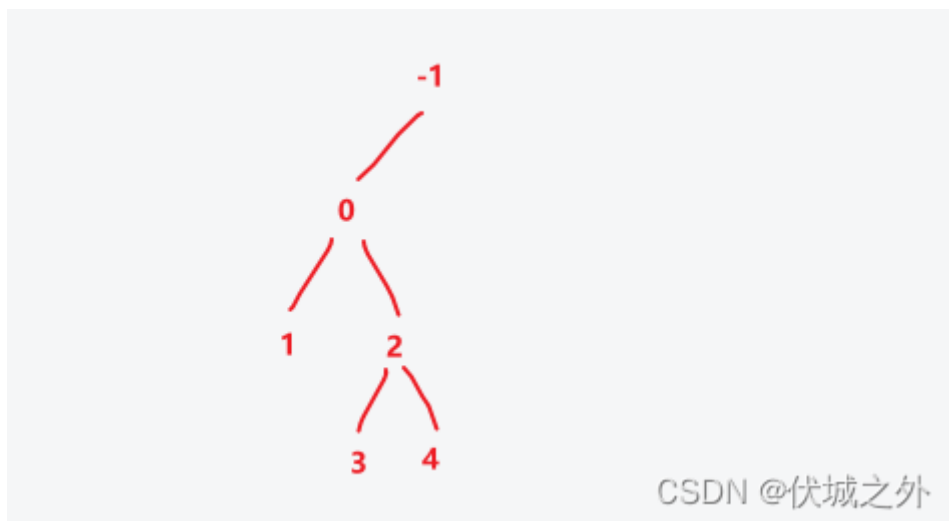
`operations[4] = [2, 1]`

即：第2行第1个创建的节点插入子节点值4



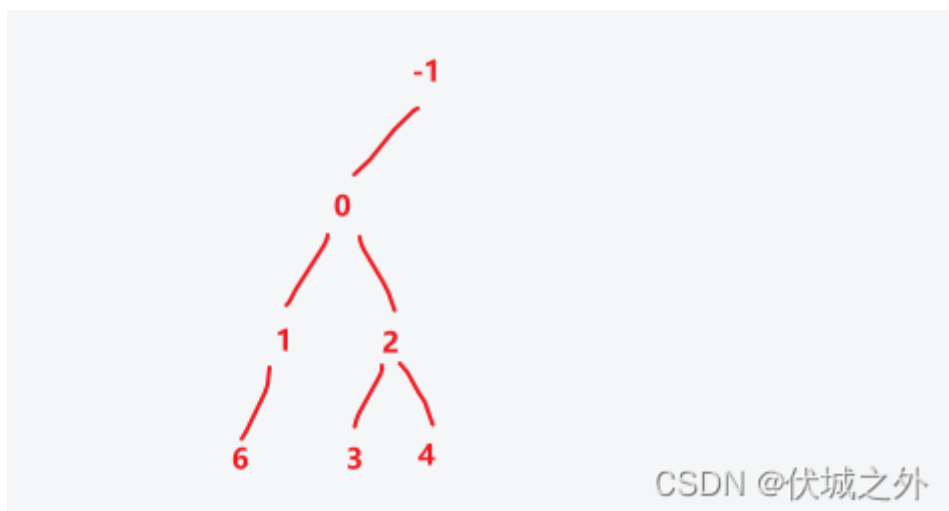
operations[5] = [2, 1]

即：第2行第1个创建的节点插入子节点值5，但是由于第2行第1个创建的节点已经有了左右子节点，因此不插入



operations[6] = [2, 0]

即：第2行第0个创建的节点插入子节点值6



operations[7] = [3, 1]

即：第3行第1个创建的节点插入子节点值7

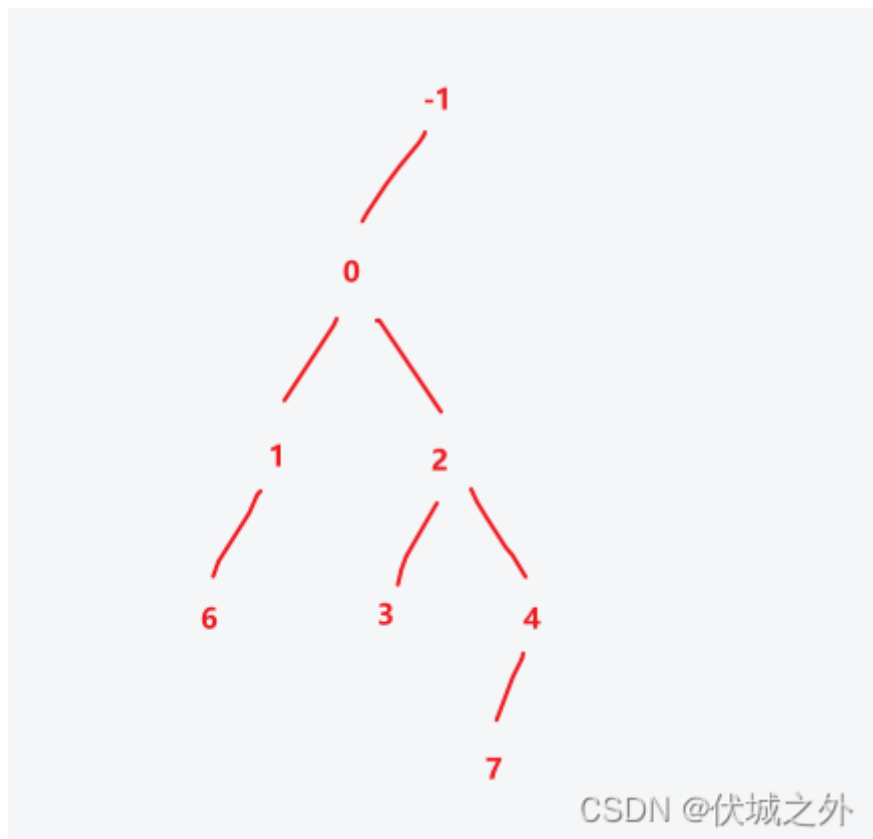
此时需要特别注意下，第3行第1个创建的**节点是上图的值3节点吗？？？*

为什么我要特别标注这句话呢？

因为题目中说：operations[i] = [height, index]，而index 指所在层的**创建顺序**。

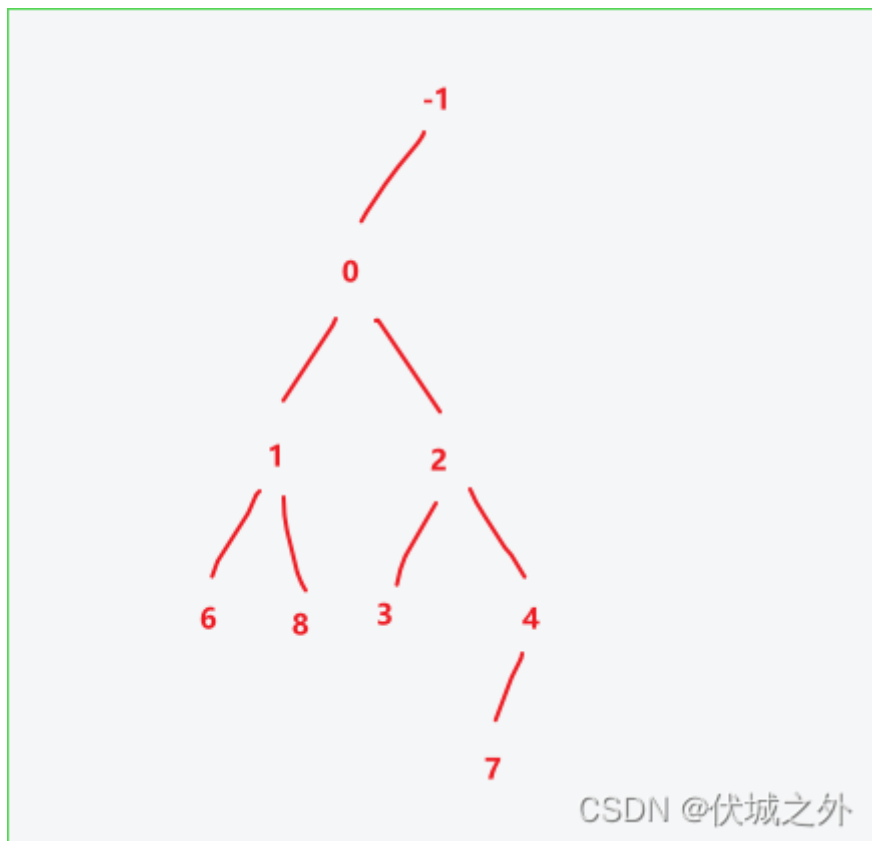
我们回到 operations[4]，可以发现***第3行第1个创建的**节点****是值4节点，而值3节点是第3行第0个创建的节点。

因此此步图示如下

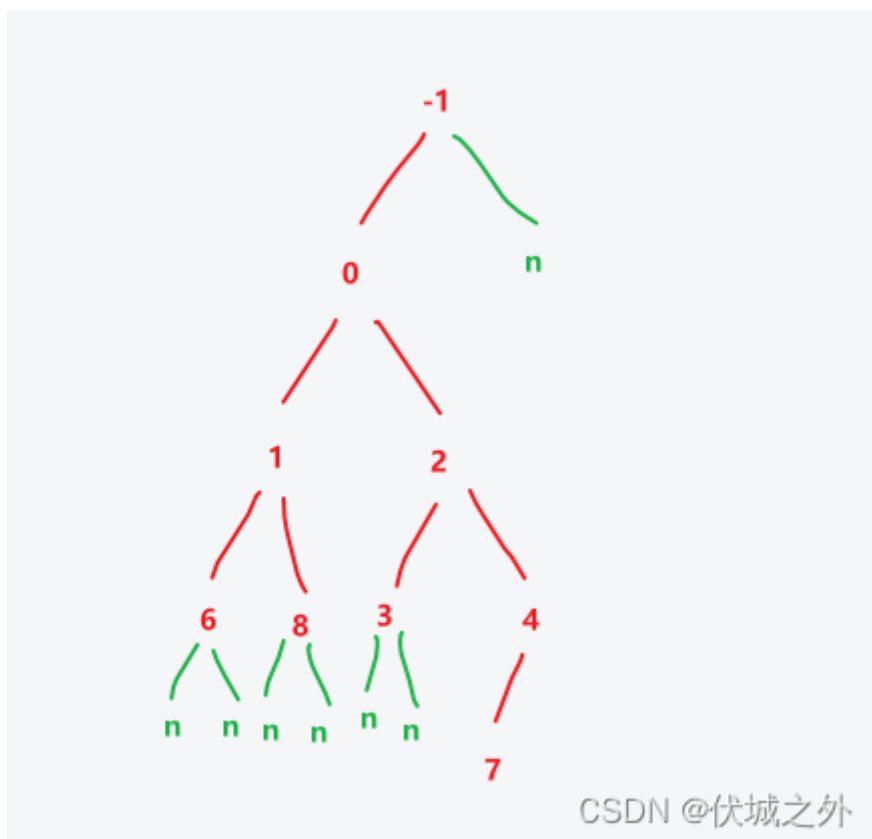


operations[8] = [2, 0]

即：第2行第0个创建的节点插入子节点值8



补充有值节点的null子节点



因此按照层序输出顺序是：

-1, 0, null, 1, 2, 6, 8, 3, 4, null, null, null, null, null, 7

当我们了解完上面两个用例的含义后，就可以做题了。

本题的难点在于如何记录每层节点的创建顺序index，如果通过传统的二叉树结构，则很难实现，原因是：

- 如何通过height（层数），index（该层第index个创建的）对应到二叉树的点

其中层数height可以通过层序遍历二叉树找到，但是index创建顺序却无法通过层序遍历找到，因此在构建二叉树时，还需要额外记录index到每个点自身，在层序遍历的时候，找到点，并取得它自身记录的index值，对比要查找的index值。

上面这个逻辑就非常麻烦了，因此不推荐这么做。

我的解题思路是，创建一个二维数组（集合，列表）tree，初始时为：[[node(-1)]]

即tree[0][0] = node(-1)，这里tree[0][0]表示height = 0, index = 0

即tree的行索引就是height层数，列索引就是index创建顺序

tree[0][0]就代表二叉树根节点。

当我们遍历operations时，operations[i] = [height, index]，其实就是找 tree[height][index]父节点，在该父节点下插入node(i)子节点，而在该父节点下插入node(i)子节点，其实就是向tree[height+1]数组中加入node(i)节点，比如

插入逻辑是：

- 若tree[height][index] 无「左子节点」，则添加左子节点；
- 若tree[height][index] 有「左子节点」，但无「右子节点」，则添加右子节点；
- 否则不作任何处理。

如果tree[height+1]不存在，需要先初始化一个数组，然后加入node(i)节点，此时node(i)节点在tree[height+1]中的索引其实就是node(i)节点，在height+1层的index创建顺序。

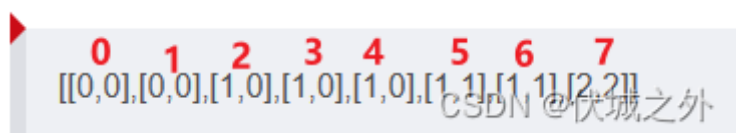
这样的话，我们就能依赖于tree二维数组结构来完成任意节点的height，index的记录了。这样用例2插入值7时，就可以快速找到第3行第1个创建的节点了。

但是，此时各节点之间并没有建立连续，因此我们还要设计Node类，来完成各节点之间的父子关系构建。这个就很简单了，具体实现看代码。

2023.03.15 经网友指正，有如下用例：

```
[[0,0],[0,0],[1,0],[1,0],[1,0],[1,1],[1,1],[2,2]]
```

其中[2,2]应该对应哪个值得节点，大家可以先思考一下



0 1 2 3 4 5 6 7
[[0,0],[0,0],[1,0],[1,0],[1,0],[1,1],[1,1],[2,2]]
CSDN @伏城之外

其实如果按照上面思路，值4节点应该是二叉树的第二行第二个创建的节点，但是它没有成功插入到二叉树中。

因此它不能算二叉树的第二行第二个创建的节点，值5接待你才能算上二叉树的第二行第二个创建的节点。

因此，前面tree记录节点创建顺序时，应该注意这种情况，要排除掉加入二叉树失败的节点。

Java算法源码

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        Integer[][] operations =
            Arrays.stream(str.substring(1, str.length() - 1).split("(?<=)"), (?<=\\
[]"))
                .map(
                    s ->
                        Arrays.stream(s.substring(1, s.length() - 1).split(", "))
                            .map(Integer::parseInt)
                            .toArray(Integer[]::new))
                .toArray(Integer[][]::new);

        System.out.println(getResult(operations));
    }

    public static String getResult(Integer[][] operations) {
        Node head = new Node(-1);

        ArrayList<Node> level0 = new ArrayList<>();
        level0.add(head);

        ArrayList<ArrayList<Node>> tree = new ArrayList<>();
        tree.add(level0);

        for (int i = 0; i < operations.length; i++) {
            int height = operations[i][0];
            int index = operations[i][1];

            if (tree.size() <= height + 1) {
                tree.add(new ArrayList<>());
            }

            Node ch = new Node(i);

            Node fa = tree.get(height).get(index);
            // 注意，tree用于记录树中加入成功的节点是第几行第几个创建的，对于加入的失败的不应该记录
            if (fa.lc == null || fa.rc == null) {
                tree.get(height + 1).add(ch);
            }
        }
    }
}
```

```

        if (fa.lc == null) fa.lc = ch;
        else if (fa.rc == null) fa.rc = ch;
    }

    LinkedList<Integer> ans = new LinkedList<>();
    LinkedList<Node> queue = new LinkedList<>();
    queue.add(tree.get(0).get(0));

    while (queue.size() > 0) {
        Node node = queue.removeFirst();

        if (node != null) {
            ans.add(node.val);
            queue.add(node.lc);
            queue.add(node.rc);
        } else {
            ans.add(null);
        }
    }

    while (true) {
        if (ans.getLast() == null) ans.removeLast();
        else break;
    }

    StringJoiner sj = new StringJoiner(", ", "[", "]");
    for (Integer an : ans) {
        sj.add(an + "");
    }

    return sj.toString();
}

class Node {
    int val;
    Node lc;
    Node rc;

    public Node(int val) {
        this.val = val;
        this.lc = null;
        this.rc = null;
    }
}

```

32. 最多等和不相交连续子序列

题目描述

给定一个数组，我们称其中连续的元素为连续子序列，称这些元素的和为连续子序列的和。

数组中可能存在几组连续子序列，组内的连续子序列互不相交且有相同的和。

求一组连续子序列，组内子序列的数目最多。

输出这个数目。

输入描述

第一行输入为数组长度N， $1 \leq N \leq 10^3$

第二行为N个用空格分开的整数 C_i ， $-10^5 \leq C_i \leq 10^5$

输出描述

第一行是一个整数M，表示满足要求的最多的组内子序列的数目。

用例

输入	10 8 8 9 1 9 6 3 9 10
输出	4
说明	四个子序列的第一个元素和最后一个元素的下标分别为2 24 45 67 7

输入	10 -1 0 4 -3 6 5 -6 5 -7 -3
输出	3
说明	三个子序列的第一个元素和最后一个元素的下标分别为：3 35 89 9

题目解析

解题步骤分为两大步：

- 1、将和相同的连续子序列的区间统计在一起
- 2、求解每个“和”下的最大不相交区间数量，保留最大数量作为题解

第一步，我们可以用[动态规划](#)前缀和的思路求出任意区间的连续子序列的和，具体请看下面博客中：一维数组前缀和的应用

[算法设计 - 前缀和 & 差分序列 伏城之外的博客-CSDN博客](#)

并记录对应区间到该“和”下面。例如

8	8	9	1	9	6	3	9	1	0	{8:[[0,0]]}	将连续子序列8的区间[0,0]记录到和8下面
8	8	9	1	9	6	3	9	1	0	{8:[[0,0]], 16:[[0,1]]}	将连续子序列8 8的区间[0,1]记录到 和16 下面
8	8	9	1	9	6	3	9	1	0	{8:[[0,0]], 16:[[0,1]], 25:[[0,2]]}	将连续子序列8 8 9的区间[0,2]记录到 25 下面

CSDN @伏城之外

JS可以直接使用对象记录不同和对应的区间，Java可以用HashMap记录。

统计完后容器应该如下：

```
{
  '0': [ [ 9, 9 ] ],
  '1': [ [ 3, 3 ], [ 8, 8 ], [ 8, 9 ] ],
  '3': [ [ 6, 6 ] ],
  '6': [ [ 5, 5 ] ],
  '8': [ [ 0, 0 ], [ 1, 1 ] ],
  '9': [ [ 2, 2 ], [ 4, 4 ], [ 5, 6 ], [ 7, 7 ] ],
  '10': [ [ 2, 3 ], [ 3, 4 ], [ 7, 8 ], [ 7, 9 ] ],
  '12': [ [ 6, 7 ] ],
  '13': [ [ 6, 8 ], [ 6, 9 ] ],
  '15': [ [ 4, 5 ] ],
  '16': [ [ 0, 1 ], [ 3, 5 ] ],
  '17': [ [ 1, 2 ] ],
  '18': [ [ 1, 3 ], [ 4, 6 ], [ 5, 7 ] ],
  '19': [ [ 2, 4 ], [ 3, 6 ], [ 5, 8 ], [ 5, 9 ] ],
  '25': [ [ 0, 2 ], [ 2, 5 ] ],
  '26': [ [ 0, 3 ] ],
  '27': [ [ 1, 4 ], [ 4, 7 ] ],
  '28': [ [ 2, 6 ], [ 3, 7 ], [ 4, 8 ], [ 4, 9 ] ],
}
```

CSDN @伏城之外

每个和下面都有对应的区间。

接下来就是求解同一个和下面的最大不相交区间数量了。

而求解给定的多个区间的最大不相交区间数量，有固定策略：

假设不相交区间数量为count，则至少为1

- 1、先将多个区间，按照右边界升序
- 2、然后取排序后第一个区间的右边界作为 t，并遍历之后的下一个区间[l, r]：
 - 如果 $l \leq t$ ，则说明当前两个区间有交集，则舍弃遍历的区间，继续遍历下一个区间
 - 如果 $l > t$ ，则说明当前两个区间无交集，即不相交，此时 $t = r$ ，并且不相交区间数量count++，然后继续下一次遍历

这样最后得到的不相交区间数量count，就是最大不相交区间数量

Java算法源码

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

int n = sc.nextInt();

int[] arr = new int[n];
for (int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}

System.out.println(getResult(arr, n));
}

public static int getResult(int[] arr, int n) {
    // 记录相同和连续子序列的区间
    HashMap<Integer, ArrayList<Integer[]>> ranges = new HashMap<>();

    // 求解arr数组的前缀和数组dp
    int[] dp = new int[n];
    dp[0] = arr[0];
    for (int i = 1; i < n; i++) {
        dp[i] = dp[i - 1] + arr[i];
    }

    // 利用前缀和求差，求出所有连续子序列的和
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            if (i == 0) {
                int sum = dp[j];
                ranges.putIfAbsent(sum, new ArrayList<>());
                ranges.get(sum).add(new Integer[] {0, j});
            } else {
                int sum = dp[j] - dp[i - 1];
                ranges.putIfAbsent(sum, new ArrayList<>());
                ranges.get(sum).add(new Integer[] {i, j});
            }
        }
    }

    //    for (int i = 0; i < n; i++) {
    //        int sum = arr[i];
    //        ranges.putIfAbsent(sum, new ArrayList<>());
    //        ranges.get(sum).add(new Integer[] {i, i});
    //    }
    //    for (int j = i + 1; j < n; j++) {
    //        sum += arr[j];
    //        ranges.putIfAbsent(sum, new ArrayList<>());
    //        ranges.get(sum).add(new Integer[] {i, j});
    //    }
    // }

    // 保存相同和不相交连续子序列的最大个数
    int max = 0;
    for (Integer key : ranges.keySet()) {
        ArrayList<Integer[]> range = ranges.get(key);
        max = Math.max(max, disjoint(range));
    }
}

```



```

        return max;
    }

    // 求不相交区间的最大个数
    public static int disjoint(ArrayList<Integer[]> ranges) {
        int count = 1; // 至少一个
        ranges.sort((a, b) -> a[1] - b[1]);

        Integer t = ranges.get(0)[1];
        for (int i = 1; i < ranges.size(); i++) {
            Integer[] range = ranges.get(i);
            Integer l = range[0];
            Integer r = range[1];

            if (t < l) {
                count++;
                t = r;
            }
        }
        return count;
    }
}

```

33. 士兵过河

题目描述

一支N个士兵的军队正在趁夜色逃亡，途中遇到一条湍急的大河。

敌军在T的时长后到达河面，没到过对岸的士兵都会被消灭。

现在军队只找到了1只小船，这船最多能同时坐上2个士兵。

1. 当1个士兵划船过河，用时为 $a[i]$ ； $0 \leq i < N$
2. 当2个士兵坐船同时划船过河时，用时为 $\max(a[j], a[i])$ 两士兵中用时最长的。
3. 当2个士兵坐船1个士兵划船时，用时为 $a[i] * 10$ ； $a[i]$ 为划船士兵用时。
4. 如果士兵下河游泳，则会被湍急水流直接带走，算作死亡。

请帮忙给出一种解决方案，保证存活的士兵最多，且过河用时最短。

输入描述

第一行：N 表示士兵数($0 < N < 1,000,000$)

第二行：T 表示敌军到达时长($0 < T < 100,000,000$)

第三行： $a[0]$ $a[1]$... $a[i]$... $a[N-1]$

$a[i]$ 表示每个士兵的过河时长。

($10 < a[i] < 100$; $0 \leq i < N$)

输出描述

第一行：“最多存活士兵数” “最短用时”

备注

- 1) 两个士兵的同时划船时，如果划速不同则会导致船原地转圈圈；所以为保持两个士兵划速相同，则需要向划的慢的士兵看齐。
- 2) 两个士兵坐船时，重量增加吃水加深，水的阻力增大；同样的力量划船速度会变慢；
- 3) 由于河水湍急大量的力用来抵消水流的阻力，所以2) 中过河用时不是 $a[i] * 2$ ，而是 $a[i] * 10$ 。

用例

输入	5 43 12 13 15 20 50
输出	3 40
说明	可以达到或小于43的一种方案： 第一步：a[0] a[1] 过河用时：13 第二步：a[0] 返回用时：12 第三步：a[0] a[2] 过河用时：15

输入	5 130 50 12 13 15 20
输出	5 128
说明	可以达到或小于130的一种方案： 第一步：a[1] a[2] 过河用时：13 第二步：a[1] 返回用时：12 第三步：a[0] a[4] 过河用时：50 第四步：a[2] 返回用时：13 第五步：a[1] a[2] 过河用时：13 第六步：a[1] 返回用时：12 第七步：a[1] a[3] 过河用时：15 所以输出为： 5 128

输入	7 171 25 12 13 15 20 35 20
输出	7 171
说明	可以达到或小于171的一种方案： 第一步：a[1] a[2] 过桥用时：13 第二步：a[1] 带火把返回用时：12 第三步：a[0] a[5] 过桥用时：35 第四步：a[2] 带火把返回用时：13 第五步：a[1] a[2] 过桥用时：13 第六步：a[1] 带火把返回用时：12 第七步：a[4] a[6] 过桥用时：20 第八步：a[2] 带火把返回用时：13 第九步：a[1] a[3] 过桥用时：15 第十步：a[1] 带火把返回用时：12 第十一步：a[1] a[2] 过桥用时：13所以输出为： 7 171

题目解析

本题是经典的吊桥谜题（过河问题，四人过桥）的变种题，

如果大家没有了解过吊桥谜题，那么此题将异常难解，建议大家可以看下这个视频[中文配音TED烧脑谜题 - 四人过桥 - 第一季01 哔哩哔哩bilibili](#)

吊桥谜题解题关键在于：

让最慢的两个人所浪费的时间最小化，这可以通过让他们一起过桥来实现，
因为需要有人提着灯（开船）返回，所以得让速度最快得两个人来完成这个任务。

因此，本题士兵过河的步骤应该如下：

1. 首先让用时最短的两个士兵A，B划船过河到对岸，用时 $A \leq B$
2. 然后让B留在对岸，让最快的士兵A划船回来本岸
3. 然后让本岸用时最长的两个士兵划船过河到对岸
4. 然后再让对岸用时最短的士兵B划船回来本岸

按以上逻辑循环，直到用时达到上限T，或者士兵全部运完。

有了以上逻辑基础，我们就可以进一步认识[动态规划](#)的状态转移方程了。

没错，本题最好的方式是基于动态规划求解。

首先，我们暂不考虑用时上限T，只考虑将所有士兵运到对岸的需要的最少时间。因此分析如下：

- 如果只有1个士兵的话，则上面步骤只能走到第1步。
- 如果只有2个士兵的话，则上面步骤也只能走到第1步。
- 如果有3个士兵的话，则上面步骤可以走到第3步，且此时是A和用时最长的士兵过河。
- 如果有4个士兵的话，则上面步骤可以走到第4步，且此时是两个用时最短的士兵A和B过河。

其中只有1个士兵和只有2个士兵的只能走到第1步的原因很容易想到，这里就不赘述了。

而造成3个士兵，和4个士兵走到不同步骤结束的原因是：

抛开用时最短的A，B士兵，剩余用时较慢的士兵是奇数个，还是偶数个。

如果是偶数个，则可以走到上面第4步，如果是奇数个，则只能走到上面第3步。

因为，当我们把士兵B留在对岸，让士兵A回来后，优先让最慢的两个士兵结伴过河，因此最慢的士兵数在不停的减2：

- 如果最慢的士兵数是奇数个的话，则最终会残留1个，此时刚好和留在本岸的士兵A一起结伴过河，提前结束。
- 如果最慢的士兵数是偶数个的话，则最终会残留0个，并且最后一组最慢士兵会将船带到对岸，因此需要对岸最快的士兵B把船开回来，然后带上A再一起走。而这也是上面第4步的由来。

我们可以定义一个dp数组，dp[i]的含义是 0~i 士兵全部过河所需的最短时间。

假设每个士兵过河所需时间记录在times数组中，且times数组已经按用时升序。

那么可得：

dp[0] = times[0]

dp[1] = times[1]

dp[i] 的取值有两种选择，如下：

dp[i] = dp[i-1] + times[0] + times[i]

dp[i] = dp[i-2] + times[0] + times[i] + times[1] + times[1]

dp[i] = dp[i-1] + times[0] + times[i] 的含义是：

抛开最快的A，B士兵，剩余较慢的士兵是奇数个，因此最后较慢士兵中会遗留1个在本岸没有人组队，其余 0 ~ i-1 士兵都已经过河（dp[i-1]代表0 ~ i-1士兵全部过河），因此我们需要让对岸最快的士兵A，即 0号士兵送船回来，此时用时times[0]，然后 0号士兵和 i 号士兵一起过河，此时用时times[i]。

dp[i] = dp[i-2] + times[0] + times[i] + times[1] + times[1] 的含义是：

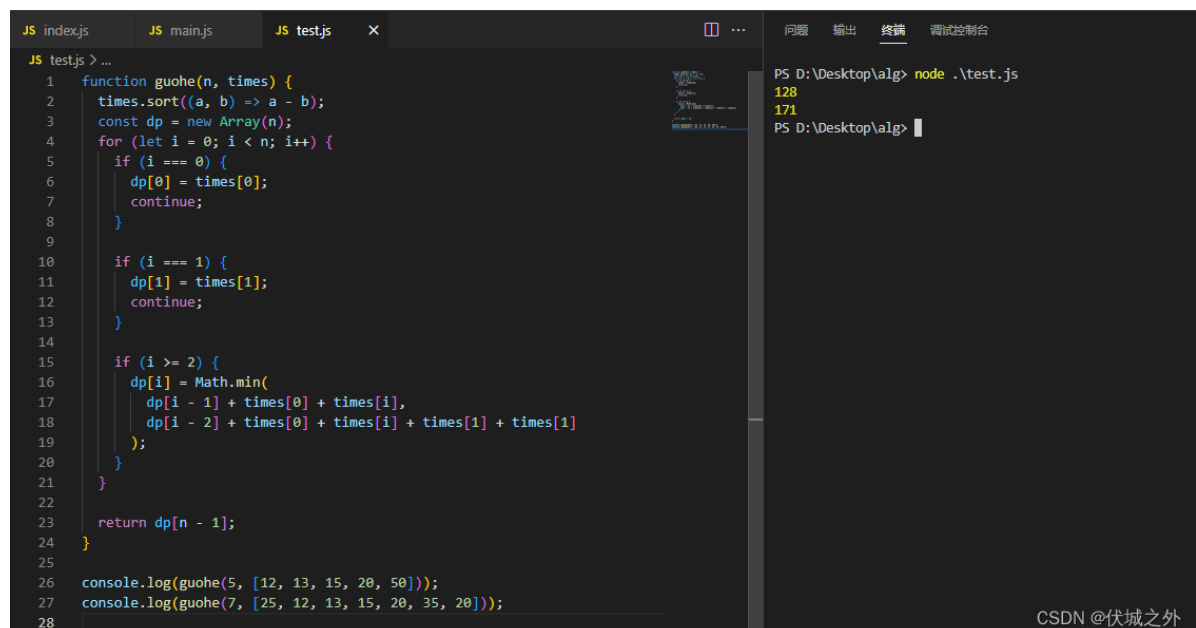
抛开最快的A，B士兵，剩余较慢的士兵是偶数个，因此最后较慢士兵会遗留0个士兵在本岸，但是此时无法找到dp状态转移关系，因此我们后退一步，即当较慢士兵会遗留2个士兵在本岸，即 0 ~ i - 2 的士兵已经全部过河（dp[i-2]代表0~i-2的士兵已经全部过河），因此我们需要让对岸最快的士兵A，即0号士兵送船回来，此时用时times[0]，然后 i - 1 号士兵和 i 号士兵划船过河，用时times[i]，然后对岸最快的B士兵，即1号士兵送船回来，用时times[1]，然后0号士兵和1号士兵再开船到对岸，用时times[1]。

为了dp状态转移方程的简单期间，这里我们不去求解：除了最快的A,B士兵，剩下较慢士兵个数是偶数个还是奇数个，而是直接让：

dp[i] = Math.min(dp[i-1] + times[0] + times[i], dp[i-2] + times[0] + times[i] + times[1] + times[1])

即取两种情况的最少用时。

上面动态规划的代码实现与测试如下：



```
JS index.js JS main.js JS test.js x
JS test.js > ...
1 function guohe(n, times) {
2   times.sort((a, b) => a - b);
3   const dp = new Array(n);
4   for (let i = 0; i < n; i++) {
5     if (i === 0) {
6       dp[0] = times[0];
7       continue;
8     }
9
10    if (i === 1) {
11      dp[1] = times[1];
12      continue;
13    }
14
15    if (i >= 2) {
16      dp[i] = Math.min(
17        dp[i - 1] + times[0] + times[i],
18        dp[i - 2] + times[0] + times[i] + times[1] + times[1]
19      );
20    }
21  }
22
23  return dp[n - 1];
24 }
25
26 console.log(guohe(5, [12, 13, 15, 20, 50]));
27 console.log(guohe(7, [25, 12, 13, 15, 20, 35, 20]));
28
```

问题 输出 终端 调试控制台

```
PS D:\Desktop\alg> node .\test.js
128
171
PS D:\Desktop\alg>
```

CSDN @伏城之外

接下来，我们就可以考虑时间限制T的因素了。

其实T就是起到一个提前终止dp[i]计算的作用。

```
function guohe(n, times, T) {

    times.sort((a, b) => a - b);

    const dp = new Array(n);

    for (let i = 0; i < n; i++) {

        if (i === 0) {

            dp[0] = times[0];

            if (dp[0] > T) return `0 0`; // 如果0~0号士兵过河用时超过T，则说明一个士兵都过不了和，用时为0

            continue;

        }

        // 能走到这一步，说明0~0号士兵是可以过河的

        if (i === 1) {

            dp[1] = times[1];
```

```

        if (dp[1] > T) return `1 ${dp[0]}`; // 如果0~1号士兵过河用时超过T，那么就只能让0
        号士兵过河，用时dp[0]

        continue;

    }

    // 能走到这步，说明0~1号士兵是可以过河的

    if (i >= 2) {

        dp[i] = Math.min(

            dp[i - 1] + times[0] + times[i],

            dp[i - 2] + times[0] + times[i] + times[1] + times[1]

        );

        if (dp[i] > T) return `${i} ${dp[i - 1]}`; // 如果0~i号士兵过河用时超过T，那么就
        是0~i-1号士兵，共计i个士兵过河，用时dp[i-1]

    }

}

```

// 能走到这一步说明，所有士兵都是可以过河的，那么过河士兵数目就是n个，用时dp[n-1]

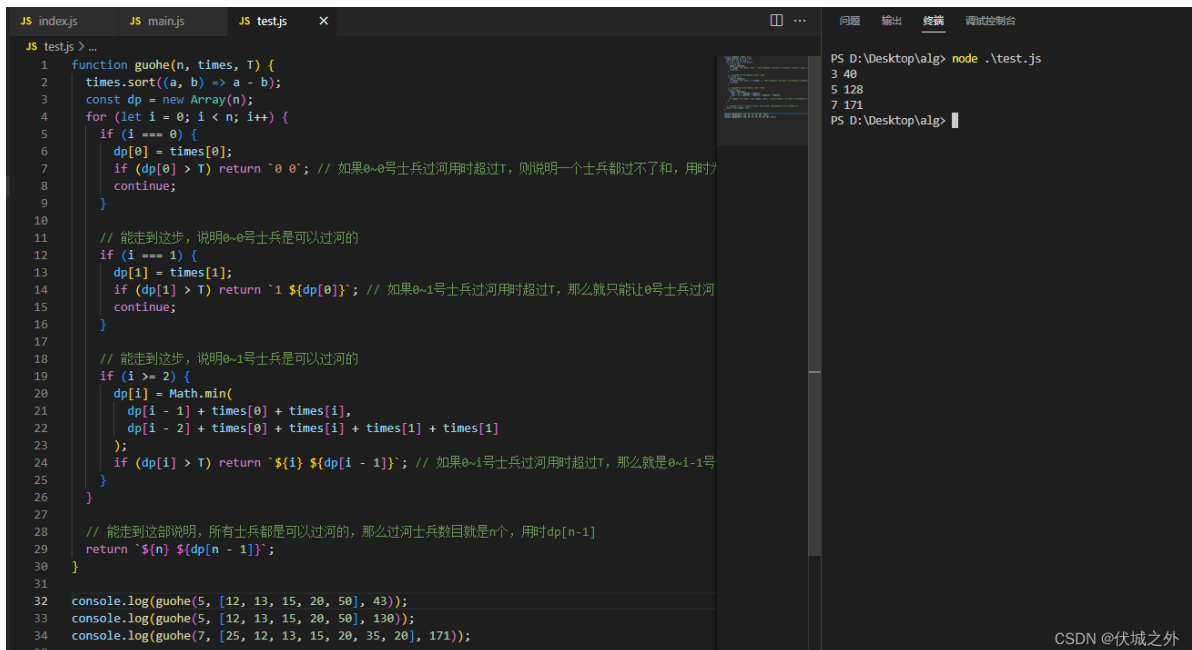
```
return `${n} ${dp[n - 1]}`;
```

```
}
```

```
console.log(guohe(5, [12, 13, 15, 20, 50], 43));
```

```
console.log(guohe(5, [12, 13, 15, 20, 50], 130));
```

```
console.log(guohe(7, [25, 12, 13, 15, 20, 35, 20], 171));
```



```
JS index.js JS main.js JS test.js X
JS test.js > ...
1 function guohe(n, times, T) {
2   times.sort((a, b) => a - b);
3   const dp = new Array(n);
4   for (let i = 0; i < n; i++) {
5     if (i === 0) {
6       dp[0] = times[0];
7       if (dp[0] > T) return '0 0'; // 如果0~0号士兵过河用时超过T，则说明一个士兵都过不了和，用时
8       continue;
9     }
10
11     // 能走到这一步，说明0~0号士兵是可以过河的
12     if (i === 1) {
13       dp[1] = times[1];
14       if (dp[1] > T) return `1 ${dp[0]}`; // 如果0~1号士兵过河用时超过T，那么就只能让0号士兵过河
15       continue;
16     }
17
18     // 能走到这一步，说明0~1号士兵是可以过河的
19     if (i >= 2) {
20       dp[i] = Math.min(
21         dp[i - 1] + times[0] + times[i],
22         dp[i - 2] + times[0] + times[i] + times[1] + times[1]
23       );
24       if (dp[i] > T) return `${i} ${dp[i - 1]}`; // 如果0~i号士兵过河用时超过T，那么就是0~i-1号
25     }
26   }
27
28   // 能走到这一步说明，所有士兵都是可以过河的，那么过河士兵数目就是n个，用时dp[n-1]
29   return `${n} ${dp[n - 1]}`;
30 }
31
32 console.log(guohe(5, [12, 13, 15, 20, 50], 43));
33 console.log(guohe(5, [12, 13, 15, 20, 50], 130));
34 console.log(guohe(7, [25, 12, 13, 15, 20, 35, 20], 171));
35
PS D:\Desktop\alg> node .\test.js
3 40
5 128
7 171
PS D:\Desktop\alg>
```

当然，上面代码只是帮助大家理解，但是形式上存在冗余，我们可以进一步优化一下，请看下面源码

另外，本题还有一个恶心的地方，那就是两个士兵划船用时间问题

1. 当2个士兵坐船同时划船过河时，用时为 $\max(a[j], a[i])$ 两士兵中用时最长的。

2. 当2个士兵坐船1个士兵划船时，用时为 $a[i]*10$ ； $a[i]$ 为划船士兵用时。

其中第2个条件，含义是，如果一个士兵划船奇慢无比，比如两个士兵的划船用时分别是10，200，那么此时如果按照第1个条件的话，则这两个士兵总用时200过河，但是如果按照第2个条件的话，可以只让用时少的士兵划船，只是用时需要乘以10，即这两个士兵总用时 $10*10=100$ 可以过河。

这个逻辑应该是考察我们关于dp状态转移方程中各组成的理解。即如下三个绿色框选部分，需要添加getMax逻辑，具体请看算法源码

$dp[i] = dp[i-1] + times[0] + times[i]$ 的含义是：

抛开最快的A，B士兵，剩余较慢的士兵是奇数个，因此最后较慢士兵中会遗留1个在本岸没有人组队，其余 $0 \sim i-1$ 士兵都已经过河（ $dp[i-1]$ 代表 $0 \sim i-1$ 士兵全部过河），因此我们需要让对岸最快的士兵A，即0号士兵送船回来，此时用时 $times[0]$ ，然后0号士兵和i号士兵一起过河，此时用时 $times[i]$ 。

$dp[i] = dp[i-2] + times[0] + times[i] + times[1] + times[1]$ 的含义是：

抛开最快的A，B士兵，剩余较慢的士兵是偶数个，因此最后较慢士兵会遗留0个士兵在本岸，但是此时无法找到dp状态转移关系，因此我们后退一步，即当较慢士兵会遗留2个士兵在本岸，即 $0 \sim i-2$ 的士兵已经全部过河（ $dp[i-2]$ 代表 $0 \sim i-2$ 的士兵已经全部过河），因此我们需要让对岸最快的士兵A，即0号士兵送船回来，此时用时 $times[0]$ ，然后i-1号士兵和i号士兵划船过河，用时 $times[i]$ ，然后对岸最快的B士兵，即1号士兵送船回来，用时 $times[1]$ ，然后0号士兵和1号士兵再开船到对岸，用时 $times[1]$ 。

为了dp状态转移方程的简单期间，这里我们不去求解：除了最快的A,B士兵，剩下较慢士兵个数是偶数个还是奇数个，而是直接让：

$dp[i] = \text{Math.min}(dp[i-1] + times[0] + times[i], dp[i-2] + times[0] + times[i] + times[1] + times[1])$

即取两种情况的最少用时。

CSDN @伏城之外

JAVA算法源码

```
import java.util.Scanner;
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int t = sc.nextInt();

        int[] times = new int[n];

        for (int i = 0; i < n; i++) {
            times[i] = sc.nextInt();
        }

        System.out.println(getResult(n, t, times));
    }

    public static String getResult(int n, int t, int[] times) {
```



```

Arrays.sort(times);

int[] dp = new int[n];

dp[0] = times[0];
if (dp[0] > t) return "0 0";

dp[1] = getMax(times[0], times[1]);
if (dp[1] > t) return 1 + " " + dp[0];

for (int i = 2; i < n; i++) {
    dp[i] = Math.min(dp[i - 1] + times[0] + getMax(times[0], times[i]),
        dp[i - 2] + times[0] + getMax(times[i - 1], times[i]) +
times[1] + getMax(times[0], times[1]));

    if (dp[i] > t) return i + " " + dp[i - 1];
}

return n + " " + dp[n - 1];
}

public static int getMax(int t1, int t2) {
    if (t1 * 10 < t2) {
        return t1 * 10;
    }
    return t2;
}
}

```

34. 组合出合法最小数

题目描述

给一个数组，数组里面都是代表非负整数的字符串，将数组里所有的数值[排列组合](#)拼接起来组成一个数字，输出拼接成的最小的数字。

输入描述

一个数组，数组不为空，数组里面都是代表非负整数的字符串，可以是0开头，例如：["13", "045", "09", "56"]。

数组的大小范围：[1, 50]

数组中每个元素的长度范围：[1, 30]

输出描述

以字符串的格式输出一个数字，

- 如果最终结果是多位数字，要优先选择输出不是“0”开头的最小数字
- 如果拼接出来的数字都是“0”开头，则选取值最小的，并且把开头部分的“0”都去掉再输出
- 如果是单位数“0”，可以直接输出“0”

用例

输入	20 1
输出	120
说明	无

输入	08 10 2
输出	10082
说明	无

输入	01 02
输出	102
说明	无

题目解析

简单的全排列求解。

在求解全排列前，我们需要对输入的数字字符串数组进行字典序排序，这样所有数字字符串就会从小到大排列，如果不考虑前导0的特殊处理的话，此时拼接出来的数值就是最小的。

但是本题需要考虑前导0，因此按字典序排序后，还要求解全排列，得到全排列后，将拼接字符串分为两类：含前导0的，和不含前导0的。

如果存在不含前导0的，则直接输出第一个。

如果不存在不含前导0的，则输出含前导0的第一个，但是按照题目要求，需要去除前导0，这里可以直接将字符串转为数字即可，比如java的可以用Integer.parseInt，比如JS可以用Number函数或者parseInt。

2023.02.03 经网友指正，本题如果使用[全排列](#)求解的话会超时。原因是本题：数组的大小范围：[1, 50]，如果求解全排列的话，则有50!种排列，这是一个巨大的数量，因此会超时。

一般来说，如果数组中元素长度都相同的话，则可以直接将数组进行字典序升序，这样数组元素按顺序拼接得到的组合数就是最小，比如数组[45, 32, 11, 31] 按照字典序升序后变为 [11, 31, 32, 45]，然后进行拼接，得到的组合数为11313245就是最小的。

但是，如果数组中元素长度不全相同的话，则此时直接字典序升序，可能无法得到最小组合数，比如数组 [3, 32, 321]，按照字典序升序后变为 [3, 32, 321]，然后进行拼接，得到组合数332321，但是这个组合数显然不是最小的，最小的组合数应该是 321323。

而本题数组元素的长度是有可能不一样的，此时我们应该采用另一种排序规则：

即直接尝试对要组合的两个字符串a, b进行组合，此时有两种组合方式：

1. a + b
2. b + a

然后，比较 (a + b) 和 (b + a) 的数值谁小，如果a+b的数值更小，则保持当前a,b顺序不变，如果b+a的数值更小，则交换a,b位置。

比如 strs = [a, b], a = "3", b = "32"

a + b = "332"

b + a = "323"

可以发现 b+a更小，因此交换a,b位置，strs变为[32, 3]，然后进行元素拼接得到 323 最小组合数。

另外，本题，还有其他限定规则：

- 如果最终结果是多位数字，要优先选择输出不是“0”开头的最小数字
- 如果拼接出来的数字都是“0”开头，则选取值最小的，并且把开头部分的“0”都去掉再输出

即排序后，数组开头元素如果以“0”开头，此时分两种情况讨论：

- 数组全部元素都是以“0”开头，则此时应该将拼接后的组合数去除前导0
- 数组有不以“0”开头的元素，则此时不应该把“0”开头的元素放在数组头部。

我的解题思路如下：

首先，按照前面的规则将数组元素排序，排序后，检查数组头元素是否以“0”开头，如果是的话，则开始遍历数组后面的元素，直到找到一个不以“0”开头的元素x，然后将元素x取出，并插入到数组头部。如果一直找不到这样的x，则说明数组元素全部是以0开头的，此时直接拼接出组合数，然后去除前导0。

关于去除前导0，这里不建议使用parseInt，或者python的int，因为对应组合数非常有可能超出int范围，这里我们应该保持组合数为字符串，实现去除前导0。

对于Java,JS而言，可以是正则表达式 `/^0+/` 来替换前导0为空字符串。

对于Python而言，可以使用lstrip方法来去除左边0

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String[] strs = sc.nextLine().split(" ");
        System.out.println(getResult(strs));
    }

    public static String getResult(String[] strs) {
        Arrays.sort(strs, (a, b) -> (a + b).compareTo(b + a));
```

```

if (strs[0].charAt(0) == '0') {
    for (int i = 1; i < strs.length; i++) {
        if (strs[i].charAt(0) != '0') {
            strs[0] = strs[i] + strs[0];
            strs[i] = "";
            break;
        }
    }
}

StringBuilder sb = new StringBuilder();
for (String str : strs) {
    sb.append(str);
}

String ans = sb.toString().replaceAll("^0+", "");

// 避免返回空串
return "".equals(ans) ? "0" : ans;
}
}

```

35. 网上商城优惠活动

题目描述

某网上商场举办优惠活动，发布了满减、打折、无门槛3种[优惠券](#)，分别为：

- 每满100元优惠10元，无使用数限制，如100~199元可以使用1张减10元，200~299可使用2张减20元，以此类推；
- 92折券，1次限使用1张，如100元，则优惠后为92元；
- 无门槛5元优惠券，无使用数限制，直接减5元。

优惠券使用限制

- 每次最多使用2种优惠券，2种优惠可以叠加（优惠叠加时以优惠后的价格计算），以购物200元为例，可以先用92折券优惠到184元，再用1张满减券优惠10元，最终价格是174元，也可以用满减券2张优惠20元为180元，再使用92折券优惠到165（165.6向下取整），不同使用顺序的优惠价格不同，以最优惠价格为准。在一次购物中，同一类型优惠券使用多张时必须一次性使用，不能分多次拆开使用（不允许先使用1张满减券，再用打折券，再使用一张满减券）。

问题

- 请设计实现一种解决方法，帮助购物者以最少的优惠券获得最优的优惠价格。优惠后价格越低越好，同等优惠价格，使用的优惠券越少越好，可以允许某次购物不使用优惠券。

约定

- 优惠活动每人只能参加一次，每个人的优惠券种类和数量是一样的。

输入描述

- 第一行：每个人拥有的优惠券数量（数量取值范围为[0,10]），按满减、打折、无门槛的顺序输入
- 第二行：表示购物的人数n（ $1 \leq n \leq 10000$ ）
- 最后n行：每一行表示某个人优惠前的购物总价格（价格取值范围(0, 1000]，都为整数）。
- 约定：输入都是符合题目设定的要求的。

输出描述

- 每行输出每个人每次购物优惠后的最低价格以及使用的优惠券总数量
- 每行的输出顺序和输入的顺序保持一致

备注

1. 优惠券数量都为整数，取值范围为[0, 10]
2. 购物人数为整数，取值范围为[1, 10000]
3. 优惠券的购物总价为整数，取值范围为 (0, 1000]
4. 优惠后价格如果是小数，则向下取整，输出都为整数。

用例

输入	3 2 5 3 100 200 400
输出	65 6 155 7 338 4
说明	输入：第一行：3种优惠券数量分别为：满减券3张，打折券2张，无门槛5张 第二行：总共3个人购物 第三行：第一个人购物优惠前价格为100元 第四行：第二个人购物优惠前价格为200元 第五行：第三个人购物优惠前价格为400元 输入3个人，输出3行结果，同输入的顺序，对应每个人的优惠结果，如下： 第一行输出：先使用1张满减券优惠到90元，再使用5张无门槛券优惠到25元，最终价格是65元，总共使用6张优惠券。 第二行输出：先使用2张满减券优惠到180元，再使用5张无门槛券优惠到25元，最终价格是155元，总共使用7张优惠券。 第三行输出：先使用1张92折券优惠到368元，再使用3张满减券优惠到30元，最终价格是338元，总共使用4张优惠券。

题目解析

本题和[华为OD机试 - 模拟商场优惠打折 伏城之外的博客-CSDN博客](#)

非常类似，但是关于满减券的使用逻辑不同，导致最终实现也不同。本题和上面链接题目应该属于AB卷题目，防止作弊的。

模拟商场优惠打折，这题关于满减券的使用逻辑：

题目描述

模拟商场优惠打折，有三种优惠券^Q可以用，满减券、打折券和无门槛券。

满减券：满100减10，满200减20，满300减30，满400减40，以此类推不限制使用；

打折券：固定折扣92折，且打折之后向下取整^Q，每次购物只能用1次；

CSDN @伏城之外

即只要符合满减要求，就可以满减，直到满减券用完，比如其用例中

而本题中，满减券使用是有限制的

题目描述

某网上商场举办优惠活动，发布了满减、打折、无门槛3种优惠券，分别为：

- 每满100元优惠10元，无使用数限制，如100~199元可以使用1张减10元，200~299可使用2张减20元，以此类推；
- 92折券，1次限使用1张，如100元，则优惠后为92元；
- 无门槛5元优惠券，无使用数限制，直接减5元。

CSDN @伏城之外

即：满100，最多使用1张减10元的券，满200最多使用2张减10元的券，满300可以使用3张减10元的券....

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
        int n = sc.nextInt();
        int k = sc.nextInt();
        int x = sc.nextInt();

        int[] arr = new int[x];
        for (int i = 0; i < x; i++) {
            arr[i] = sc.nextInt();
        }

        getResult(arr, m, n, k);
    }

    public static void getResult(int[] arr, int m, int n, int k) {
        for (int i = 0; i < arr.length; i++) {
            Integer[][] ans = new Integer[4][2]; // 4的含义对应4种使用券的方式：
            // MN, NM, MK, NK, 2的含义对应每种方式下：剩余总价，剩余券数量
            int price = arr[i];

            int[] resM = M(price, m); // 先满减
```

```

int[] resN = N(price, n); // 先打折

// MN
int[] resMN_N = N(resM[0], n); // 满减后打折
ans[0] =
    new Integer[] {
        resMN_N[0], m + n - resM[1] - resMN_N[1]
    }; // resMN_N[0]是“满减后打折”的剩余总价， m + n - resM[1] - resMN_N[1]
是 该种用券方式的：总券数 m+n， 剩余券数
// resM[1] + resMN_N[1]， 因此使用掉的券数： m+n - (resM[1] + resMN_N[1])

// NM
int[] resNM_M = M(resN[0], m); // 打折后满减
ans[1] = new Integer[] {resNM_M[0], n + m - resN[1] - resNM_M[1]};

// MK
int[] resMK_K = K(resM[0], k); // 满减后无门槛
ans[2] = new Integer[] {resMK_K[0], m + k - resM[1] - resMK_K[1]};

// NK
int[] resNK_K = K(resN[0], k); // 打折后无门槛
ans[3] = new Integer[] {resNK_K[0], n + k - resN[1] - resNK_K[1]};

Arrays.sort(
    ans,
    (a, b) ->
        a[0].equals(b[0])
        ? a[1] - b[1]
        : a[0] - b[0]); // 对ans进行排序，排序规则是：优先按剩余总价升序，如果
剩余总价相同，则再按“使用掉的券数量”升序
System.out.println(ans[0][0] + " " + ans[0][1]);
}
}

/**
 * @param price 总价
 * @param m 满减券数量
 * @return 总价满减后结果，对应数组含义是 [用券后剩余总价， 剩余满减券数量]
 */
public static int[] M(int price, int m) {
    int maxCount = price / 100; // 满100最多用1张满减券，满200最多用2张满减券....，
price总价最多使用price/100张券
    int count = Math.min(maxCount, m); // 实际可使用的满减券数量

    price -= count * 10; // 每张满减券只能减10元
    m -= count;

    return new int[] {price, m};
}

/**
 * @param price 总价
 * @param n 打折券数量
 * @return 总价打折后结果，对应数组含义是 [用券后剩余总价， 剩余打折券数量]
 */

```

```

public static int[] N(int price, int n) {
    if (n >= 1) {
        price = (int) Math.floor((price * 0.92));
    }
    return new int[] {price, n - 1};
}

/**
 * @param price 总价
 * @param k 无门槛券数量
 * @return 无门槛券后结果，对应数组含义是 [用券后剩余总价, 剩余无门槛券数量]
 */
public static int[] K(int price, int k) {
    while (price > 0 && k > 0) {
        price -= 5;
        price = Math.max(price, 0); // 感谢m0_71826536提供的思路，当无门槛券过多时，是有可能导致优惠后总价低于0的情况的，此时我们应该避免总价小于0的情况
        k--;
    }
    return new int[] {price, k};
}
}

```

36. 羊、狼、农夫过河

题目描述

羊、狼、农夫都在岸边，当羊的数量小于狼的数量时，狼会攻击羊，农夫则会损失羊。农夫有一艘容量固定的船，能够承载固定数量的动物。

要求求出不损失羊情况下将全部羊和狼运到对岸需要的最小次数。

只计算农夫去对岸的次数，回程时农夫不会运送羊和狼。

备注：农夫在或农夫离开后羊的数量大于狼的数量时狼不会攻击羊。

输入描述

第一行输入为M，N，X， 分别代表羊的数量，狼的数量，小船的容量。

输出描述

输出不损失羊情况下将全部羊和狼运到对岸需要的最小次数（若无法满足条件则输出0）。

用例

输入	5 3 3
输出	3

输入	5 3 3
说明	第一次运2只狼第二次运3只羊第三次运2只羊和1只狼

输入	5 4 1
输出	0
说明	如果找不到不损失羊的运送方案，输出0

题目解析

本题求不损失羊的前提下，将羊和狼全部运到对岸的最小次数。

首先，要搞清楚，如何保证不损失羊？

农夫在或农夫离开后羊的数量大于狼的数量时狼不会攻击羊。

这里有个文字断句陷阱，到底是这样断句

- 农夫在时，狼不会攻击羊
- 农夫离开后羊的数量大于狼的数量时，狼不会攻击羊

还是这样断句

- 农夫在，羊的数量大于狼的数量时，狼不会攻击羊
- 农夫离开后，羊的数量大于狼的数量时，狼不会攻击羊

经过一位网友的真实机考反馈，上面画删除线的断句理解是错误的。

那么“农夫在时，狼不会攻击羊”，这句话到底会有什么影响呢？

只计算农夫去对岸的次数，回程时农夫不会运送羊和狼。

通过上面这句话，我们可以理解，农夫回程不会带动物。因此可以认定：

- 农夫如果在本岸带走的动物后，如果本岸羊 $\leq$ 狼了，在农夫离开后，羊就会被攻击
- 农夫如果在对岸离开后，对岸的羊 $\leq$ 狼，羊就会被攻击

因此，“农夫在时，狼不会攻击羊”，这句话只会影响：船上，羊和狼的关系，即农夫在船上时，如果羊数量 $\leq$ 狼数量，此时因为农夫在，因此狼不会攻击羊。

本题没有什么好的解题思路，只能通过暴力枚举所有羊、狼数量情况，只需要满足下面三个条件：

- 农夫离开后，本岸羊 $>$ 本岸狼
- 农夫离开后，对岸羊 $>$ 对岸狼
- 船上由于农夫始终在，因此羊、狼什么数量都可以，只要不超过船载量

Java算法源码

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Scanner;
```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
        int n = sc.nextInt();
        int x = sc.nextInt();

        System.out.println(getResult(m, n, x));
    }

    /**
     * @param sheep 本岸羊数量
     * @param wolf 本岸狼数量
     * @param boat 船负载
     * @return 最少运送次数
     */
    public static int getResult(int sheep, int wolf, int boat) {
        ArrayList<Integer> ans = new ArrayList<>();
        dfs(sheep, wolf, boat, 0, 0, 0, ans);

        if (ans.size() > 0) {
            return Collections.min(ans);
        } else {
            return 0;
        }
    }

    public static void dfs(
        int sheep,
        int wolf,
        int boat,
        int oppo_sheep,
        int oppo_wolf,
        int count,
        ArrayList<Integer> ans) {
        if (sheep == 0 && wolf == 0) {
            ans.add(count);
            return;
        }

        if (sheep + wolf <= boat) {
            ans.add(count + 1);
            return;
        }

        // i 代表船上羊数量, 最多Math.min(boat, sheep)
        for (int i = 0; i <= Math.min(boat, sheep); i++) {
            // j 代表船上狼数量, 最多Math.min(boat, wolf)
            for (int j = 0; j <= Math.min(boat, wolf); j++) {
                // 空运
                if (i + j == 0) continue;
                // 超载
                if (i + j > boat) break;
            }
        }
    }
}

```

```
// 本岸羊 <= 本岸狼, 说明狼运少了
if (sheep - i <= wolf - j && sheep - i != 0) continue;

// 对岸羊 <= 对岸狼, 说明狼运多了
if (oppo_sheep + i <= oppo_wolf + j && oppo_sheep + i != 0) break;

// 对岸没羊, 但是对岸狼已经超过船载量, 即下次即使整船都运羊, 也无法保证对岸羊 > 对岸狼
if (oppo_sheep + i == 0 && oppo_wolf + j >= boat) break;

dfs(sheep - i, wolf - j, boat, oppo_sheep + i, oppo_wolf + j, count + 1,
ans);
    }
    }
}
```

37. 获取最大软件版本号

题目描述

Maven 版本号定义, <主版本>.<次版本>.<增量版本>-<里程碑版本>, 举例3.1.4-beta

其中, 主版本和次版本都是必须的, 主版本, 次版本, 增量版本由多位数字组成, 可能包含前导零, 里程碑版本由字符串组成。

<主版本>.<次版本>.<增量版本>: 基于**数字**比较;

里程碑版本: 基于**字符串**比较, 采用**字典序**;

比较版本号时, 按从左到右的顺序依次比较。

基于数字比较, 只需比较**忽略任何前导零后的整数值**。

输入2个版本号, 输出**最大版本号**。

输入描述

输入2个版本号, 换行分割, 每个版本的最大长度小于50

输出描述

版本号相同时输出第一个输入版本号

用例

输入	2.5.1-C 1.4.2-D
输出	2.5.1-C
说明	无

输入	1.05.1 1.5.01
输出	1.05.1
说明	版本号相同，输出第一个版本号

输入	1.5 1.5.0
输出	1.5.0
说明	主次相同，存在增量版本大于不存在

输入	1.5.1-A 1.5.1-a
输出	1.5.1-a
说明	里程碑版本号，基于字典序比较，a 大于 A

题目解析

[简单排序](#)题，考察数组排序

本题和[华为机试 - 比较两个版本号的大小](#) [伏城之外的博客-CSDN博客](#)

类似，做完本题，可以再试试上面这个真题

2022.12.20 补充

这题看走眼了。

题目中说：“主版本和次版本都是必须的”，也就说后面几个版本可能是没有的，因此如下正则表达式：`const regExp = /^(\d+).(\d+).(\d+)-(.+)$/;`

获取缺失了增量版本和里程碑版本的字符串时，是存在问题的。

这里更新了下正则

```
const reg = /^(\d+)(.(\d+))(.(\d+))?(-.+)?$/;
```

在增量版本捕获组后面加了？，以及在里程碑版本后面加了？，表示这两个版本可以没有。

同时为了直到缺失的增量版本，还是里程碑版本，我们需要将"."和 "-" 加入捕获组中，来帮助我们辨别。

最终效果如下：

```

> const reg = /^(\d+)(\.(?!\d+))?(?!\d+)?(\-?.+)?$/;
< undefined

> reg.exec('1.5')
< (7) ['1.5', '1', '.5', '5', undefined, undefined, undefined, index: 0, input: '1.5', groups: undefined]

> reg.exec('1.5.0')
< (7) ['1.5.0', '1', '.5', '5', '.0', '0', undefined, index: 0, input: '1.5.0', groups: undefined]

> reg.exec('1.5-a')
< (7) ['1.5-a', '1', '.5', '5', undefined, undefined, '-a', index: 0, input: '1.5-a', groups: undefined]

```

CSDN @伏城之外

需要注意的是捕获组的顺序就是正则中括号从左到右的顺序，比如

```

> const reg = /^(\d+)(\.(?!\d+))(\.(?!\d+))?(?!\d+)?$/;
< undefined

> reg.exec('1.5.5-A')
< (7) ['1.5.5-A', '1', '.5', '5', '.5', '5', '-A', index: 0, input: '1.5.5-A', groups: undefined]

```

CSDN @伏城之外

字符串操作解法

Java算法源码

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String s1 = sc.next();
        String s2 = sc.next();

        System.out.println(getResult(s1, s2));
    }

    public static String getResult(String s1, String s2) {
        String[] v1 = getVersion(s1);
        String[] v2 = getVersion(s2);

        /*
         * 主要版本比较
         */
        int major1 = Integer.parseInt(v1[0]), major2 = Integer.parseInt(v2[0]);
        if (major1 > major2) return s1;
        else if (major1 < major2) return s2;

        /*
         * 次要版本比较
         */
    }
}

```

```

int minor1 = Integer.parseInt(v1[1]), minor2 = Integer.parseInt(v2[1]);
if (minor1 > minor2) return s1;
else if (minor1 < minor2) return s2;

/*
 * 增量版本比较
 * */
String patch1 = v1[2], patch2 = v2[2];
if (!"".equals(patch1) || !"".equals(patch2)) {
    if ("".equals(patch2)) return s1;
    else if ("".equals(patch1)) return s2; // 走到这步, s2的增量版本确定不是空串, 则
    如果只有s1的增量版本是空串, 则返回s2

    // 走到此步, 说明s1,s2的增量版本都不是空串, 此当成数字进行比较
    int v1_patch = Integer.parseInt(patch1);
    int v2_patch = Integer.parseInt(patch2);
    if (v1_patch > v2_patch) return s1;
    else if (v1_patch < v2_patch) return s2;
}

/*
 * 里程碑版本比较
 * */
String mile1 = v1[3], mile2 = v2[3];
// s1,s2的里程碑的空串判断, 同增量版本
if ("".equals(mile2)) {
    return s1;
} else if ("".equals(mile1)) {
    return s2;
}

// 如果s1, s2的里程碑版本都不是空串, 则按照字典序比较
return mile1.compareTo(mile2) >= 0 ? s1 : s2;
}

public static String[] getVersion(String s) {
    String major = "";
    String minor = "";
    String patch = "";
    String mile = "";

    // 找到第一个 "-", 先把里程碑解析出来, 因为里程碑中可能含有 "-" 和 ".", 因此必须先处理掉
    int i = s.indexOf("-");

    String major_minor_patch = "";
    if (i != -1) {
        mile = s.substring(i + 1);
        major_minor_patch = s.substring(0, i);
    } else {
        major_minor_patch = s;
    }

    // 之后再处理非里程碑部分
    String[] split = major_minor_patch.split("\\.");
    major = split[0];

```

```

        minor = split[1];

        if (split.length > 2) {
            patch = split[2];
        }

        return new String[] {major, minor, patch, mile};
    }
}

```

正则解法

Java算法源码

```

import java.util.*;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str1 = sc.next();
        String str2 = sc.next();

        System.out.println(getResult(new String[]{str1, str2}));
    }

    public static String getResult(String[] versions) {
        String reg = "^((\\d+)(\\.((\\d+))\\.((\\d+)))?(\\-\\.+)?)$";
        Pattern p = Pattern.compile(reg);

        Arrays.sort(versions, (v1, v2) -> {
            Matcher m1 = p.matcher(v1);
            Matcher m2 = p.matcher(v2);

            if (m1.find() && m2.find()) {
                Integer major1 = Integer.parseInt(m1.group(1));
                Integer major2 = Integer.parseInt(m2.group(1));

                if (major1 != major2) return major2 - major1;

                Integer minor1 = Integer.parseInt(m1.group(3));
                Integer minor2 = Integer.parseInt(m2.group(3));

                if (minor1 != minor2) return minor2 - minor1;

                String patch1 = m1.group(5);
                String patch2 = m2.group(5);

                if (patch1 != null && patch2 != null) {
                    int patch1_intVal = Integer.parseInt(patch1);
                    int patch2_intVal = Integer.parseInt(patch2);
                    if (patch1_intVal != patch2_intVal) {

```

```

        return Integer.parseInt(patch2) -
Integer.parseInt(patch1);
    }
    } else if(patch1 != null) {
        return -1;
    } else if(patch2 != null) {
        return 1;
    }

    String mile1 = m1.group(6);
    String mile2 = m2.group(6);

    if(mile1 != null && mile2 != null) {
        return mile2.compareTo(mile1);
    } else if(mile1 != null) {
        return -1;
    } else if(mile2 != null) {
        return 1;
    } else {
        return 0;
    }
}

return 0;
});

return versions[0];
}
}

```

38.单向链表中间节点

题目描述

求[单向链表](#)中间的节点值，如果奇数个节点取中间，偶数个取偏右边的那个值。

输入描述

第一行 链表头节点地址 后续输入的节点数n

后续输入每行表示一个节点，格式 节点地址 节点值 下一个节点地址(-1表示[空指针](#))

输入保证链表不会出现环，并且可能存在一些节点不属于链表。

输出描述

单向链表中间的节点值

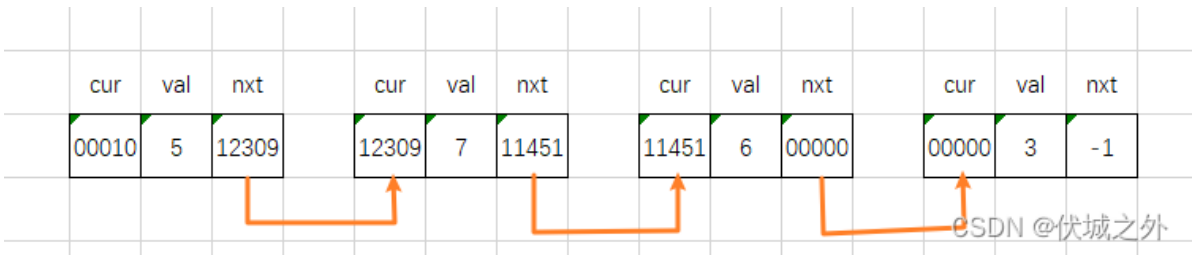
用例

输入	00010 4 00000 3 -1 00010 5 12309 11451 6 00000 12309 7 11451
输出	6
说明	无

输入	10000 3 76892 7 12309 12309 5 -1 10000 1 76892
输出	7
说明	无

题目解析

用例1示意图如下



JS本题可以利用数组模拟链表

基于链表数据结构解题

Java算法源码

需要注意的是Java中LinkedList类的get(index)方法的时间复杂度不是O(1)，而是O(n)，这题建议使用ArrayList代替

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String head = sc.next();
        int n = sc.nextInt();

        HashMap<String, String[]> nodes = new HashMap<>();
```

```

        for (int i = 0; i < n; i++) {
            String addr = sc.next();
            String val = sc.next();
            String nextAddr = sc.next();
            nodes.put(addr, new String[] {val, nextAddr});
        }

        System.out.println(getResult(head, nodes));
    }

    public static String getResult(String head, HashMap<String, String[]> nodes) {
        //    LinkedList<String> link = new LinkedList<>();
        ArrayList<String> link = new ArrayList<>();

        String[] node = nodes.get(head);
        while (node != null) {
            String val = node[0];
            String next = node[1];

            link.add(val);
            node = nodes.get(next);
        }

        int len = link.size();
        int mid = len / 2;
        return link.get(mid);
    }
}

```

快慢指针解题

链表数据结构本质上来说没有索引概念，因为其在内存上不是一段连续的内存，因此索引对于链表结构而言没有意义。

但是从使用上来说，我又经常需要去获取链表结构的第几个元素，因此大部分语言都为链表结构提高了“假索引”，比如Java的LinkedList类，虽然提高了get(index)方法，但是其底层是通过[遍历链表](#)（通过next属性找到下一个节点）来找到对应“假索引”的元素的，即LinkedList每次都需要O(n)的时间复杂度才能找到index位置上的元素。

另外，链表还有一个常考问题，那就是链表长度未知的情况下，我们如何找到链表的中间节点？

此时，就要用到快慢指针。

所谓快慢指针，即通过两个指针遍历链表，慢指针每次步进1个节点，快指针每次步进2个节点，这样快指针必然先到达链表尾部，而当快指针到达链表尾部时，慢指针其实刚好就是在链表中间节点的位置（奇数个节点取中间，偶数个取偏右边的那个值）。

本题虽然给出了节点数，但是这些节点不一定属于同一个链表结构，因此本题的链表长度也是未知的，而本题要求的链表中间节点要求刚好和快慢指针找的中间节点吻合，因此本题最佳策略是使用快慢指针。

Java算法源码

```
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String head = sc.next();
        int n = sc.nextInt();

        HashMap<String, String[]> nodes = new HashMap<>();
        for (int i = 0; i < n; i++) {
            String addr = sc.next();
            String val = sc.next();
            String nextAddr = sc.next();
            nodes.put(addr, new String[] {val, nextAddr});
        }

        System.out.println(getResult(head, nodes));
    }

    public static String getResult(String head, HashMap<String, String[]> nodes) {
        String[] slow = nodes.get(head);
        String[] fast = nodes.get(slow[1]);

        while (fast != null) {
            slow = nodes.get(slow[1]);

            fast = nodes.get(fast[1]);
            if (fast != null) {
                fast = nodes.get(fast[1]);
            } else {
                break;
            }
        }

        return slow[0];
    }
}
```

39.新学校选址

题目描述

为了解新学期学生暴涨的问题,小乐村要建立所新学校,
考虑到学生上学安全问题,需要所有学生家到学校的距离最短。
假设学校和所有学生家都走在一条直线之上,请问学校建立在什么位置,
能使得学校到各个学生家的距离和最短。

输入描述

第一行: 整数 n 取值范围 [1 ,1000],表示有 n户家庭。
第二行: 一组整数 m 取值范围 [0, 10000] , 表示每户家庭的位置, 所有家庭的位置都不相同。

输出描述

一个整数, 确定的学校的位置。
如果有多个位置, 则输出最小的。

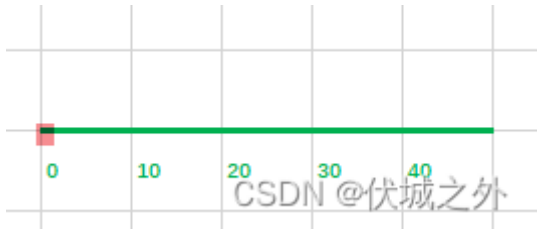
用例

输入	5 0 20 40 10 30
输出	20
说明	20到各个家庭的距离分别为20 0 20 10 10, 总和为60, 最小

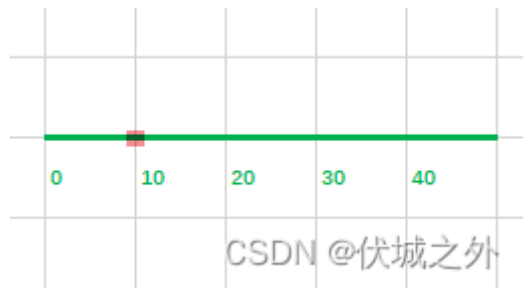
输入	1 20
输出	20
说明	只有一组数据, 20到20距离最小, 为0

输入	2 0 20
输出	0
说明	有多个地方可选, 但是0数值最小

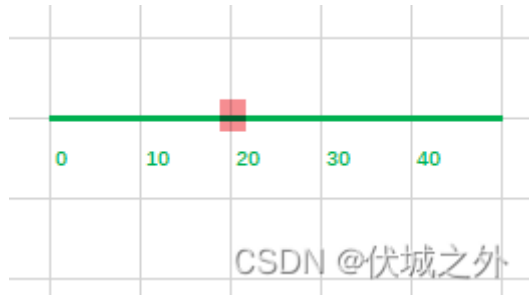
题目解析



$$0 + 10 + 20 + 30 + 40 = 100$$



$$10 + 0 + 10 + 20 + 30 = 70$$



$$20 + 10 + 0 + 10 + 20 = 50$$

将学校建在30，40点上，其实和建在10，0点上相同，此处不再赘述。

因此，我们发现，将学校建在所有学生家位置的共同中心点位置的距离最短。

本题其实就是[中位数](#)定理。

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.println(getResult(arr));
    }

    public static int getResult(int[] arr) {
        Arrays.sort(arr);

        int len = arr.length;
        if (len % 2 == 0) {
            int mid = len / 2;
            return arr[mid - 1];
        }
    }
}
```

```
    } else {
        return arr[len / 2];
    }
}
}
```

40.通信误码

题目描述

信号传播过程中会出现一些误码，不同的数字表示不同的误码ID，取值范围为1~65535，用一个数组记录误码出现的情况，
每个误码出现的次数代表误码频度，请找出记录中包含频度最高误码的最小子数组长度。

输入描述

误码总数目：取值范围为0~255，取值为0表示没有误码的情况。
误码出现频率数组：误码ID范围为1~65535，数组长度为1~1000。

输出描述

包含频率最高的误码最小子数组长度

用例

输入	5 1 2 2 4 1
输出	2
说明	频度最高的有1和2，频度是2（出现的次数都是2）。可以包含频度最高的记录数组是[2 2]和[1 2 2 4 1]，最短是[2 2]，最小长度为2。

输入	7 1 2 2 4 2 1 1
输出	4
说明	频度最高的是1和2，最短的是[2 2 4 2]

题目解析

简单的排序题。

首先，我们统计出误码数组各个误码的出现过的索引值，假设计到idxs对象中，属性是误码，属性值是数组，记录误码出现过的索引位置。

然后将idxs对象的所有属性值（各个误码出现过的索引位置数组）拎出来，即Object.values，然后对这些索引位置数组，进行排序，先按照索引位置数组长度进行排序，长度越长，说明频率越高，排序越靠前，如果两个数组长度相同，则看索引跨度，即索引数组的头元素索引和尾元素索引的差距，差距越小，越靠前。这样排序后，得到的首元素数组的首尾索引跨度就是题解。

Java算法源码

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Scanner;
import java.util.stream.Collectors;

public class Main {
    static ArrayList<Integer[]> nodes;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.println(getResult(arr));
    }

    /**
     * @param arr 误码出现频率数组
     * @return 包含频率最高的误码最小子数组长度
     */
    public static int getResult(int[] arr) {
        HashMap<Integer, ArrayList<Integer>> idxs = new HashMap<>();

        for (int i = 0; i < arr.length; i++) {
            Integer code = arr[i];
            idxs.putIfAbsent(code, new ArrayList<>());
            idxs.get(code).add(i);
        }

        ArrayList<Integer> countList =
            idxs.values().stream()
                .sorted(
                    (a, b) -> {
                        if (a.size() != b.size()) {
                            return b.size() - a.size();
                        } else {
                            int alen = a.get(a.size() - 1) - a.get(0);
                            int blen = b.get(b.size() - 1) - b.get(0);
                            return alen - blen;
                        }
                    }
                )
                .limit(1)
                .collect(Collectors.toList())
    }
}
```

```
        .get(0);

        return countList.get(countList.size() - 1) - countList.get(0) + 1;
    }
}
```

41. 任务总执行时长

题目描述

任务编排服务负责对任务进行组合调度。

参与编排的任务有两种类型，其中一种执行时长为taskA，另一种执行时长为taskB。

任务一旦开始执行不能被打断，且任务可连续执行。

服务每次可以编排num个任务。

请编写一个方法，生成每次编排后的任务所有可能的总执行时长。

输入描述

第1行输入分别为第1种任务执行时长taskA，

第2种任务执行时长taskB，

这次要编排的任务个数num，以逗号分隔。

注：每种任务的数量都大于本次可以编排的任务数量

- $0 < \text{taskA}$
- $0 < \text{taskB}$
- $0 \leq \text{num} \leq 100000$

输出描述

数组形式返回所有总执行时时长，需要按从小到大排列。

用例

输入	1,2,3
输出	[3, 4, 5, 6]
说明	无

题目解析

本题看上去是求解[全排列](#)，但是实际上无论如何排列，其实就只是两种类型任务的排列，而且本题要求任务编排的执行总时长，这其实就是就每种排列的和，因此其实我们不需要求解排列，只需要求解该排列对应的组合即可

比如用例中，有三个任务，那么有如下组合：

- 三个1
- 两个1，一个2
- 一个1，两个2
- 三个2

无论每个组合，能编排成几个排列，其实执行总时长，即排列的和都是一样的

- 三个1：和为3
- 两个1，一个2：和为4
- 一个1，两个2：和为5
- 三个2：和为6

因此，本题其实就是求解两种类型任务各自可能的数量。

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;
import java.util.TreeSet;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.next();
        Integer[] arr =
Arrays.stream(str.split(",")).map(Integer::parseInt).toArray(Integer[]::new);

        System.out.println(getResult(arr[0], arr[1], arr[2]));
    }

    public static String getResult(int taskA, int taskB, int num) {
        if (num == 0) return "[]";

        if (taskA == taskB) {
            return Arrays.toString(new int[] {taskA * num});
        }

        TreeSet<Integer> ans = new TreeSet<>();
        for (int i = 0; i <= num; i++) {
            ans.add(taskA * i + taskB * (num - i));
        }

        return ans.toString();
    }
}
```

42.区块链文件转储系统

题目描述

[区块链](#)底层存储是一个链式文件系统，由顺序的N个文件组成，每个文件的大小不一，依次为 $F_1, F_2, \dots, F_n$ 。随着时间的推移，所占存储会越来越大。

[云平台](#)考虑将区块链按文件转储到廉价的SATA盘，只有连续的区块链文件才能转储到SATA盘上，且转储的文件之和不能超过SATA盘的容量。

假设每块[SATA](#)盘容量为M，求能转储的最大连续文件之和。

输入描述

第一行为SATA盘容量M， $1000 \leq M \leq 1000000$

第二行为区块链文件大小序列 $F_1, F_2, \dots, F_n$ 。其中 $1 \leq n \leq 100000$ ， $1 \leq F_i \leq 500$

输出描述

求能转储的最大连续文件大小之和

用例

输入	1000 100 300 500 400 400 150 100
输出	950
说明	最大序列和为950，序列为[400,400,150]

输入	1000 100 500 400 150 500 100
输出	1000
说明	最大序列和为1000，序列为[100,500,400]

题目解析

由于本题需要求解最大连续文件大小之和，因此可以考虑使用双指针+滑动窗口来解题。

本题的滑动窗口的左边界l,右边界r的运动逻辑如下：

- 如果滑动窗口内部和 < m，则r++
- 如果滑动窗口内部和 > m，则l++
- 如果滑动窗口内部和 = m，则已得到最大值，直接返回m即可。

在计算滑动窗口内部和的过程中，如果r++，则说明内部和可能会增大产生最大值，因此我们需要在r++时，判断并保留最大值。

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = Integer.parseInt(sc.nextLine());
        Integer[] f =
            Arrays.stream(sc.nextLine().split("
")).map(Integer::parseInt).toArray(Integer[]::new);

        System.out.println(getResult(m, f));
    }

    public static int getResult(int m, Integer[] f) {
        int l = 0, r = 0;
        int sum = 0, max = 0;

        int n = f.length;

        while (r < n) {
            // 尝试右指针右移一下的新和（注意初始时右指针右移后指向0）
            int newSum = sum + f[r];

            // 如果新和超过了m，即SATA盘容量，则右指针不能右移，并且还需要左指针右移来减少旧和
            if (newSum > m) {
                sum -= f[l++]; // 左指针右移只会减少旧和，因此不会产生最大值
            }
            // 如果新和小于m，则当前尝试的右指针右移可行，因此 sum += f[r]，并且我们下一步还可以继续尝试让右指针右移，即r++
            else if (newSum < m) {
                sum += f[r++];
                max = Math.max(sum, max); // 右指针右移时会增加旧和，因此可能会产生最大值
            }
            // 如果新和等于m，那么说明已经找到了一个容量和SATA盘相同的连续文件大小，即此时已经是最大值了，可以直接返回
            else {
                return m;
            }
        }

        return max;
    }
}
```

43.最快到达医院的方法

题目描述

新型冠状病毒疫情的肆虐，使得家在武汉的大壮不得不思考自己家和附近定点医院的具体情况。

经过一番调查，大壮明白了距离自己家最近的定点医院有两家。其中：

- 医院A和自己的距离是X公里
- 医院B和自己的距离是Y公里

由于武汉封城，公交停运，私家车不能上路，交通十分不便。现在：

- 到达医院A只能搭乘志愿者计程车，已知计程车的平均速度是M米/分钟，上车平均等待时间为L分钟。
- 到达医院B只能步行，平均速度是N米/分钟；

给出X, Y, M, L, N的数据，请问大壮到达哪家医院最快？

输入描述

一行，5个数。

分别是到达A医院的距离，到达B医院的距离，计程车平均速度，上车等待时间，步行速度。

输出描述

一行，计程车（Taxi）、步行（Walk）、相等（Same）

用例

输入	50 5 500 30 90
输出	Walk
说明	无

题目解析

本题简单的有点过分。

根据题目意思，去A只能Taxi，去B只能Walk，而对应的距离和速度都给出来了，因此解题逻辑应该没有什么悬念。

这题难道在考察细心程度？就是距离单位是公里，速度单位是米/分钟？

Java算法源码

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double x = sc.nextDouble();
        double y = sc.nextDouble();
        double m = sc.nextDouble();
        double l = sc.nextDouble();
        double n = sc.nextDouble();

        System.out.println(getResult(x, y, m, l, n));
    }

    /**
     * @param x 到达A医院的距离（公里）
     * @param y 到达B医院的距离（公里）
     * @param m 计程车平均速度（米/分钟）
     * @param l 上车等待时间（分钟）
     * @param n 步行速度（米/分钟）
     * @return 请问大壮到达哪家医院最快
     */
    public static String getResult(double x, double y, double m, double l, double
n) {
        double taxiToA = x * 1000 / m + l;
        double walkToB = y * 1000 / n;

        if (taxiToA == walkToB) {
            return "Same";
        } else if (taxiToA > walkToB) {
            return "Walk";
        } else {
            return "Taxi";
        }
    }
}
```

44. 回文字符串

题目描述

如果一个字符串正读和反读都一样（大小写敏感），则称它为一个「[回文串](#)」，例如：

- leVel是一个「回文串」，因为它的正读和反读都是leVel；同理a也是「回文串」
- art不是一个「回文串」，因为它的反读tra与正读不同
- Level不是一个「回文串」，因为它的反读level与正读不同（因大小写敏感）

给你一个仅包含大小写字母的字符串，请用这些字母构造出一个最长的回文串，若有多个最长的，返回其中[字典序](#)最小的回文串。

字符串中的每个位置的字母最多备用一次，也可以不用。

输入描述

无

输出描述

无

用例

输入	abczcccddzz
输出	ccdazdcc
说明	无

输入	ABabBabA
输出	ABabbaBA
说明	无

题目解析

回文串必然是对称的，可以分为三部分，即： 左边部分， 中间部分， 右边部分

其中左边部分 和 右边部分 互为倒序

而中间部分 可以是空串，可以是单字母。

比如 [aba](#)，其中左边部分是a，右边部分也是a，而中间部分是b

再比如 aa，其中左边部分是a，右边部分也是a，而中间部分是""

因此，我的解题思路如下：

统计输入字符串各字母出现的次数：

- 如果字母出现次数 ≥ 2 ：
 1. 字母出现次数为偶数，则可以平均分到左边部分，和右边部分
 2. 字母出现次数为奇数，则平均分到左，右部分后，必然还会剩余一个无法成对
- 如果字母出现次数 $= 1$ ，则无法成对

对于剩余无法成对的字母，我们记录字典序最小的到mid中（题目要求返回其中字典序最小的回文串）

对于可以平均分配到左，右部分的字母，我们可以将可以成对的字母记录到ans数组，将ans字典序升序，拼接字符串即为回文串左边部分，ans字典序降序，拼接字符串即为回文串右边部分。

最终拼接：回文串左边部分 + 中间部分 + 回文串右边部分，即为题解。

Java算法源码

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.next();
        System.out.println(getResult(str));
    }

    private static String getResult(String str) {
        HashMap<Character, Integer> count = new HashMap<>();

        // 统计各字母个数
        for (int i = 0; i < str.length(); i++) {
            char c = str.charAt(i);
            count.put(c, count.getDefault(c, 0) + 1);
        }

        ArrayList<Character> half = new ArrayList<>();
        String mid = "";

        for (char c : count.keySet()) {
            int n = count.get(c);
            // 如果字母数量大于等于2，则可以成对出现，
            if (n >= 2) {
                for (int i = 0; i < n / 2; i++) half.add(c);
            }

            // 如果字母数量只有1个，或者字母数量大于2但是为奇数，则最后必然只剩单个字母可用，此时我们
            // 应该在无法成对的单字母中选择一个字典序最小的
            if (n % 2 != 0 && ("".equals(mid) || mid.compareTo(c + "") > 0)) {
                mid = c + "";
            }
        }

        half.sort((a, b) -> a - b);
        StringBuilder sb = new StringBuilder();
        for (Character c : half) sb.append(c);

        return sb + mid + sb.reverse(); // 回文串左边部分 + 中间单字母 + 回文串右边部分
    }
}
```

45. 计算网络信号、信号强度

题目描述

网络信号经过传递会逐层衰减，且遇到阻隔物无法直接穿透，在此情况下需要计算某个位置的网络信号值。

注意:网络信号可以绕过阻隔物。

array[m][n] 的二维数组代表网格地图，

array[i][j] = 0代表i行j列是空旷位置;

array[i][j] = x(x为正整数)代表i行j列是信号源，信号强度是x;

array[i][j] = -1代表i行j列是阻隔物。

信号源只有1个，阻隔物可能有0个或多个

网络信号衰减是上下左右相邻的网格衰减1

现要求输出**对应位置的网络信号值**。

输入描述

输入为三行，第一行为 m、n，代表输入是一个 $m \times n$ 的数组。

第二行是一串 $m \times n$ 个用空格分隔的整数。

每连续 n 个数代表一行，再往后 n 个代表下一行，以此类推。

对应的值代表对应的网格是空旷位置，还是信号源，还是阻隔物。

第三行是 i、j，代表需要计算array[i][j]的网络信号值。

注意：此处 i 和 j 均从 0 开始，即第一行 i 为 0。

例如

6 5

0 0 0 -1 0 0 0 0 0 0 0 -1 4 0 0 0 0 0 0 0 0 -1 0 0 0 0 0

1 4

代表如下地图

0	0	0	-1	0
0	0	0	0	0
0	0	-1	4	0
0	0	0	0	0
0	0	0	0	-1
0	0	0	0	0

需要输出第1行第4列的网络信号值，如下图，值为2。

输出描述

输出对应位置的网络信号值，如果网络信号未覆盖到，也输出0。
一个网格如果可以途径不同的传播衰减路径传达，取较大的值作为其信号值。

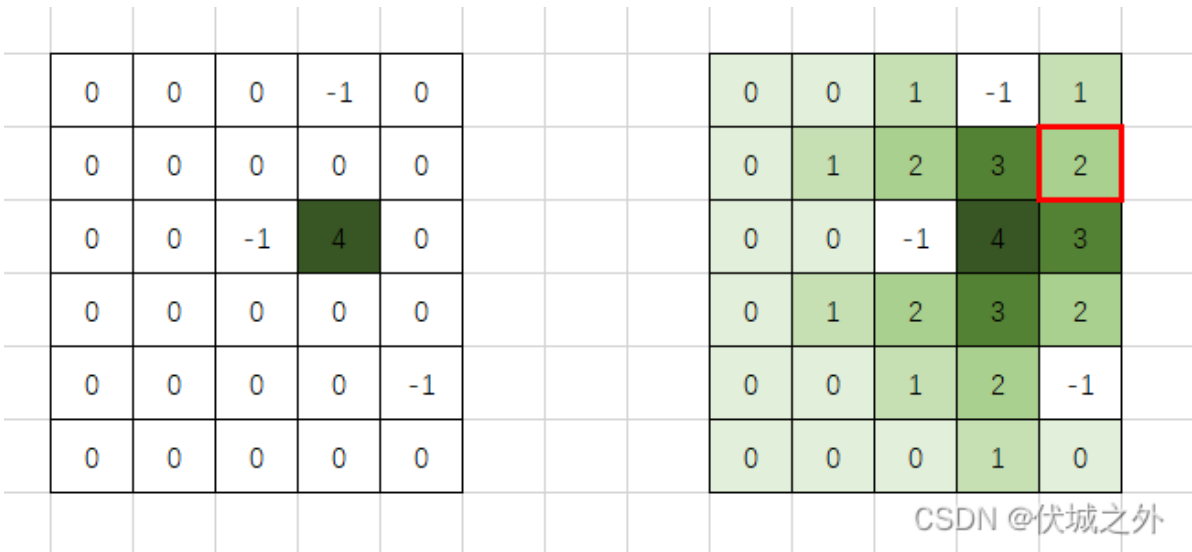
用例

输入	6 5 0 0 0 -1 0 0 0 0 0 0 0 0 -1 4 0 0 0 0 0 0 0 0 0 0 0 0 -1 0 0 0 0 0 1 4
输出	2
说明	无

输入	6 5 0 0 0 -1 0 0 0 0 0 0 0 0 -1 4 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 1
输出	0
说明	无

题目解析

用例图示如下



因此[1,4]位置（如上图红框位置）的值为2。

通过图示，我们可以很容易想到解题思路是图的多源BFS。

因此，题解请参考：

[华为机试 - 计算疫情扩散时间 伏城之外的博客-CSDN博客](#)

[华为机试 - 污染水域 伏城之外的博客-CSDN博客](#)

Java算法源码

```
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
        int n = sc.nextInt();

        int[] arr = new int[m * n];
        for (int i = 0; i < m * n; i++) {
            arr[i] = sc.nextInt();
        }

        int tarI = sc.nextInt();
        int tarJ = sc.nextInt();

        System.out.println(getResult(arr, m, n, tarI, tarJ));
    }

    public static int getResult(int[] arr, int m, int n, int tarI, int tarJ) {
        LinkedList<Integer[]> queue = new LinkedList<>();

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (arr[i * n + j] > 0) {
                    queue.addLast(new Integer[] {i, j});
                    break;
                }
            }
        }

        // 上下左右偏移量
        int[][] offsets = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};

        while (queue.size() > 0) {
            Integer[] pos = queue.removeFirst();
            int i = pos[0];
            int j = pos[1];

            int x = arr[i * n + j] - 1;

            if (x == 0) break;

            for (int[] offset : offsets) {
                int newI = i + offset[0];
                int newJ = j + offset[1];

                if (newI >= 0 && newI < m && newJ >= 0 && newJ < n && arr[newI * n + newJ] == 0) {
                    arr[newI * n + newJ] = x;
                    queue.addLast(new Integer[] {newI, newJ});
                }
            }
        }
    }
}
```

```
    }  
    }  
}  
  
    return arr[tarI * n + tarJ];  
}  
}
```

46. Linux发行版的数量

题目描述

Linux操作系统有多个发行版，distrowatch.com提供了各个发行版的资料。这些发行版互相存在关联，例如Ubuntu基于[Debian](#)开发，而Mint又基于Ubuntu开发，那么我们认为Mint同Debian也存在关联。

发行版集是一个或多个相关存在关联的操作系统发行版，集合内不包含没有关联的发行版。

给你一个 $n * n$ 的矩阵 `isConnected`，其中 `isConnected[i][j] = 1` 表示第 i 个发行版和第 j 个发行版直接关联，而 `isConnected[i][j] = 0` 表示二者不直接相连。

返回最大的发行版集中发行版的数量。

输入描述

第一行输入发行版的总数量 N ，

之后每行表示各发行版间是否直接相关

输出描述

输出最大的发行版集中发行版的数量

备注

$1 \leq N \leq 200$

用例

输入	4 1 1 0 0 1 1 1 0 0 1 1 0 0 0 0 1
输出	3
说明	Debian(1)和 Unbuntu (2)相关Mint(3)和Ubuntu(2)相关，EeulerOS(4)和另外三个都不相关，所以存在两个发行版集，发行版集中发行版的数量分别是3和1，所以输出3。

题目解析

本题可以利用[并查集](#)求解，本题要求的就是各个连通分量的节点数，并输出最大的连通分量的节点数。

如果大家对并查集还不了解，可以看下这个入门视频：

[《算法训练营》进阶篇 01 并查集哔哩哔哩bilibili](#)

学会并查集数据结构后，本题的解题难度就很小了，本题题解可以参考

[华为OD机试 - 发广播伏城之外的博客-CSDN博客信道分配 华为od](#)

解决完本题，可以继续尝试2022.Q4题库的其他并查集算法题：

[华为OD机试 - 计算快递主站点 伏城之外的博客-CSDN博客](#)

[华为OD机试 - 开心消消乐 伏城之外的博客-CSDN博客](#)

[华为OD机试 - 机器人 伏城之外的博客-CSDN博客](#)

[华为OD机试 - 快递业务站 伏城之外的博客-CSDN博客](#)

Java算法源码

```
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[][] matrix = new int[n][n];

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        System.out.println(getResult(matrix, n));
    }

    public static int getResult(int[][] matrix, int n) {
        UnionFindSet ufs = new UnionFindSet(n);

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) { // j从i+1开始，是因为矩阵是对称的
                if (matrix[i][j] == 1) {
                    ufs.union(i, j);
                }
            }
        }

        // connected的key代表某个连通分量的顶级父节点，value代表该连通分量下的节点个数
```

```

HashMap<Integer, Integer> connected = new HashMap<>();

for (int i = 0; i < n; i++) {
    Integer fa = ufs.find(ufs.fa[i]);
    connected.put(fa, connected.getOrDefault(fa, 0) + 1);
}

// 返回最大节点数
return connected.values().stream().max((a, b) -> a - b).get();
}
}

// 并查集实现
class UnionFindSet {
    int[] fa;
    int count;

    public UnionFindSet(int n) {
        this.count = n;
        this.fa = new int[n];
        for (int i = 0; i < n; i++) this.fa[i] = i;
    }

    public int find(int x) {
        if (x != this.fa[x]) {
            return (this.fa[x] = this.find(this.fa[x]));
        }
        return x;
    }

    public void union(int x, int y) {
        int x_fa = this.find(x);
        int y_fa = this.find(y);

        if (x_fa != y_fa) {
            this.fa[y_fa] = x_fa;
            this.count--;
        }
    }
}

```

47.开放日活动、取出尽量少的球

题目描述

某部门开展**Family Day**开放日活动，其中有个从桶里取球的游戏，游戏规则如下：

有N个容量一样的小桶等距排开，

且每个小桶都默认装了数量不等的小球，

每个小桶装的小球数量记录在数组 bucketBallNums 中，

游戏开始时，要求所有桶的小球总数不能超过SUM，

如果小球总数超过SUM，则需对所有的小桶统一设置一个容量最大值 maxCapacity，

并需将超过容量最大值的小球拿出来，直至小桶里的小球数量小于 maxCapacity;

请您根据输入的数据，计算从每个小桶里拿出的小球数量。

限制规则一：

所有小桶的小球总和小于SUM，则无需设置容量值maxCapacity，并且无需从小桶中拿球出来，返回结果[]

限制规则二：

如果所有小桶的小球总和大于SUM，则需设置容量最大值maxCapacity，并且需从小桶中拿球出来，返回从每个小桶拿出的小球数量组成的数组；

输入描述

第一行输入2个正整数，数字之间使用空格隔开，其中第一个数字表示SUM，第二个数字表示 bucketBallNums数组长度；
第二行输入N个正整数，数字之间使用空格隔开，表示bucketBallNums的每一项；

输出描述

找到一个maxCapacity，来保证取出尽量少的球，并返回从每个小桶拿出的小球数量组成的数组。

用例

输入	14 7 2 3 2 5 5 1 4
输出	[0,1,0,3,3,0,2]
说明	小球总数为22，SUM=14，超出范围了，需从小桶取球，maxCapacity=1，取出球后，桶里剩余小球总和为7，远小于14 maxCapacity=2，取出球后，桶里剩余小球总和为13，maxCapacity=3，取出球后，桶里剩余小球总和为16，大于14 因此maxCapacity为2，每个小桶小球数量大于2的都需要拿出来；

输入	3 3 1 2 3
输出	[0,1,2]
说明	小球总数为6，SUM=3，超出范围了，需从小桶中取球，maxCapacity=1，则小球总数为3，从1号桶取0个球，2号桶取1个球，3号桶取2个球；

输入	6 2 3 2
输出	[]

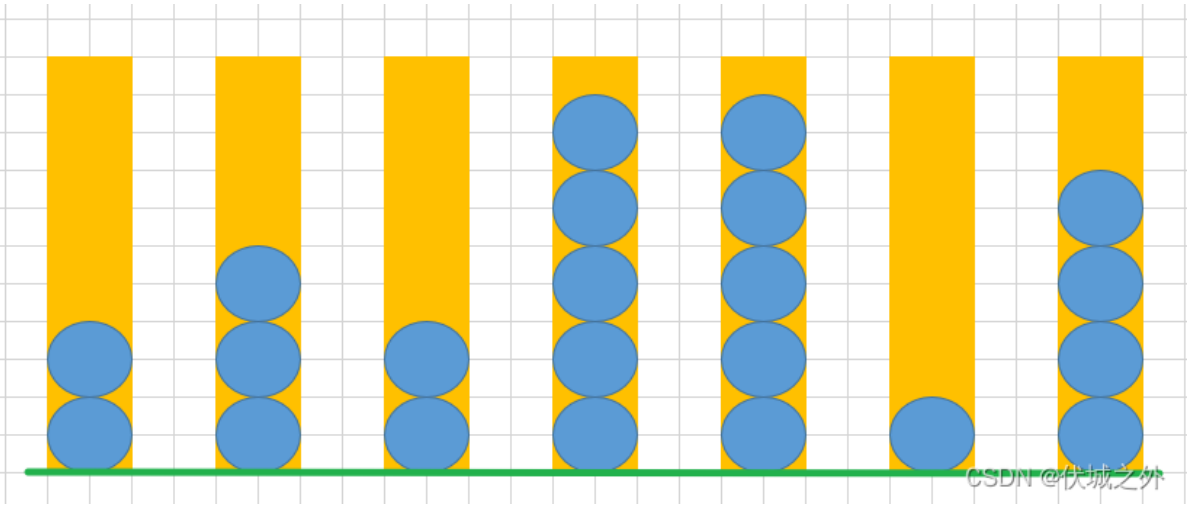
输入	6 2 3 2
说明	小球总数为5, SUM=6, 在范围内, 无需从小桶取球;

题目解析

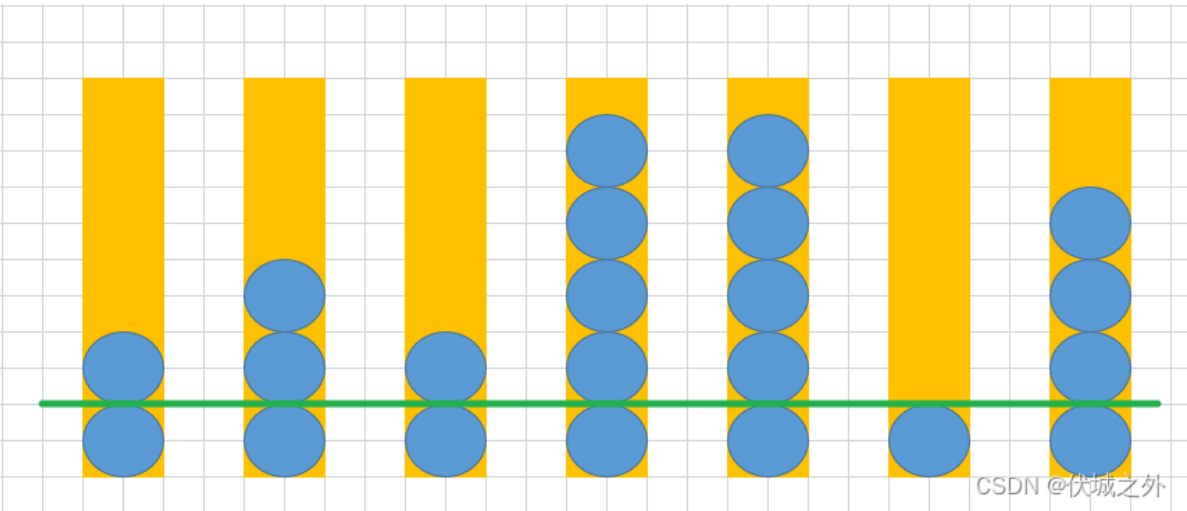
用例示意图如下:

由于所有桶中的球数之和超过了14, 因此我们需要设置一个maxCapacity来限制每个桶中球的数量。

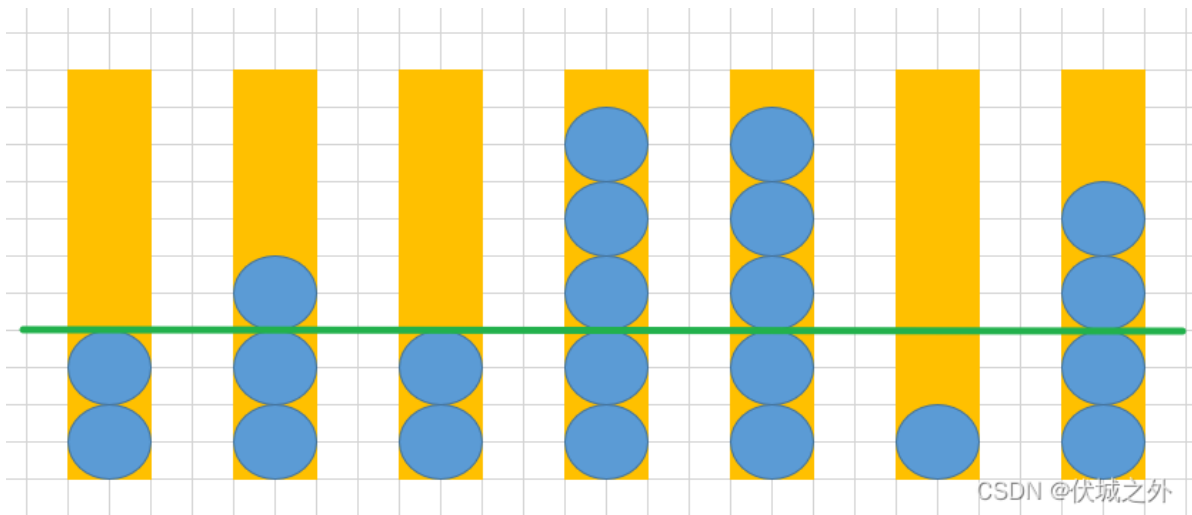
如果maxCapacity值设置为0, 则所有桶中的球都需要取出, 因此剩余球总数为0, 小于sum=14,因此符合要求



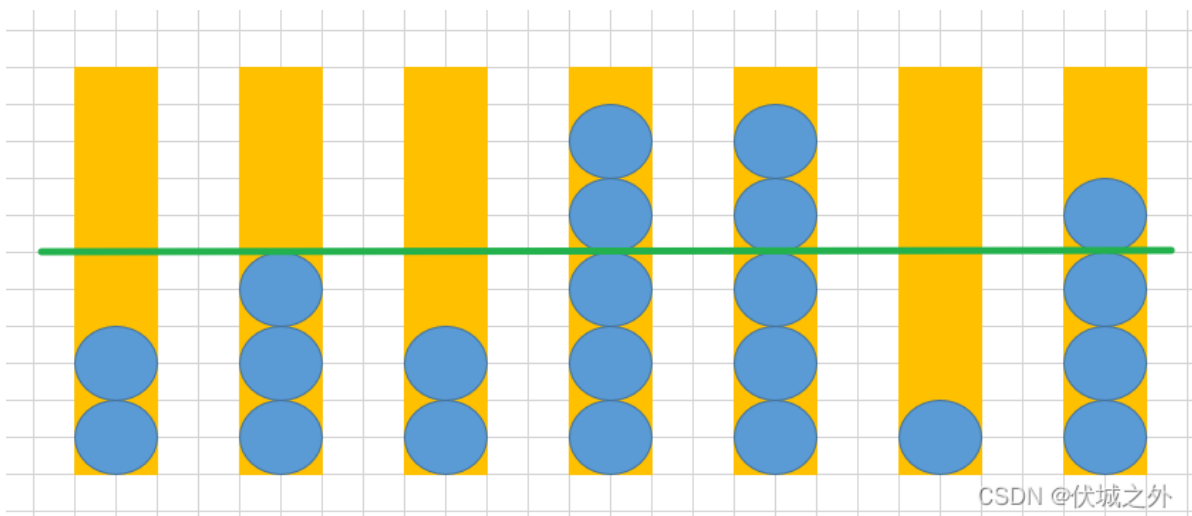
如果maxCapacity值设置为1, 则所有桶中的球最多只保留1个, 如下图所示, 剩余球总数7个, 也符合要求



如果maxCapacity值设置为2, 则所有桶中的球最多只保留2个, 如下图所示, 剩余球总数13个, 也符合要求



如果maxCapacity值设置为3，则所有桶中的球最多只保留3个，如下图所示，剩余球总数17个，不符合要求



因此，我们可以发现，maxCapacity取值2时，剩余球数最多，总数量小于SUM=14，符合要求，且取出的球最少，分别为0，1，0，3，3，0，2。

那么我们是否需要从maxCapacity=0开始找呢？

答案是不需要，我们完全可以使用 $SUM / bucketBallNums.length$ 求得一个最理想值。

比如用例中SUM=14，bucketBallNums.length=7，则每个桶中球数量的最理想值是 $14/7=2$ 。

我们可以将此时最理想值作为maxCapacity的起始值。然后向后查找。

但是上面这种算法在应对较大数量级，可能会超时，因此改进策略是使用[二分查找](#)，具体逻辑请看

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```



```

int sum = sc.nextInt();
int n = sc.nextInt();

Integer[] arr = new Integer[n];
for (int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}

System.out.println(getResult(sum, arr, n));
}

public static String getResult(int sum, Integer[] arr, int n) {
    int total = Arrays.stream(arr).reduce((p, c) -> p + c).get();
    if (total <= sum) return "[]";

    int max_maxCapacity = Arrays.stream(arr).max((a, b) -> a - b).get();
    int min_maxCapacity = sum / n;

    final int min_maxCapacity_copy = min_maxCapacity;
    Integer[] ans =
        Arrays.stream(arr)
            .map(count -> count > min_maxCapacity_copy ? count -
min_maxCapacity_copy : 0)
            .toArray(Integer[]::new);

    while (max_maxCapacity - min_maxCapacity > 1) {
        int maxCapacity = (max_maxCapacity + min_maxCapacity) / 2;

        // tmp数组保存的是每个桶移除的球的数量
        Integer[] tmp = new Integer[n];
        int remain = total;
        for (int i = 0; i < arr.length; i++) {
            // r是每个桶需要移除的球的个数，如果桶内球数超过maxCapacity，则需要移除超出部分，否则不需要移除
            int r = arr[i] > maxCapacity ? arr[i] - maxCapacity : 0;
            remain -= r;
            tmp[i] = r;
        }

        if (remain > sum) {
            max_maxCapacity = maxCapacity;
        } else if (remain < sum) {
            min_maxCapacity = maxCapacity;
            ans = tmp;
        } else {
            ans = tmp;
            break;
        }
    }

    return Arrays.toString(ans);
}
}

```

48.人数最多的站点

题目描述

公园园区提供小火车单向通行，从园区站点编号最小到最大通行如1~2~3~4~1，然后供员工在各个办公园区穿梭，通过对公司N个员工调研统计到每个员工的坐车区间，包含前后站点，请设计一个程序计算出小火车在哪个园区站点时人数最多。

输入描述

第1个行，为调研员工人数

第2行开始，为每个员工的上车站点和下车站点。

使用数字代替每个园区用空格分割，如3 5表示从第3个园区上车，在第5个园区下车

输出描述

人数最多时的园区站点编号，最多人数相同时返回编号最小的园区站点

用例

输入	3 1 3 2 4 1 4
输出	2
说明	无

题目解析

本题其实就是求解最大重叠区间个数的变种题。

即，我们只要找到具有最大重叠部分的区间的起点就是本题题解。

关于最大重叠区间个数求解，请看[年年岁岁都容易挂的算法高频面试题，一线大厂经典面试题之堆和最大线段重合问题](#)[哔哩哔哩bilibili](#)

上面视频的核心思想其实就是：

- 首先，将所有区间按开始位置升序
- 然后，遍历排序后区间，并将小顶堆中小于遍历区间起始位置的区间弹出（小顶堆实际存储区间结束位置），此操作后，小顶堆中剩余的区间个数，就是和当前遍历区间重叠数。

我们只需要在求解最大重叠数时，保留遍历的区间的起始位置即可。

对于JS而言，没有原生堆结构（即[优先队列](#)），因此我们需要手写小顶堆代码，关于优先队列，大家可以参考：

[LeetCode - 1705 吃苹果的最大数目](#) [伏城之外的博客-CSDN博客](#)

但是本题是100分值的，可能不会存在大数量级，因此大家可以先尝试用[有序数组](#)代替小顶堆，如果不行，再实现小顶堆。

另外，本题和[华为OD机试 - 最大化控制资源成本 伏城之外的博客-CSDN博客](#)

很像，大家可以继续尝试做下上面这题。

2023.2.1根据网友指正，本题小火车是有可能走回程的，比如，坐车区间为 [3,2]，即从站点3 -> 站点4 -> 站点1 -> 站点2。

对于这种情况，就会破坏上面对于区间的排序。因此，我们需要将：该情况的坐车区间拆分为 [3, 4]和 [1, 2]

Java算法源码

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.PriorityQueue;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[][] ranges = new int[n][2];

        for (int i = 0; i < n; i++) {
            ranges[i][0] = sc.nextInt();
            ranges[i][1] = sc.nextInt();
        }

        System.out.println(getResult(ranges));
    }

    public static int getResult(int[][] ranges) {
        // 由于题目并未说有几个站点，因此需要计算出最后一个站点
        Integer last =
            Arrays.stream(ranges)
                .map(range -> Math.max(range[0], range[1]))
                .max((a, b) -> a - b)
                .orElse(0);

        // 如果存在 [3,2] 这种坐车区间，即从站点3 -> 站点4 -> 站点1 -> 站点2，则此时将该坐车区
        // 间拆分为 [3, 4]和[1, 2]
        ArrayList<Integer[]> tmp = new ArrayList<>();
        for (int[] range : ranges) {
            int s = range[0];
            int e = range[1];

            if (s > e) {
                tmp.add(new Integer[] {s, last});
            }
        }
    }
}
```

```

        tmp.add(new Integer[] {1, e});
    } else {
        tmp.add(new Integer[] {s, e});
    }
}

// 最大重叠区间个数求解
tmp.sort((a, b) -> a[0] - b[0]);

PriorityQueue<Integer> end = new PriorityQueue<>((a, b) -> a - b);

int max = 0;
int ans = tmp.get(0)[0];
for (Integer[] range : tmp) {
    int s = range[0];
    int e = range[1];

    while (end.size() > 0) {
        Integer top = end.peek();

        if (top < s) {
            end.poll();
        } else {
            break;
        }
    }

    end.offer(e);

    if (end.size() > max) {
        max = end.size();
        ans = s;
    }
}
return ans;
}
}

```

49.基站维护工程师

题目描述

小王是一名[基站](#)维护工程师，负责某区域的基站维护。

某地方有 n 个基站($1 < n < 10$)，已知各基站之间的距离 $s(0 < s < 500)$ ，并且基站 x 到基站 y 的距离，与基站 y 到基站 x 的距离并不一定会相同。

小王从基站 1 出发，途经每个基站 1 次，然后返回基站 1，需要你为他选择一条距离最短的路。

输入描述

站点数n和各站点之间的距离(均为整数)

输出描述

最短路程的数值

用例

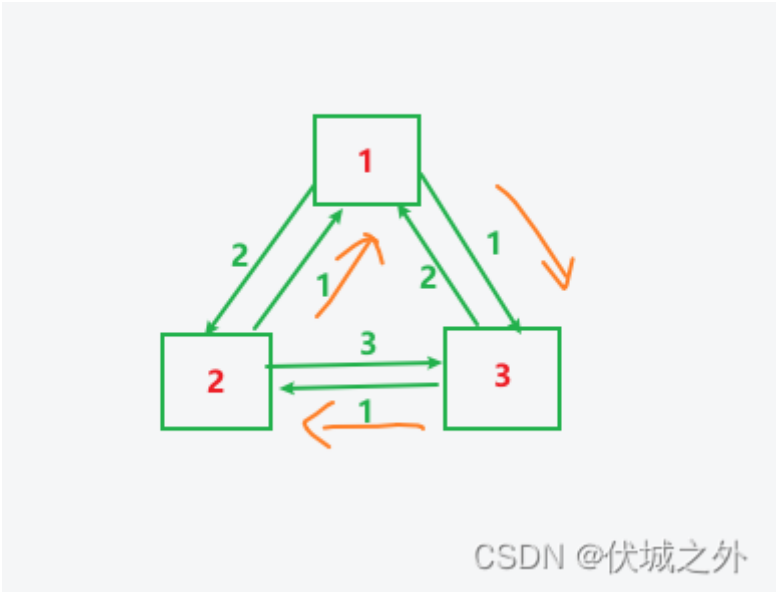
输入	3 0 2 1 1 0 2 2 1 0
输出	3
说明	无

题目解析

用例输入含义是,

3 //有3个基站,
0 2 1 // 站点1到站点1的距离0, 到站点2的距离2, 到站点3的距离1
1 0 2 // 站点2到站点1的距离1, 到站点2的距离0, 到站点3的距离2
2 1 0 // 站点3到站点1的距离2, 到站点2的距离1, 到站点3的距离0

图示如下



可以发现, 1 → 3 → 2 → 1 的路线距离是最短的, 只有3距离。

题目中说:

小王从基站 1 出发, 途经每个基站 1 次, 然后返回基站 1

并且按照题目输入来看，每个站点都与剩下的其他站点相连，因此本题其实就是求解n-1个站点（即2~n站点，起始站点1）的[全排列](#)。

比如用例一共三个站点，从1站点出发，即求2，3站点的全排列：23，32

因此一共有两种途径选择：1 → 2 → 3 → 1 和 1 → 3 → 2 → 1

我们只要比较各排列路径中距离最小的即为题解。

两个站点i，j之间距离，即为matrix[i-1][j-1]，比如求解1 → 2距离，起始就是matrix[0][1]。

题目中说 $1 < n < 10$ ，也就是说最多有9个站点，而我们求解n-1个站点的全排列，即8个站点的全排列，一共有 $8! = 40320$ 个，每个排列求解距离要进行一个O(n)的遍历，即9次遍历。因此一共是差不多40w次循环，好在没什么计算量。

我测试了一下10*10矩阵的用时为200ms左右，应该符合要求。

```
PS D:\Desktop\alg> node .\index.js
10
0 1 2 3 4 1 2 3 4 1
2 0 3 4 1 2 3 3 4 2
3 4 0 1 2 3 4 1 2 3
1 2 3 0 1 2 3 3 4 2
3 3 4 2 0 2 3 3 4 2
2 1 1 2 3 0 3 2 4 2
3 0 0 1 3 4 0 2 3 3
4 2 2 0 3 2 2 0 3 3
3 2 1 2 1 2 3 2 0 3
3 2 1 1 3 3 4 2 2 0
12
test: 214.232ms

PS D:\Desktop\alg>
```

CSDN @伏城之外

Java算法源码

```
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[][] matrix = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        System.out.println(getResult(matrix, n));
    }
}
```

```

public static int getResult(int[][] matrix, int n) {
    boolean[] used = new boolean[n];
    LinkedList<Integer> path = new LinkedList<>();
    int[] ans = {Integer.MAX_VALUE};

    dfs(n, used, path, ans, matrix);

    return ans[0];
}

public static void dfs(
    int n, boolean[] used, LinkedList<Integer> path, int[] ans, int[][]
matrix) {
    if (path.size() == n - 1) {
        int dis = matrix[0][path.get(0)];
        for (int i = 0; i < path.size() - 1; i++) {
            int p = path.get(i);
            int c = path.get(i + 1);
            dis += matrix[p][c];
        }
        dis += matrix[path.getLast()][0];
        ans[0] = Math.min(ans[0], dis);
        return;
    }

    for (int i = 1; i < n; i++) {
        if (!used[i]) {
            path.push(i);
            used[i] = true;
            dfs(n, used, path, ans, matrix);
            used[i] = false;
            path.pop();
        }
    }
}

```

50.最大数字

题目描述

给定一个由纯数字组成以字符串表示的数值，现要求字符串中的每个数字最多只能出现2次，超过的需要进行删除；

删除某个重复的数字后，其它数字相对位置保持不变。

如"34533"，数字3重复超过2次，需要删除其中一个3，删除第一个3后获得最大数值"4533"

请返回经过删除操作后的最大的数值，以字符串表示。

输入描述

第一行为一个纯数字组成的字符串，长度范围：[1,100000]

输出描述

输出经过删除操作后的最大的数值

用例

输入	34533
输出	4533
说明	无

输入	5445795045
输出	5479504
说明	无

题目解析

本题最优解题思路是利用栈结构。

首先，我们需要定义两个map，分别是unused和[reserve](#)，其中：

- unused含义是：每个数字剩余可用个数
- reserve含义是：每个数字已保留的个数

然后对他们进行初始化，初始时unused就是统计输入字符串中各数字字符的出现次数，而reserve每个数字字符的个数都初始化为0。

接下来，遍历出输入字符串的每个字符c：

如果此时stack栈顶没有元素，则将遍历的c直接压入栈，然后unused[c]--，reserve[c]++

如果此时stack栈顶有元素，假设为top，则：

- c <= top 的话，则直接压入栈，然后unused[c]--，reserve[c]++
- c > top的话，则需要考虑将栈顶top弹出，而弹出是有条件的，因为题目说，每个数字最多出现两次，那么表示每个数字出现次数>=2的话，我们就保留该数字2个，此时整体数值才会最长，最长才有可能最大。如果此时我们将top弹出，那么需要看 unused[top] + reserve[top] - 1是不是 >= 2，只有成立时，我们才能弹出top，否则对应数字字符的数量将不足2个，最终影响整体数值的长度，进而影响大小。

Java算法源码

```
import java.util.HashMap;
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.next();

        System.out.println(getResult(str));
    }

    public static String getResult(String str) {
        // 每个数字字符的可用个数
        HashMap<Character, Integer> unused = new HashMap<>();
        // 每个数字字符的保留个数
        HashMap<Character, Integer> reserve = new HashMap<>();

        // 初始时，每个数字有多少个，就可用多少个，由于还未使用，因此保留个数为0
        for (int i = 0; i < str.length(); i++) {
            char c = str.charAt(i);
            unused.put(c, unused.getOrDefault(c, 0) + 1);
            reserve.putIfAbsent(c, 0);
        }

        // 定义一个栈
        LinkedList<Character> stack = new LinkedList<>();

        // 遍历输入字符串的每个数字字符c
        for (int i = 0; i < str.length(); i++) {
            char c = str.charAt(i);

            // 如果该字符已经保留了2个了，则后续再出现该数字字符可以不保留
            if (reserve.get(c) == 2) {
                // 则可用c数字个数--
                unused.put(c, unused.get(c) - 1);
                continue;
            }

            // 比较当前数字c和栈顶数字top，如果c>top，那么需要考虑将栈顶数字弹出
            while (stack.size() > 0) {
                char top = stack.getLast();

                // 如果栈顶数字被弹出后，已保留的top字符数量和未使用的top字符数量之和大于等于2，则可以弹出，否则不可以
                if (top < c && unused.get(top) + reserve.get(top) - 1 >= 2) {
                    stack.removeLast();
                    reserve.put(top, reserve.get(top) - 1);
                } else {
                    break;
                }
            }
        }
    }
}
```

```

        // 选择保留当前遍历的数字c
        stack.add(c);
        // 则可用c数字个数--
        unused.put(c, unused.get(c) - 1);
        // 已保留c数字个数++
        reserve.put(c, reserve.get(c) + 1);
    }

    StringBuilder sb = new StringBuilder();
    for (Character c : stack) {
        sb.append(c);
    }
    return sb.toString();
}
}

```

51.快速开租建站

题目描述

当前IT部门支撑了子公司颗粒化业务，该部门需要实现为子公司快速开租建站的能力，建站是指在一个全新的环境部署一套IT服务。

- 每个站点开站会由一系列部署任务项构成，每个任务项部署完成时间都是固定和相等的，设为1。
- 部署任务项之间可能存在依赖，假如任务2依赖任务1，那么等任务1部署完，任务2才能部署。
- 任务有多个依赖任务则需要等所有依赖任务都部署完该任务才能部署。
- 没有依赖的任务可以并行部署，优秀的员工们会做到完全并行无等待的部署。

给定一个站点部署任务项和它们之间的依赖关系，请给出一个站点的最短开站时间。

输入描述

第一行是任务数taskNum,第二行是任务的依赖关系数relationsNum

接下来 relationsNum 行，每行包含两个id，描述一个依赖关系，格式为：IDi IDj，表示部署任务i部署完成了，部署任务j才能部署，IDi 和 IDj 值的范围为：[0, taskNum)

注：输入保证部署任务之间的依赖不会存在环。

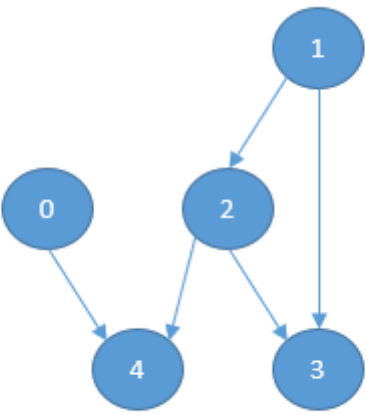
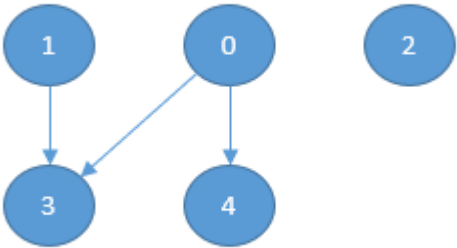
输出描述

一个整数，表示一个站点的最短开站时间。

备注

- $1 < \text{taskNum} \leq 100$
- $1 \leq \text{relationsNum} \leq 5000$

用例

输入	5 5 0 4 1 2 1 3 2 3 2 4		
输出	3		
说明	有5个部署任务项，5个依赖关系，如下图所示。  我们可以先同时部署任务项0和任务项1，然后部署任务项2，最后同时部署任务项3和任务项4.最短开战时间为3。		
输入	5 3 0 3 0 4 1 3		
输出	2		
说明	有5个部署任务项，3个依赖关系，如下图所示。  我们可以先同时部署任务项0，任务项1，任务项2。然后再同时部署任务项3和任务项4.最短开战时间为2。		

题目解析

本题是经典的[拓扑排序](#)问题。因为题目中各任务之间有较为明显的前后依赖关系。

对于拓扑排序，我们可以看下面博客的解析：

[LeetCode - 207 课程表 伏城之外的博客-CSDN博客](#)

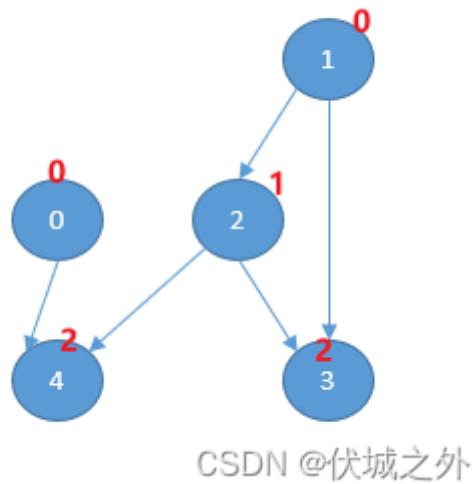
[华为OD机试 - 跳格子游戏 伏城之外的博客-CSDN博客跳格子 华为机试](#)

本题要求的并非是让我们判断拓扑结构中是否有环结构，因为题目已经说明了

输入保证部署任务之间的依赖不会存在环

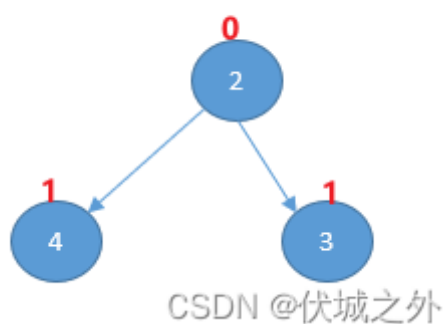
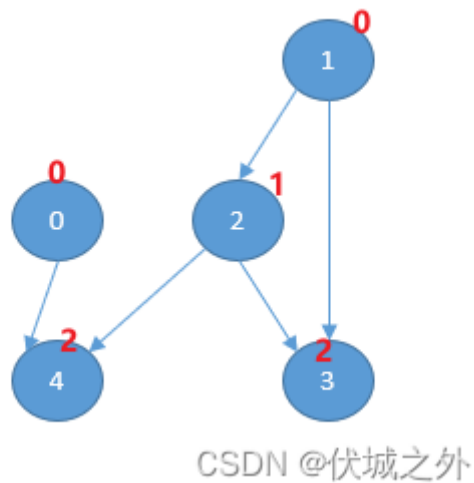
本题是要我们求解最短开战时间，这里最短开战时间其实就是剥离入度0点的次数，如下图所示：

用例1中各顶点的入度值已用红色值标出



我们可以不断地剥离入度为0地顶点，因为题目说：

没有依赖的任务可以并行部署，优秀的员工们会做到完全并行无等待的部署。





CSDN @伏城之外

可以只需要剥离三次，而这也是最短建站时间。

Java算法源码

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int taskNum = sc.nextInt();
        int relationsNum = sc.nextInt();

        int[][] relations = new int[relationsNum][2];
        for (int i = 0; i < relationsNum; i++) {
            relations[i][0] = sc.nextInt();
            relations[i][1] = sc.nextInt();
        }

        System.out.println(getResult(relations, taskNum));
    }

    public static int getResult(int[][] relations, int taskNum) {
        HashMap<Integer, ArrayList<Integer>> next = new HashMap<>();
        int[] inDegree = new int[taskNum];

        for (int[] relation : relations) {
            int a = relation[0];
            int b = relation[1];

            next.putIfAbsent(a, new ArrayList<>());
            next.get(a).add(b); // a的下一个顶点有b
            inDegree[b]++; // b顶点的入度++
        }

        LinkedList<Integer[]> queue = new LinkedList<>();
        int t = 1;

        for (int i = 0; i < taskNum; i++) {
            if (inDegree[i] == 0) {
```

```

        queue.add(new Integer[] {i, t}); // i含义是入度为0的顶点，t含义是该顶点所处建站
        时间
    }
}

while (queue.size() > 0) {
    Integer[] tmp = queue.removeFirst(); // 注意这里为了维持t，一定要使用队列先进先
    出，出队代表删除某顶点
    int task = tmp[0];
    int time = tmp[1];

    if (next.containsKey(task) && next.get(task).size() > 0) {
        for (Integer nxt : next.get(task)) {
            // 该顶点被删除，则其后继点的入度值--，若--后入度为0，则将成为新的出队点
            if (--inDegree[nxt] == 0) {
                t = time + 1; // 此时建站时间+1
                queue.add(new Integer[] {nxt, t});
            }
        }
    }
}

return t;
}
}

```

52.简单的解压缩算法

题目描述

现需要实现一种算法，能将一组压缩字符串还原成原始字符串，**还原规则**如下：

- 1、字符后面加数字N，表示重复字符N次。例如：压缩内容为A3，表示原始字符串为AAA。
- 2、花括号中的字符串加数字N，表示花括号中的字符重复N次。例如压缩内容为{AB}3，表示原始字符串为ABABAB。
- 3、字符加N和花括号后面加N，支持**任意的嵌套**，包括**互相嵌套**，例如：压缩内容可以{A3B1{C}3}3

输入描述

输入一行压缩后的字符串

输出描述

输出压缩前的字符串

备注

- 输入保证，数字不会为0，花括号中的内容不会为空，保证输入的都是合法有效的压缩字符串
- 输入输出字符串区分大小写
- 输入的字符串长度范围为[1, 10000]
- 输出的字符串长度范围为[1, 100000]
- 数字N范围为[1, 10000]

用例

输入	{A3B1{C}3}3
输出	AAABCCCAAABCCCAAABCCC
说明	{A3B1{C}3}3代表A字符重复3次，B字符重复1次，花括号中的C字符重复3次，最外层花括号中的AAABCCC重复3次。

题目解析

本题似曾相识，和下面两个题目很像

[华为机试 - 一种字符串压缩表示的解压](#) [伏城之外的博客-CSDN博客](#)

[华为机试 - 报文解压缩](#) [伏城之外的博客-CSDN博客](#)

做完本题后，可以尝试再做做上面这两个真题。

本题可以使用栈结构来解题。

我的解题思路是，从左到右遍历输入字符串的每一个字符：

若发现非数字的字符，则直接压入栈stack中；

另外，发现字符“{”时，需要记录它在stack栈中位置到idxs数组中；

若发现数字字符c，则需要观察此时stack栈顶元素：

- 如果栈顶不是 “}” 字符，则取出栈顶元素，重复c次后，重新压入栈中。
- 如果栈顶是 “}” 字符，则弹出，并取出idxs中记录的最后一次的 “{” 压stack栈的位置idx，然后通过splice操作，将stack栈中idx往后的元素都删除取出为frag，然后重复frag片段c次，后重新压入stack栈中。

按照此逻辑，即可完成本题要求的解压缩。

但是，本题有两个疑点：

- 是否存在异常情况？由于本题没说如何处理异常，因此我这里默认无异常情况，即输入都是合法的。

- 数字N会不会出现多位数，比如A13这种情况，如果允许多位数字，则本题会变得更加复杂，由于用例和题目描述中都没有特别说明这种情况，因此我们默认这里的数字不会有多位数。

2023.01.29

本题新增了备注说明，因此上面两个疑点得到了解答。

1、备注说明：保证输入的都是合法有效的压缩字符串

2、数字N范围为[1, 10000]

Java算法源码

Java这里最好使用LinkedList模拟栈，但是LinkedList模拟的栈是严格栈，即只能每次弹出栈顶元素，不能一下将某个范围元素全部删除，因此在取出栈中{ 和 }中间内容时，比JS略显复杂，但是处理逻辑相同。

```
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.next();
        System.out.println(getResult(str));
    }

    public static String getResult(String str) {
        LinkedList<String> stack = new LinkedList<>();
        // idxs记录 { 出现的索引位置
        LinkedList<Integer> idxs = new LinkedList<>();
        StringBuilder repeat = new StringBuilder();
        str += " ";

        for (int i = 0; i < str.length(); i++) {
            char c = str.charAt(i);
            if (c >= '0' && c <= '9') { // 如果压栈遇到数字
                // 如果出现A13,或者{ABC}99这种情况，我们需要把多位数解析出来
                repeat.append(c);
                continue;
            }

            if (repeat.length() > 0) {
                int n = Integer.parseInt(repeat.toString());
                repeat = new StringBuilder();
                if ("}".equals(stack.getLast())) { // 如果此时栈顶是 } ，则需要将{,} 中间的内容整体重复repeat次
                    int left = idxs.removeLast();
                    stack.remove(left); // 去掉 {
                    stack.removeLast(); // 去掉 }
                    updateStack(stack, left, n); // 将 {,} 中间部分重复 repeat次后重新压栈
                } else { // 如果此时栈顶不是 } ，则只需要将栈顶元素重复repeat次即可。
                    updateStack(stack, stack.size() - 1, n);
                }
            }
        }
    }

    private static void updateStack(LinkedList<String> stack, int left, int n) {
        for (int i = 0; i < n; i++) {
            String s = stack.get(left);
            for (int j = 0; j < s.length(); j++) {
                stack.add(s.charAt(j));
            }
        }
    }
}
```



```

    }
}

// 记录 { 出现的索引位置
if (c == '{') {
    idxs.addLast(stack.size());
}

// 数字外的字符都压入栈中，其中{,}需要再重复操作时删除
stack.addLast(c + "");
}

StringBuilder sb = new StringBuilder();
for (String c : stack) {
    sb.append(c);
}
return sb.toString().trim();
}

// 将stack，从left索引开始到最后的内容，弹栈，并整体重复repeat次后，再重新压入栈
public static void updateStack(LinkedList<String> stack, int left, int repeat)
{
    int count = stack.size() - left;

    // frag用于存储弹栈数据
    String[] frag = new String[count];

    while (count-- > 0) {
        frag[count] = stack.removeLast();
    }

    // 由于重复的是弹栈内容的整体，而不是每个，因此需要将弹栈内容合并
    StringBuilder sb = new StringBuilder();
    for (String s : frag) {
        sb.append(s);
    }

    // 将弹栈内容合并后重复repeat次，再重新压入栈中
    String fragment = sb.toString();
    StringBuilder ans = new StringBuilder();

    for (int i = 0; i < repeat; i++) {
        ans.append(fragment);
    }

    stack.addLast(ans.toString());
}
}

```

53. 硬件产品销售方案

题目描述

某公司目前推出了AI开发者套件，AI加速卡，AI加速模块，AI服务器，智能边缘多种硬件产品，每种产品包含若干个型号。

现某合作厂商要采购金额为 $amount$ 元的硬件产品搭建自己的AI基座。

例如当前库存有 N 种产品，每种产品的库存量充足，给定每种产品的价格，记为 $price$ （不存在价格相同的产品型号）。

请为合作厂商列出所有可能的产品组合。

输入描述

输入包含采购金额 $amount$ 和产品价格列表 $price$ 。第一行为 $amount$ ，第二行为 $price$ ，例如：

500
[100, 200, 300, 500]

输出描述

输出为组合列表。例如：

[[100, 100, 100, 100, 100], [100, 100, 100, 200], [100, 100, 300], [100, 200, 200], [200, 300], [500]]

用例

输入	500 [100, 200, 300, 500, 500]
输出	[[100, 100, 100, 100, 100], [100, 100, 100, 200], [100, 100, 300], [100, 200, 200], [200, 300], [500], [500]]
说明	无

题目解析

简单的可重复元素组合求解。

Java算法源码

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

int amount = Integer.parseInt(sc.nextLine());

String str = sc.nextLine();
Integer[] prices =
    Arrays.stream(str.substring(1, str.length() - 1).split(", "))
        .map(Integer::parseInt)
        .toArray(Integer[]::new);

System.out.println(getResult(amount, prices));
}

public static String getResult(int amount, Integer[] prices) {
    ArrayList<String> res = new ArrayList<>();
    LinkedList<Integer> path = new LinkedList<>();

    dfs(amount, prices, 0, 0, path, res);
    return res.toString();
}

public static void dfs(
    int total,
    Integer[] arr,
    int index,
    int sum,
    LinkedList<Integer> path,
    ArrayList<String> res) {
    if (sum >= total) {
        if (sum == total) res.add(path.toString());
        return;
    }

    for (int i = index; i < arr.length; i++) {
        path.addLast(arr[i]);
        dfs(total, arr, i, sum + arr[i], path, res);
        path.removeLast();
    }
}
}

```

54.最多几个直角三角形

题目描述

有N条线段，长度分别为a[1]-a[n]。

现要求你计算这N条线段最多可以组合成几个直角三角形。

每条线段只能使用一次，每个三角形包含三条线段。

输入描述

第一行输入一个正整数T ($1 \leq T \leq 100$)，表示有T组测试数据.

对于每组测试数据，接下来有T行，

每行第一个正整数N，表示线段个数 ($3 \leq N \leq 20$)，接着是N个正整数，表示每条线段长度，($0 < a[i] < 100$)。

输出描述

对于每组测试数据输出一行，每行包括一个整数，表示最多能组合的直角三角形个数

用例

输入	1 7 3 4 5 6 5 12 13
输出	2
说明	可以组成2个直角三角形 (3, 4, 5)、(5, 12, 13)

题目解析

简单的全组合求解，即从n个数中选3个，并通过勾股定理验证选择的3个数是否可以组成直角三角形。

需要注意的是，用例中有两个5，理论上可以形成两组3, 4, 5, 和两组5, 12, 13, 即最终有四个符合要求的组合，但是这里用例只输出两个符合组合，因此需要对线段长度去重。

这里的去重可以在求解全组合之前做，即依赖于Set去重，也可以在求解全组合的过程中做，即依赖于树层去重。其中树层去重的性能更好。

另外判断直接三角形时，我们需要选择的线段升序排序，因为根据勾股定理可知，斜边必然要大于两个直角边，因此升序后，最后一个元素就是最长的线段，即斜边。

根据网友指正，本题并不是简单的全组合求解，比如我们用用例1：

```
1
7 3 4 5 6 5 12 13
```

如果单纯以全组合角度来求解组成直角三角形的线段的话，有如下情况：

- 3 4 5
- 3 4 5
- 5 12 13
- 5 12 13

一共四组，造成重复的原因是，存在两个5。

现在要求的是，以给的的线段，能组合出来的最多直角三角形数量。这句话的意思是，我们能用3 4 5 6 5 12 13组合出最多几个直角三角形？

比如，我们已经用了3 4 5组成一个直角三角形，那么给的线段还剩下6 5 12 13，而剩下的线段中，只能组合出一个直角三角形5 12 13。

那么该如何实现一种算法找出最多的呢？

我的解题思路如下，首先用全组合，求出所有直角三角形的组合可能：

- 3 4 5
- 3 4 5
- 5 12 13
- 5 12 13

然后统计出给的各种长度线段对应的数量，比如用例1可以统计如下：

```
{
  3: 1, // 长度为3的线段有1个
  4: 1,
  5: 2,
  6: 1,
  12: 1,
  13: 1
}
```

然后通过回溯算法，比如遍历出（前面全组合求解出来的）第一个可能的直角三角形组合3 4 5，然后上面统计数量变为：

```
{
  3: 0,
  4: 0,
  5: 1,
  6: 1,
  12: 1,
  13: 1
}
```

然后，继续遍历下一个可能直角三角形组合3 4 5，发现统计的3的数量已经为0了，因此这个三角形无法组合，继续遍历下一个可能直角三角形组合 5 12 13，然后统计数量变为

```
{
  3: 0,
  4: 0,
  5: 0,
  6: 1,
  12: 0,
  13: 0
}
```

然后继续遍历下一个可能组合5 12 13，发现对应长度线段的数量都变为了0，因此无法组合。

继续遍历，发现没有下一个可能的组合了，因此，计算出该种情况可以得到2个直角三角形组合：3 4 5 以及5 12 13

下面通过回溯算法，开始从第二个可能的组合开始向后遍历。

最终，最多的[组合数](#)就是题解。

我们可以尝试下这个自测用例：

```
1
7 3 4 5 12 13 84 85
```

其中有三种可能得直角三角形组合，分别是：

- 3 4 5
- 5 12 13
- 13 84 85

如果我们选择组合3 4 5，则还可以组合一个13 84 85

如果我们选择组合5 12 13，则无法组合出其他直角三角形

因此，最终本用例返回2，最多可以组合出2个直角三角形，分别是3 4 5和13 84 85

Java算法源码

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int t = sc.nextInt();
        int[][] cases = new int[t][];

        for (int i = 0; i < t; i++) {
            int n = sc.nextInt();
            int[] arr = new int[n];
            for (int j = 0; j < n; j++) {
                arr[j] = sc.nextInt();
            }
            cases[i] = arr;
        }

        getResult(cases);
    }

    public static void getResult(int[][] cases) {
        for (int[] arr : cases) {
            // 对每组测试线段升序排序
            Arrays.sort(arr);

            ArrayList<Integer[]> res = new ArrayList<>();
            dfs(arr, 0, new LinkedList<>(), res);

            int[] count = new int[100];
```

```

        for (int i : arr) {
            count[i]++;
        }

        ArrayList<Integer> ans = new ArrayList<>();
        canCombine(res, 0, count, 0, ans);
        System.out.println(ans.stream().max((a, b) -> a - b).orElse(0));
    }
}

// 全组合求解，即n个数中选3个
public static void dfs(int[] arr, int index, LinkedList<Integer> path,
ArrayList<Integer[]> res) {
    if (path.size() == 3) {
        if (isRightTriangle(path)) {
            res.add(path.toArray(new Integer[3]));
        }
        return;
    }

    for (int i = index; i < arr.length; i++) {
        path.add(arr[i]);
        dfs(arr, i + 1, path, res);
        path.removeLast();
    }
}

// 判断三条边是否可以组成直角三角形
public static boolean isRightTriangle(LinkedList<Integer> path) {
    // 注意，path中元素是升序的，因为path是取自arr的组合，而arr是升序的
    int x = path.get(0);
    int y = path.get(1);
    int z = path.get(2);

    return x * x + y * y == z * z;
}

// 求解当前直角三角形中不超过用线段的最多组合数
public static void canCombine(
    ArrayList<Integer[]> ts, int index, int[] count, int num,
    ArrayList<Integer> ans) {
    if (index >= ts.size()) {
        ans.add(num);
        return;
    }

    for (int i = index; i < ts.size(); i++) {
        Integer[] tri = ts.get(i);
        int a = tri[0];
        int b = tri[1];
        int c = tri[2];

        if (count[a] > 0 && count[b] > 0 && count[c] > 0) {
            count[a]--;
            count[b]--;

```

```
        count[c]--;
        num++;
        canCombine(ts, i + 1, count, num, ans);
        num--;
        count[a]++;
        count[b]++;
        count[c]++;
    }
}

ans.add(num);
}
}
```

55. 猜字谜

题目描述

小王设计了一个简单的猜字谜游戏，游戏的谜面是一个错误的单词，比如nesw，玩家需要猜出谜底库中正确的单词。猜中的要求如下：

对于某个谜面和谜底单词，满足下面任一条件都表示猜中：

1. 变换顺序以后一样的，比如通过变换w和e的顺序，“nwes”跟“news”是可以完全对应的；
2. 字母去重以后是一样的，比如“woood”和“wood”是一样的，它们去重后都是“wod”

请你写一个程序帮忙在谜底库中找到正确的谜底。谜面是多个单词，都需要找到对应的谜底，如果找不到的话，返回“not found”

输入描述

1. 谜面单词列表，以“,”分隔
2. 谜底库单词列表，以","分隔

输出描述

- 匹配到的正确单词列表，以","分隔
- 如果找不到，返回"not found"

备注

1. 单词的数量N的范围： $0 < N < 1000$
2. 词汇表的数量M的范围： $0 < M < 1000$
3. 单词的长度P的范围： $0 < P < 20$
4. 输入的字符只有小写英文字母，没有其他字符

用例

输入	conection connection,today
输出	connection
说明	无

输入	bdni,woood bind,wrong,wood
输出	bind,wood
说明	无

题目解析

本题有点歧义，那就是：

对于某个谜面和谜底单词，满足下面任一条件都表示猜中：

1. 变换顺序以后一样的，比如通过变换w和e的顺序，“nwes”跟“news”是可以完全对应的；
2. 字母去重以后是一样的，比如“woood”和“wood”是一样的，它们去重后都是“wod”

那么如果两个条件都满足的话，算不算猜中呢？

如果两个条件都满足也算猜中的话，那我们直接对谜底和谜面单词进行去重+[字典序排序](#)，然后对比即可。

如果两个条件都满足不算猜中，只有一个条件满足才算猜中的话，则需要对单词分别进行去重和字典序排序，然后对比两次。

这里我给出两个情况的实现。

2023.02.21 根据机考网友反馈，本题使用“两个条件都满足”的解法可以获得100%通过率

Java算法源码

两个条件都满足也算猜中

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String[] issues = sc.nextLine().split(",");
        String[] answers = sc.nextLine().split(",");

        System.out.println(getResult(issues, answers));
    }

    public static String getResult(String[] issues, String[] answers) {
        ArrayList<String> ans = new ArrayList<>();
```

```

        for (String issue : issues) {
            String str1 = getSortedAndDistinctStr(issue);
            boolean find = false;

            for (String answer : answers) {
                String str2 = getSortedAndDistinctStr(answer);
                if(str1.equals(str2)) {
                    ans.add(answer);
                    find = true;
                    // break; // 如果一个谜面对应多个谜底，这里就不能break，如果一个谜面
                    // 只对应一个谜底，那这里就要break，考试的时候都试下
                }
            }

            if(!find) {
                ans.add("not found");
            }
        }

        StringJoiner sj = new StringJoiner(",","","");
        for (String an : ans) {
            sj.add(an);
        }
        return sj.toString();
    }

    public static String getSortedAndDistinctStr(String str) {
        // HashSet不会记录元素加入顺序，会按照hash算法排序，因此不需要额外排序
        HashSet<Character> set = new HashSet<>();
        for (char c : str.toCharArray()) set.add(c);
        return set.toString();
    }
}

```

唯一条件满足才算猜中

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String[] issues = sc.nextLine().split(",");
        String[] answers = sc.nextLine().split(",");

        System.out.println(getResult(issues, answers));
    }

    public static String getResult(String[] issues, String[] answers) {
        ArrayList<String> ans = new ArrayList<>();

        for (String issue : issues) {
            String[] issueDeal = getSortedAndDistinctStr(issue);
            boolean find = false;

```

```

        for (String answer : answers) {
            String[] answerDeal = getSortedAndDistinctStr(answer);

            if(issueDeal[0].equals(answerDeal[0]) ||
issueDeal[1].equals(answerDeal[1])) {
                ans.add(answer);
                find = true;
                // break; // 如果一个谜面对应多个谜底，这里就不能break，如果一个谜面
                // 只对应一个谜底，那这里就要break，考试的时候都试下
            }
        }

        if(!find) {
            ans.add("not found");
        }
    }

    StringJoiner sj = new StringJoiner(",","","");
    for (String an : ans) {
        sj.add(an);
    }
    return sj.toString();
}

public static String[] getSortedAndDistinctStr(String str) {
    char[] arr = str.toCharArray();
    Arrays.sort(arr);
    String sorted_str = new String(arr); // 字典序排序后的字符串

    LinkedHashSet<Character> set = new LinkedHashSet<>();
    for (char c : str.toCharArray()) set.add(c);
    String distinct_str = set.toString(); // 去重后的字符串

    return new String[]{sorted_str, distinct_str};
}
}

```

56.寻找相似单词

题目描述

给定一个可存储若干单词的字典，找出指定单词的所有相似单词，并且按照单词名称从小到大排序输出。

单词仅包括字母，但可能大小写并存（大写不一定只出现在首字母）。

相似单词说明：给定一个单词X，如果通过任意交换单词中字母的位置得到不同的单词Y，那么定义Y是X的相似单词，如abc、bca即为相似单词（大小写是不同的字母，如a和A算两个不同字母）。

字典序排序：大写字母<小写字母。同样大小写的字母，遵循26字母顺序大小关系。

即A<B<C<...<X<Y<Z<a<b<c<...<x<y<z. 如Bac<aBc<acB<cBa.

输入描述

第一行为给定的单词个数N（N为非负整数）

从第二行到地N+1行是具体的单词（每行一个单词）

最后一行是指定的待检测单词（用于检测上面给定的单词中哪些是与该指定单词是相似单词，该单词可以不是上面给定的单词）

输出描述

从给定的单词组中，找出指定单词的相似单词，并且按照从小到大字典序排列输出，中间以空格隔开

如果不存在，则输出null（字符串null）

用例

输入	4 abc dasd tad bca abc
输出	abc bca
说明	在给定的输入种，与abc是兄弟单词的是abc bca，且输出按照字典序大小排序，输出的所有单词以空格隔开

输入	4 abc dasd tad bca abd
输出	null
说明	给定的单词组中，没有与给定单词abd是兄弟单词，输出为null（字符串null）

题目解析

简单的排序题，逻辑请看代码。

Java算法源码

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Scanner;
import java.util.StringJoiner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        String[] words = new String[n];
        for (int i = 0; i < n; i++) {
```

```

        words[i] = sc.next();
    }

    String target = sc.next();

    System.out.println(getResult(words, target));
}

public static String getResult(String[] words, String target) {
    String sorted_target = sortStr(target);

    ArrayList<String> ans = new ArrayList<>();
    for (String word : words) {
        String sorted_word = sortStr(word);

        if (sorted_target.equals(sorted_word)) {
            ans.add(word);
        }
    }

    if (ans.size() > 0) {
        ans.sort(String::compareTo);

        StringJoiner sj = new StringJoiner(" ");
        for (String an : ans) sj.add(an);
        return sj.toString();
    } else {
        return "null";
    }
}

public static String sortStr(String word) {
    char[] chars = word.toCharArray();
    Arrays.sort(chars);
    return new String(chars);
}
}

```

57.完美走位

题目描述

在第一人称射击游戏中，玩家通过键盘的A、S、D、W四个按键控制游戏人物分别向左、向后、向右、向前进行移动，从而完成走位。

假设玩家每按动一次键盘，游戏任务会向某个方向移动一步，如果玩家在操作一定次数的键盘并且各个方向的步数相同时，此时游戏任务必定会回到原点，则称此次走位为完美走位。

现给定玩家的走位（例如：ASDA），请通过更换其中**一段连续走位的方式**使得原走位能够变成一个完美走位。其中待更换的连续走位可以是相同长度的任何走位。

请返回待更换的连续走位的最小可能长度。

如果原走位本身是一个完美走位，则返回0。

输入描述

输入为由键盘字母表示的走位s，例如：ASDA

输出描述

输出为待更换的连续走位的最小可能长度。

用例

输入	WASDAASD
输出	1
说明	将第二个A替换为W，即可得到完美走位

输入	AAAA
输出	3
说明	将其中三个连续的A替换为WSD，即可得到完美走位

题目解析

题目要求，保持W,A,S,D字母个数平衡，即相等，如果不相等，可以从字符串中选取一段连续子串替换，来让字符串平衡。

比如：WWWWAAAASSSS

字符串长度12，W,A,S,D平衡的话，则每个字母个数应该是3个，而现在W,A,S各有4个，也就是说各超了1个。

因此我们应该从字符串中，选取一段包含1个W，1个A，1个S的子串，来替换为D。

WWWWAAAASSSS

WWWWAAAASSSS

WWWWAAAASSSS

.....

WWWWAAAASSSS

而符合这种要求的子串可能很多，我们需要找出其中最短的，即WAAAAS。

本题其实就是求最小覆盖子串，同[LeetCode - 76 最小覆盖子串 伏城之外的博客-CSDN博客](#)

题目解析请看上面链接博客。

Java算法源码

```
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println(getResult(sc.next()));
    }

    public static int getResult(String str) {
        // count用于记录W,A,S,D字母的数量
        HashMap<Character, Integer> count = new HashMap<>();

        for (int i = 0; i < str.length(); i++) {
            Character c = str.charAt(i);
            count.put(c, count.getOrDefault(c, 0) + 1);
        }

        // 平衡状态时，W,A,S,D应该都是avg数量
        int avg = str.length() / 4;

        // total用于记录多余字母个数
        int total = 0;

        // flag表示当前是否为平衡状态，默认是
        boolean flag = true;

        for (Character c : count.keySet()) {
            if (count.get(c) > avg) {
                // 如果有一个字母数量超标，则平衡打破
                flag = false;
                // 此时count记录每个字母超过avg的数量
                count.put(c, count.get(c) - avg);
                total += count.get(c);
            } else {
                count.put(c, 0); // 此时count统计的其实是多余字母，如果没有超过avg，则表示没有多余
字母
            }
        }

        // 如果平衡，则输出0
        if (flag) return 0;

        int i = 0;
        int j = 0;
        int minLen = str.length() + 1;

        while (j < str.length()) {
            Character jc = str.charAt(j);

            if (count.get(jc) > 0) {
                total--;
            }
        }
    }
}
```

```

        count.put(jc, count.get(jc) - 1);

        while (total == 0) {
            minLen = Math.min(minLen, j - i + 1);

            Character ic = str.charAt(i);
            if (count.get(ic) >= 0) {
                total++;
            }
            count.put(ic, count.get(ic) + 1);

            i++;
        }
        j++;
    }
    return minLen;
}
}

```

58.最多获得的短信条数、云短信平台优惠活动

题目描述

某云短信厂商，为庆祝国庆，推出充值优惠活动。
现在给出客户预算，和优惠售价序列，求最多可获得的短信总条数。

输入描述

第一行客户预算M，其中 $0 \leq M \leq 10^6$
第二行给出售价表， $P_1, P_2, \dots, P_n$ ，其中 $1 \leq n \leq 100$ ，
 P_i 为充值 i 元获得的短信条数。 $1 \leq P_i \leq 1000$ ， $1 \leq n \leq 100$

输出描述

最多获得的短信条数

用例

输入	6 10 20 30 40 60
输出	70
说明	分两次充值最优， 1 元、 5 元各充一次。总条数 $10 + 60 = 70$

输入	15 10 20 30 40 60 60 70 80 90 150
----	-----------------------------------

输入	15 10 20 30 40 60 60 70 80 90 150
输出	210
说明	分两次充值最优，10 元 5 元各充一次，总条数 $150 + 60 = 210$

题目解析

本题是[完全背包问题](#)。

如果大家不是很清楚[完全背包](#)的求解，可以看[算法设计 - 01背包问题的状态转移方程优化，以及完全背包问题 伏城之外的博客-CSDN博客](#)

本题中：

- 客户预算M相当于背包的承重，
- 出售价表：
 1. i 元相当于物品的重量，
 2. P_i 短信条数相当于物品的价值

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = Integer.parseInt(sc.nextLine());
        Integer[] p =
            Arrays.stream(sc.nextLine().split("
")).map(Integer::parseInt).toArray(Integer[]::new);

        System.out.println(getResult(m, p));
    }

    public static int getResult(int m, Integer[] p) {
        int[] dp = new int[m + 1];

        for (int i = 0; i <= p.length; i++) {
            for (int j = 0; j <= m; j++) {
                if (i == 0 || j == 0 || j < i) continue;
                dp[j] = Math.max(dp[j], dp[j - i] + p[i - 1]);
            }
        }

        return dp[m];
    }
}
```

59.最大利润

题目描述

商人经营一家店铺，有number种商品，
由于仓库限制每件商品的`最大持有数量是item[index]`
每种商品的价格是`item-price[item_index][day]`
通过对商品的买进和卖出获取利润
请给出商人在days天内能获取的最大的利润
注：同一件商品可以反复买进和卖出

输入描述

第一行输入商品的数量number，比如3
第二行输入商品售货天数 days，比如3
第三行输入仓库限制每件商品的`最大持有数量是item[index]`，比如4 5 6

后面继续输入number行days列，含义如下：
第一件商品每天的价格，比如1 2 3
第二件商品每天的价格，比如4 3 2
第三件商品每天的价格，比如1 5 3

输出描述

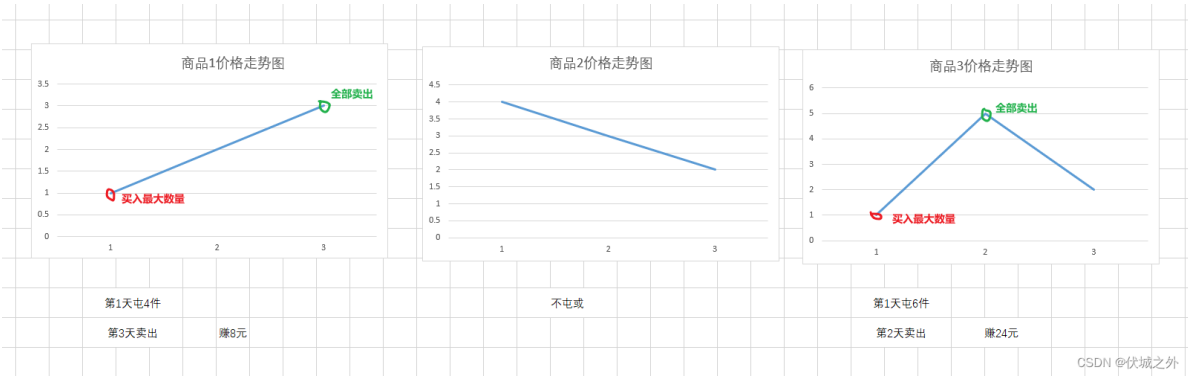
输出商人在这段时间内的最大利润。

用例

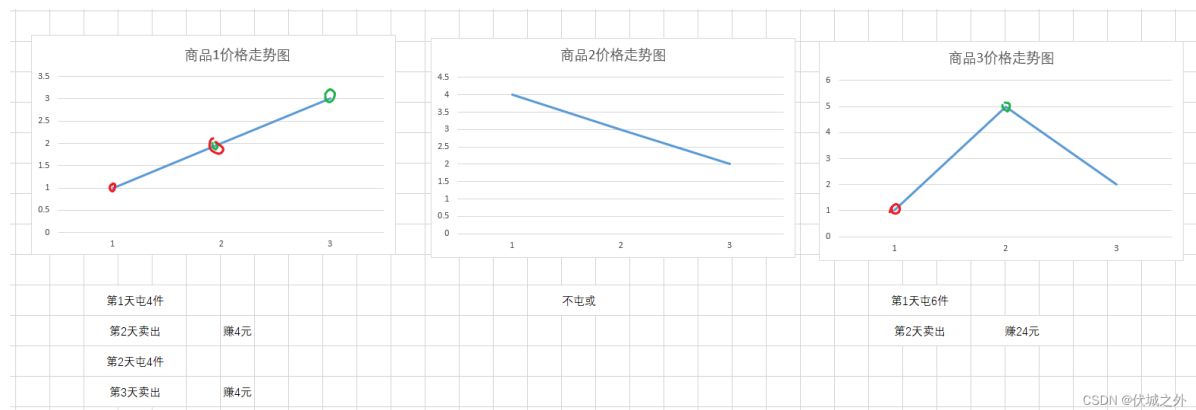
输入	3 3 4 5 6 1 2 3 4 3 2 1 5 2
输出	32
说明	无

题目解析

用例含义如下所示



另外，本题描述说：同一件商品可以反复买进和卖出，因此上面图示买卖操作还可以变为这样：



本题算是[华为机试 - 最大股票收益](#) [伏城之外的博客-CSDN博客](#) 的变种题。

我们只要找到商品价格走势的上升区段，然后低价all in买入，高价all out卖出即可求得最大利润。

那么如何找到上升区段呢？

我们假设商品1的第 i 天的价格为 $price1[i]$ ，那么只要 $price1[i] < price1[i+1]$ ，则说明当前处于上升区段的低价位，因此可以all in，然后到 $i+1$ 天的时候all out。

Java算法源码

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try { // 此段try...catch不影响正常业务逻辑，只是有网友说可能存在如输入行数校验要返回0，
            因此加了
            int number = sc.nextInt();
            int days = sc.nextInt();

            int[] maxCount = new int[number];
            for (int i = 0; i < number; i++) {
                maxCount[i] = sc.nextInt();
            }

            int[][] prices = new int[number][days];
            for (int i = 0; i < number; i++) {
                for (int j = 0; j < days; j++) {
                    prices[i][j] = sc.nextInt();
                }
            }

            System.out.println(getResult(number, days, maxCount, prices));
        } catch (Exception e) {
            System.out.println(0);
        }
    }
}
```

```

/**
 * @param number 几种商品
 * @param days 几天
 * @param maxCount 每种商品的最大囤货数量
 * @param prices 每种商品的在days天内的价格变动情况
 * @return 最大利润
 */
public static int getResult(int number, int days, int[] maxCount, int[][]
prices) {
    int ans = 0;
    for (int i = 0; i < number; i++) {
        int[] price = prices[i];
        for (int j = 0; j < days - 1; j++) {
            if (price[j] < price[j + 1]) {
                ans += (price[j + 1] - price[j]) * maxCount[i];
            }
        }
    }
    return ans;
}
}

```

60.简单的自动曝光、平均像素值

题目描述

一个图像有n个像素点，存储在一个长度为n的数组img里，每个像素点的取值范围[0,255]的正整数。请你给图像每个像素点值加上一个整数k（可以是负数），得到新图newImg，使得新图newImg的所有像素平均值最接近中位值128。请输出这个整数k。

输入描述

n个整数，中间用空格分开

输出描述

一个整数k

备注

- $1 \leq n \leq 100$
- 如有多个整数k都满足，输出小的那个k；
- 新图的像素值会自动截取到[0,255]范围。当新像素值<0，其值会更改为0；当新像素值>255，其值会更改为255；

例如newImg="-1 -2 256",会自动更改为"0 0 255"

用例

输入	0 0 0 0
输出	128
说明	四个像素值都为0

输入	129 130 129 130
输出	-2
说明	-1的均值128.5，-2的均值为127.5，输出较小的数-2

题目解析

本题如果用暴力法求解的话思路如下：

首先输入的老图片的像素值，应该是符合要求的，即像素点应都在[0,255]范围内，因此像素点最小值为0，最大值为255。

那么如果我们想让0接近中位值128，则需要加上128。

如果我们想让255接近中位值128，则需要减去127。

因此，k的取值范围应该在-127到128之间，这样的话，就可以保证每一个点都能接近到中位值。

那么我们就从-127遍历到128，然后将遍历值加到老图片的每一个像素值上，然后[求平均值](#)。

需要注意的是，如果新图片的像素点值低于0，则取0，高于255，则取255

```
function getResult(arr) {
  const ans = [];
  for (let k = -127; k <= 128; k++) {
    const res =
      arr
        .map((num) => {
          const newNum = num + k;
          return newNum < 0 ? 0 : newNum > 255 ? 255 : newNum;
        })
        .reduce((p, c) => p + c) / arr.length;
    ans.push([k, res]);
  }
  console.table(ans);
}

getResult([10, 20, 100, 200, 250]);
```

如上面代码中，我们可以得到各种k加到图片像素点上后的图片像素平均值

135	8	123.4
136	9	124.2
137	10	125
138	11	125.8
139	12	126.6
140	13	127.4
141	14	128.2
142	15	129
143	16	129.8
144	17	130.6
145	18	131.4

上面测试代码结果显示，k取14时，老图片像素点的平均值为128.2，最接近中位值128。

上面[算法时间复杂度](#)是外层256，内层100（ $1 \leq n \leq 100$ ），差不多两三万次循环，可以接受。

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        Integer[] arr = Arrays.stream(str.split("
")).map(Integer::parseInt).toArray(Integer[]::new);

        System.out.println(getResult(arr));
    }

    public static int getResult(Integer[] arr) {
        int len = arr.length;
        double minDiff = Integer.MAX_VALUE;
        Integer ans = null;

        for (int k = -127; k <= 128; k++) {
            double sum = 0;
            for (Integer val : arr) {
                int newVal = val + k;
                // 新图的像素值会自动截取到[0,255]范围。当新像素值<0，其值会更改为0；当新像素值
                >255，其值会更改为255;
                newVal = Math.max(0, Math.min(newVal, 255));
                sum += newVal;
            }

            double diff = Math.abs(sum / len - 128);

            if (diff < minDiff) {
                minDiff = diff;
                ans = k;
            } else if (diff == minDiff && ans != null) {
                // 如有多个整数k都满足，输出小的那个k
                ans = Math.min(ans, k);
            }
        }
    }
}
```

```
    }  
    }  
  
    return ans;  
    }  
}
```

61.插队

题目描述

某银行将客户分为了若干个优先级，1 级最高，5 级最低，当你需要在银行办理业务时，优先级高的人随时可以插队到优先级低的人的前面。

现在给出一个人员到来和银行办理业务的时间序列，请你在每次银行办理业务时输出客户的编号。

如果同时有多位优先级相同且最高的客户，则按照先来后到的顺序办理。

输入描述

输入第一行是一个正整数 n ,表示输入的序列中的事件数量。 ($1 \leq n \leq 500$)

接下来有 n 行，每行第一个字符为 a 或 p 。

当字符为 a 时，后面会有两个的正整数 num 和 x ,表示到来的客户编号为 num ,优先级为 x ；

当字符为 p 时，表示当前优先级最高的客户去办理业务。

输出描述

输出包含若干行，对于每个 p ，输出一行，仅包含一个正整数 num ,表示办理业务的客户编号。

用例

输入	4 个 1 个 3 个 2 个 2 个 3 个 2 个
输出	2
说明	无

题目解析

简单的[优先队列](#)应用。

但是本题需要注意几个问题：

1、客户编号应该不是先来后到的顺序，输入顺序才是

2、会不会存在p数量超过a数量的情况？我这边考虑了这种情况，如果p时，优先队列已经没有客户了，那么就输出空

Java算法源码

```
import java.util.PriorityQueue;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(sc.nextLine());

        String[][] arr = new String[n][];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextLine().split(" ");
        }

        getResult(n, arr);
    }

    public static void getResult(int n, String[][] arr) {
        PriorityQueue<int[]> pq =
            new PriorityQueue<>((a, b) -> a[0] != b[0] ? a[0] - b[0] : a[1] - b[1]);

        for (int i = 0; i < n; i++) {
            String[] tmp = arr[i];
            switch (tmp[0]) {
                case "a":
                    int num = Integer.parseInt(tmp[1]);
                    int x = Integer.parseInt(tmp[2]);
                    pq.offer(new int[] {x, i, num}); // i 是先后到顺序
                    break;
                case "p":
                    int[] poll = pq.poll();
                    if (poll != null) System.out.println(poll[2]);
                    else System.out.println("");
            }
        }
    }
}
```

62.分奖金

题目描述

公司老板做了一笔大生意，想要给每位员工分配一些奖金，想通过游戏的方式来决定每个人分多少钱。

按照员工的工号顺序，每个人随机抽取一个数字。

按照工号的顺序往后排列，遇到第一个数字比自己数字大的，那么，前面的员工就可以获得“距离数字差值”的奖金。

如果遇不到比自己数字大的，就给自己分配随机数数量的奖金。

例如，按照工号顺序的随机数字是：2,10,3。

那么第2个员工的数字10比第1个员工的数字2大，
所以，第1个员工可以获得 $1(10-2) = 8$ 。第2个员工后面没有比他数字更大的员工，
所以，他获得他分配的随机数数量的奖金，就是10。
第3个员工是最后一个员工，后面也没有比他更大数字的员工，所以他得到的奖金是3。
请帮老板计算一下每位员工最终分到的奖金都是多少钱。

输入描述

第一行n表示员工数量（包含最后一个老板）
第二是每位员工分配的随机数字

输出描述

最终每位员工分到的奖金数量

注：随机数字不重复，员工数量（包含老板）范围1~10000，随机数范围1~100000

用例

输入	3 2 10 3
输出	8 10 3
说明	无

题目解析

本题最简单的思路是双重for，但是时间复杂度是 $O(m^2)$ ，而m取值1~10000，这个数量级非常有可能超时。

因此我们需要考虑更优的解法：

上面算法其实就是找每个数组元素的下一个更大值，因此可以参考

[华为机试 - 找朋友 伏城之外的博客-CSDN博客找朋友算法](#)

的解题思路，利用栈结构通过 $O(n)$ 时间，找出数组每一个元素的下一个更大值。

暴力解法（100%通过）

Java算法源码

```
import java.util.ArrayList;
import java.util.Scanner;
import java.util.StringJoiner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();

        int[] arr = new int[m];
        for (int i = 0; i < m; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.println(getResult(arr, m));
    }

    public static String getResult(int[] arr, int m) {
        ArrayList<Integer> ans = new ArrayList<>();

        outer:
        for (int i = 0; i < m; i++) {
            for (int j = i + 1; j < m; j++) {
                if (arr[j] > arr[i]) {
                    ans.add((j - i) * (arr[j] - arr[i]));
                    continue outer;
                }
            }
            ans.add(arr[i]);
        }

        StringJoiner sj = new StringJoiner(" ");
        for (Integer an : ans) sj.add(an + "");
        return sj.toString();
    }
}
```

单调栈解法

Java算法源码

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();

        int[] arr = new int[m];
        for (int i = 0; i < m; i++) {
```

```

        arr[i] = sc.nextInt();
    }

    System.out.println(getResult(arr, m));
}

public static String getResult(int[] arr, int m) {
    LinkedList<Integer[]> stack = new LinkedList<>();
    Integer[] nextBigger = new Integer[m];
    Arrays.fill(nextBigger, -1);

    for (int i = 0; i < arr.length; i++) {
        while (stack.size() > 0) {
            Integer[] top = stack.peek();
            int idx = top[0];
            int val = top[1];

            if (arr[i] > val) {
                stack.pop();
                nextBigger[idx] = i;
            } else {
                break;
            }
        }

        Integer[] ele = {i, arr[i]};
        stack.push(ele);
    }

    ArrayList<Integer> ans = new ArrayList<>();

    for (int i = 0; i < arr.length; i++) {
        Integer idx = nextBigger[i];

        if (idx == -1) {
            ans.add(arr[i]);
        } else {
            ans.add((idx - i) * (arr[idx] - arr[i])); // 距离 * 数字差值
        }
    }

    StringJoiner sj = new StringJoiner(" ", "", "");
    for (Integer an : ans) sj.add(an + "");
    return sj.toString();
}
}

```

63.不含101的数

题目描述

小明在学习二进制时，发现了一类不含 101 的数，也就是：

将数字用二进制表示，不能出现 101。

现在给定一个整数区间 $[l,r]$ ，请问这个区间包含了多少个不含 101 的数？

输入描述

输入的唯一一行包含两个正整数 l, r ($1 \leq l \leq r \leq 10^9$)。

输出描述

输出的唯一一行包含一个整数，表示在 $[l,r]$ 区间内一共有几个不含 101 的数。

用例

输入	1 10
输出	8
说明	区间 $[1,10]$ 内，5 的二进制表示为 101，10 的二进制表示为 1010，因此区间 $[1,10]$ 内有 $10-2=8$ 个不含 101 的数。

输入	10 20
输出	7
说明	区间 $[10,20]$ 内，满足条件的数字有 $[12,14,15,16,17,18,19]$ 因此答案为 7。

题目解析

本题如果用暴力法求解，很简单

```
/* JavaScript Node ACM模式 控制台输入获取 */
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

rl.on("line", (line) => {
  const [l, r] = line.split(" ").map(Number);

  console.log(getResult(l, r));
});
```

```
function getResult(l, r) {
    let count = r - l + 1;
    for (let i = l; i <= r; i++) {
        if (Number(i).toString(2).indexOf("101") !== -1) {
            count--;
        }
    }
    return count;
}
```

但是本题的 $1 \leq l \leq r \leq 10^9$ ，也就是说区间范围最大是 $1 \sim 10^9$ ，那么上面 $O(n)$ 时间复杂度的算法会超时。

因此，我们需要找到一个性能更优的算法。

本题需要使用[数位DP](#)算法，具体逻辑原理请看

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int L = sc.nextInt();
        int R = sc.nextInt();
        int count = digitSearch(R) - digitSearch(L - 1);
        System.out.println(count);
    }

    public static int digitSearch(int num) {
        Integer[] arr =
            Arrays.stream(Integer.toString(num).split(""))
                .map(Integer::parseInt)
                .toArray(Integer[]::new);

        int[][][] f = new int[arr.length][2][2];

        return dfs(0, true, f, arr, 0, 0);
    }

    public static int dfs(int p, boolean limit, int[][][] f, Integer[] arr, int
pre, int prepre) {
        if (p == arr.length) return 1;

        if (!limit && f[p][pre][prepre] != 0) return f[p][pre][prepre];

        int max = limit ? arr[p] : 1;
        int count = 0;

        for (int i = 0; i <= max; i++) {
            if (i == 1 && pre == 0 && prepre == 1) continue;
```

```
count += dfs(p + 1, limit && i == max, f, arr, i, pre);
}

if (!limit) f[p][pre][prepre] = count;

return count;
}
}
```

64.服务中心选址

题目描述

一个快递公司希望在一条街道建立新的服务中心。公司统计了该街道中所有区域在地图上的位置，并希望能够以此为依据为新的服务中心选址：使服务中心到所有区域的距离的总和最小。

给你一个数组positions，其中positions[i] = [left, right] 表示第 i 个区域在街道上的位置，其中left代表区域的左侧的起点，right代表区域的右侧终点，假设服务中心的位置为location：

- 如果第 i 个区域的右侧终点right满足 $right < location$ ，则第 i 个区域到服务中心的距离为 $location - right$ ；
- 如果第 i 个区域的左侧起点left 满足 $left > location$ ，则第 i 个区域到服务中心的距离为 $left - location$ ；
- 如果第 i 个区域的两侧left, right满足 $left \leq location \leq right$ ，则第 i 个区域到服务中心的距离为 0

选择最佳的服务中心位置为location，请返回最佳的服务中心位置到所有区域的距离总和的最小值。

输入描述

第一行，一个整数N表示区域个数。

后面N行，每行两个整数，表示区域的左右起点终点。

输出描述

运行结果输出一个整数，表示服务中心位置到所有区域的距离总和的最小值。

用例

输入	3 1 2 3 4 10 20
输出	8
说明	无

题目解析

本题考察的是：中位数最短距离定理

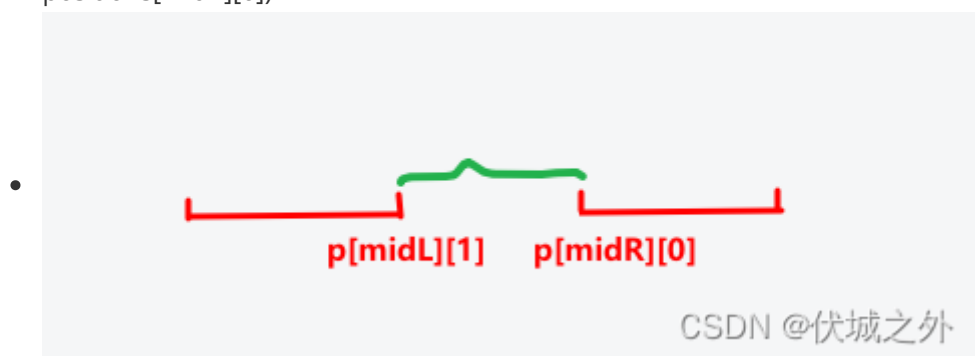
如果你还不知道这个定理，请看：[中位数的性质（数学真是有趣啊）太强了 - xidian mao - 博客园 \(cnblogs.com\)](http://cnblogs.com/xidian-mao/)

而中位数是指按照一定顺序（从大到小或从小到大）排列后中间的一个数（这组数所个数为奇数个时）或中间两个数的平均数（当这组数据为偶数个时），求解公式如下：

- 有序数列arr的元素有奇数n个，那么中点 $\text{mid} = \text{floor}(n/2)$ ，中位数 $\text{midVal} = \text{arr}[\text{mid}]$
- 有序数列arr的元素有偶数n个，那么中点就有两个 $\text{midR} = \text{floor}(n/2)$ ， $\text{midL} = \text{midR} - 1$ ，中位数也有两个，分别是 $\text{arr}[\text{midL}]$ ， $\text{arr}[\text{midR}]$ ，如果中位数可以不是数列元素，那么中位数 $= (\text{arr}[\text{L}] + \text{arr}[\text{R}]) / 2$

本题特殊在，不是依据多个点来求解中位数，而是依据多个区间来求解中位数，这个其实不难，我们只要将多个区间进行排序，然后取中间区间到中位数即为所有区间的中位数：

- 如果区间是奇数n个，那么中间位置就是 $\text{mid} = \text{floor}(n/2)$ ，中间区间就是 $\text{positions}[\text{mid}]$ ，而该区间的中位数 $\text{midPos} = (\text{positions}[\text{mid}][0] + \text{positions}[\text{mid}][1]) / 2$
- 如果区间是偶数n个，那么中间位置就是 $\text{midR} = \text{floor}(n/2)$ ， $\text{midL} = \text{midR} - 1$ ，中间区间就是 $\text{positions}[\text{midL}]$ 和 $\text{positions}[\text{midR}]$ ，这两个区间的中位数 $\text{midPos} = (\text{positions}[\text{midL}][1] + \text{positions}[\text{midR}][0]) / 2$



求得中位数后，接下来就是遍历所有区间 $[l, r]$ ，比较 l, r 和 midPos 的关系

如果 $l > \text{midPos}$ ，那么该区间到中位数的距离为： $l - \text{midPos}$

如果 $r < \text{midPos}$ ，那么该区间到中位数的距离为： $\text{midPos} - r$

如果 $l \leq \text{midPos} \leq r$ ，那么该区间到中位数的距离为：0

累加上面的距离就是题解。

根据网友反馈，上面逻辑的代码通过率为0。

我分析了一下原因，上面代码只针对各区域不存在交集时有效，如果各区域之间存在交集，特别是中间区域存在交集时，则上面逻辑会有问题。

因此，本题中各区域应该是有交集的。

我想了很久，如何求解某个点到有交集区域的最小距离和，但是没有什么好的办法，直到我死心准备用暴力法求解时，发现了一丝丝生机。下面是暴力法测试过程：

测试用例：含区域交集情况

```
11
-10 10
1 2
3 4
5 10
6 8
7 12
9 13
15 20
31 41
22 35
34 50
```

Java暴力实现

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        double[][] positions = new double[n][2];
        for (int i = 0; i < n; i++) {
            positions[i][0] = sc.nextInt();
            positions[i][1] = sc.nextInt();
        }

        System.out.println(getResult(n, positions));
    }

    public static double getResult(int n, double[][] positions) {
        ArrayList<Double> tmp = new ArrayList<>();
        for (double[] pos : positions) {
            tmp.add(pos[0]);
            tmp.add(pos[1]);
        }
        tmp.sort(Double::compareTo);

        double min = tmp.get(0);
        double max = tmp.get(tmp.size() - 1);
        double ans = Double.MAX_VALUE;

        for (double i = min; i <= max; i += 0.5) {
            double dis = 0;
            for (double[] pos : positions) {
```



```

        double l = pos[0];
        double r = pos[1];
        if (r < i) dis += i - r;
        else if (i < l) dis += l - i;
    }

    System.out.println(i + "\t" + dis);

    ans = Math.min(ans, dis);
}

return ans;
}
}

```

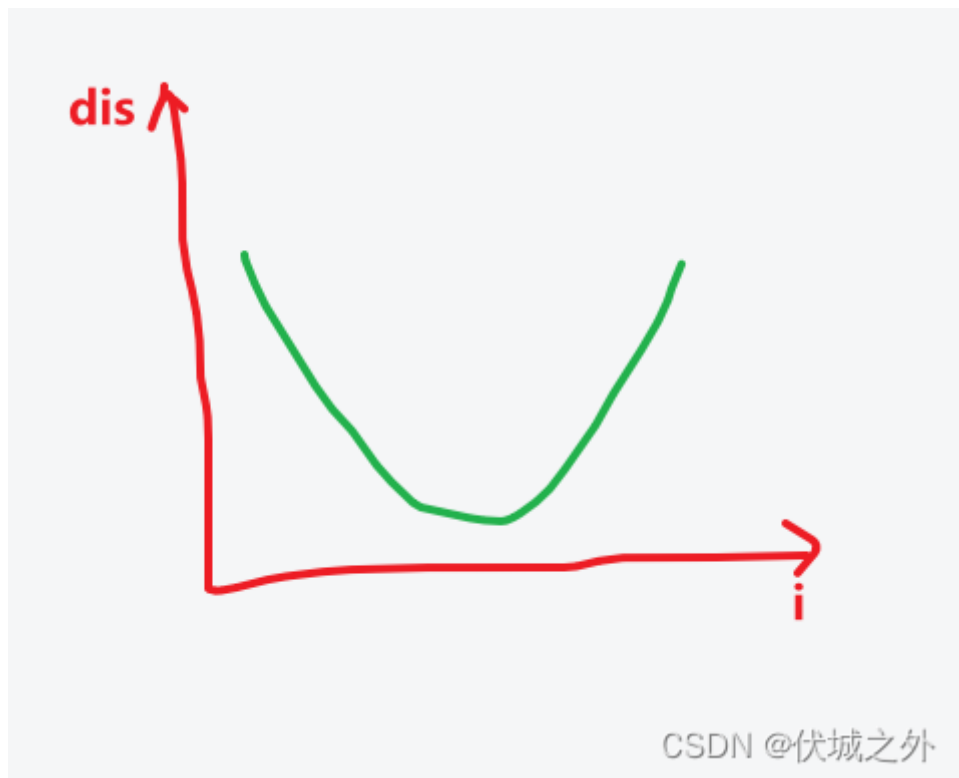
The screenshot shows an IDE with a Java file named 'Main.java' and a 'Run' window displaying the output. The code defines a method 'getResult' that takes the number of points 'n' and a 2D array 'positions' of their coordinates. It iterates through each point 'i' as a potential service center, calculates the sum of distances 'dis' to all other points, and keeps track of the minimum 'ans'.

The 'Run' window shows the following output:

i	dis
1.0	110.0
2.0	114.0
2.5	110.0
3.0	106.0
3.5	102.5
4.0	99.0
4.5	96.0
5.0	93.0
5.5	90.5
6.0	88.0
6.5	86.0
7.0	84.0
7.5	82.5
8.0	81.0
8.5	80.0
9.0	79.0
9.5	78.5
10.0	78.0
10.5	78.5
11.0	79.0
11.5	79.5
12.0	80.0
12.5	81.0
13.0	82.0
13.5	83.5
14.0	85.0
14.5	86.5
15.0	88.0
15.5	90.0
16.0	92.0
16.5	94.0
17.0	96.0
17.5	98.0
18.0	100.0
18.5	102.0
19.0	104.0
19.5	106.0
20.0	108.0
20.5	110.5
21.0	113.0
21.5	115.5

The minimum value, 78.0 at i=10.0, is highlighted with a red box in the original image.

上面暴力法过程中，我首先获得了所有区间中最左边的点min，和最右边的点max，并遍历这两个点之间每一个点作为服务中心地址 i，并求每个服务中心地址到各区域的距离之和 dis，然后将它们成对打印出来，结果发现一个现象：



随着 服务中心位置 i 的变化，服务中心到各区域的距离之和 dis 呈现上图U型曲线。

即，一定存在一个 i ，其左边点 $i-0.5$ 的，和其右边点 $i+0.5$ 到各区域的距离和大于它。

因此，我想是否可以用[二分法](#)求解，即取min点和max点的中间点mid作为服务中心位置，：

1. 如果 $dis(mid - 0.5) \geq dis(mid)$ && $dis(mid + 0.5) \geq dis(mid)$ ，则 mid 就是所求的点
2. 如果 $dis(mid - 0.5) > dis(mid) > dis(mid + 0.5)$ ，则说明当前 mid 点处于上图的下降区间，此时我们应该将 mid 作为新的min值，然后重新取min，max的中间点作为新 mid
3. 如果 $dis(mid - 0.5) < dis(mid) < dis(mid + 0.5)$ ，则说明当前 mid 点处于上图的上升区间，此时我们应该将 mid 作为新的max值，然后重新取min，max的中间点作为新 mid

这样一搞，我们最终就可以找到最低 dis 的 mid 点。

Java算法源码

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        double[][] positions = new double[n][2];
        for (int i = 0; i < n; i++) {
            positions[i][0] = sc.nextInt();
            positions[i][1] = sc.nextInt();
        }

        System.out.println(getResult(n, positions));
    }
}
```

```

public static double getResult(int n, double[][] positions) {
    ArrayList<Double> tmp = new ArrayList<>();
    for (double[] pos : positions) {
        tmp.add(pos[0]);
        tmp.add(pos[1]);
    }
    tmp.sort(Double::compareTo);

    double min = tmp.get(0);
    double max = tmp.get(tmp.size() - 1);

    while (min < max) {
        double mid = Math.ceil((min + max) / 2);

        double midDis = getDistance(mid, positions);
        double midLDis = getDistance(mid - 0.5, positions);
        double midRDis = getDistance(mid + 0.5, positions);

        if (midDis <= midLDis && midDis <= midRDis) {
            return midDis;
        }

        if (midDis < midLDis) {
            min = mid + 0.5;
            continue;
        }

        if (midDis < midRDis) {
            max = mid - 0.5;
        }
    }

    return 0;
}

public static double getDistance(double t, double[][] positions) {
    double dis = 0;
    for (double[] pos : positions) {
        double l = pos[0];
        double r = pos[1];
        if (r < t) dis += t - r;
        else if (t < l) dis += l - t;
    }
    return dis;
}
}

```

65.优雅子数组

题目描述

如果一个数组中出现次数最多的元素出现大于等于K次，被称为 *k-优雅数组*，k也可以被称为优雅阈值。
例如，数组1, 2, 3, 1、2, 3, 1，它是一个3-优雅数组，因为元素1出现次数大于等于3次，
数组[1, 2, 3, 1, 2]就不是一个3-优雅数组，因为其中出现次数最多的元素是1和2，只出现了2次。

给定一个数组A和k，请求出A有多少子数组是k-优雅子数组。

子数组是数组中一个或多个连续元素组成的数组。

例如，数组[1,2,3,4]包含10个子数组，分别是：
[1], [1,2], [1,2,3], [1,2,3,4], [2], [2,3], [2,3,4], [3], [3, 4], [4]。

输入描述

第一行输入两个数字，以空格隔开，含义是：A数组长度 k值

第二行输入A数组元素，以空格隔开

输出描述

输出A有多少子数组是k-优雅子数组

用例

输入	7 3 1 2 3 1 2 3 1
输出	1
说明	无

输入	7 2 1 2 3 1 2 3 1
输出	10
说明	无

题目解析

我的解题思路如下：

利用[双指针](#)（即一个双重for）找到所有子数组（有点暴力），外层 i 指针指向子数组左边界，内层 j 指针指向子数组右边界，然后统计子数组内部各数字出现个数，若有数字出现次数大于等于k，则该子数组符合要求，统计结果ans++。

上面算法逻辑中，找所有子数组的逻辑基本无法优化，但是统计每个子数组内部各数字出现次数的逻辑却可以优化。

当j指针移动到尾巴时，就可以i++，然后开始新一轮的统计，即统计以索引i元素开头的所有子数组中符合要求的个数，例如

1	2	3	1	2	3	1	count{2:1}
	↑↑						
1	2	3	1	2	3	1	count{2:1, 3:1}
	↑	↑					
1	2	3	1	2	3	1	count{2:1, 3:1, 1:1}
	↑		↑				
1	2	3	1	2	3	1	count{2:2, 3:1, 1:1}
	↑			↑			

$n - j = 7 - 4 = 3 \uparrow$

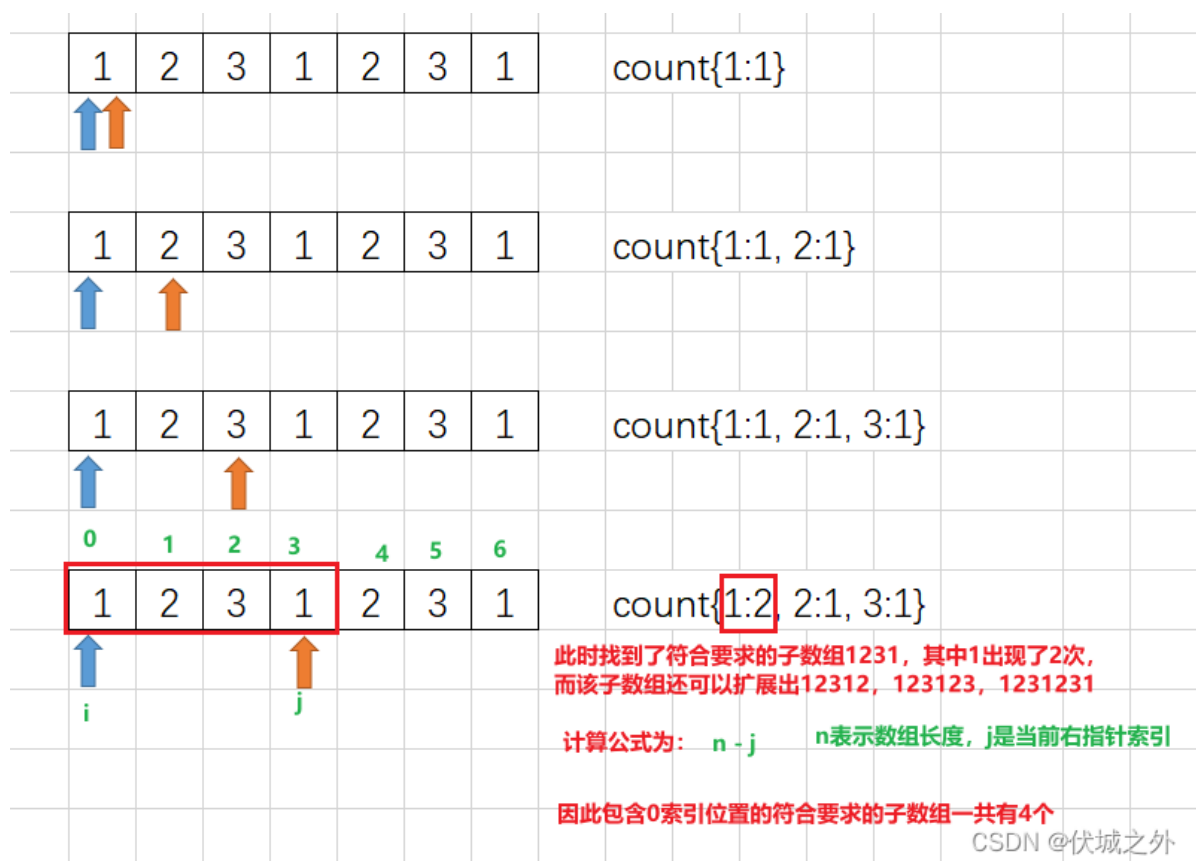
CSDN @伏城之外

这种前缀思想可以避免大量的重复统计工作。

同时，每轮统计我们只需要一个对象保存统计结果，一轮统计结束，则丢弃统计结果，不会占用太多内存。

2023.02.22 上面逻辑有机考同学反馈，通过率只有33%，后面用例不通过原因是超时。

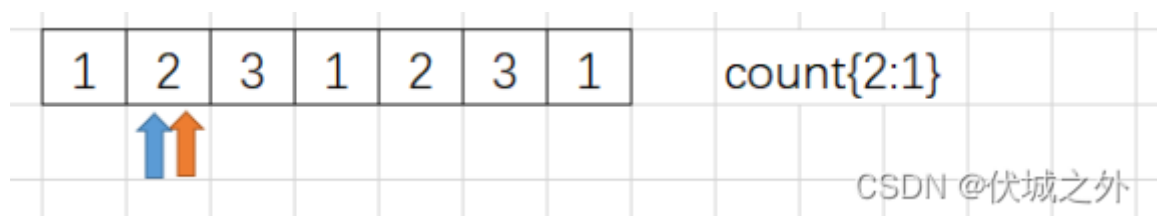
其实上面逻辑超时的原因很容易发现，就是暴力枚举了所有子数组，因此必须优化子数组获取逻辑，和一位网友讨论后，发现上面逻辑有一个可以优化的点



那就是上图最找到符合要求的子串后，下一步的 i, j 指针运动逻辑，按照前面逻辑思路是：

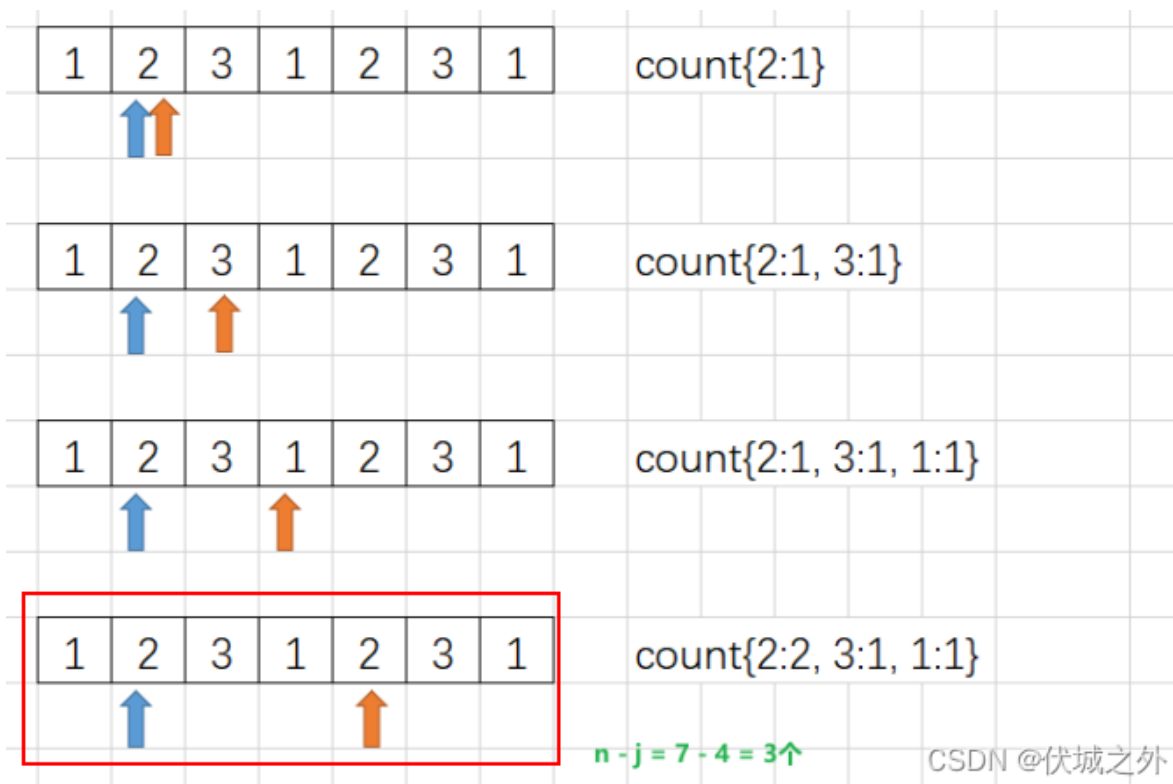
- i 后移一位
- j 重新指向 i 新位置

即如下图所示，此时count也重新统计

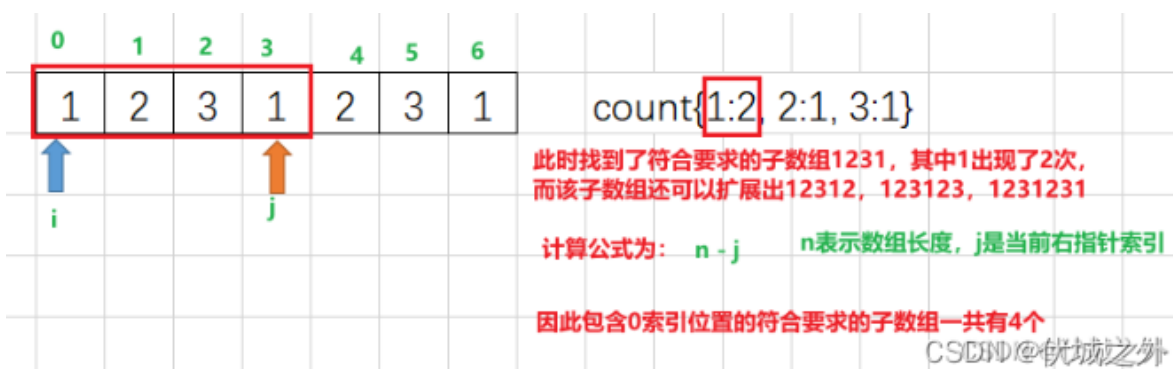


但是真的有必要吗？

我们继续往下看，按照上面逻辑，我们必然会走到下图画红圈这一步，然后重新找到符合要求的子串



但是我们再回到下图第一轮时找到符合要求子串时的图示

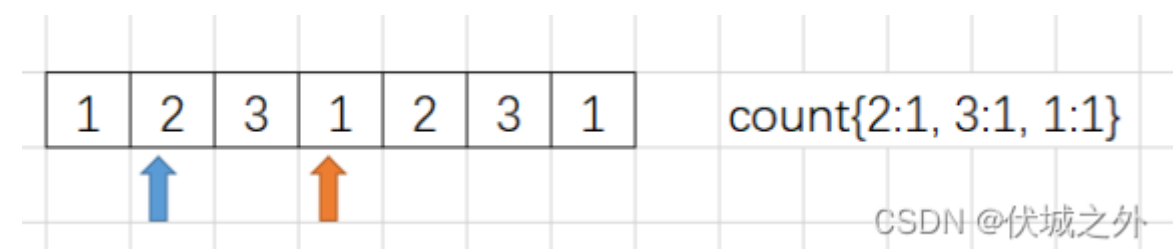


有没有发现什么端倪呢？

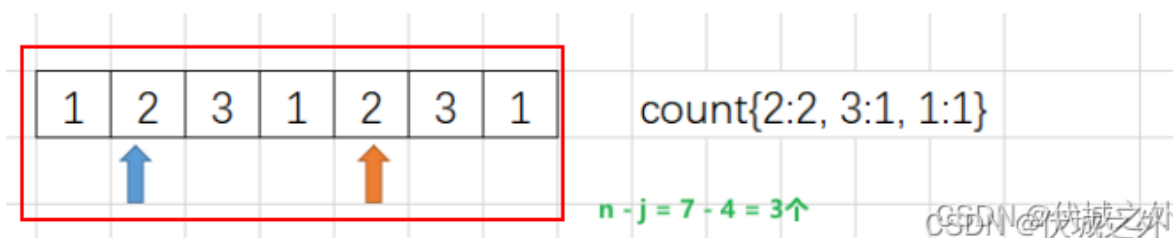
其实，我们在第一轮结束时，不需要将 i, j 重头开始，而是可以直接：

- $i++$

即变为下图



然后下一步 $j++$ ，很快就会进入，前面第二轮经过多次指针运动后才能进入的状态

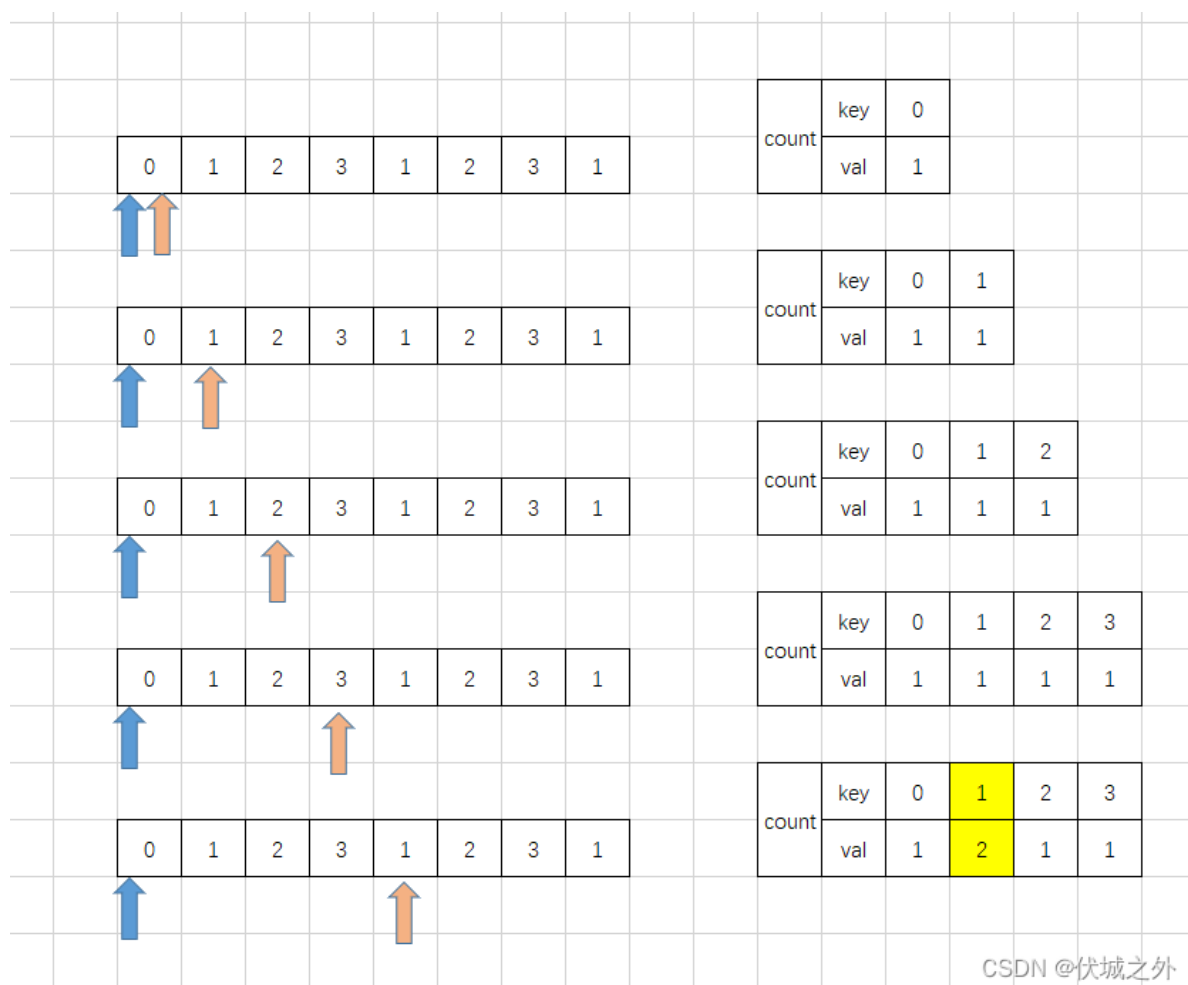


这个逻辑就非常节省时间了。

但是上面逻辑还是不够完善，比如大家看下面这个用例：

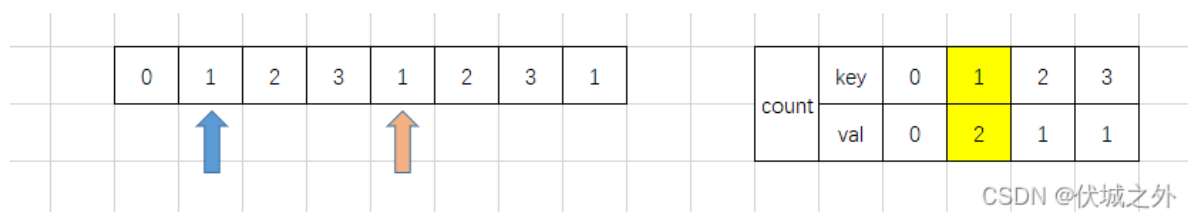
8 2
0 1 2 3 1 2 3 1

我按照上面新逻辑画图演示下双指针运动过程



当走到最后一步时，我们就找到符合要求的子数组。

那么按照前面逻辑，下一步我们应该 $i++$ （注意 $i++$ 的时，对应的子数组就失去一个 $arr[i]$ ，因此对应 $count[arr[i]]$ 数量要减少）



然后又发现了符合要求的子数组，这一步在逻辑上没有问题，但是在代码实现上可能会发生问题，比如看下面伪代码：

```
l = 0 // 左指针
r = 0 // 右指针

// n是数组arr长度
while(l < n && r < n) {
```

```

c = arr[r] // r指针指向的数组元素就是将要加入的左右指针内部的元素

// count用于统计左右指针内部（子数组）c字符出现的次数
count[c]++

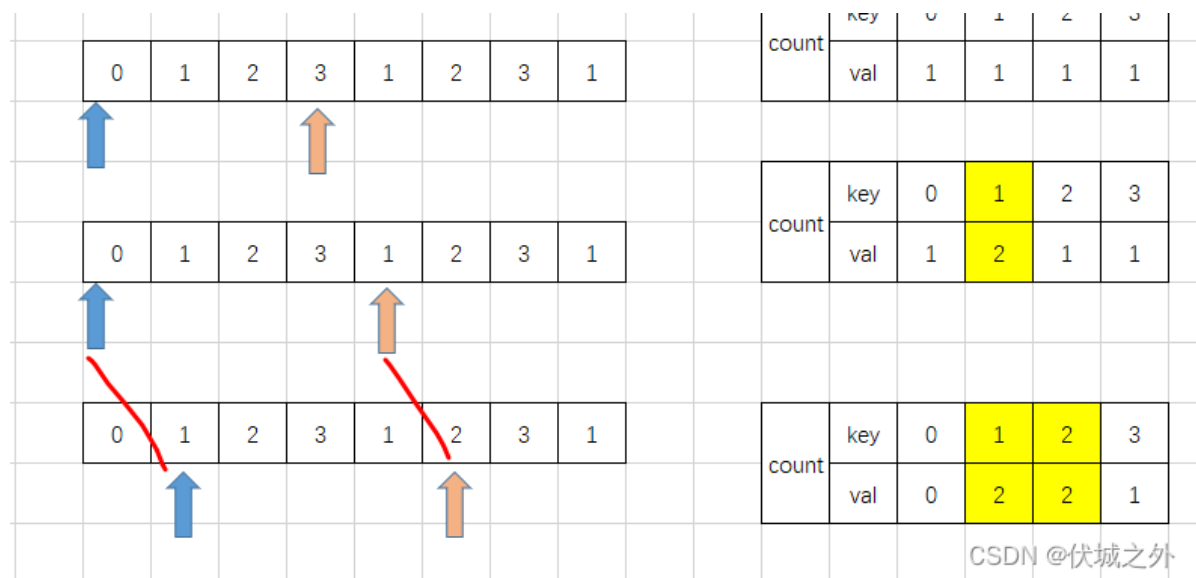
// k阈值
if(count[c] >= k) {
    ans += n - r // 发现符合要求的子数组后，可以拓展出n-r个符合要求的子数组

    count[arr[l]]-- // 左指针右移一格，但是右移前要减去左指针指向字符的统计次数
    l++
}
r++ // 右指针移动
}

```

上面代码大家有发现问题吗？

问题出在左指针L右移一格后，右指针R也右移了一格，即如下图所示



CSDN @伏城之外

此时其实就遗漏部分符合要求的子数组情况。大家可以对照前面正确运动过程想一下。

那么该怎么改呢？

```

l = 0 // 左指针
r = 0 // 右指针

// n是数组arr长度
while(l < n && r < n) {
    c = arr[r] // r指针指向的数组元素就是将要加入的左右指针内部的元素

    // count用于统计左右指针内部（子数组）c字符出现的次数
    count[c]++

    // k阈值
    if(count[c] >= k) {
        ans += n - r // 发现符合要求的子数组后，可以拓展出n-r个符合要求的子数组

        count[arr[l]]-- // 左指针右移一格，但是右移前要减去左指针指向字符的统计
        l++

        count[c]--
        r--
    }
    r++ // 右指针移动
}

```

CSDN @伏城之外

即在R指针右移前，做一次左移即可。

Java算法源码

```

import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int k = sc.nextInt();

        Integer[] arr = new Integer[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.println(getResult(arr, n, k));
    }

    public static Integer getResult(Integer[] arr, Integer n, Integer k) {
        int ans = 0;

        int l = 0;
        int r = 0;
        HashMap<Integer, Integer> count = new HashMap<>();

        while (l < n && r < n) {
            Integer c = arr[r];
            count.put(c, count.getDefault(c, 0) + 1);
            if (count.get(c) >= k) {
                ans += n - r;

                count.put(c, count.get(c) - 1);
                l++;
            }
            r++;
        }

        return ans;
    }
}

```

```
count.put(arr[l], count.get(arr[l]) - 1);
l++;

count.put(c, count.get(c) - 1);
r--;
}
r++;
}

return ans;
}
}
```

66.农场施肥

题目描述

某农场主管理了一大片果园， $fields[i]$ 表示不同果林的面积，单位： m^2 ，现在要为所有的果林施肥且必须在 n 天之内完成，否则影响收成。小布是果林的工作人员，他每次选择一片果林进行施肥，且一片果林施肥完后当天不再进行施肥作业。

假设施肥机的能效为 k ，单位： m^2/day ，请问至少租赁能效 k 为多少的施肥机才能确保不影响收成？如果无法完成施肥任务，则返回-1。

输入描述

第一行输入为 m 和 n ， m 表示 $fields$ 中的元素个数， n 表示施肥任务必须在 n 天内（含 n 天）完成；

第二行输入为 $fields$ ， $fields[i]$ 表示果林 i 的面积，单位： m^2

输出描述

对于每组数据，输出最小施肥机的能效 k ，无多余空格。

备注

- $1 \leq fields.length \leq 10^4$
- $1 \leq n \leq 10^9$
- $1 \leq fields[i] \leq 10^9$

用例

输入	5 7 5 7 9 15 10
输出	9

输入	5 7 5 7 9 15 10
说明	当能效k为9时，fields[0]需要1天，fields[1]需要1天，fields[2]需要1天，fields[3]需要2天，fields[4]需要2天，共需要7天，不会影响收成。

输入	3 1 2 3 4
输出	-1
说明	由于一天最多完成一片果林的施肥，无论k为多少都至少需要3天才能完成施肥，因此返回-1。

题目解析

本题中能效k其实和果林面积fields[i]是强相关的，比如用例1中fields = [5,7,9,15,10]：

能效k取5的话，则面积5的果林只需要1天，其他大于面积5的果林需要 $\text{Math.ceil}(\text{fields}[i] / k)$

能效k取7的话，则面积5、7的果林只需要1天，其他大于面积7的果林需要 $\text{Math.ceil}(\text{fields}[i] / k)$

因此，k的取值范围其实就是果林最小面积min ~ 最大面积max之间，我们完全可以遍历min~max之间的所有可能给k，然后求出能效k施肥完所有果林需要的天数spend，然后求出spend===n的所有情况中最小的k作为题解。

但是，本题min~max之间有 $1 \leq \text{fields}[i] \leq 10^9$ ，并且求解每种k对应的spend都需要遍历 $1 \leq \text{fields.length} \leq 10^4$ 次，因此上面算法的时间复杂度是 $10^9 * 10^4 = 10^{13}$ ，这是必然超时的。

因此，我们可以采用[二分查找](#)的方式来找k，即每次取min、max的中间值作为k，如果k能效需要的spend时间和指定天数n时间的关系：

- spend > n：说明k能效太低了，花费的时间超过了n，因此要提高k，即min = k，这样下次二分查找的值就会增大k
- spend < n：说明k能效太高了，花费的时间少于n，因此要降低k，即max = k，这样下次二分查找的值就会减小k
- spend == n：说明k能效刚刚好，花费的时间等于n，但是此时可能并非最优解，我们还需继续找更小的k，即max = k
- 2023.02.07 经网友指正，输入描述中说：施肥任务必须在n天内（含n天），即施肥任务可以少于n天时间完成，因此spend < n时的k也是符合要求的k。因此spend < n和spend == n的情况可以合并处理，即当 spend <= n时，说明k能效足够了，花费的时间小于等于n，但是此时可能并非最优解，我们还需继续找更小的k，即max = k

此时，[算法时间复杂度](#)为 $\log(10^9) * 10^4 = 9 * 10^4$ ，大大的优化了。

关于本题K的取小数还是取整数的思考，请大家先看下面的用例：

3 5

1 1 10

如果取小数的话，上面用例K的最小取值是无穷小数3.3333....
因此，K值如果取小数的话，必然要说明精度，但是本题没有。
因此，个人认为本题K应该取整数。
当然考试的时候，还是要灵活应对。

补充一个用例，关于初始二分，左边界的取值

```
1 10
9
```

即只有一个果林，面积为9，现在要求10天完成施肥，此时最低能效K应该为1。

因此，前面逻辑中，二分查找能效K时，应该从1~max中二分，而不是min~max，即能效K的最低值可能为1。

另外，本题已经找到原型题

[875. 爱吃香蕉的珂珂 - 力扣 \(LeetCode\)](#)

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
        int n = sc.nextInt();

        int[] fields = new int[m];
        for (int i = 0; i < m; i++) {
            fields[i] = sc.nextInt();
        }

        System.out.println(getResult(n, fields));
    }

    /**
     * @param n 表示施肥任务必须在n天内（含n天）
     * @param fields fields[i]表示果林 i 的面积
     * @return 最小施肥机的能效 k
     */
    public static int getResult(int n, int[] fields) {
        double min = 1; // 最小果林面积
        double max = Arrays.stream(fields).max().orElse(0); // 最大果林面积

        int ans = -1;

        // 我们需要找min,max中间值作为k效率，如果min,max间距小于1，则没有中间值，循环结束
        while (min <= max) {
```

```

        int k = (int) Math.ceil((min + max) / 2);

        int res = check(k, n, fields);

        if (res > 0) min = k + 1; // k的效率太低，导致花费的时间超过了n天，因此要提高k效率
        else {
            ans = k; // k的效率太高或刚好，花费的时间<=n天，则此时k效率就是一个题解，但可能不是
            最优解
            max = k - 1; // 继续尝试找更小的k
        }
    }

    return ans;
}

/**
 * @param k 能效k
 * @param n 指定天数
 * @param fields 所有果林面积
 * @return 基于能效k需要spend天施肥完所有果林，返回spend - n
 */
public static int check(int k, int n, int[] fields) {
    int spend = 0;
    for (int field : fields) {
        if (k >= field) spend++; // k效率比field果林面积大，则field果林只需要一天
        else spend += Math.ceil(field / (double) k); // k效率比field果林面积小，则需要
        Math.ceil(field / k)天
    }

    return spend - n;
}
}

```

67.无向图染色

题目描述

给一个无向图染色，可以填红黑两种颜色，必须保证相邻两个节点不能同时为红色，输出有多少种不同的染色方案？

输入描述

第一行输入M(图中节点数) N(边数)

后续N行格式为：V1 V2表示一个V1到V2的边。

数据范围：1 <= M <= 15, 0 <= N <= M * 3，不能保证所有节点都是连通的。

输出描述

输出一个数字表示染色方案的个数。

用例

输入	4 4 1 2 2 4 3 4 1 3
输出	7
说明	4个节点，4条边，1号节点和2号节点相连，2号节点和4号节点相连，3号节点和4号节点相连，1号节点和3号节点相连，若想必须保证相邻两个节点不能同时为红色，总共7种方案。

输入	3 3 1 2 1 3 2 3
输出	4
说明	无

输入	4 3 1 2 2 3 3 4
输出	8
说明	无

输入	4 3 1 2 1 3 2 3
输出	8
说明	无

题目解析

2022.12.25 更正解析说明，感谢Andy__Zhong指出错误。

本题其实就是求解[连通图](#)的染色方案，

目前我想到的最好方式是暴力法，即通过[回溯算法](#)，求解出染红节点的全组合，

n个数的全组合数量一共有 $(2^n) - 1$ 。

比如：1,2,3的全组合情况有：1、2、3、12、13、23、123，即 $(2^3) - 1 = 7$ 个。

本题中节点一共有m个，而 $1 \leq m \leq 15$ ，即最多有 $(2^{15}) - 1 = 32767$ 个组合情况，这个数量级不算多。因此暴力法可行。

我们需要尝试对组合中的节点进行染红色，但是相邻节点不能同时染成红色。因此，在求解全组合时，还可以进行剪枝优化，即判断新加入的节点 是否和 已存在的节点相邻，如果相邻，则剪枝，如果不相邻则继续递归。

```
// 连通图的染色方案数求解
function getDyeCount(arr, m) {
  // connect用于存放每个节点的相邻节点
  const connect = {};

  for (let [v1, v2] of arr) {
    connect[v1] ? connect[v1].add(v2) : (connect[v1] = new Set([v2]));
    connect[v2] ? connect[v2].add(v1) : (connect[v2] = new Set([v1]));
  }

  // 必有一种全黑的染色方案
  let count = 1;

  // 求解染红节点的全组合情况
  function dfs(m, index, path) {
    if (path.length === m) return;

    outer: for (let i = index; i <= m; i++) {
      // 如果新加入节点和已有节点相邻，则说明新加入节点不能染成红色，需要进行剪枝
      for (let j = 0; j < path.length; j++) {
        if (path[j].has(i)) continue outer;
      }

      count++;

      path.push(connect[i]);
      dfs(m, i + 1, path);
      path.pop();
    }
  }

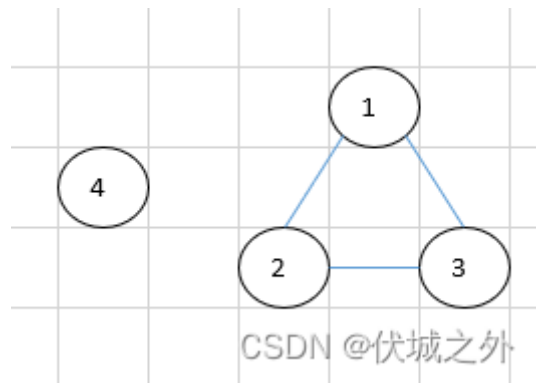
  // 节点从1开始
  dfs(m, 1, []);

  return count;
}
```

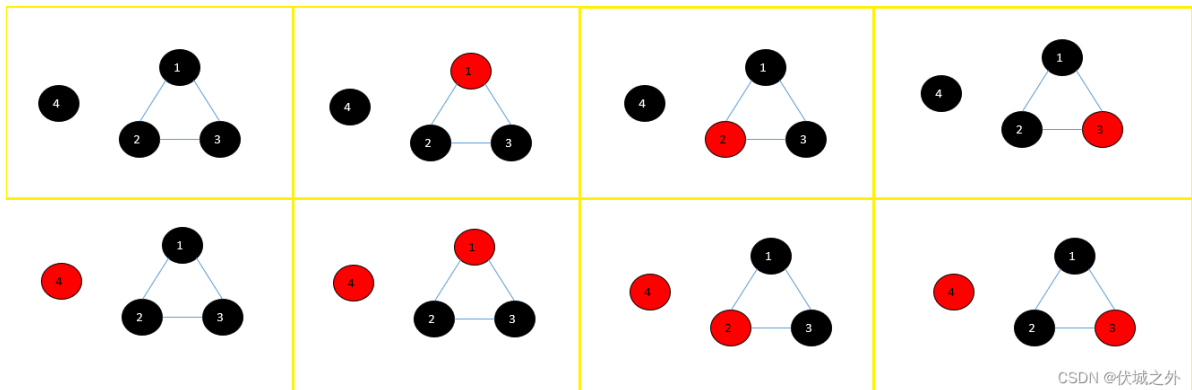
本题，到此还未结束，因为题目中有一句话：

不能保证所有节点都是连通的

这说明什么呢？即对应用例4的情况，用例4对应的无向图如下：



此时一共有8种染色方案如下：



其实就是先求解无向图的各个连通分量，比如用例4的无向图就有两个连通分量，分别是：

- {4}
- {1, 2, 3}

然后求解各连通分量各自的染色方案，比如

- {4} 有2种染色方案
- {1, 2, 3} 有4种染色方案

那么总染色方案数目就是 $2 \times 4 = 8$ 种

因此，本题还考察了连通分量的求解。

连通分量的求解可以使用并查集，关于并查集知识请看：[华为机试 - 发广播 伏城之外的博客-CSDN博客](#)

但是本题实现上可以取巧，即不需要使用并查集去求解连通分量，而是完全依赖于暴力，因为不管节点是否在一个连通分量中，还是不在一个连通分量中，他们的染色都要满足：

相邻节点不能同时为红色

因此，处于两个连通分量中的节点必然不相连，则必然可以同时染红，因此直接用前面求染红节点组合就可以，不需要用并查集。

补充一个边界用例情况：

4 3
2 3
2 4
3 4

输出应该是8

但是节点1和任何其他节点不相连，也没有在边，因此下面代码，统计connect时，即统计每个节点的相邻节点，必然统计不到节点1，即connect[1]的值为null，因此后续获取节点1的相邻节点时会得到null，此时我们应该要特殊处理null。

Java算法源码

直接利用节点间相邻关系暴力枚举所有染色方案。该方案实现上更简单，但是性能没有基于并查集求出各连通分量后分别求解染色方案的好。

```
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
        int n = sc.nextInt();

        int[][] edges = new int[n][2];
        for (int i = 0; i < n; i++) {
            edges[i][0] = sc.nextInt();
            edges[i][1] = sc.nextInt();
        }

        System.out.println(getResult(edges, m));
    }

    /**
     * @param edges 边，即[v1, v2]
     * @param m 点数量
     * @return
     */
    public static int getResult(int[][] edges, int m) {
        // connect用于存放每个节点的相邻节点
        HashMap<Integer, HashSet<Integer>> connect = new HashMap<>();

        for (int[] edge : edges) {
            connect.putIfAbsent(edge[0], new HashSet<>());
            connect.get(edge[0]).add(edge[1]);

            connect.putIfAbsent(edge[1], new HashSet<>());
            connect.get(edge[1]).add(edge[0]);
        }
    }
}
```

```

// 节点从index=1开始，必有count=1个的全黑染色方案
return dfs(connect, m, 1, 1, new LinkedList<>());
}

// 该方法用于求解给定多个节点染红的全组合数
public static int dfs(
    HashMap<Integer, HashSet<Integer>> connect,
    int m,
    int index,
    int count,
    LinkedList<HashSet<Integer>> path) {
    if (path.size() == m) return count;

    outer:
    for (int i = index; i <= m; i++) {
        // 如果新加入节点i和已有节点j相邻，则说明新加入节点不能染成红色，需要进行剪枝
        for (HashSet<Integer> p : path) {
            if (p.contains(i)) continue outer;
        }

        count++;

        if (connect.containsKey(i)) {
            path.addLast(connect.get(i));
            count = dfs(connect, m, i + 1, count, path);
            path.removeLast();
        } else {
            count = dfs(connect, m, i + 1, count, path);
        }
    }

    return count;
}
}

```

68.荒地

题目描述

祖国西北部有一片大片荒地，其中零星的分布着一些湖泊，保护区，矿区；

整体上常年光照良好，但是也有一些地区光照不太好。

某电力公司希望在这里建设多个光伏电站，生产清洁能源对每平方公里的土地进行了发电评估，其中不能建设的区域发电量为0kw，可以发电的区域根据光照，地形等给出了每平方公里年发电量x千瓦。

我们希望能够找到其中集中的矩形区域建设电站，能够获得良好的收益。

输入描述

第一行输入为调研的地区长，宽，以及准备建设的电站【长宽相等，为正方形】的边长最低要求的发电量
之后每行为调研区域每平方公里的发电量

输出描述

输出为这样的区域有多少个

用例

输入	2 5 2 6 1 3 4 5 8 2 3 6 7 1
输出	4
说明	输入含义：调研的区域大小为长2宽5的矩形，我们要建设的电站的边长为2，建设电站最低发电量为6.输出含义：长宽为2的正方形满足发电量大于等于6的区域有4个。

输入	2 5 1 6 1 3 4 5 8 2 3 6 7 1
输出	3
说明	无

题目解析

本题和[华为OD机试 - 探索地块建立_伏城之外的博客-CSDN博客](#)

一模一样。题解请参考该博客。

还是有区别的，本题中有一句比较有歧义的话

其中不能建设的区域发电量为0kw

这里不能建设是什么意思？不能建设发电站吗？还是说单纯只是产出0发电量？

如果含义是不能建设发电站，那意思应该是，正方形发电站区域如果含0的话，则不能建设发电站。

如果含义是单纯发电量0，那应该没有什么影响。

下面我源码中，我对这两种情况都做了实现了。考试时，大家可以都试试。

Java算法源码

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int r = sc.nextInt();
        int c = sc.nextInt();

        int s = sc.nextInt();
        int min = sc.nextInt();

        int[][] matrix = new int[r][c];
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        System.out.println(getResult(matrix, r, c, s, min));
    }

    /**
     * @param matrix 调研区域每单元面积的发电量矩阵
     * @param r 调研区域的长
     * @param c 调研区域的宽
     * @param s 正方形电站的边长
     * @param min 正方形电站的最低发电量
     * @return 调研区域内有几个符合要求的正方形发电站
     */
    public static int getResult(int[][] matrix, int r, int c, int s, int min) {
        // 压缩后的行数
        int zip_r = r - s + 1;
        // 压缩后的列数
        int zip_c = c - s + 1;

        // 先进行列压缩
        int[][] zip_col_dps = new int[r][zip_c];

        for (int i = 0; i < matrix.length; i++) {
            int[] row = matrix[i];

            for (int j = 0; j < s; j++) {
                zip_col_dps[i][0] += row[i];
            }

            for (int j = 1; j < zip_c; j++) {
                zip_col_dps[i][j] = zip_col_dps[i][j - 1] - row[j - 1] + row[j + s - 1];
            }
        }

        // 更新矩阵
        matrix = zip_col_dps;
    }
}
```

```

int ans = 0;

// 再进行行压缩
int[][] zip_col_row_dps = new int[zip_r][zip_c];

for (int j = 0; j < zip_c; j++) {
    for (int i = 0; i < s; i++) {
        zip_col_row_dps[0][j] += matrix[i][j];
    }
    // 压缩后的每一个区块都是一个边长为s的正方形，只要其发电量>=min即可
    if (zip_col_row_dps[0][j] >= min) ans++;

    for (int i = 1; i < zip_r; i++) {
        zip_col_row_dps[i][j] = zip_col_row_dps[i - 1][j] - matrix[i - 1][j] +
matrix[i + s - 1][j];
        // 压缩后的每一个区块都是一个边长为s的正方形，只要其发电量>=min即可
        if (zip_col_row_dps[i][j] >= min) ans++;
    }
}

return ans;
}
}

```

69.最大化控制资源成本

题目描述

公司创新实验室正在研究如何最小化资源成本，最大化资源利用率，请你设计算法帮他们解决一个任务混部问题：

有taskNum项任务，每个任务有开始时间（startTime），结束时间（endTime），并行度（parallelism）三个属性，

并行度是指这个任务运行时将会占用的服务器数量，一个服务器在每个时刻可以被任意任务使用但最多被一个任务占用，任务运行完立即释放（结束时刻不占用）。

任务混部问题是指给定一批任务，让这批任务由同一批服务器承载运行，

请你计算完成这批任务混部最少需要多少服务器，从而最大化控制资源成本。

输入描述

第一行输入为taskNum，表示有taskNum项任务

接下来taskNum行，每行三个整数，表示每个任务的

开始时间（startTime），结束时间（endTime），并行度（parallelism）

输出描述

一个整数，表示最少需要的服务器数量

备注

- `1 <= taskNum <= 100000`
- `0 <= startTime < endTime <= 50000`
- `1 <= parallelism <= 100`

用例

输入	3 2 3 1 6 9 2 0 5 1
输出	2
说明	一共有三个任务，第一个任务在时间区间[2, 3]运行，占用1个服务器，第二个任务在时间区间[6, 9]运行，占用2个服务器，第三个任务在时间区间[0, 5]运行，占用1个服务器，需要最多服务器的时间区间为[2, 3]和[6, 9]，需要2个服务器。

输入	2 3 9 2 4 7 3
输出	5
说明	一共两个任务，第一个任务在时间区间[3, 9]运行，占用2个服务器，第二个任务在时间区间[4, 7]运行，占用3个服务器，需要最多服务器的时间区间为[4, 7]，需要5个服务器。

题目解析

本题其实是求解最大区间重叠个数的变种题。

关于最大区间重叠个数求解，请看[华为OD机试 - 人数最多的站点 伏城之外的博客-CSDN博客](#)

本题并不是要求解最大区间重叠个数，而是要求解重叠区间的权重和。因此，我们需要定义一个变量sum来记录重叠区间的权重和，当[小顶堆](#)弹出不重叠区间时，sum需要减去被弹出区间的权重，当我们向小顶堆压入重叠区间时，则sum需要加上被压入区间的权重。

Java算法源码

```
import java.util.Arrays;
import java.util.PriorityQueue;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```



```

int n = sc.nextInt();

int[][] ranges = new int[n][3];

for (int i = 0; i < n; i++) {
    ranges[i][0] = sc.nextInt();
    ranges[i][1] = sc.nextInt();
    ranges[i][2] = sc.nextInt();
}

System.out.println(getResult(ranges));
}

public static int getResult(int[][] ranges) {
    Arrays.sort(ranges, (a, b) -> a[0] - b[0]);

    PriorityQueue<Integer[]> end = new PriorityQueue<>((a, b) -> a[0] - b[0]);

    int max = 0;
    int sum = 0;
    for (int[] range : ranges) {
        int s = range[0];
        int e = range[1];
        int p = range[2];

        while (end.size() > 0) {
            Integer[] top = end.peek();

            if (top[0] <= s) {
                Integer[] poll = end.poll();
                sum -= poll[1];
            } else {
                break;
            }
        }

        end.offer(new Integer[] {e, p});
        sum += p;

        if (sum > max) {
            max = sum;
        }
    }
    return max;
}
}

```

70.最佳对手

题目描述

游戏里面，队伍通过匹配实力相近的对手进行对战。但是如果匹配的队伍实力相差太大，对于双方游戏体验都不会太好。

给定n个队伍的实力值，对其进行两两实力匹配，两支队伍实例差距在允许的最大差距d内，则可以匹配。

要求在匹配队伍最多的情况下匹配出的各组实力差距的总和最小。

输入描述

第一行，n，d。队伍个数n。允许的最大实力差距d。

- $2 \leq n \leq 50$
- $0 \leq d \leq 100$

第二行，n个队伍的实力值空格分割。

- $0 \leq \text{各队伍实力值} \leq 100$

输出描述

匹配后，各组对战的实力差值的总和。若没有队伍可以匹配，则输出-1。

用例

输入	6 30 81 87 47 59 81 18
输出	57
说明	18与47配对，实力差距29 59与81配对，实力差距22 81与87配对，实力差距6 总实力差距29+22+6=57

输入	6 20 81 87 47 59 81 18
输出	12
说明	最多能匹配成功4支队伍。 47与59配对，实力差距12， 81与81配对，实力差距0。总实力差距12+0=12

输入	4 10 40 51 62 73
输出	-1
说明	实力差距都在10以上， 没有队伍可以匹配成功。

动态规划求解

题目解析

这题我的动态规划思路是参考的[LeetCode - 198 打家劫舍 伏城之外的博客-CSDN博客](#)

首先，我们将实力数组进行升序，比如自测用例

5 5

1 2 10 18 20

实力数组升序后如下：

[1, 2, 10, 18, 20]

此时，两两相邻的队伍之间匹配可得最小实力差值，比如1和2匹配，要比1和其他队伍匹配获得的实力差更小，比如2和1匹配，2和10匹配，2队伍的最小实力差只会在这两个中产生。

因此，我们需要对上面两两相邻的队伍尝试进行匹配，比如n个队伍，就可以产生n-1个匹配，上面用例可得匹配如下：

- [1, 2]
- [2, 10]
- [10, 18]
- [18, 20]

另外题目要求，匹配队伍的实力差不能大于d，因此上面匹配中，[2,10]、[10,18]匹配是不符合要求的需要剔除。

我们将上面匹配再转化一下，变为实力差即为：

- 1
- 8
- 8
- 2

此时，一个实力差即为一个匹配。

接下来，就是将本题逻辑向leetcode 198靠拢的过程。

在leetcode 198 [打家劫舍](#)中，题意是这么描述的：

有相连的多个屋子，每个屋子都有钱，我们用数组nums表示，而小偷偷盗时，不能连着偷两个相邻的屋子，比如偷了第i间屋子，那么第i+1间屋子就不能再偷了，否则会触发警报。

如果我们将本题的实力差数组，类比为打家劫舍的nums，是不是有一样的限制逻辑呢？即我选择第i个实力差了，就不能再选择第i+1个实力差，原因是：

- 第i个实力差 等价于 $a[i] - a[i-1]$ ，即第i个队伍和第i-1个队伍进行匹配
- 第i+1个实力差 等价于 $a[i+1] - a[i]$ ，即第i+1个队伍和第i个队伍进行匹配

此时，我们发现实力差数组元素不能连着选的原因是，相邻的实力差有重叠的队伍。

因此本题实力差数组和leetcode打家劫舍一样，要跳着选。

首先，请大家先去把leetcode 打家劫舍看会了，再来看接下来的解析。

[LeetCode - 198 打家劫舍 伏城之外的博客-CSDN博客](#)

如果，你打家劫舍已经看明白了。那么就可以继续看下面的解析了。

本题中，两两相连队伍匹配产生的实力差非常有可能大于d，即不符合要求，此时无疑增加了我们“跳着”选的逻辑设计难度，即不仅要考虑相邻实力差是否已经选过了，还要考虑当前实力差是否大于d，即是否可用，如果不可用还要继续跳。

为了减小难度，我们应该将实力差分段，即

- 1
- 8
- 8
- 2

上面两个8的实力差是不符合要求的，那么我们就将实力差分段，分为[1], [2]，然后针对每一段求解匹配最多，实力差之和最小的选择方式。

这样就容易很多了。

好的，到了这里，基本上，本题已经解决大半了，接下来是本题的动态规划转移方程的推导。

本题第i个实力差选与不选，并不完全等价于leetcode 打家劫舍的 第i个屋子偷与不偷的逻辑。

因为本题要求：

匹配队伍最多的情况下匹配出的各组实力差距的总和最小

即：实力差选择最多的情况下，实力差之和还要最小。

因此，我们需要定义一个二维dp。

- dp[i][0] 记录：0~i中，已选了几个实力差，即实力差数量
- dp[i][1] 记录：0~i中，已选的实力差的和

我们需要首先保证dp[i][0]最大，即选与不选第i个实力差时，选择能够使dp[i][0]最大的，如果选与不选对应的dp[i][0]相同，则再选择能够使dp[i][1]最小的。

本题的动态规划方程具体请看源码，源码中已添加了详细的注释说明。

Java算法源码

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int d = sc.nextInt();

        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.println(getResult(n, d, arr));
    }

    /**
     * @param n 队伍个数n
     * @param d 允许的最大实力差距d
     * @param arr n个队伍的实力值数组
     * @return 匹配队伍最多的情况下匹配出的各组实力差距的总和最小
     */
    public static int getResult(int n, int d, int[] arr) {
        // 实力数组升序
        Arrays.sort(arr);

        // ans记录各组实力差之和
        int ans = 0;

        // 此flag标记是否没有队伍可以匹配，true表示没有队伍可以匹配，此时应该返回-1
        boolean flag = true;

        // 下面这段逻辑是为了进行相邻实力组队，并且进行分段组队，分段的规则如下例子：
        /**
         比如用例： 5 5 1 2 10 18 20 其中1和2可以组队，但是2和10不能组队 其中18和20可以组队，
         但是10和18不能组队 因此，这里可以将实力数组分为两段：1 2 以及
         18 20，然后针对每一段进行相邻两队尝试匹配，并计算出实力差（可以认为一个实力差值就是一个组
         队匹配）
         为什么要做上面这步逻辑呢？ 因为，向往leetcode 198 打家劫舍 的逻辑靠拢
         */

        // segment用于保存分段
        ArrayList<Integer> segment = new ArrayList<>();
        for (int i = 1; i < n; i++) {
            // 相邻两队的实力差diff
            int diff = arr[i] - arr[i - 1];

            // 如果diff大于d，那么说明 i - 1 无法和 i 组队匹配
            if (diff > d) {
                // 此时分段开始
            }
        }
    }
}
```

```

        if (segment.size() > 0) {
            flag = false;
            ans += getMaxCountMinSum(segment); // 计算分段部分的组队匹配的：匹配队伍最多的
            情况下匹配出的各组实力差距的最小总和
            segment = new ArrayList<>(); // 开始记录新的分段
        }
    } else {
        segment.add(diff); // 如果diff不大于d，那么说明 i - 1 可以和 i 组队匹配，此时将
        实力差（相当于一个组队匹配）计入分段
    }
}

// 最后一个分段也要记得处理，上面逻辑无法将最后一个分段处理到
if (segment.size() > 0) {
    flag = false;
    ans += getMaxCountMinSum(segment);
}

if (flag) {
    return -1;
}

return ans;
}

/**
 * 此函数逻辑参考leetcode 198 打家劫舍
 *
 * @param segment 记录的都是两两相邻队伍的匹配实力差，比如队伍实力数组为：1 2 5 8 9，则对
    应的segment = [1, 3, 3, 1]，即 [2-1, 5-2,
    *      8-5, 9-8]，一个实力差就相当于一个组队匹配，这里和打家劫舍的相同点在于：小偷不能同时
    偷相邻的屋子
    * @return 匹配队伍最多的情况下匹配出的各组实力差距的最小总和
    */
public static int getMaxCountMinSum(ArrayList<Integer> segment) {
    int n = segment.size();

    // dp[i][0] 表示在0 ~ i 实力差中，匹配队伍的最多数量（优先级更高）
    // dp[i][1] 表示在0 ~ i 实力差中，匹配队伍的最小实力差之和（优先级低）
    int[][] dp = new int[n][2];

    // 如果只有一个实力差，即一个匹配队伍，那么就只能选择，
    dp[0][0] = 1; // 此时匹配队伍的数量为1
    dp[0][1] = segment.get(0); // 此时匹配队伍的实力差之和，就是这个匹配队伍的实力差

    if (n == 1) return dp[0][1];

    // 如果有两个实力差，即两个匹配队伍，那么只能二选一
    dp[1][0] = 1; // 此时匹配队伍的数量依旧为1
    dp[1][1] = Math.min(segment.get(0), segment.get(1)); // 此时应该选择实力差更小的
    那支匹配队伍

    if (n == 2) return dp[1][1];

    // 如果有多个实力差，即多支匹配队伍

```

```

    for (int i = 2; i < n; i++) {
        // 第 i 个队伍选择的话（第 i 间屋子偷的话），此时会多出1个匹配队伍数量，但是第 i - 1
        个队伍就不能选择了，接下来考虑第 i - 2 个队伍的选与不选（偷与不偷）
        int steal_count = dp[i - 2][0] + 1; // 匹配队伍数量+1
        int steal_val = dp[i - 2][1] + segment.get(i); // 匹配队伍的实力差之和 +
        segment.get(i)

        // 第 i 个队伍不选的话（第 i 间屋子不偷的话），此时匹配队伍数量不变，但是第 i - 1 个队
        伍就可以选择了
        int not_steal_count = dp[i - 1][0];
        int not_steal_val = dp[i - 1][1];

        // 此时不同于leetcode 198的是，本题需要的是：匹配队伍最多的情况下匹配出的各组实力差距的
        最小总和

        // 如果选第i个匹配得到的最终匹配数量 > 不选第i个匹配得到的最终匹配数量，那么因为题目要求
        匹配队伍最多，因此我们应该选第i个匹配
        if (steal_count > not_steal_count) {
            dp[i][0] = steal_count;
            dp[i][1] = steal_val;
        }
        // 如果选第i个匹配得到的最终匹配数量 == 不选第i个匹配得到的最终匹配数量，那么因为题目要
        求：匹配队伍最多的情况下匹配出的各组实力差距的最小总和
        else if (steal_count == not_steal_count) {
            // 因此实力差之和越小的，就应该被选
            if (steal_val < not_steal_val) {
                dp[i][0] = steal_count;
                dp[i][1] = steal_val;
            } else {
                dp[i][0] = not_steal_count;
                dp[i][1] = not_steal_val;
            }
        } else {
            dp[i][0] = not_steal_count;
            dp[i][1] = not_steal_val;
        }
    }

    return dp[n - 1][1];
}
}

```

组合求解（超时）

题目解析

下面逻辑代码通过率65%，原因是求解组合过程超时

本题要求两两组队，并且两个队伍的实力差值必须小于等于d才能组队。

假设现在有4队，实力值分别为 10, 20, 30, 40，而d=20

因此10可以和20组队，也可以和30组队，但是不能和40组队。

那么这里，我们到底是让10和20组队，还是让10和30组队呢？

假设，10和20组队了，那么剩下的只能30和40组队

假设，10和30组队了，那么剩下的只能20和40组队

现在，题目要求：

要求在匹配队伍最多的情况下匹配出的各组实力差距的总和最小。

这里有两个条件，但是最优先的是要匹配队伍最多

那么上面例子中，两种组队方式，哪种能产生最多匹配队伍呢？

这里其实就是贪心思维，如果10和30组队，那么20和40组队的风险就增加了，因为30和40组队要比20和40组队的风险更低，即实力差值更小，更有可能组队成功。

因此，本题，我们可以先将队伍按照实力值从小到大排序，比如用例1：

18 47 59 81 81 87

然后，求出所有相邻队伍实力差值，并只保留实力差之小于等于d的组合，例如用例（升序后）可得如下组合：

- [0, 1, 29] // 含义是：队伍0（18实力）和队伍1（47实力）组合，实力差为29
- [1, 2, 12]
- [2, 3, 22]
- [3, 4, 0]
- [4, 5, 6]

然后开始进行组合之间的组合，而组合之间进行组合的条件，即不存在重复队伍，比如

[0, 1, 29] 和 [1, 2, 12] 不能进行组合，因为有重复队伍1。

最终我们会得到多个多组合，此时只要取组合数最多的，如果存在组合数相同的多组合，则看实力差值最小的。

增加一个用例：

5 5
1 2 5 8 9

答案应该是2

Java算法源码

下面代码65%通过率，原因是超时

```
import java.util.*;  
  
public class Main {
```



```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    int d = sc.nextInt();

    int[] arr = new int[n];
    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
    }

    System.out.println(getResult(n, d, arr));
}

public static int getResult(int n, int d, int[] arr) {
    Arrays.sort(arr);

    ArrayList<Integer[]> diffs = new ArrayList<>();

    for (int i = 1; i < arr.length; i++) {
        int diff = arr[i] - arr[i - 1];
        if (diff <= d) diffs.add(new Integer[] {i - 1, i, diff});
    }

    if (diffs.size() == 0) {
        return -1;
    }

    ArrayList<Integer[]> res = new ArrayList<>();
    dfs(0, diffs, new LinkedList<>(), res);

    res.sort((a, b) -> Objects.equals(a[0], b[0]) ? a[1] - b[1] : b[0] - a[0]);
    return res.get(0)[1];
}

public static void dfs(
    int index, ArrayList<Integer[]> diffs, LinkedList<Integer[]> path,
    ArrayList<Integer[]> res) {
    for (int i = index; i < diffs.size(); i++) {
        if (path.size() == 0 || path.getLast()[1] < diffs.get(i)[0]) {
            path.add(diffs.get(i));
            dfs(i + 1, diffs, path, res);

            int count = path.size();
            int sumDiff = path.stream().map(e -> e[2]).reduce((p, c) -> p +
c).orElse(0);

            res.add(new Integer[] {count, sumDiff});

            path.removeLast();
        }
    }
}

```

71.最小调整顺序次数、特异性双端队列

题目描述

有一个特异性的双端队列，该队列可以从头部或尾部添加数据，但是只能从头部移出数据。

小A依次执行2n个指令往队列中添加数据和移出数据。其中n个指令是添加数据（可能从头部添加、也可能从尾部添加），依次添加1到n；n个指令是移出数据。

现在要求移除数据的顺序为1到n。

为了满足最后输出的要求，小A可以在任何时候调整队列中数据的顺序。

请问 小A 最少需要调整几次才能够满足移除数据的顺序正好是1到n；

输入描述

第一行一个数据n，表示数据的范围。

接下来的2n行，其中有n行为添加数据，指令为：

- "head add x" 表示从头部添加数据 x，
- "tail add x" 表示从尾部添加数据x，

另外 n 行为移出数据指令，指令为："remove" 的形式，表示移出1个数据；

$1 \leq n \leq 3 * 10^5$ 。

所有的数据均合法。

输出描述

一个整数，表示 小A 要调整的最小次数。

用例

输入	5 head add 1 tail add 2 remove head add 3 tail add 4 head add 5 remove remove remove remove
输出	1
说明	无

题目解析

本题重在题目意思理解，本题最后要求：最小的调整顺序次数。而不是最小的交换次数。因此本题的难度大大降低了。

比如用例：

1	head add 1	queue = [1]
2	tail add 2	queue = [1,2]
3	remove	此时删除头部，顺序符合要求，因此不需要调整顺序，删除后，queue=[2]
4	head add 3	queue = [3,2]
5	tail add 4	queue = [3,2,4]
6	head add 5	queue = [5,3,2,4]
7	remove	此时删除头部的元素应该是2，但实际是5，因此需要调整顺序，queue=[2,3,4,5]，然后再删除头部，queue = [3,4,5]
8	remove	此时删除头部3
9	remove	此时删除头部4
10	remove	此时删除头部5

因此，只需要在第7步调整一次顺序。

本题不需要模拟出一个队列，因为那样需要频繁的验证队列元素顺序，以及调整顺序，非常不划算。

我们可以总结规律：

如果队列为空，即size==0，此时无论head add，还是tail add，都不会破坏队列顺序性。

如果队列不为空，即size!=0，此时tail add不会破坏顺序性，head add会破坏顺序性。

我们定义一个变量isSorted表示队列是否有序，初始时isSorted = true，表示初始时队列有序。当有序性被破坏，即isSorted = false。

head add和tail add会导致size++，remove会导致size--。

我们定义一个count变量来记录调整顺序的次数，初始为0。

当remove时，如果isSorted为false，则我们需要调整顺序，即count++，并更新isSorted = true。

当head add时，如果size为0，则不破坏顺序性，isSorted为true，如果size不为0，则会破坏顺序性，即isSorted=false。另外size++。

当tail add时, 仅size++。

Java算法源码

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(sc.nextLine());
        int m = n * 2;

        String[] cmds = new String[m];
        for (int i = 0; i < m; i++) {
            cmds[i] = sc.nextLine();
        }

        System.out.println(getResult(cmds));
    }

    public static int getResult(String[] cmds) {
        int size = 0;
        boolean isSorted = true;
        int count = 0;

        for (int i = 0; i < cmds.length; i++) {
            String cmd = cmds[i];
            if (cmd.startsWith("head add")) {
                if (size > 0 && isSorted) isSorted = false;
                size++;
            } else if (cmd.startsWith("tail add")) {
                size++;
            } else {
                if (size == 0) continue;
                if (!isSorted) {
                    count++;
                    isSorted = true;
                }
                size--;
            }
        }

        return count;
    }
}
```

72.匿名信

题目描述

电视剧《分界线》里面有一个片段，男主为了向警察透露案件细节，且不暴露自己，于是将报刊上的字减下来，剪拼成匿名信。

现在又一名举报人，希望借鉴这种手段，使用英文报刊完成举报操作。

但为了增加文章的混淆度，只需满足每个单词中字母数量一致即可，不关注每个字母的顺序。

解释：单词on允许通过单词no进行替代。

报纸代表newspaper，匿名信代表anonymousLetter，求报纸内容是否可以拼成匿名信。

输入描述

第一行输入newspaper内容，包括1-N个字符串，用空格分开

第二行输入anonymousLetter内容，包括1-N个字符串，用**空格**分开。

newspaper和anonymousLetter的字符串由小写英文字母组成，且每个字母只能使用一次；

newspaper内容中的每个字符串字母顺序可以任意调整，但必须保证字符串的完整性（每个字符串不能有多余字母）

$1 < N < 100,$

$1 \leq \text{newspaper.length}, \text{anonymousLetter.length} \leq 10^4$

输出描述

如果报纸可以拼成匿名信返回true，否则返回false

用例

输入	ab cd ab
输出	true
说明	无

输入	ab ef aef
输出	false
说明	无

输入	ab bcd ef cbd fe
输出	true
说明	无

输入	ab bcd ef cd ef
输出	false

输入	ab bcd ef cd ef
说明	无

题目解析

用例1的意思是：

报纸上有两个单词：ab、cd，而要写的匿名信需要一个单词ab，因此可以直接使用报纸上的单词ab，所以可以写出匿名信。

用例2的意思是：

报纸上有两个单词：ab、ef，而要写的匿名信需要一个aef，根据题目意思

只需满足每个单词中字母数量一致即可

如果想用报纸上的单词，代替匿名信上的单词，则这两个单词的字母数量必须一致。

因此，对于匿名信单词aef，有三个字母，而报纸上没有有三个字母的单词，因此输出false。

我增加一个自测用例，说明下面这个特点

不关注每个字母的顺序。单词on允许通过单词no进行替代。

比如，报纸单词ba、cd，匿名信需要单词ab，而题目说

不关注每个字母的顺序

因此匿名信的单词ab可以用报纸的单词ba代替，因此输出true。

本题需要关注下数量级：1 <= newspaper.length, anonymousLetter.length <= 10⁴

因此双重for的O(n²)时间复杂度会高达一亿次循环，会超时。

我的策略如下，统计出newspaper中每个单词出现的次数到count对象中，统计前，对每个单词进行字典序排序，这样就可以忽略字母顺序了。

比如，用例1的newspaper会统计出count: { "ab":1, "cd": 1}，这个时间复杂度是O(n)

然后，再遍历anonymousLetter每个单词，并对单词进行字典序排序，忽略字母顺序，然后用排序后单词去count中找，如果count[letter] > 0，则可以找到，然后count[letter]--

遵循题目意思：每个字母只能使用一次

如果count[letter]不存在，或者count[letter]===0，则说明匿名信中的某个单词在报纸单词中找不到替代，因此可以直接返回false。

如果全部都可以找到，则返回true。

这个过程的时间复杂度是O(n)。

因此总的时间复杂度是O(n)。

Java算法源码

```
import java.util.Arrays;
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String[] newspaper = sc.nextLine().split(" ");
        String[] anonymousLetter = sc.nextLine().split(" ");

        System.out.println(getResult(newspaper, anonymousLetter));
    }

    public static boolean getResult(String[] newspaper, String[] anonymousLetter)
    {
        HashMap<String, Integer> count = new HashMap<>();
        for (String str : newspaper) {
            String newStr = strSort(str);
            count.put(newStr, count.getDefault(newStr, 0) + 1);
        }

        boolean flag = true;
        for (String str : anonymousLetter) {
            String newStr = strSort(str);

            if (count.containsKey(newStr) && count.get(newStr) > 0) {
                count.put(newStr, count.get(newStr) - 1);
            } else {
                flag = false;
                break;
            }
        }

        return flag;
    }

    public static String strSort(String str) {
        char[] cArr = str.toCharArray();
        Arrays.sort(cArr);
        return new String(cArr);
    }
}
```

73.最优高铁城市修建方案

题目描述

高铁城市圈对人们的出行、经济的拉动效果明显。每年都会规划新的高铁城市圈建设。

在给定：城市数量，可建设高铁的两城市间的修建成本列表、以及结合城市商业价值会固定建设的两城市建高铁。

请你设计算法，达到修建城市高铁的最低成本。

注意，需要满足城市圈内城市间两两互联可达(通过其他城市中转可达也属于满足条件)。

输入描述

- 第一行，包含此城市圈中城市的数量、可建设高铁的两城市间修建成本列表数量、必建高铁的城市列表。三个数字用空格间隔。
- 可建设高铁的两城市间的修建成本列表，为多行输入数据，格式为3个数字，用空格分隔，长度不超过1000。
- 固定要修建的高铁城市列表，是上面参数2的子集，可能为多行，每行输入为2个数字，以空格分隔。

城市id从1开始编号，建设成本的取值为正整数，取值范围均不会超过1000

输出描述

修建城市圈高铁的最低成本，正整数。如果城市圈内存在两城市之间无法互联，则返回-1。

用例

输入	3 3 0 1 2 10 1 3 100 2 3 50
输出	60
说明	3 3 0城市圈数量为3，表示城市id分别为1,2,3 1 2 10城市1，2间的高铁修建成本为10 1 3 100城市1，3间的高铁修建成本为100 2 3 50城市2，3间的高铁修建成本为50 满足修建成本最低，只需要建设城市1，2间，城市2，3间的高铁即可，城市1，3之间可通过城市2中转来互联。这样最低修建成本就是60。

输入	3 3 1 1 2 10 1 3 100 2 3 50 1 3
输出	110
说明	无

题目解析

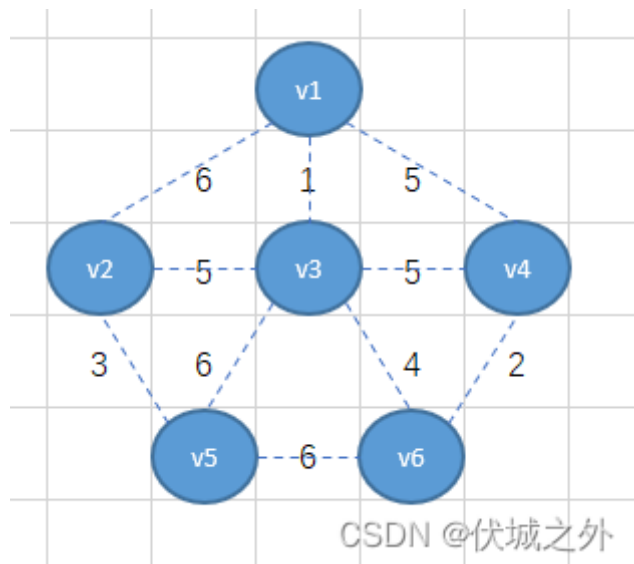
本题是求解[最小生成树](#)的变种题。

在了解最小生成树概念前，我们需要先了解[生成树](#)的概念：

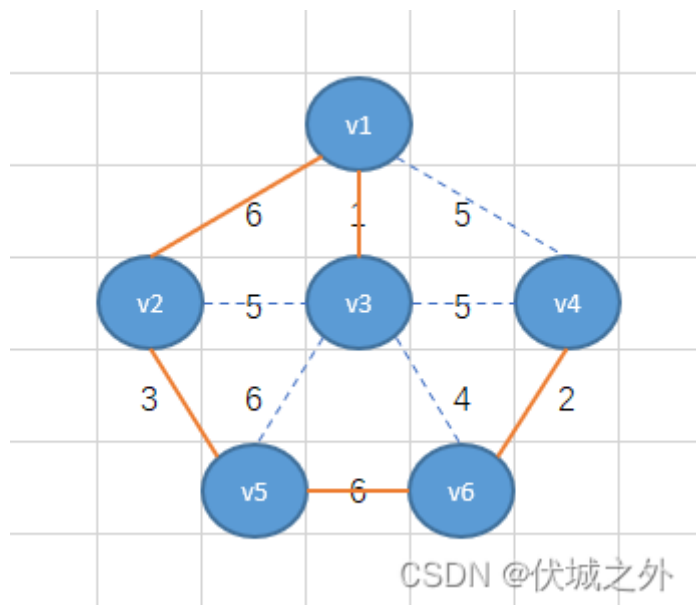
在无向[连通图](#)中，生成树是指包含了全部顶点的极小连通子图。

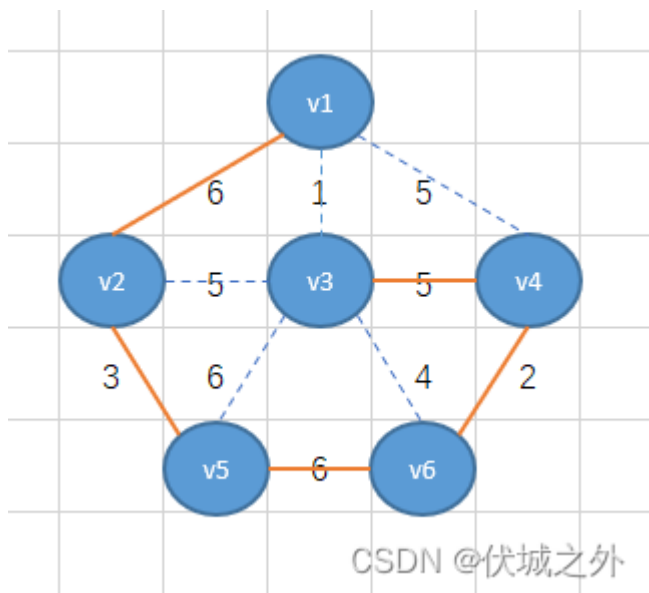
生成树包含原图全部的 n 个顶点和 $n-1$ 条边。（注意，边的数量一定是 $n-1$ ）

比如下面无向连通图例子：



根据生成树概念，我们可以基于上面无向连通图，产生多个生成树，下面举几个生成树例子：





如上图我们用 $n-1$ 条橙色边连接了 n 个顶点。这样就从无向连通图中产生了生成树。

为什么生成树只能由 $n-1$ 条边呢？

因为少一条边，则生成树就无法包含所有顶点。多一条边，则生成树就会形成环。

而生成树最重要的两个特性就是：

- 1、包含所有顶点
- 2、无环

了解了生成树概念后，我们就可以进一步学习最小生成树了。

我们回头看看无向连通图，可以发现每条边都有权重值，比如 $v1-v2$ 权重值是6， $v3-v6$ 权重值是4。

最小生成树指的是，生成树中 $n-1$ 条边的权重值之和最小。

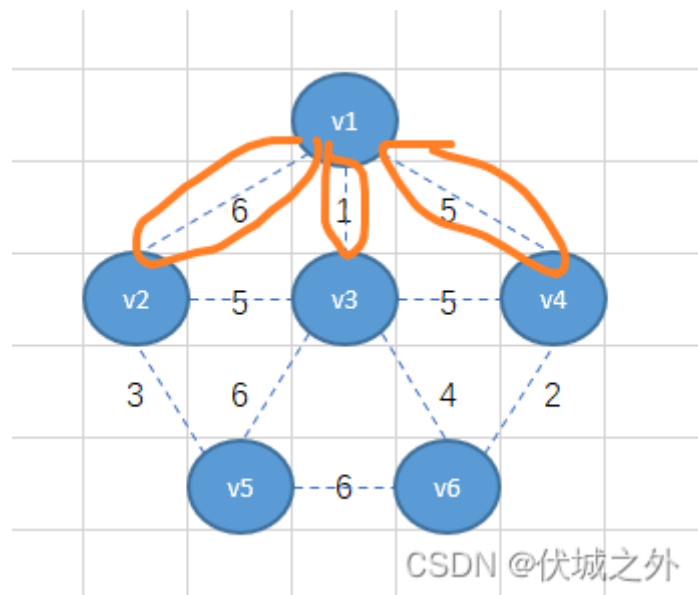
那么如何才能准确的找出一个无向连通图的最小生成树呢？

有两种算法：Prim算法和Kruskal算法。

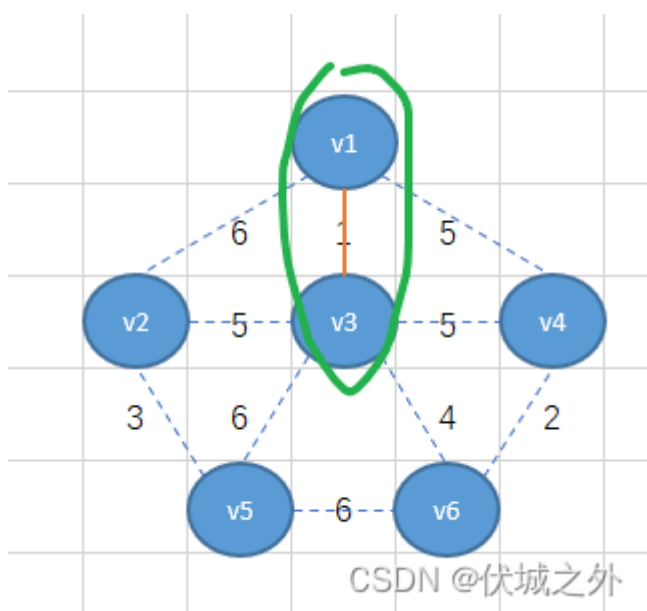
Prim算法是基于顶点找最小生成树。Kruskal是基于边找最小生成树。

首先，我们介绍Prim算法：

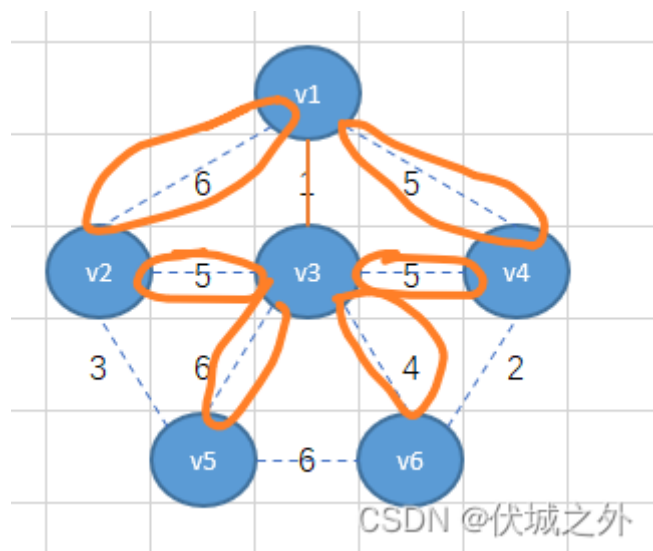
我们可以选择无向连通图中的任意一个顶点作为起始点，比如我们选 $v1$ 顶点为起始点



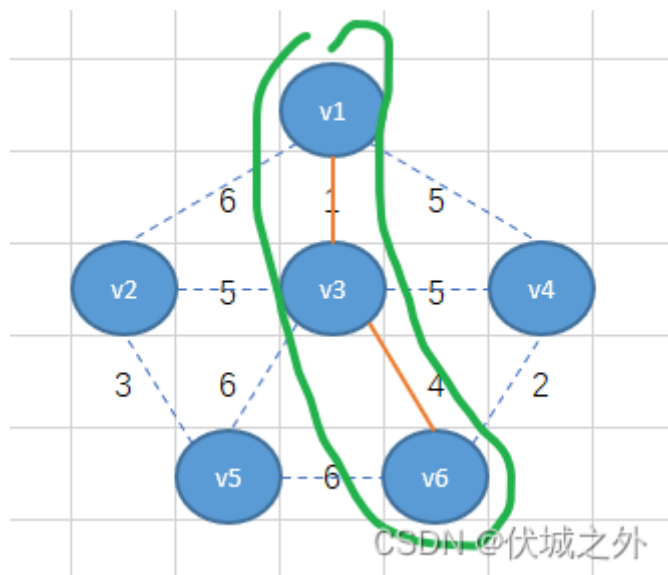
从v1顶点出发，有三条边，我们选择权重最小的1，即将v1-v3相连



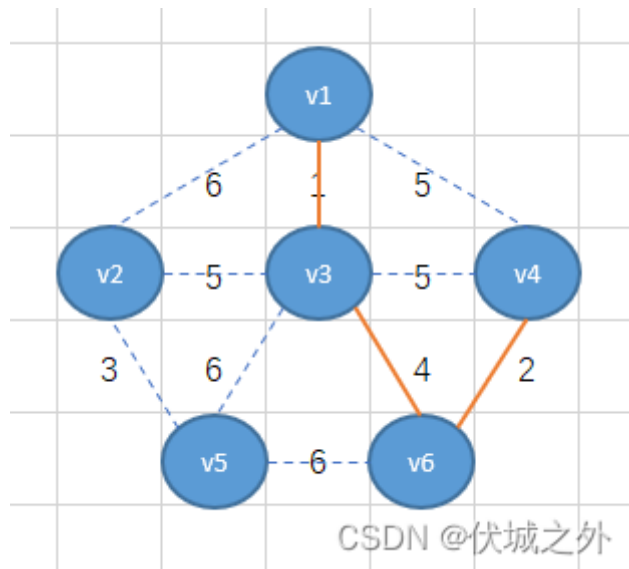
此时我们需要将v1-v3看成一个整体，然后继续找这个整体出发的所有边里面的最小的，



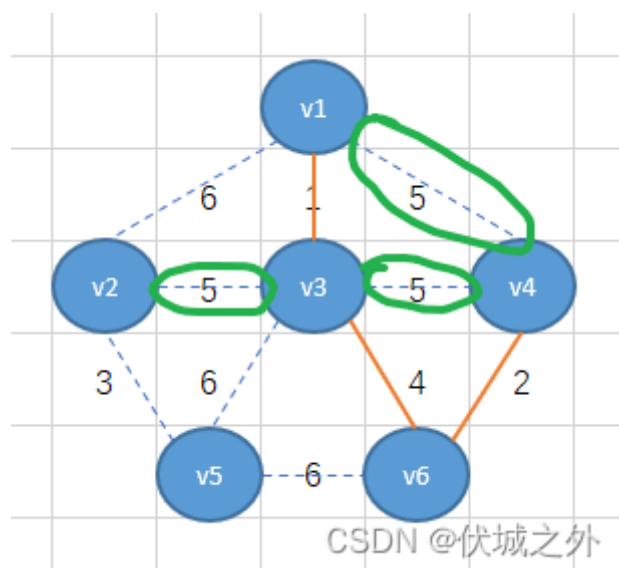
可以发现为最小权重为4，因此，将v3-v6相连



接着将v1-v3-v6看出一个整体，找这个整体出发的所有边里面的最小的，可以找到最小权重2，因此将v6-v4相连



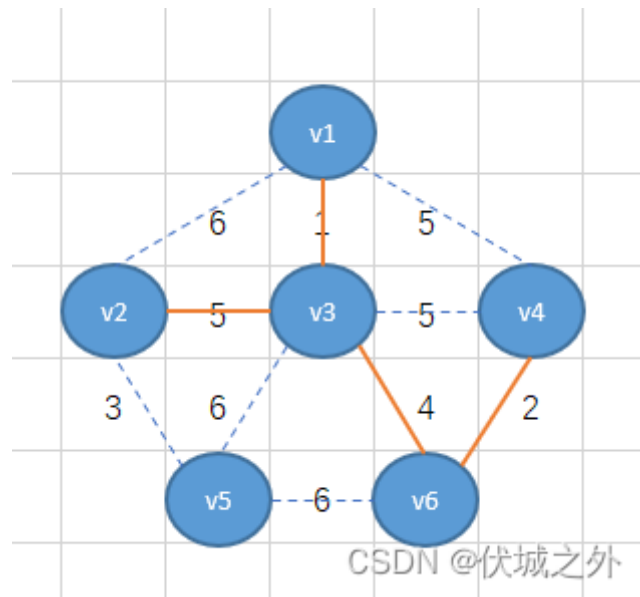
但是接下来，我们会发现，从v1-v3-v6-v4整体出发的所有边里面同时有三个最小权重5，那么该如何选择呢？



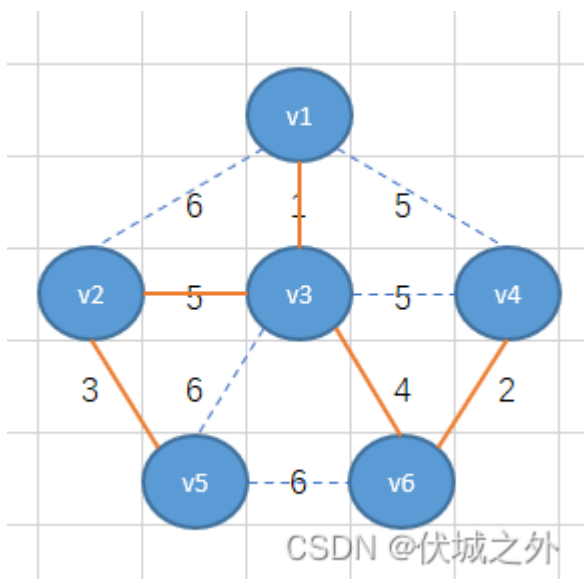
其实不难看出，如果选择v4-v3，或者v4-v1相连，则对应的生成树就形成了环结构，因此就不符合生成树特性了，因此我们只能选择v3-v2。

(注意：如果有多个相同的最小权重边可选，并且都不会产生环结构，则可以选择其中任意一条边，最终得到结果都是最小生成树)

其实，不仅仅在上面遇到相同权重边时，需要判断是否形成环，在前选择每一条边时都需要判断是否形成环，一旦选择的边能够形成环，那么我们就应该舍弃它，选择第二小的权重边，并继续判断。



按照上面逻辑，我们可以继续找到v1-v2-v3-v4-v6整体出发所有边中的最小权重边3，即将v2-v5相连，并且连接后不会形成环

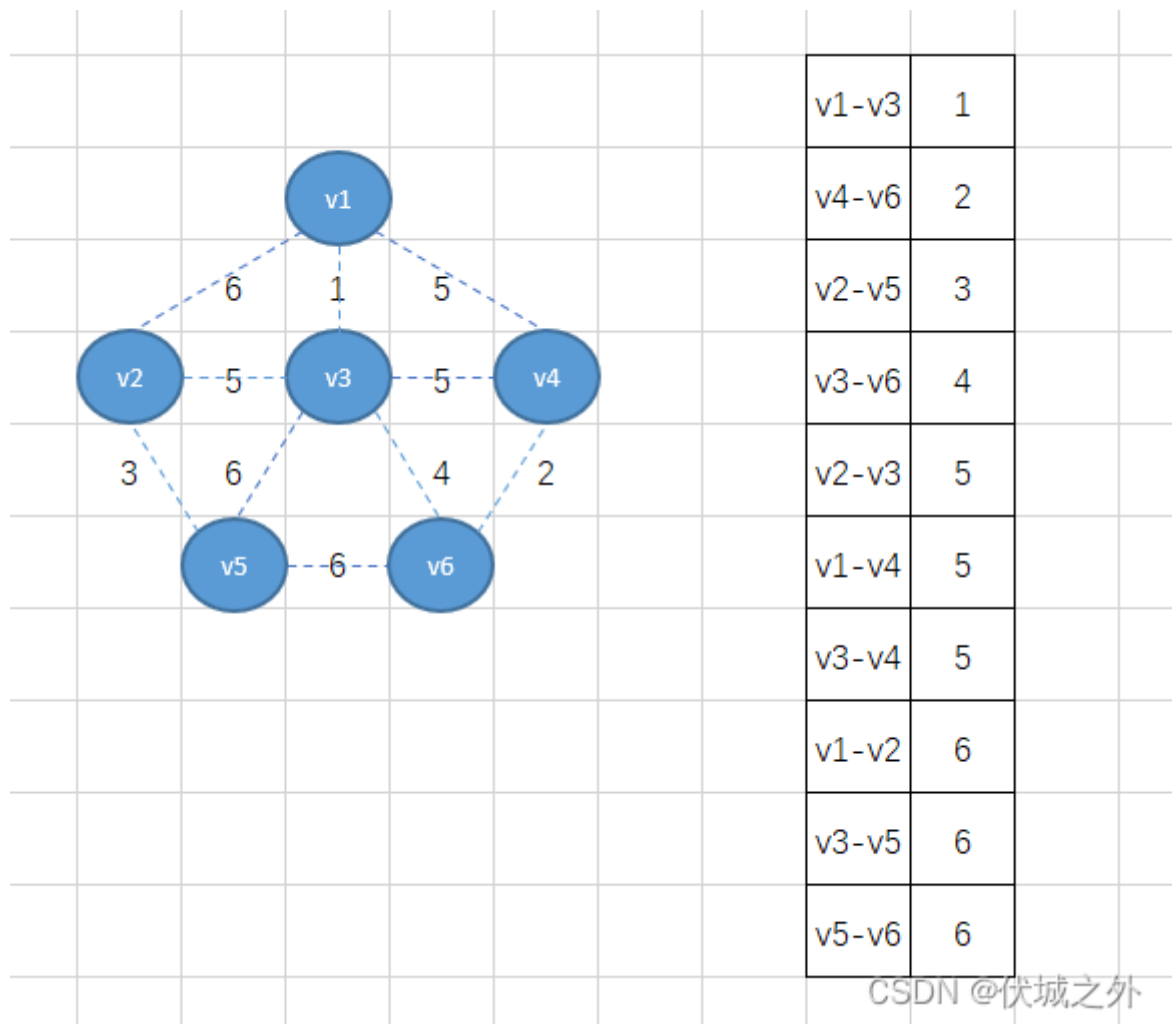


此时选择的边数已经达到了n-1条，因此可以结束逻辑，而现在得到的就是最小生成树。我们可以将这个最小生成树的所有边的权重值之和计算出来为20。

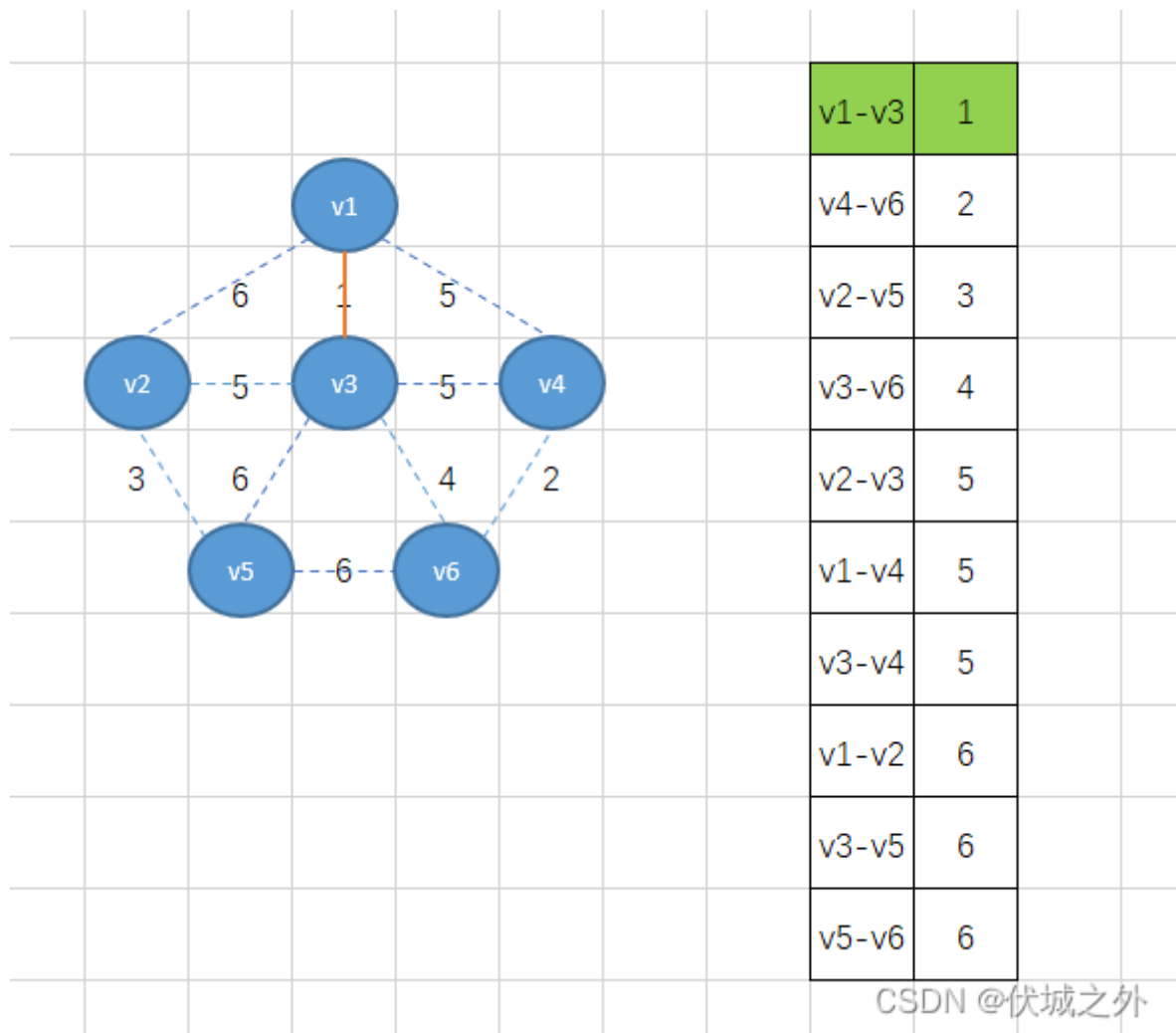
上面这种基于顶点的找最小生成树的方式就是Prim算法。

接下来介绍Kruskal算法：

Kruskal算法要求我们将所有的边按照权重值升序排序，因此可得：

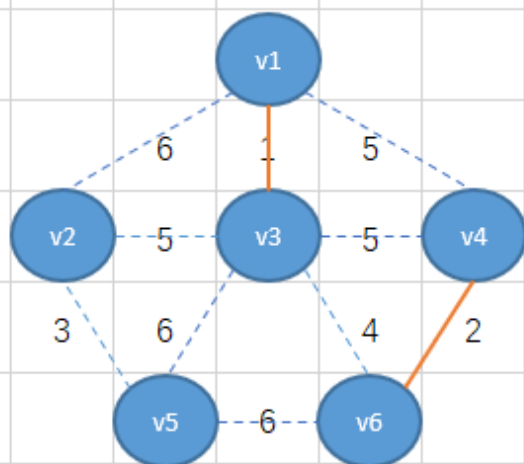


首先，我们将权重最小的边v1-v3加入，得到下图



CSDN @伏城之外

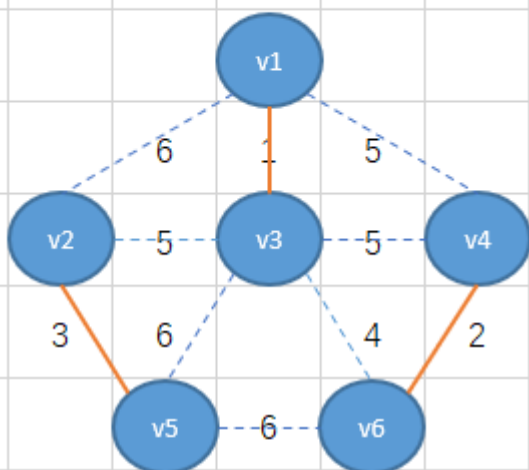
接着将下个最小权重2的边v4-v6加入



v1-v3	1
v4-v6	2
v2-v5	3
v3-v6	4
v2-v3	5
v1-v4	5
v3-v4	5
v1-v2	6
v3-v5	6
v5-v6	6

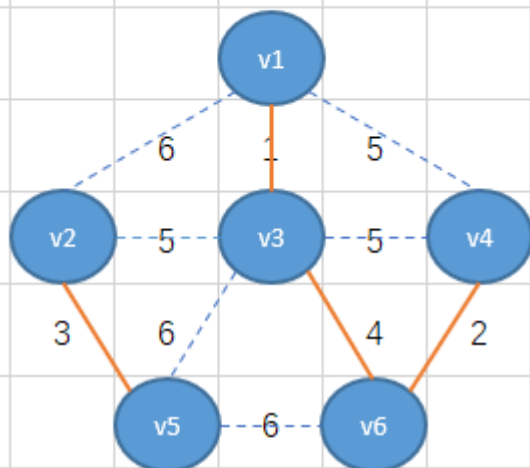
CSDN @伏城之外

接着继续加最小权重边



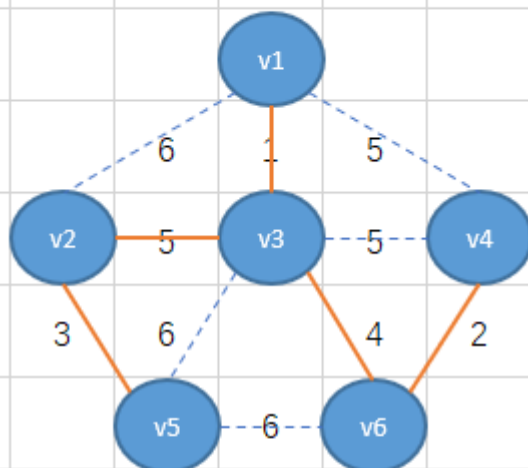
v1-v3	1
v4-v6	2
v2-v5	3
v3-v6	4
v2-v3	5
v1-v4	5
v3-v4	5
v1-v2	6
v3-v5	6
v5-v6	6

CSDN @伏城之外



v1-v3	1
v4-v6	2
v2-v5	3
v3-v6	4
v2-v3	5
v1-v4	5
v3-v4	5
v1-v2	6
v3-v5	6
v5-v6	6

CSDN @伏城之外

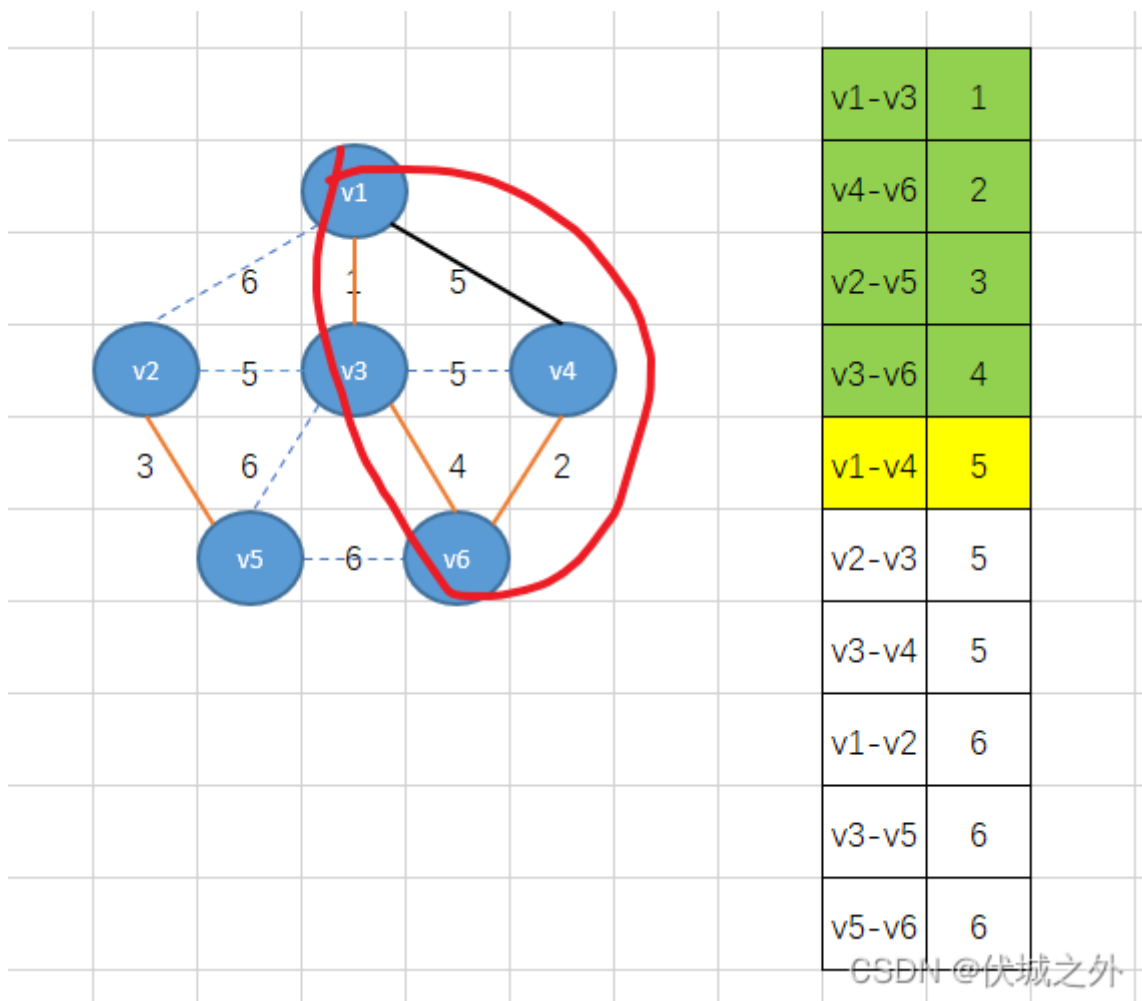


v1-v3	1
v4-v6	2
v2-v5	3
v3-v6	4
v2-v3	5
v1-v4	5
v3-v4	5
v1-v2	6
v3-v5	6
v5-v6	6

CSDN @伏城之外

此时边数已经达到 $n-1$ ，而刚好这个过程中也没有环的形成，因此得到的就是最小生成树。

但是这里有巧合因素在里面，因为最后一步中，最小权重5的边有多条，如果并不是v2-v3排在前面呢，比如是v1-v4呢？



可以发现，形成了环，因此我们应该舍弃这条边，继续找剩下的最小权重边。最后总能找到v2-v3。

通过上面对于Prim算法和Kruskal算法的分析，我们发现一个关键点，那就是必须实时判断是否产生了环？

那么判断环的存在就是实现上面Prim算法和Kruskal算法的关键点！

其实，生成树就是一个连通分量，初始时，生成树这个连通分量只有一个顶点（Prim），或者两个顶点（Kruskal），后面会不断合入新的顶点进来，来扩大连通分量范围。

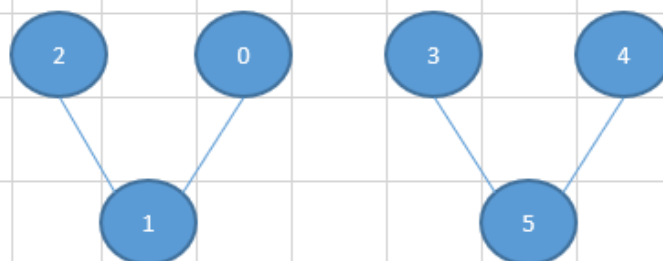
而连通分量可以使用并查集表示，

并查集本质就是一个长度为n的数组（n为无向图的顶点数），数组索引值代表图中某个顶点child，数组索引指向的元素值，代表child顶点的祖先顶点father。

初始时，每个child的father都是自己。即初始时，默认有n个连通分量。

比如 arr = [1,1,1,5,5,5] 数组就可以模拟出一个无向图

arr	索引	0	1	2	3	4	5
	元素	1	1	1	5	5	5



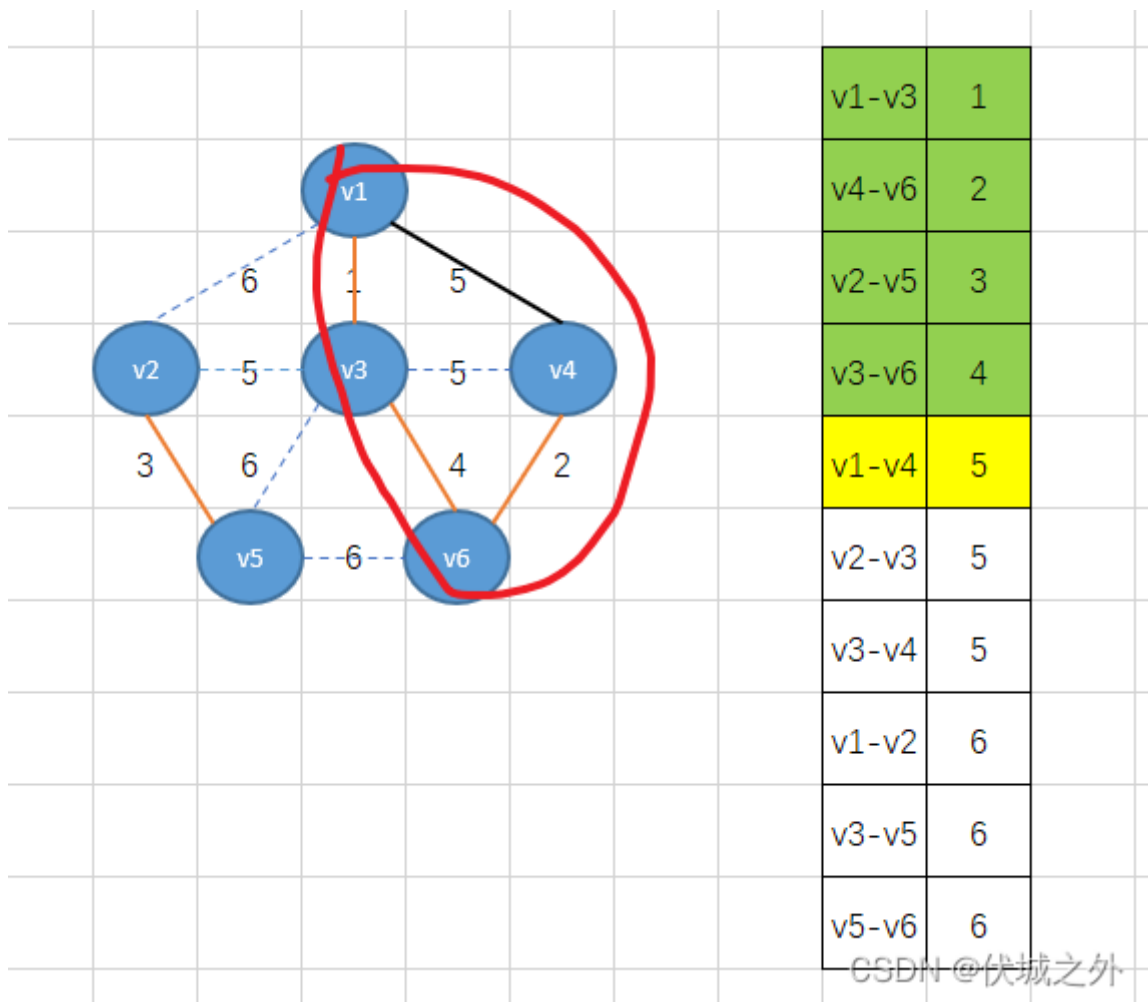
CSDN @伏城之外

- 0顶点的祖先是1顶点 =》 arr[0] = 1
- 1顶点的祖先是1顶点 =》 arr[1] = 1
- 2顶点的祖先是1顶点 =》 arr[2] = 1

如果大家还不理解并查集结构，可以看[华为机试 - 发广播伏城之外的博客-CSDN博客服务器广播 华为机试](#)

我们可以用father指代一个连通分量。比如上面arr = [1,1,1,5,5,5]就有两个连通分量，分别是father为1的连通分量和father为5的连通分量。

最小生成树中的顶点必然都处于同一个连通分量中，因此每加入一个新的顶点child_new，我们就可以看它的father是否已经是连通分量对应的father，如果是，则说明顶点child_new其实已经存在于最小生成树中了，因此就产生了环，比如下面例子：



我们定义一个数组arr，来表示并查集结构，初始时，

arr	索引	v1	v2	v3	v4	v5	v6
	元素	v1	v2	v3	v4	v5	v6

如果达到上图橙色边连接状态，则arr变为

arr	索引child	v1	v2	v3	v4	v5	v6
	元素father	v1	v2	v1	v1	v2	v1

接下来加入黑色边，即v4顶点的father改成v1，但是实际上v4的father已经是v1，那么此时如果再强行加入的话，那么就形成了环。

下面，我们来解答本题：

本题要求的最低成本，其实就是最小生成树所有边相加得到的最小权重和。

但是本题，又不是一个纯粹的求最小生成树的题。

因为，本题中规定了一些“必建高铁的两个城市”，那么多个“必建高铁的两个城市”是非常有可能形成环的。

但是，我们并不需要纠结这个，因为我们要求的不是最小生成树，而是最小权重和，只是这个最小权重和中某些值已经固定了，这些固定的值就是“必建高铁的两个城市”对应的费用。

我们定义一个minFee来代表最低成本，首先我们需要将必建高铁的成本计算进去。并且在计算的过程中，将必建高铁的两个城市（两个顶点）纳入一个连通分量中（基于并查集）。

之后，我们在将“可以建高铁”的列表按照成本费用升序排序（Kruskal算法），然后遍历排序后列表，依次将“可以建高铁”的两个城市（两个顶点）尝试纳入连通分量中，如果有环，则不纳入，无环，则纳入，纳入的话，则将成本费用计入minFee中。

那么上面逻辑何时结束呢？1、所有顶点已经纳入到一个连通分量中；2、循环结束。

循环结束后，

如果并查集中的连通分量数量为1，则说明所有城市（顶点）已经通车，且是最低成本。此时返回minFee就是题解。

如果并查集中的连通分量数量不为1，则说明所有城市（顶点）无法连通，返回-1。

Java算法源码

```
import java.util.Arrays;
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int can = sc.nextInt();
        int must = sc.nextInt();

        int[][] cans = new int[can][3];
        for (int i = 0; i < can; i++) {
            cans[i][0] = sc.nextInt();
            cans[i][1] = sc.nextInt();
            cans[i][2] = sc.nextInt();
        }

        int[][] musts = new int[must][2];
        for (int i = 0; i < must; i++) {
            musts[i][0] = sc.nextInt();
            musts[i][1] = sc.nextInt();
        }

        System.out.println(getResult(n, cans, musts));
    }
}
```

```

/**
 * @param n 一共几个城市
 * @param cans 哪些城市之间可以修建高铁，以及修建费用
 * @param musts 哪些城市之间必须修建高铁
 * @return 最少费用
 */
public static int getResult(int n, int[][] cans, int[][] musts) {
    UnionFindSet ufs = new UnionFindSet(n);

    // 为了方便统计“必建高铁”的费用，我们需要将cans数组改造为cansMap，key为'1-2' 两个城市，
    // val为 这两个城市建高铁的费用
    HashMap<String, Integer> cansMap = new HashMap<>();
    for (int[] can : cans) {
        int city1 = can[0], city2 = can[1], fee = can[2];
        String key = city1 < city2 ? city1 + "-" + city2 : city2 + "-" + city1;
        cansMap.put(key, fee);
    }

    int minFee = 0;
    for (int[] must : musts) {
        // 计入必建高铁的费用到minFee中
        String key = must[0] < must[1] ? must[0] + "-" + must[1] : must[1] + "-" +
must[0];
        minFee += cansMap.get(key);
        // 并将必建高铁的两个城市纳入同一个连通分量重
        ufs.union(must[0], must[1]);
    }

    // 如果必建高铁本身已经满足所有城市通车了，那么直接返回minFee
    if (ufs.count == 1) return minFee;

    // 否则，按照求解最小生成树的Kruskal算法，将高铁线（即图的边）按照成本费用（即边的权重）升
    序
    Arrays.sort(cans, (a, b) -> a[2] - b[2]);

    // 遍历排序后的cans，每次得到的都是当前的最小权重边
    for (int[] can : cans) {
        int city1 = can[0], city2 = can[1], fee = can[2];
        // 如果对应城市已经接入高铁线（即处于连通分量中）则再次合入就会产生环，因此不能合入，否则
        就可以合入
        //      if (ufs.fa[city1] != ufs.fa[city2]) {
        if (ufs.find(city1) != ufs.find(city2)) {
            ufs.union(city1, city2);
            // 若可以合入，则将对应的建造成本计入minFee
            minFee += fee;
        }

        // 如果此时，所有城市都通车了（即并查集中只有一个连通分量），则提前结束循环
        if (ufs.count == 1) break;
    }

    // 如果循环完，发现并查集中还有多个连通分量，那么代表有的城市无法通车，因此返回-1
    if (ufs.count > 1) return -1;

    return minFee;
}

```

```

    }
}

// 并查集
class UnionFindSet {
    int[] fa;
    int count;

    public UnionFindSet(int n) {
        this.fa = new int[n + 1];
        this.count = n;
        for (int i = 0; i < n; i++) this.fa[i] = i;
    }

    public int find(int x) {
        if (x != this.fa[x]) {
            return (this.fa[x] = this.find(this.fa[x]));
        }
        return x;
    }

    public void union(int x, int y) {
        int x_fa = this.find(x);
        int y_fa = this.find(y);

        if (x_fa != y_fa) {
            this.fa[y_fa] = x_fa;
            this.count--;
        }
    }
}

```

74.工单调度策略

题目描述

当小区通信设备上报警时，系统会自动生成待处理的工单，华为工单调度系统需要根据不同的策略，调度外线工程师（[FME](#)）上站修复工单对应的问题。

根据与运营商签订的合同，不同严重程度的工单被处理并修复的时长要求不同，这个要求被修复的时长我们称之为SLA时间。

假设华为和运营商A签订了运维合同，部署了一套调度系统，只有1个外线工程师（FME），每个工单根据问题严重程度会给一个评分，在SLA时间内完成修复的工单，华为获得工单评分对应的积分，超过SLA完成的工单不获得积分，但必须完成该工单。运营商最终会根据积分进行付款。

请设计一种调度策略，根据现状得到调度结果完成所有工单，让这个外线工程师处理的工单获得的总积分最多。

假设从某个调度时刻开始，当前工单数量为N，不会产生新的工单，每个工单处理修复耗时为1小时，请设计你的调度策略，完成业务目标。

不考虑外线工程师在小区之间行驶的耗时。

输入描述

第一行为一个整数N，表示工单的数量。

接下来N行，每行包括两个整数。第一个整数表示工单的SLA时间（小时），第二个数表示该工单的积分。

输出描述

输出一个整数表示可以获得的最大积分。

备注

- 工单数量 $N \leq 10^6$
- SLA时间 $\leq 7 * 10^5$
- 答案的最大积分不会超过2147483647

用例

假设有7个工单的SLA时间（小时）和积分如下：

工单编号	SLA	积分
1	1	6
2	1	7
3	3	2
4	3	1
5	2	4
6	2	5
7	6	1

输入	7 1 6 1 7 3 2 3 1 2 4 2 5 6 1
输出	15
说明	最多可获得15积分，其中一个调度结果完成工单顺序为2, 6, 3, 1, 7, 5, 4（可能还有其他顺序）

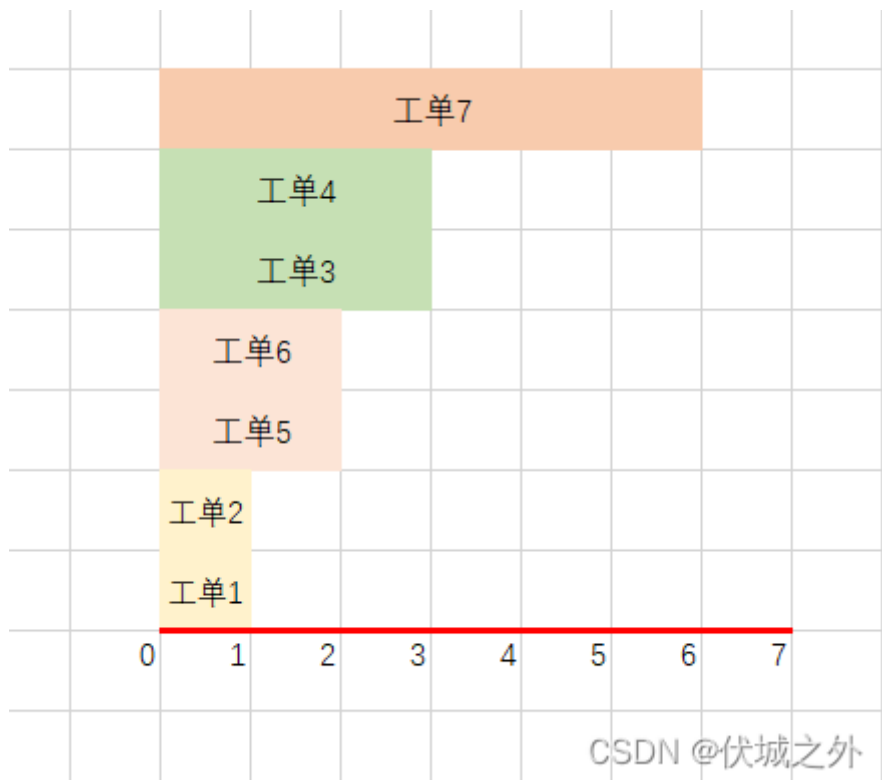
题目解析

用例的含义如下

从现在开始：

- 编号1和2的工单，需要在1小时内完成
- 编号5和6的工单，需要在2小时内完成
- 编号3和4的工单，需要在3小时内完成
- 编号7的工单，需要在6小时内完成

如果现在是0点，那么有时间线如下：



但是每个工单都需要花费1小时来修复，因此：

- 0~1点之间，工单1和工单2是最优先的，因为超过这个时间，他们即使修复了也没有积分了，但是由于每个工单都需要1小时，因此这个时间段内，只有一个工单可以被修复，另一个工单需要被放弃，此时我们应该选择积分最多的工单2（7个积分）来优先修复。

因此这个时间段可以获得7个积分。

- 1~2点之间，工单5和工单6是最优先的，但是由于只有1小时，因此只能修复一个工单，优先选择积分高的工单6（5个积分）来修复。而工单5因为超时所以放弃。

因此这个时间段可以获得5个积分。

- 2~3点之间，工单3和工单4是最优先的，但是由于只有1小时，因此只能修复一个工单，优先选择积分高的工单3（2个积分）来修复。而工单4因为超时所以放弃。

因此这个时间段可以获得2个积分。

- 3~5点之间，没有紧急工单，这个时间可以处理2个工单，而之前放弃的工单数量有3个，因此我们可以在这段时间内选择处理任意两个放弃掉的工单，但是没有积分拿。

- 5~6点之间，有一个紧急工单7，因此我们修复工单7，拿1个积分。

因此这个时间段可以获得1个积分。

- 6~7之间，没有紧急工单，这个时间可以处理1个工单，而之前放弃的工单数量还剩一个，因此我们可以在这段时间选择处理剩下一个被放弃工单，但是没有积分拿。

最终可以拿到 $7 + 5 + 2 + 1 = 15$ 个积分。

另外，还有一些其他网友提供的用例如：

```
3
1 1
2 10
2 20
```

当0~1点时，虽然工单1是最紧急的，但是放弃工单1，而是在0~2点执行工单2和3可以获得最大积分30。

那么上面逻辑该如何实现呢？

本题的解题思路可以参照[LeetCode - 630 课程表III 伏城之外的博客-CSDN博客](#)

我的解题思路如下：

首先将所有工单wos按照截止时间升序，即最紧急的排在最前面。

然后创建一个优先队列pq（按照积分升序，即积分少的工单在堆顶），定义一个当前时间curTime = 0，定义一个ans记录拿到的积分，然后遍历升序后的wos的每一个wo并尝试加入pq中，此时会出现两种情况：

- 要加入pq的工单wo的截止时间endTime $\geq$ curTime + 1，表示当前花一小时修复该工单后，依旧在工单的有效期内，此时我们可以将wo加入pq，并且获得该工单的积分 ans += score，然后 curTime++
- 要加入pq的工单wo的截止时间endTime < curTime + 1，表示当前花一个小时修复该工单后，超出了该工单的有效期，此时我们需要比较 优先队列堆顶的工单A的积分（优先队列中最小积分）和 要加入的B工单的积分：
 1. 如果B > A，则说明当前B工单更具有修复价值，因此我们应该把修复工单A的一小时用来修复B工单，因此优先队列弹出顶部工单A，并加入工单B。此时 ans += B.score - A.score，而curTime不变，因为只是将原本用于修复A的一小时，换到修复B上。
 2. 如果B <= A，则说明当前B工单不比A更具有修复价值。

对于JS而言没有原生的优先队列实现类，基于堆结构实现的优先队列请看：[LeetCode - 1705 吃苹果的最大数目 伏城之外的博客-CSDN博客](#)

如果觉得实现过于麻烦，也可以使用有序数组实现优先队列，但是维持优先级的时间复杂度将从O(lgN)升到O(N)

Java算法源码

```
import java.util.Arrays;
import java.util.PriorityQueue;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[][] wos = new int[n][2];
        for (int i = 0; i < n; i++) {
            wos[i][0] = sc.nextInt();
            wos[i][1] = sc.nextInt();
        }

        System.out.println(getResult(n, wos));
    }

    /**
     * @param n 工单数量
     * @param wos 工单的 [SLA, 积分]
     * @return 可以获得的最大积分
     */
    public static int getResult(int n, int[][] wos) {
        Arrays.sort(wos, (a, b) -> a[0] - b[0]);
        PriorityQueue<Integer> pq = new PriorityQueue<>((a, b) -> a - b);

        int curTime = 0;
        int ans = 0;
        for (int[] wo : wos) {
            int endTime = wo[0];
            int score = wo[1];

            if (endTime >= curTime + 1) {
                pq.offer(score);
                ans += score;
                curTime++;
            } else {
                if (pq.size() == 0) {
                    continue;
                }

                int min_score = pq.peek();
                if (score > min_score) {
                    pq.poll();
                    pq.offer(score);
                    ans += score - min_score;
                }
            }
        }

        return ans;
    }
}
```

```
}  
}
```

75.采样过滤

题目描述

在做物理实验时，为了计算物体移动的速率，通过相机等工具周期性的采样物体移动距离。

由于工具故障，采样数据存在误差甚至相误的情况。

需要通过一个算法过滤掉不正确的采样值，不同工具的故意模式存在差异，算法的各关门限会根据工具类型做相应的调整。

请实现一个算法，计算出给定一组采样值中正常值的最长连续周期。

判断第1个周期的采样数据S0是否正确的规则如下(假定物体移动速率不超过10个单元前一个采样周期S[i-1]):

```
S[i]<=0, 即为错误值`  
`S[i]<S[i-1], 即为错误值`  
`S[i]-S[i-1]>=10, 即为错误值。  
其它情况为正常值
```

判断工具是否故障的规则如下：

在M个周期内，采样数据为错误值的次数为T(次数可以不连续)，则工具故障。

判断故障恢复的条件如下：

产生故障后的P个周期内，采样数据一直为正常值，则故障恢复

错误采样数据的处理方式：

检测到故障后，丢弃从故障开始到故障恢复的采样数据

在检测到工具故障之前，错误的采样数据，则由最近一个正常值代替;如果前面没有正常的采样值，则丢弃此采样数据

给定一段周期的采样数据列表S，计算正常值的最长连续周期。

输入描述

故障确认周期数和故障次数门限分别为M和T，故障恢复周期数为P。

第i个周期，检测点的状态为S[i]。

输入为两行，格式如下：

M T F`
`s1 s2 s3 ...`
`M、t 和 e的取值范围为[1, 100000]`
`s1取值范围为[0, 100000]，从0开始编号`

输出描述

一行，输出正常值的最长连续周期。

用例

输入	10 6 3 -1 1 2 3 100 10 13 9 10
输出	8
说明	无

题目解析

输入描述似乎有误，第一行输入的值应该对应M T P，而不是M T F，因为题目描述中没有说F是啥。

第二行输入的应该就是每个周期的采样数据，

根据题目描述，判断一个采样数据是否为错误的，由如下三个条件组成：

- $S[i] \leq 0$ ，即为错误值
- $S[i] < S[i-1]$ ，即为错误值
- $S[i] - S[i-1] \geq 10$ ，即为错误值

因此可以判断用例第二行的数据哪些是错误的，并且错误数据的处理规则如下：

- 1、在检测到工具故障之前，错误的采样数据，则由最近一个正常值代替;如果前面没有正常的采样值，则丢弃此采样数据
- 2、检测到故障后，丢弃从故障开始到故障恢复的采样数据

-1	1	2	3	100	10	13	9	10	$S[i] < 0$, 因此为错误值, 并且它前面没有值, 因此直接舍弃
↑									
	1	2	3	100	10	13	9	10	符合要求
	↑								
	1	2	3	100	10	13	9	10	符合要求
		↑							
	1	2	3	100	10	13	9	10	符合要求
			↑						
	1	2	3	100 ³	10	13	9	10	$S[i] - S[i-1] \geq 10$, 因此错误, 因为它前面有值, 因此用它前面的值代替
			↑						
	1	2	3	3	10	13	9	10	符合要求
				↑					
	1	2	3	3	10	13	9	10	符合要求
					↑				
	1	2	3	3	10	13	9 ¹³	10	$S[i] < S[i-1]$, 因此错误, 因为它前面有值, 因此用它前面的值代替
						↑			
	1	2	3	3	10	13	13	10 ¹³	$S[i] < S[i-1]$, 因此错误, 因为它前面有值, 因此用它前面的值代替
							↑		

CSDN @伏城之外

需要注意的是, 由于检测故障每10个周期的数据检查一次, 但是如果不到10个周期, 比如当前第二行输入只有9个周期, 也要看错误数据个数是否达到了门限值 $T = 6$, 而根据上面图示错误数据个数只有4个, 因此不会触发故障清理。

因此, 最终可以保留8个连续正常周期。

但是这个用例过于特殊, 不能说明任何问题。

我自己设计了一个测试用例:

10 6 3

-1 -1 -1 1 2 3 100 4 6 10 100 3 5 8 9 19 22 14 15 25

用例含义是:

每10个周期内进行故障判断, 如果10个周期内出现了6次错误, 则第6次错误时可以判定发现了故障,

之后故障开始, 并进入故障恢复判断, 故障恢复的条件是要出现连续3个正常数据。

一旦判定故障恢复, 则从故障开始~故障恢复这段时间的数据都要丢弃。

上面是我的个人理解, 这道题目描述太宽泛了, 题目想表达的意思不好把握

我按照我的理解进行图示演示

错误数据，并且前面没有数据，因此丢弃。我们用一个指针i来标记正常周期范围的起始位置																				
数据	-1	-1	-1	1	2	3	100	4	6	10	100	3	5	8	9	19	22	14	15	25
	↑																			
周期	1	2	3	4	5	6	7	8	9	10										

CSDN @伏城之外

CSDN @伏城之外

	同前面																			
数据	-1	-1	-1	1	2	3	100	4	6	10	100	3	5	8	9	19	22	14	15	25
		↑																		
周期	1	2	3	4	5	6	7	8	9	10										

CSDN @伏城之外

CSDN @伏城之外

			同前面																	
数据	-1	-1	-1	1	2	3	100	4	6	10	100	3	5	8	9	19	22	14	15	25
			↑																	
周期	1	2	3	4	5	6	7	8	9	10										

CSDN @伏城之外

CSDN @伏城之外

				正确数据																
数据	-1	-1	-1	1	2	3	100	4	6	10	100	3	5	8	9	19	22	14	15	25
				↑																
周期	1	2	3	4	5	6	7	8	9	10										

CSDN @伏城之外

CSDN @伏城之外

由于遇到了正确的开头数据，因此 i 指针代表的正常连续区间的左边界确定，现在开始确定右边界，我们用 j 指针表示右边界																				
此时 j 指针指向的数据2，满足要求，是正常数据																				
数据	-1	-1	-1	1	2	3	100	4	6	10	100	3	5	8	9	19	22	14	15	25
																				
周期	1	2	3	4	5	6	7	8	9	10										

CSDN @伏城之外

CSDN @伏城之外

					右指针是正确数据															
数据	-1	-1	-1	1	2	3	100	4	6	10	100	3	5	8	9	19	22	14	15	25
																				
周期	1	2	3	4	5	6	7	8	9	10										
CSDN @伏城之外																				

CSDN @伏城之外

				此时右指针指向错误数据，但是前面有数据，因此可以替换为前面的数据																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																												</
--	--	--	--	----------------------------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----

CSDN @伏城之外

								右指针指向的是正确数据																	
数据	-1	-1	-1	1	2	3	3	4	6	10	100	3	5	8	9	19	22	14	15	25					
				↑				↑																	
周期	1	2	3	4	5	6	7	8	9	10															

CSDN@伏城之外

CSDN @伏城之外

	右指针指向是正确数据																			
数据	-1	-1	-1	1	2	3	3	4	6	10	100	3	5	8	9	19	22	14	15	25
																				
周期	1	2	3	4	5	6	7	8	9	10										

CSDN @伏城之外

CSDN @伏城之外

右指针指向是正确数据，并且到了m个周期，可以开始故障判断了，由于出现的错误数据个数只有4个，少于门限值6，因此没有故障，后续可以继续下一个m周期判断																				
数据	-1	-1	-1	1	2	3	3	4	6	10	100	3	5	8	9	19	22	14	15	25
																				
周期	1	2	3	4	5	6	7	8	9	10										

CSDN @伏城之外

CSDN @伏城之外

错误数据，但是前面有值，因此可以替换为前面的数据																				
数据	-1	-1	-1	1	2	3	3	4	6	10	100	3	5	8	9	19	22	14	15	25
周期											1	2	3	4	5	6	7	8	9	10

CSDN @ 伏城之外

CSDN @伏城之外

错误数据，但是前面有数据，因此可以替换																				
数据	-1	-1	-1	1	2	3	3	4	6	10	10	10	5	8	9	19	22	14	15	25
周期											1	2	3	4	5	6	7	8	9	10

CSDN @ 伏城之外

CSDN @伏城之外

错误数据，但是前面有数据，因此可以替换																				
数据	-1	-1	-1	1	2	3	3	4	6	10	10	10	10	8	9	19	22	14	15	25
周期											1	2	3	4	5	6	7	8	9	10

<

CSDN @伏城之外

错误数据，但是可以替换																				
数据	-1	-1	-1	1	2	3	3	4	6	10	10	10	10	8	9	19	22	14	15	25
周期											1	2	3	4	5	6	7	8	9	10

CSDN @伏城之外

CSDN @伏城之外

错误数据，但是可以替换																				
数据	-1	-1	-1	1	2	3	3	4	6	10	10	10	10	10	19	22	14	15	25	
周期											1	2	3	4	5	6	7	8	9	10

CSDN @伏城之外

																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																					</
--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----

CSDN @伏城之外

数据	-1	-1	-1	1	2	3	3	4	6	10	10	10	10	10	19	22	14	15	25
周期										1	2	3	4	5	6	7	8	9	10

CSDN @伏城之外

数据	-1	-1	-1	1	2	3	3	4	6	10	10	10	10	10	19	22	22	15	25	
周期											1	2	3	4	5	6	7	8	9	10

错误数据, 替换为前面数据

注意，这里错误次数已经达到了门限 t ，因此判定为发现了故障，后面就是故障恢复判断了，因此此时的 $i-j$ 范围就是一个连续正常范围区间，长度15

由于故障恢复期间的数据无论正确错误都要丢弃，因此造成了连续正常区间的中断，即指针右移

数据	-1	-1	-1	1	2	3	3	4	6	10	10	10	10	10	10	19	22	22	15	25
周期											1	2	3	4	5	6	7	8	9	10

CSDN @伏城之外

数据	-1	-1	-1	1	2	3	3	4	6	10	10	10	10	10	19	22	22	22	25
周期										1	2	3	4	5	6	7	8	9	10

22

CSDN @伏城之外

	-1	-1	-1	1	2	3	3	4	6	10	10	10	10	10	10	19	22	22	25	正确数据
数据																				
周期											1	2	3	4	5	6	7	8	9	10

CSDN @伏城之外

注意，在故障恢复阶段，左指针要一直跟随右指针，因为这个阶段的数据无论对错，都会丢弃。

因此，最终最长连续周期长度是15

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Integer[] mtp =
            Arrays.stream(sc.nextLine().split("
"))
                .map(Integer::parseInt)
                .toArray(Integer[]::new);

        Integer[] s =
            Arrays.stream(sc.nextLine().split("
"))
                .map(Integer::parseInt)
                .toArray(Integer[]::new);
```

```

        System.out.println(getResult(mtp[0], mtp[1], mtp[2], s));
    }

    /**
     * @param m 故障确认周期数
     * @param t 故障次数门限值
     * @param p 故障恢复周期数
     * @param s 数组，元素是每个周期的采样数据
     * @return 最长连续周期的长度
     */
    public static int getResult(Integer m, Integer t, Integer p, Integer[] s) {
        int i = 0; // 连续正常周期的起始位置
        int fault = 0; // m个周期内错误数据的个数
        int cycle = 0; // 处于m个周期内第几个周期
        int ans = 0; // 最终结果，即最长连续正常周期长度

        // 如果采样数据一开始就是错误的，则直接丢弃
        while (s[i] <= 0) {
            // i指针后移，表示前面区间不属于连续正常周期范围
            i++;
            // 虽然错误数据被丢弃，但是仍然属于m周期内的出现的错误数据，因此需要计数
            fault++;
            cycle++;

            // 如果在m周期范围内，错误数据数量达到门限t，则工具故障，并进入故障恢复检测阶段
            if (cycle <= m) {
                if (cycle == m) {
                    cycle = 0;
                    fault = 0;
                }

                if (fault == t) {
                    cycle = 0;
                    fault = 0;
                    int p1 = p;
                    while (i < s.length && p1 > 0) {
                        // 故障恢复条件是，p个周期内一直都是正常数据，并要求连续，如果不连续，则重新计数
                        if (isFault(s, i)) {
                            p1 = p;
                        } else {
                            p1--;
                        }
                        i++;
                    }
                }
            }
        }

        // 这个cycle计数对应第i周期的
        cycle++;
        int j = i + 1;

        while (j < s.length) {
            cycle++; // 这个cycle计数对应第j周期的
            if (isFault(s, j)) {

```

```

        s[j] = s[j - 1];
        fault++;
    }

    j++;

    if (cycle <= m) {
        if (cycle == m) {
            cycle = 0;
            fault = 0;
        }

        if (fault == t) {
            cycle = 0;
            fault = 0;
            ans = Math.max(ans, j - i); // 注意此时的j必然是故障开始的j，因此不计入正常连续周期中，因此求长度时不需要+1
            // 发现故障，故障开始，进行故障恢复判断
            int p1 = p;
            while (j < s.length && p1 > 0) {
                // 故障恢复条件是，p个周期内一直都是正常数据，要求连续
                if (isFault(s, j)) {
                    p1 = p;
                } else {
                    p1--;
                }
                i = j;
            }
        }
    }

    ans = Math.max(ans, j - i); // 注意这里的j必然是越界的j，因此求长度时不需要+1
    return ans;
}

// 该方法用于判断数据是否错误
public static boolean isFault(Integer[] s, int j) {
    return s[j] <= 0 || (j >= 1 && (s[j] < s[j - 1] || s[j] - s[j - 1] >= 10));
}
}

```

76.最大平分数组

题目描述

给定一个数组nums，可以将元素分为若干个组，使得每组和相等，求出满足条件的所有分组中，最大的平分组个数。

输入描述

第一行输入 m
接着输入m个数，表示此数组
数据范围:1<=M<=50, 1<=nums[i]<=50

输出描述

最大的平分组数个数

用例

输入	7 4 3 2 3 5 2 1
输出	4
说明	可以等分的情况有：4 个子集 (5) , (1,4) , (2,3) , (2,3) 2 个子集 (5, 1, 4) , (2,3, 2,3) 最大的平分组数个数为4个。

输入	9 5 2 1 5 2 1 5 2 1
输出	4
说明	可以等分的情况有：4 个子集 (5, 1) , (5, 1) , (5, 1) , (2, 2, 2) 2 个子集 (5, 1, 5,1) , (2,2, 2,5,1) 最大的平分组数个数为4个。

题目解析

本题和

[LeetCode - 698 划分为k个相等的子集 伏城之外的博客-CSDN博客](#)

[华为机试 - 叠积木伏城之外的博客-CSDN博客叠积木 算法](#)

[华为机试 - 等和子数组最小和 伏城之外的博客-CSDN博客](#)

属于类似，解题步骤基本相同。

题解请看leetcode划分划分k个相等子集。

Java算法源码

```
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);

int m = sc.nextInt();

LinkedList<Integer> link = new LinkedList<>();
for (int i = 0; i < m; i++) {
    link.add(sc.nextInt());
}

System.out.println(getResult(link, m));
}

public static int getResult(LinkedList<Integer> link, int m) {
    link.sort((a, b) -> b - a);

    int sum = 0;
    for (Integer ele : link) {
        sum += ele;
    }

    while (m >= 1) {
        LinkedList<Integer> link_cp = new LinkedList<>(link);
        if (canPartitionMSubsets(link_cp, sum, m)) return m;
        m--;
    }

    return 1;
}

public static boolean canPartitionMSubsets(LinkedList<Integer> link, int sum,
int m) {
    if (sum % m != 0) return false;

    int subSum = sum / m;

    if (subSum < link.get(0)) return false;

    // while (link.get(0) == subSum) {
    while (link.size() > 0 && link.get(0) == subSum) {
        link.removeFirst();
        m--;
    }

    int[] buckets = new int[m];
    return partition(link, 0, buckets, subSum);
}

public static boolean partition(LinkedList<Integer> link, int index, int[]
buckets, int subSum) {
    if (index == link.size()) return true;

    int select = link.get(index);

    for (int i = 0; i < buckets.length; i++) {
        if (i > 0 && buckets[i] == buckets[i - 1]) continue;

```

```
        if (select + buckets[i] <= subSum) {
            buckets[i] += select;
            if (partition(link, index + 1, buckets, subSum)) return true;
            buckets[i] -= select;
        }
    }

    return false;
}
```

77.递增字符串

题目描述

[定义字符串](#)完全由 'A' 和 'B' 组成，当然也可以全是 'A' 或全是 'B'。如果字符串从前往后都是以字典序排列的，那么我们称之为严格递增字符串。

给出一个字符串 s ，允许修改字符串中的任意字符，即可以将任何的 'A' 修改成 'B'，也可以将任何的 'B' 修改成 'A'，

求可以使 s 满足严格递增的最小修改次数。

$0 < s \text{ 的长度} < 100000$ 。

输入描述

输入一个字符串：“AABBA”

输出描述

输出：1

用例

输入	AABBA
输出	1
说明	修改最后一位得到AABBB。

题目解析

新增一个用例：

AAABAAABBBBABB

该用例只需要修改两次。

我的解题思路如下：

首先，我们需要记录字符串长度为total，然后统计字符串中A的数量，记为A。

然后用[动态规划](#)，即定义一个dp数组，dp[i]的含义是 [0,i]闭区间内有多少个A。

统计好后，我们可以假设将 0 ~ i 闭区间内全部变成A，需要修改多少次？

0~i闭区间内理论会有 i+1 个A，而目前有dp[i] 个A，因此需要修改 $i + 1 - dp[i]$ 次。

接着考虑将 i+1 ~ str.length-1 区间内全部变成B，需要修改多少次？

dp[i]表示 0~i闭区间内A的数量，那么 i+1~str.length-1 区间内A的数量= A - dp[i]

因此需要修改 A - dp[i]次

因此，一共需要修改 $i + 1 - dp[i] + A - dp[i]$ 次

我们从 0 ~ str.length-1 遍历出每一个 i 带入上面公式，保留最小的结果，就是题解。

另外，本题 $0 < s \text{ 的长度} < 100000$ ，如果使用dp数组的话，最坏需要定义一个100000长度的数组，这是有可能爆内存的，因此，下面代码中我不是dp数组，直接使用leftA变量。

补充一下：上面逻辑遗漏考虑两个分界线位置：



如上图所示，分别是-1索引位置和n索引位置，即上图两个绿色箭头位置。

前面逻辑，只考虑分界线位置在数组索引的0~n-1，因此当用例如下时：

BBABB

使用上面逻辑会产生问题。

此时，我们应该记录将字符串全部修改为A的次数，和全部修改为B的次数，即上面两个绿色箭头分界线。

Java算法源码

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.next();
        System.out.println(getResult(str));
    }

    public static int getResult(String str) {
        int total = str.length();
```

```
int A = str.replaceAll("B", "").length(); // 字符串str中共有多少个A

// 如果全B或全A, 则直接返回0
if (A == 0 || A == total) return 0;

// 修改为全A或者全B的次数取最小值
int ans = Math.min(A, total - A);

int leftA = 0;
for (int i = 0; i < total; i++) {
    if (str.charAt(i) == 'A') leftA++;
    ans = Math.min(i + 1 - leftA + A - leftA, ans);
}

return ans;
}
```

78.租车骑绿岛

题目描述

部门组织绿岛骑行团建活动。租用公共双人自行车，每辆自行车最多坐两人，最大载重M。给出部门每个人的体重，请问最多需要租用多少双人自行车。

输入描述

第一行两个数字m、n，分别代表自行车限重，部门总人数。

第二行，n个数字，代表每个人的体重，体重都小于等于自行车限重m。

- $0 < m \leq 200$
- $0 < n \leq 1000000$

输出描述

最小需要的双人自行车数量。

用例

输入	3 4 3 2 2 1
输出	3
说明	无

题目解析

本题需要最少的车辆，即尽可能组合出重量小于等于m的两人组。

首先，我们可以将所有人按体重升序，然后将最大体重和m比较，若最大体重大于等于m，则这个人只能一人占一辆车，车数量count++，然后将最大体重弹出，继续将剩下体重中最大的和m比较，逻辑同上，直到最大体重小于m时，停止弹出。

在剩余体重中，我们利用[双指针](#)，i指针指向最小体重，j指针指向最大体重，然后组合它们，即arr[i]+arr[j]，和m比较，若小于等于m，则说明这两个人可以共享一辆车，车数量count++，然后i++，j--。如果arr[i]+arr[j]>m，则说明两个人无法共享一辆车，我们只能优先将这里车分配给较大体重的人，此时车数量count++，然后j--。

按上面逻辑移动双指针，最后可能会出现两种情况：

- $i > j$ 此情况下所有人均分配到了车，因此可以直接输出count作为题解
- $i == j$ 此情况下还有一个人未分配到车，因此需要count++，为这个人单独分配一辆车

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
        int n = sc.nextInt();

        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.println(getResult(arr, m));
    }

    public static int getResult(int[] arr, int m) {
        Arrays.sort(arr);

        int count = 0;

        int i = 0;
        int j = arr.length - 1;

        while (i < j) {
            if (arr[i] + arr[j] <= m) i++;
            j--;
            count++;
        }

        if (i == j) count++;

        return count;
    }
}
```

```
    }

    if (i == j) count++;

    return count;
}
}
```

79.整理扑克牌

题目描述

给定一组数字，表示扑克牌的牌面数字，忽略扑克牌的花色，请按如下规则对这一组扑克牌进行整理：

步骤1. 对扑克牌进行分组，形成组合牌，规则如下：

- 当牌面数字相同张数大于等于4时，组合牌为“炸弹”；
- 3张相同牌面数字 + 2张相同牌面数字，且3张牌与2张牌不相同，组合牌为“葫芦”；
- 3张相同牌面数字，组合牌为“三张”；
- 2张相同牌面数字，组合牌为“对子”；
- 剩余没有相同的牌，则为“单张”；

步骤2. 对上述组合牌进行由大到小排列，规则如下：

- 不同类型组合牌之间由大到小排列规则：“炸弹” > “葫芦” > “三张” > “对子” > “单张”；
- 相同类型组合牌之间，除“葫芦”外，按组合牌全部牌面数字加总由大到小排列；
- “葫芦”则先按3张相同牌面数字加总由大到小排列，3张相同牌面数字加总相同时，再按另外2张牌面数字加总由大到小排列；
- 由于“葫芦”>“三张”，因此如果能形成更大的组合牌，也可以将“三张”拆分为2张和1张，其中的2张可以和其它“三张”重新组合成“葫芦”，剩下的1张为“单张”

步骤3. 当存在多个可能组合方案时，按如下规则排序取最大的一个组合方案：

- 依次对组合方案中的组合牌进行大小比较，规则同上；
- 当组合方案A中的第n个组合牌大于组合方案B中的第n个组合牌时，组合方案A大于组合方案B；

输入描述

第一行为空格分隔的N个正整数，每个整数取值范围[1,13]，N的取值范围[1,1000]

输出描述

经重新排列后的扑克牌数字列表，每个数字以空格分隔

用例

输入	1 3 3 3 2 1 5
输出	3 3 3 1 1 5 2

输入	1 3 3 3 2 1 5
说明	无

输入	4 4 2 1 2 1 3 3 3 4
输出	4 4 4 3 3 2 2 1 1 3
说明	无

题目解析

我的解题思路如下：

首先，将给定牌中，炸弹，三张，对子，单子先统计出来，即先不处理葫芦。

统计逻辑很简单，就是看某个牌面的数量：

- ≥ 4 ，那么该牌面可以组成炸弹
- $\text{count} == 3$ ，那么该牌面可以组成三张
- $\text{count} == 2$ ，那么该牌面可以组成对子
- $\text{count} == 1$ ，那么该牌面可以组成单张

统计完后，我们就可以先对炸弹进行排序，排序规则是：全部牌面数字加总由大到小排列

接着可以组合葫芦了，组合逻辑如下：

首先，需要先对三张、对子按照加总降序

然后，选取一个最大的三张，并比较第二大的三张的牌面和第一大的对子的牌面

- 如果对子牌面大，则直接组合最大的三张和最大对子为忽略
- 如果第二大三张牌面大，则拆分该三张为一个对子，一个单张，其中对子和最大的三张组合成葫芦，单张加入前面统计的单张数组

按照上面规则组合葫芦，直到三张用完。

注意上面逻辑是三张用完结束，而不是对子用完，因为还有一种情况就是对子先用完了，但是三张还有多个，此时我们要继续拆分小的三张来组合大三张为葫芦。

组合完葫芦后。

我们就可以对单张进行加总降序排序了，因为组合葫芦过程中，很可能产生新的单张。

最后，依次将统计并排序后的炸弹、葫芦、三张、对子、单张，打印出来

Java算法源码

2023.1.2 根据网友指正，发现[Java版本](#)代码存在问题，即LinkedList的push方法是头插，这将会导致葫芦的排序出错，换成add方法即可改正。

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

String str = sc.nextLine();
Integer[] arr = Arrays.stream(str.split("
")).map(Integer::parseInt).toArray(Integer[]::new);

System.out.println(getResult(arr));
}

public static String getResult(Integer[] arr) {
    HashMap<Integer, Integer> card = new HashMap<>();

    // 统计各种牌面的数量
    for (Integer num : arr) {
        if (card.containsKey(num)) {
            int val = card.get(num);
            card.put(num, ++val);
        } else {
            card.put(num, 1);
        }
    }

    // 统计组合, 4代表炸弹, 3+2代表葫芦, 3代表三张, 2代表对子, 1代表单张
    HashMap<String, LinkedList<Integer[]>> combine = new HashMap<>();
    combine.put("4", new LinkedList<Integer[]>());
    combine.put("3+2", new LinkedList<Integer[]>());
    combine.put("3", new LinkedList<Integer[]>());
    combine.put("2", new LinkedList<Integer[]>());
    combine.put("1", new LinkedList<Integer[]>());

    // 首先将初始组合统计出来
    Set<Integer> cardKeys = card.keySet();
    for (Integer num : cardKeys) {
        switch (card.get(num)) {
            case 3:
                combine.get("3").add(new Integer[] {num});
                break;
            case 2:
                combine.get("2").add(new Integer[] {num});
                break;
            case 1:
                combine.get("1").add(new Integer[] {num});
                break;
            default:
                combine
                    .get("4")
                    .add(
                        new Integer[] {
                            num, card.get(num)
                        }); // 由于炸弹可能有4张以上相同牌面组成, 因此既需要统计牌面num, 也需要统计牌数card[num]
        }
    }

    // 炸弹排序, 按照牌面值总和大小排序, 总和越大, 越高前, 牌面总和 = 牌面值 * 牌数
    combine.get("4").sort((a, b) -> b[0] * b[1] - a[0] * a[1]);
}

```

```

// 三张排序，牌面值越大，三张越大
combine.get("3").sort((a, b) -> b[0] - a[0]);

// 对子排序，牌面值越大，对子越大
combine.get("2").sort((a, b) -> b[0] - a[0]);

// 尝试组合出葫芦
while (combine.get("3").size() > 0) {
    // 如果对子用完，三张还有一个，那么可以直接结束循环
    if (combine.get("2").size() == 0 && combine.get("3").size() == 1) break;

    // 否则，选取一个最大的三张
    Integer san_top = combine.get("3").removeFirst()[0];

    Integer tmp;
    // 如果此时没有对子了，胡总和第二大的三张的牌面，比最大的对子牌面大，则可以拆分三张，组合出葫芦
    if (combine.get("2").size() == 0
        || (combine.get("3").size() > 0
            && combine.get("3").get(0)[0] > combine.get("2").get(0)[0])) {
        tmp = combine.get("3").removeFirst()[0];
        // 拆分三张为对子的话，会多出一个单张
        combine.get("1").add(new Integer[] {tmp});
    } else {
        // 如果对子牌面比三张大，则不需要拆分三张，直接使用对子组合出葫芦
        tmp = combine.get("2").removeFirst()[0];
    }
    combine.get("3+2").add(new Integer[] {san_top, tmp}); // 葫芦元素含义：[三张牌面，对子牌面]
}

// 处理完葫芦后，就可以对单张进行降序了（因为组合葫芦的过程中，可能产生新的单张，因此单张排序要在葫芦组合得到后进行）
combine.get("1").sort((a, b) -> b[0] - a[0]);

// ans存放题解
ArrayList<Integer> ans = new ArrayList<>();

// 首先将炸弹放到ans中
for (Integer[] vals : combine.get("4")) {
    int score = vals[0];
    int count = vals[1];
    for (int i = 0; i < count; i++) {
        ans.add(score);
    }
}

// 然后将葫芦放大ans中
for (Integer[] vals : combine.get("3+2")) {
    int san = vals[0];
    int er = vals[1];
    for (int i = 0; i < 3; i++) ans.add(san);
    for (int i = 0; i < 2; i++) ans.add(er);
}

```

```

// 之后将三张放到ans中
for (Integer[] vals : combine.get("3")) {
    for (int i = 0; i < 3; i++) ans.add(vals[0]);
}

// 接着是对子放到ans中
for (Integer[] vals : combine.get("2")) {
    for (int i = 0; i < 2; i++) ans.add(vals[0]);
}

// 最后是单张放到ans中
for (Integer[] vals : combine.get("1")) {
    ans.add(vals[0]);
}

StringJoiner sj = new StringJoiner(" ", "", "");
for (Integer an : ans) {
    sj.add(an + "");
}

return sj.toString();
}
}

```

80.查找充电设备组合

题目描述

某个充电站，可提供 n 个充电设备，每个充电设备均有对应的输出功率。任意个充电设备组合的输出功率总和，均构成功率集合 P 的1个元素。功率集合 P 的最优元素，表示最接近充电站最大输出功率 $p_{\max}$ 的元素。

输入描述

输入为3行：

- 第1行为充电设备个数 n
- 第2行为每个充电设备的输出功率
- 第3行为充电站最大输出功率 $p_{\max}$

输出描述

功率集合 P 的最优元素

备注

- 充电设备个数 $n > 0$
- 最优元素必须小于或等于充电站最大输出功率 $p_{\max}$

用例

输入	4 50 20 20 60 90
输出	90
说明	当充电设备输出功率50、20、20组合时，其输出功率总和为90，最接近充电站最大充电输出功率，因此最优元素为90。

输入	2 50 40 30
输出	0
说明	所有充电设备的输出功率组合，均大于充电站最大充电输出功率30，此时最优元素值为0。

题目解析

本题可以当成[01背包问题](#)处理。其中：

第3行充电站最大输出功率 $p_{\max}$ 可以当成 背包承重

第2行每个充电设备的输出功率 可以当成 不同物品的重量以及价值，即重量=价值

现在要求：背包承重下能装入的最大价值。

关于01背包问题，大家可以看[算法设计 - 01背包问题_伏城之外的博客-CSDN博客](#)

Java算法源码

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] p = new int[n];
        for (int i = 0; i < n; i++) {
            p[i] = sc.nextInt();
        }
    }
}
```

```

int p_max = sc.nextInt();

System.out.println(getResult(n, p, p_max));
}

public static int getResult(int n, int[] p, int p_max) {
    int[][] dp = new int[n + 1][p_max + 1];

    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= p_max; j++) {
            if (i == 0 || j == 0) continue;

            // 注意这里p[i-1]中i-1不会越界，因为i=0时不会走到这一步，另外i-1是为了防止尾越界，因为dp矩阵的行数是n+1，因此p[i]可能会尾越界，而p[i-1]就不会越界了
            if (p[i - 1] > j) {
                dp[i][j] = dp[i - 1][j];
            } else {
                dp[i][j] = Math.max(dp[i - 1][j], p[i - 1] + dp[i - 1][j - p[i - 1]]);
            }
        }
    }

    return dp[n][p_max];
}
}

```

81.上班之路

题目描述

Jungle 生活在美丽的蓝鲸城，大马路都是方方正正，但是每天马路的封闭情况都不一样。

地图由以下元素组成：

- 1) "." — 空地，可以达到；
- 2) "" — 路障，不可达到；
- 3) "S" — Jungle的家；
- 4) "T" — 公司。

其中我们会限制Jungle拐弯的次数，同时Jungle可以清除给定个数的路障，现在你的任务是计算Jungle* 是否可以从家里出发到达公司。

输入描述

输入的第一行为两个整数t,c ($0 \leq t, c \leq 100$) ,t代表可以拐弯的次数，c代表可以清除的路障个数。

输入的第二行为两个整数n,m ($1 \leq n, m \leq 100$) ,代表地图的大小。

接下来是n行包含m个字符的地图。n和m可能不一样大。

我们保证地图里有S和T。

输出描述

输出是否可以从家里出发到达公司，是则输出YES，不能则输出NO。

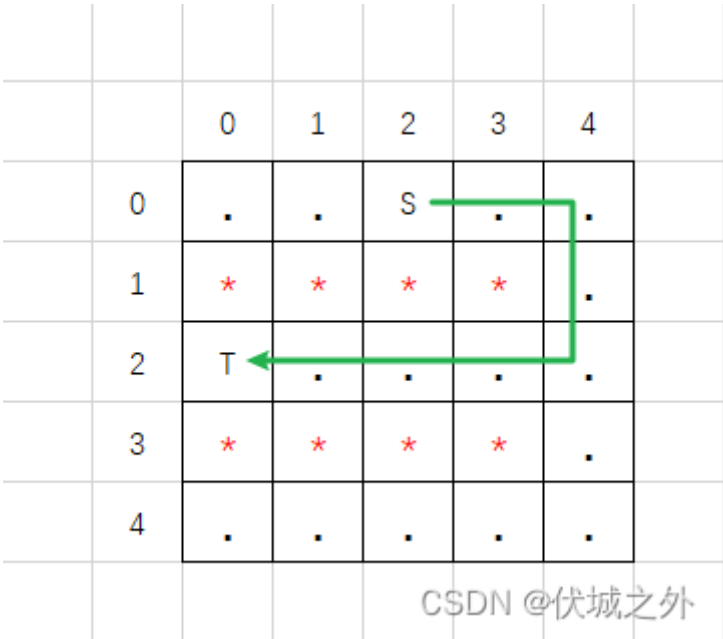
用例

输入	2 0 5 5 ..S.. T....
输出	YES
说明	无

输入	1 2 5 5 .S. * ..*.. * T....
输出	NO
说明	该用例中，至少需要拐弯1次，清除3个路障，所以无法到达

题目解析

用例1图示



用例2图示

		0	1	2	3	4	
	0	.	*	S	*	.	
	1	*	*	*	*	*	
	2	.	.	*	.	.	
	3	*	*	*	*	*	
	4	T	.	.	.	.	
CSDN @伏城之外							

本题和[迷宫问题](#)很像，都可以使用深度优先搜索来做，相较于其他迷宫问题，本题对找终点的运动路径做了如下限制：

- 1、最多只能变更t次数运动方向
- 2、支持破壁，即清除障碍，但是最多只能破壁c次数。

因此，我们在[深度优先搜索](#)之前，需要定义两个变量：

- ut：已变更了几次运动方向
- uc：已破壁几次

如果深度优先搜索的下一步的代价是 $ut > t$ ，或者 $uc > c$ ，则说明下一步无法继续走了。

Java算法源码

```
import java.util.HashSet;
import java.util.Scanner;

public class Main {
    static int t, c, n, m;
    static String[][] matrix;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        t = sc.nextInt();
        c = sc.nextInt();

        n = sc.nextInt();
        m = sc.nextInt();

        matrix = new String[n][m];
        for (int i = 0; i < n; i++) {
            matrix[i] = sc.next().split("");
        }
    }
}
```

```

        System.out.println(getResult());
    }

    public static String getResult() {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if ("S".equals(matrix[i][j])) {
                    HashSet<Integer> path = new HashSet<>();
                    path.add(i * m + j);
                    return dfs(i, j, 0, 0, 0, path) ? "YES" : "NO";
                }
            }
        }
        return "NO";
    }
}

```

// 元素含义【行偏移，列偏移，运动方向】，运动方向：1上，2下，3左，4右

```
static int[][] offsets = {{-1, 0, 1}, {1, 0, 2}, {0, -1, 3}, {0, 1, 4}};
```

```
/**
```

```
 * @param si 当前位置横坐标
```

```
 * @param sj 当前位置纵坐标
```

```
 * @param ut 已拐弯次数
```

```
 * @param uc 已破壁次数
```

```
 * @param lastDirect 上一次运动方向，初始为0，表示第一次运动不会造成拐弯
```

```
 * @param path 行动路径，用于记录走过的位置，避免走老路
```

```
 * @return 终点是否可达
```

```
 */
```

```
public static boolean dfs(int si, int sj, int ut, int uc, int lastDirect,
    HashSet<Integer> path) {
```

```
    // 如果当前位置就是目的地，则返回true
```

```
    if ("T".equals(matrix[si][sj])) {
```

```
        return true;
```

```
    }
```

```
    // 有四个方向选择走下一步
```

```
    for (int[] offset : offsets) {
```

```
        int direct = offset[2];
```

```
        int newI = si + offset[0];
```

```
        int newJ = sj + offset[1];
```

```
        // flag1记录本次运动是否拐弯
```

```
        boolean flag1 = false;
```

```
        // flag2记录本次运动是否破壁
```

```
        boolean flag2 = false;
```

```
        // 如果下一步位置没有越界，则进一步检查
```

```
        if (newI >= 0 && newI < n && newJ >= 0 && newJ < m) {
```

```
            // 如果下一步位置已经走过了，则是老路，可以不走
```

```
            int pos = newI * m + newJ;
```

```
            if (path.contains(pos)) continue;
```

```
            // 如果下一步位置需要拐弯
```

```
            if (lastDirect != 0 && lastDirect != direct) {
```

```
                // 如果拐弯次数用完了，则不走
```

```

        if (ut + 1 > t) continue;
        // 否则就走
        flag1 = true;
    }

    // 如果下一步位置需要破壁
    if ("*".equals(matrix[newI][newJ])) {
        // 如果破壁次数用完了，则不走
        if (uc + 1 > c) continue;
        // 否则就走
        flag2 = true;
    }

    // 记录已走过的位置
    path.add(pos);

    // 继续下一步递归
    boolean res = dfs(newI, newJ, ut + (flag1 ? 1 : 0), uc + (flag2 ? 1 : 0), direct, path);

    // 如果某路径可以在给定的t,c下，到达目的地T，则返回true
    if (res) return true;

    // 否则，回溯
    path.remove(pos);
}
}
return false;
}
}

```

82.天然蓄水库

题目描述

公元2919年，人类终于发现了一颗宜居星球——X星。

现在想在X星一片连绵起伏的山脉间建一个天然蓄水库，如何选取水库边界，使蓄水量最大？

要求：

- 山脉用正整数数组s表示，每个元素代表山脉的高度。
- 选取山脉上两个点作为蓄水库的边界，则边界内的区域可以蓄水，蓄水量需排除山脉占用的空间
- 蓄水量的高度为两边界的最小值。
- 如果出现多个满足条件的边界，应选取距离最近的一组边界。

输出边界下标（从0开始）和最大蓄水量；如果无法蓄水，则返回0，此时不返回边界。

例如，当山脉为s=[3,1,2]时，则选取s[0]和s[2]作为水库边界，则蓄水量为1，此时输出：0 2:1

当山脉s=[3,2,1]时，不存在合理的边界，此时输出：0。

输入描述

一行正整数，用空格隔开，例如输入

1 2 3

表示s=[1,2,3]

输出描述

当存在合理的水库边界时，输出左边界、空格、右边界、英文冒号、蓄水量；例如

0 2:1

当不存在合理的书库边界时，输出0；例如

0

备注

- $1 \leq \text{length}(s) \leq 10000$
- $0 \leq s[i] \leq 10000$

用例

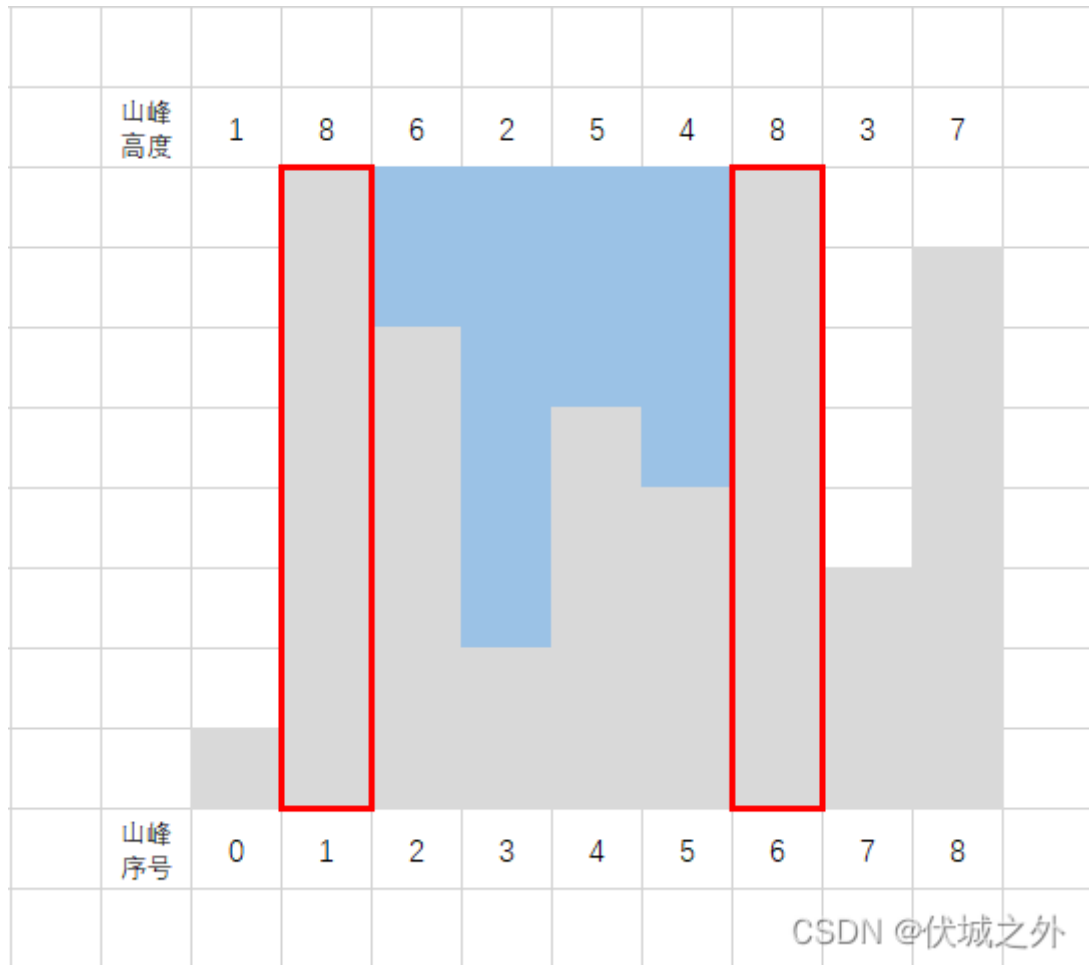
输入	1 9 6 2 5 4 9 3 7
输出	1 6:19
说明	经过分析，选取s[1]和s[6]，水库蓄水量为19（3+7+4+5）

输入	1 8 6 2 5 4 8 3 7
输出	1 6:15
说明	经过分析，选取s[1]和s[8]时，水库蓄水量为15；同样选取s[1]和s[6]时，水库蓄水量也为15。由于后者下标距离小（为5），故应选取后者。

输入	1 2 3
输出	0
说明	不存在合理的水库边界

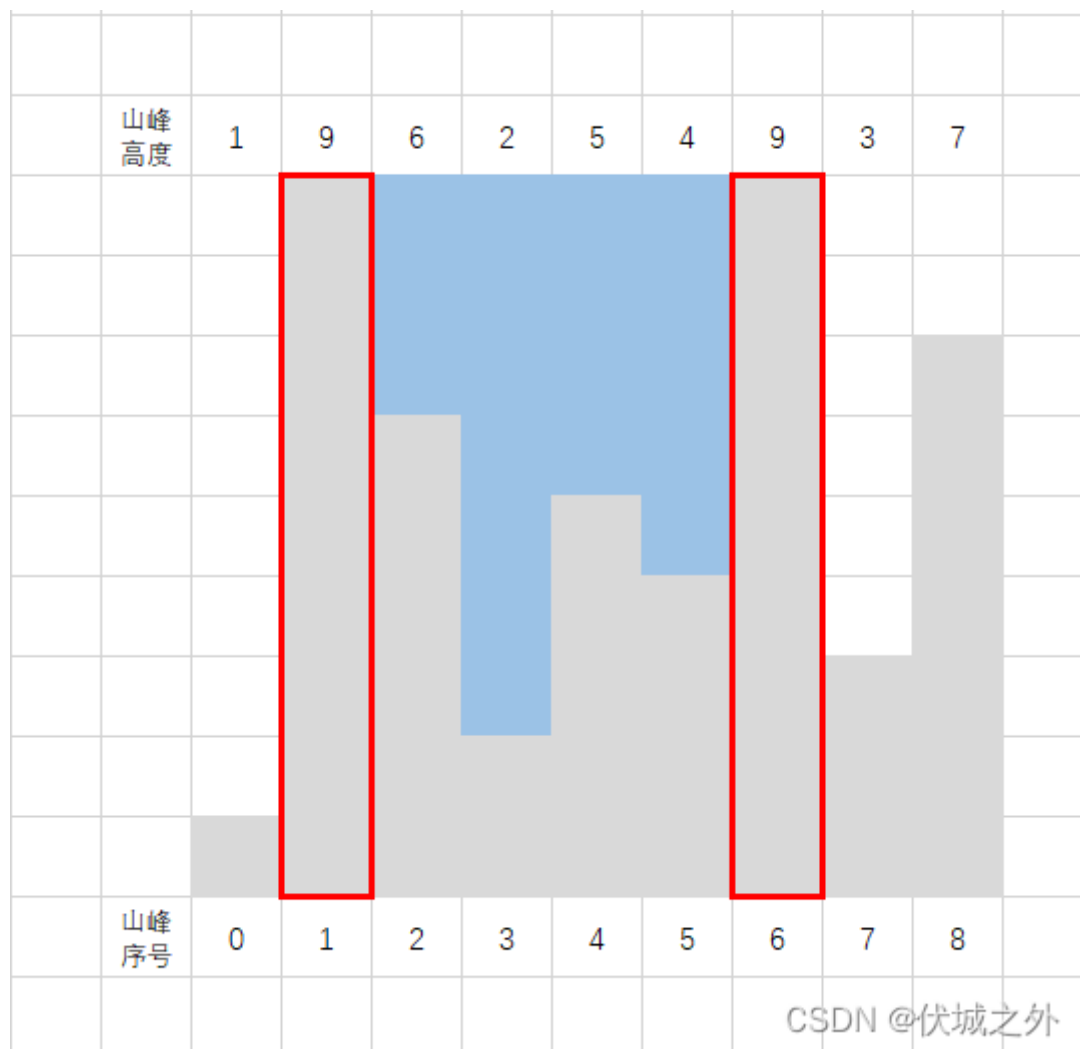
题目解析

用例1图示如下:



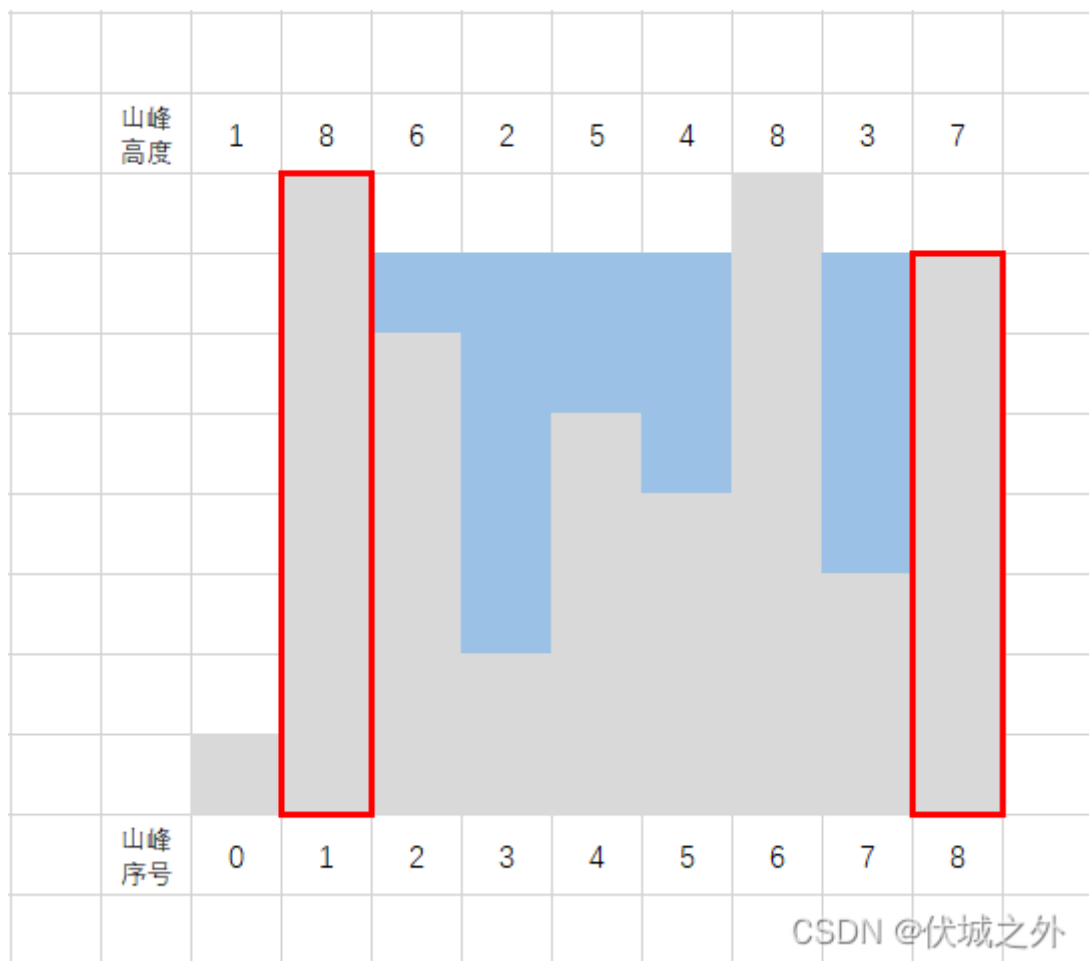
选择山峰1和山峰6作为边界, 则可获得最大蓄水量19

用例2图示如下



选择山峰1和山峰6作为边界，则可获得最大蓄水量15

其实用例2还可以选择山峰1和山峰8作为边界，也可以获得最大蓄水量15，如下图所示



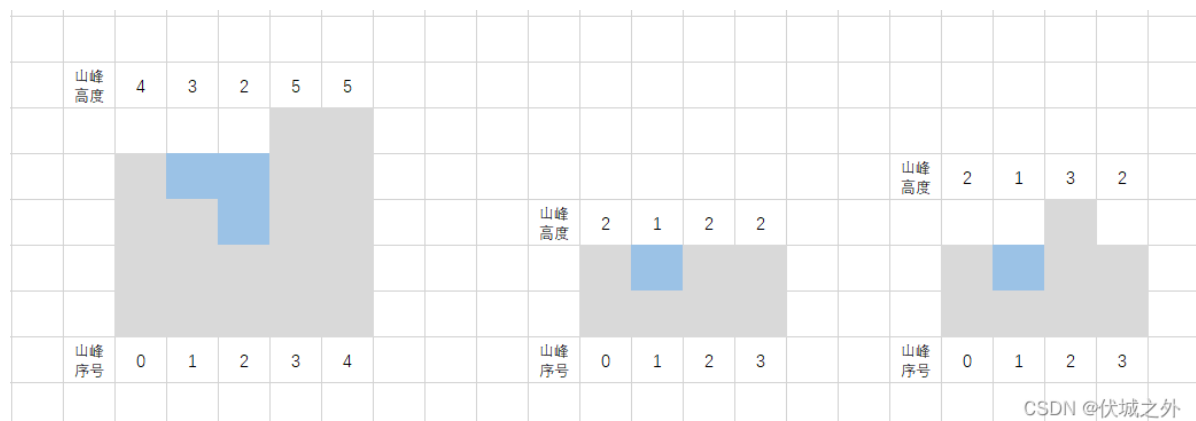
但是此时两边界山峰的距离是6，相较于选择山峰1，6作为边界时距离4而言，更远。

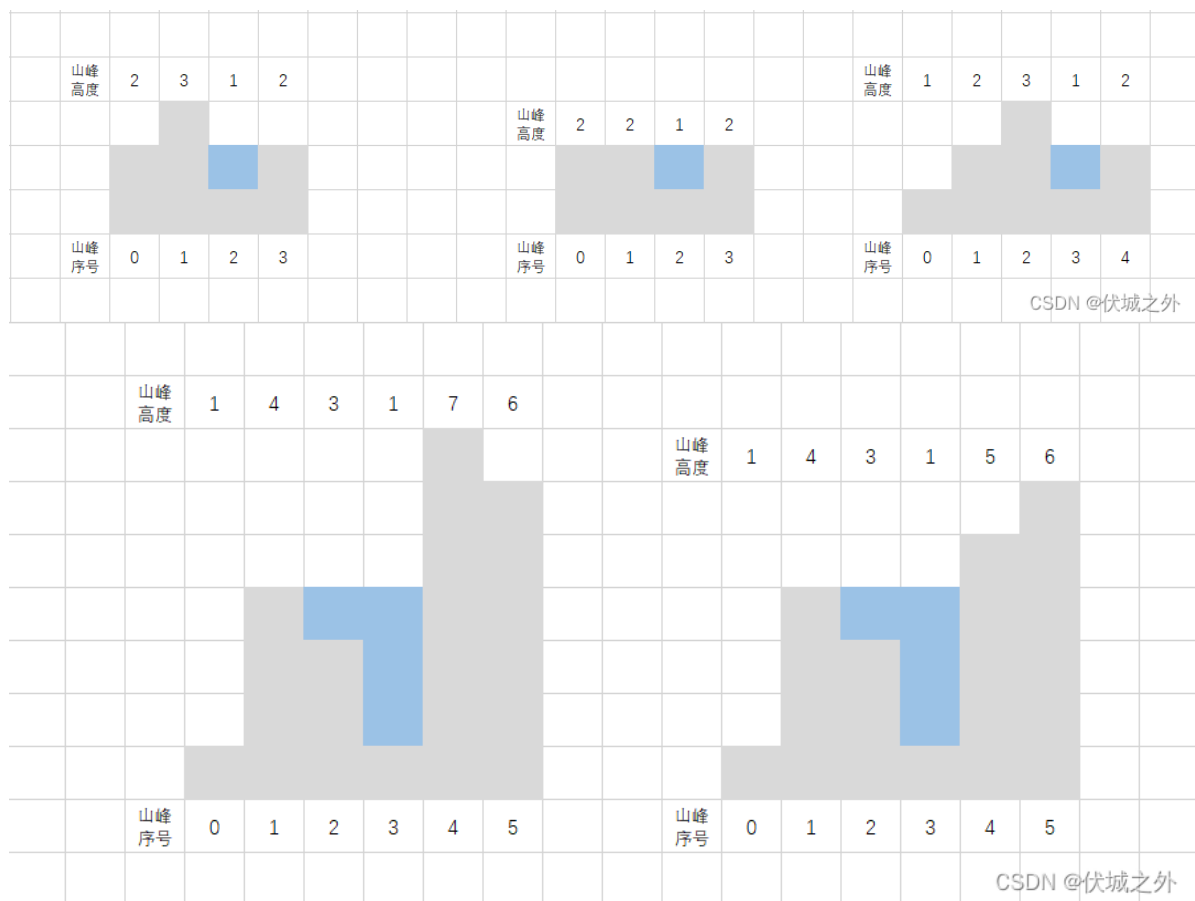
按照题目要求，我们需要找到：蓄水量最大的，且距离最近的两个边界山峰。

我一开始的解题思路是[双指针](#)，类似于下面这题

[华为OD机试 - 太阳能板最大面积 \(java & JS & Python\) 伏城之外的博客-CSDN博客](#)

但是经过如下几个用例测试，发现本题无法像上面链接题目一样找到一个 $O(n)$ 的解法，双边指针无法找到一个固定的策略进行运动。

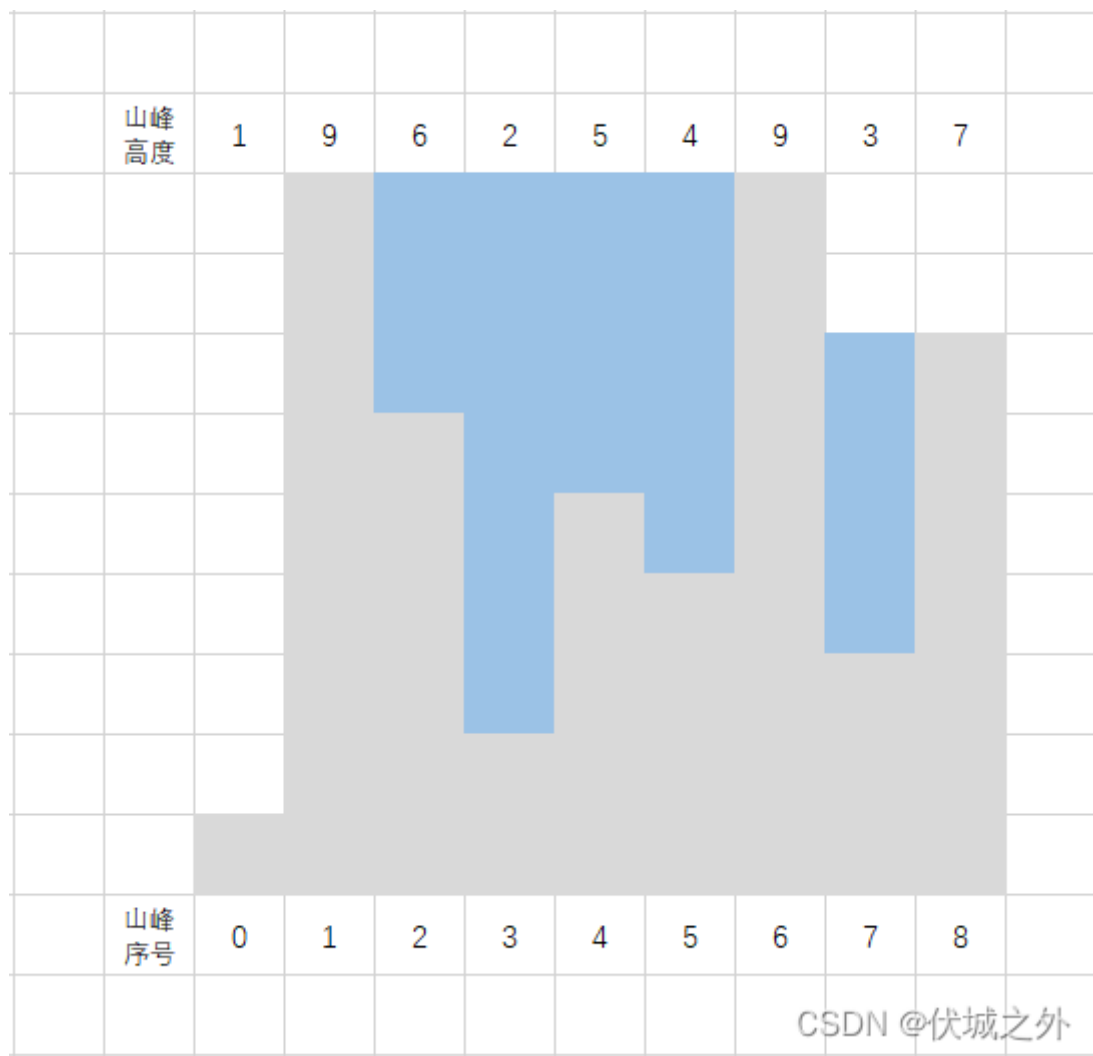




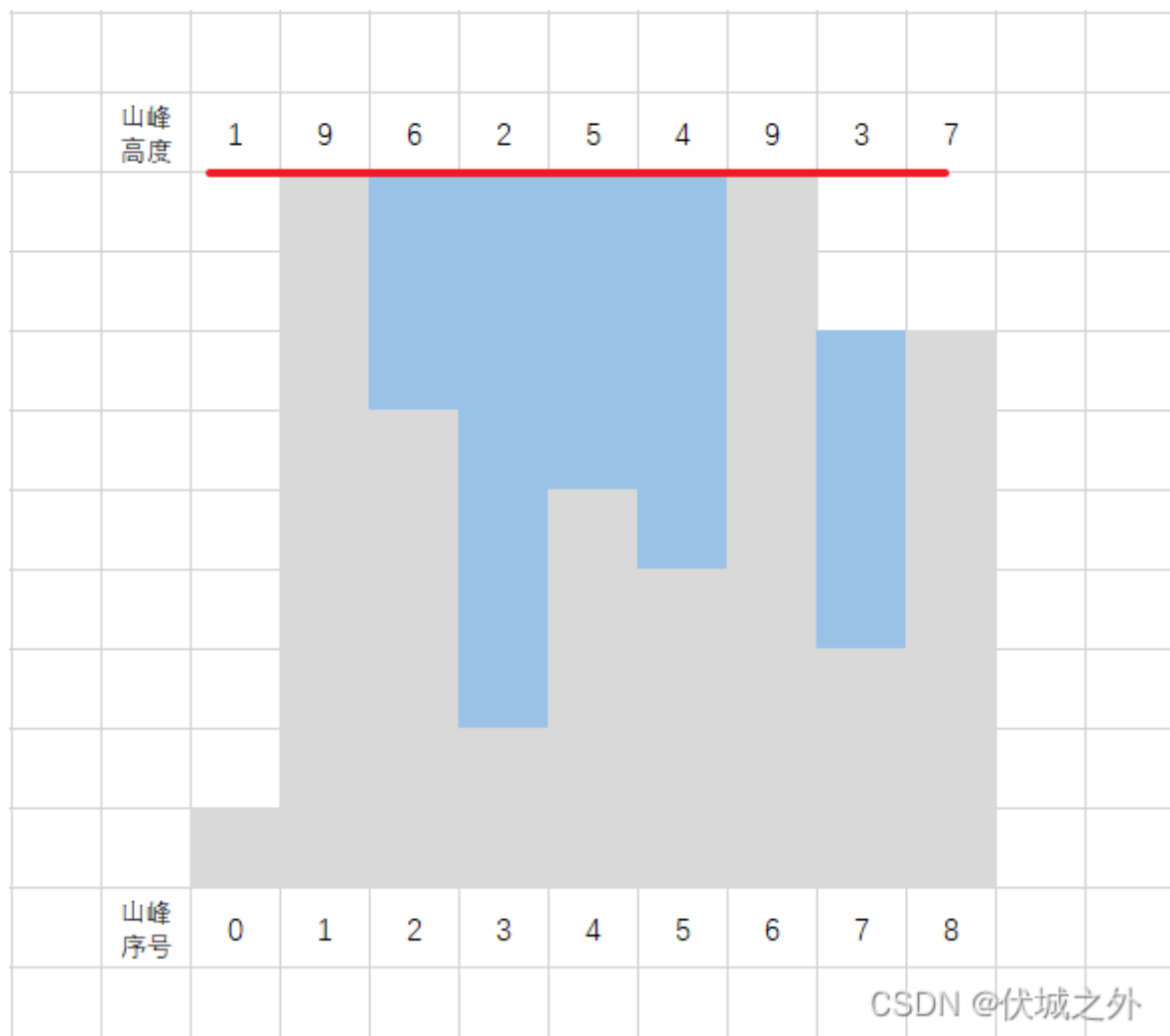
因此，我开始寻找其他的思路，直到发现了[LeetCode - 42 接雨水 伏城之外的博客-CSDN博客](#)

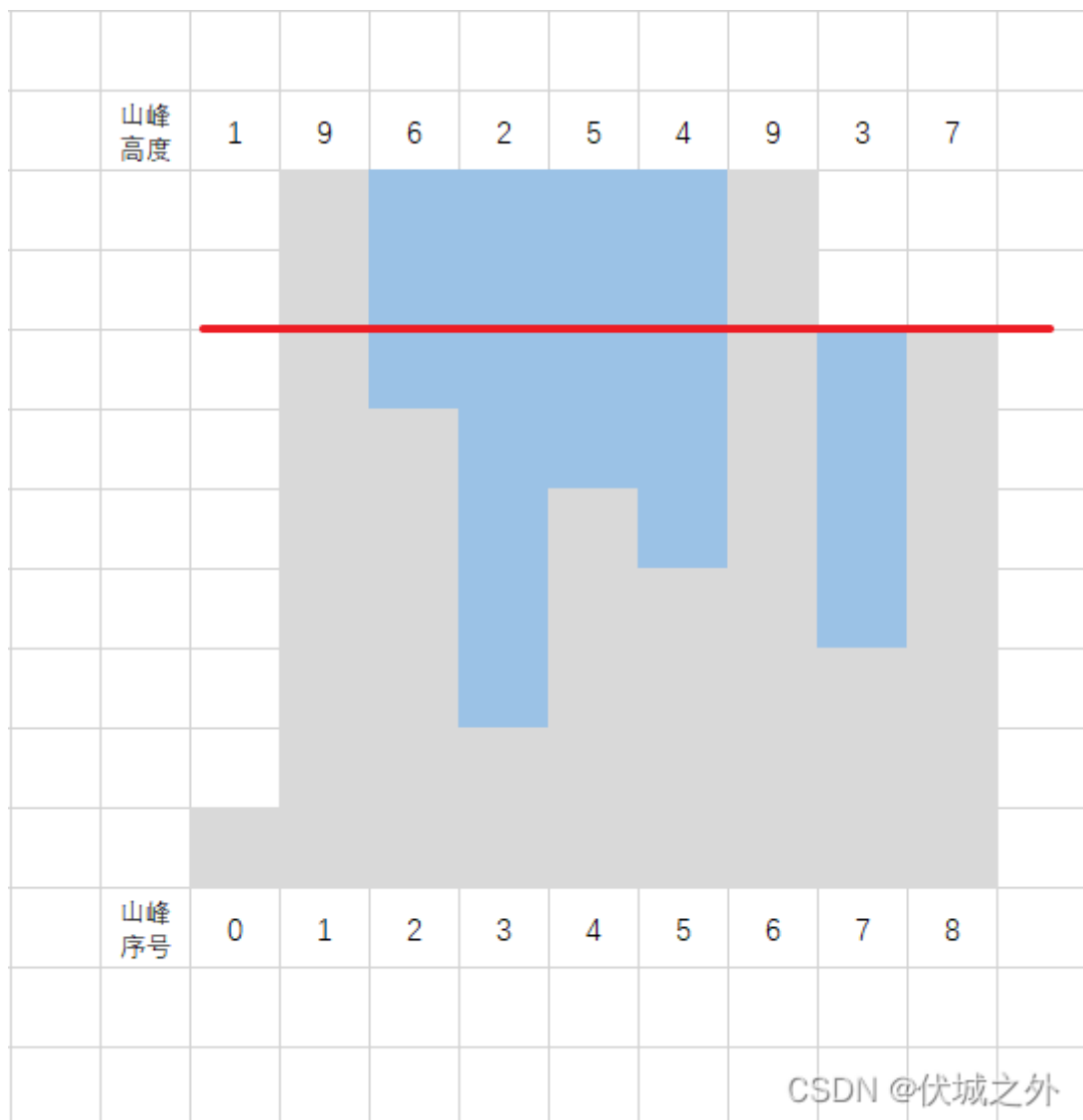
其实，我们不应该从横向来思考本题，可以从纵向来思考本题。什么意思呢？

我们按照接雨水那个思路，把用例1中所有能接水的山峰全部接满，即如下图所示



此时从纵向来看只有有两条水位线，如下图所示

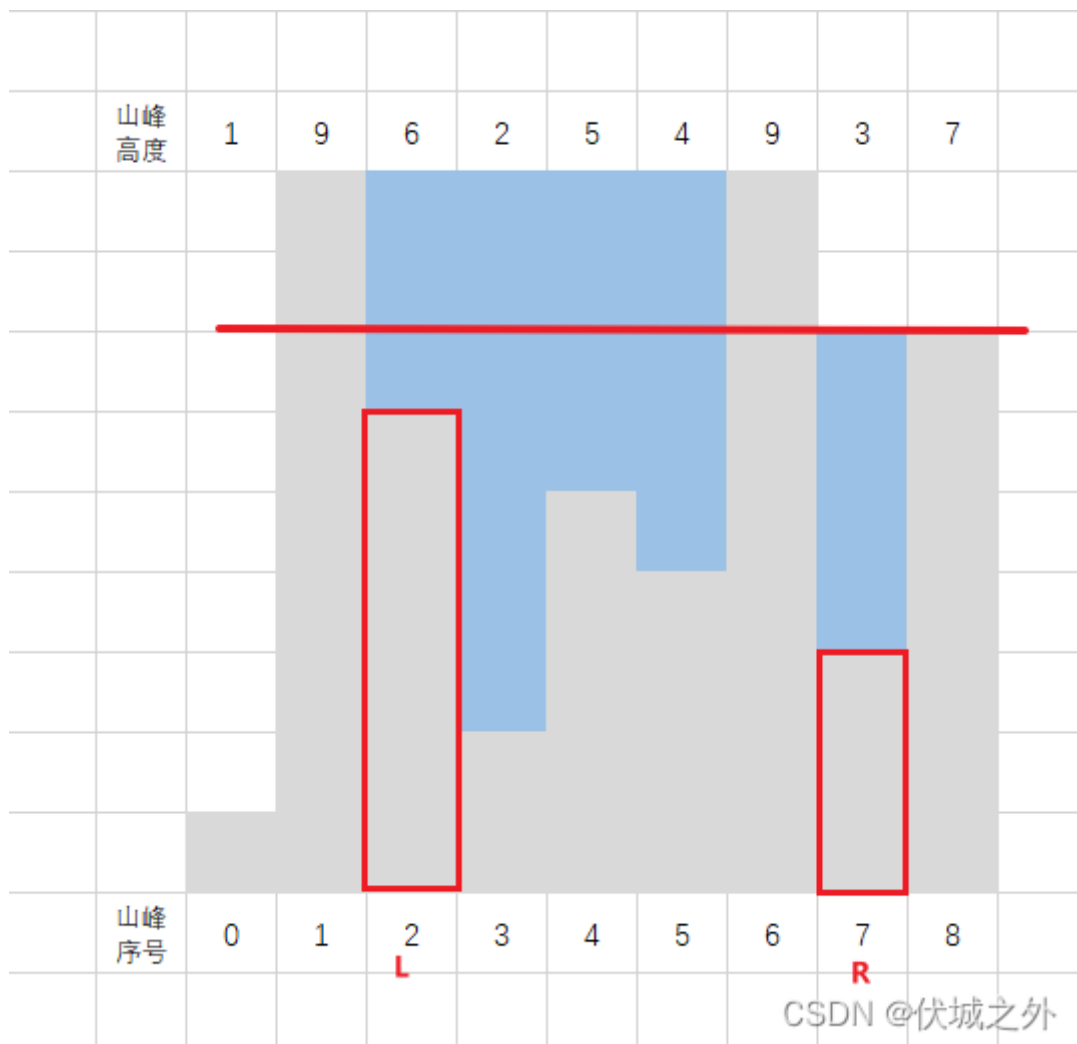




从上图可以看出，每条水位线都有都可能与多个山峰相交，但是我们只需要关注：

- 两端的
- 能够达到该水位线要求得山峰

如下图所示：



上图中，L山峰和R山峰是可以达到该水位线要求的最外层的两端山峰，此时这两座山峰之间的每个山峰的储水量就是该水位线最大的储水量。

而此时边界山峰为L-1，和R+1。

Java算法源码

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Integer[] h =
            Arrays.stream(sc.nextLine().split("
"))
                .map(Integer::parseInt)
                .toArray(Integer[]::new);

        System.out.println(getResult(h));
    }

    public static String getResult(Integer[] h) {
        int n = h.length;

        // left[i] 记录 第 i 个山峰左边的最高山峰
        int[] left = new int[n];
```

```

for (int i = 1; i < n; i++) left[i] = Math.max(left[i - 1], h[i - 1]);

// right[i] 记录 第 i 个山峰右边的最高山峰
int[] right = new int[n];
for (int i = n - 2; i >= 0; i--) right[i] = Math.max(right[i + 1], h[i + 1]);

// lines[i] 记录 第 i 个山峰的水位线高度
int[] lines = new int[n];
// lineSet记录有哪些水位线
HashSet<Integer> lineSet = new HashSet<>();
for (int i = 1; i < n - 1; i++) {
    int water = Math.max(0, Math.min(left[i], right[i]) - h[i]); // water 记录
    第 i 个山峰可以储存多少水

    if (water != 0) {
        // 第 i 个山峰的水位线高度
        lines[i] = water + h[i];
        lineSet.add(lines[i]);
    }
}

// ans数组含义: [左边界, 右边界, 储水量]
int[] ans = {0, 0, 0};

// 遍历每一个水位线
for (int line : lineSet) {

    // 满足该水位线的最左侧山峰位置l
    int l = 0;
    while (lines[l] < line || h[l] >= line) {
        l++;
    }

    // 满足该水位线的最右侧山峰位置r
    int r = n - 1;
    while (lines[r] < line || h[r] >= line) {
        r--;
    }

    // 该水位线的总储水量
    int sum = 0;
    for (int i = l; i <= r; i++) {
        sum += Math.max(0, line - h[i]);
    }

    // 记录最大的储水量
    if (sum > ans[2]) {
        ans[0] = l - 1;
        ans[1] = r + 1;
        ans[2] = sum;
    }

    // 如果有多个最多储水量选择, 则选择边界山峰距离最短的
    else if (sum == ans[2]) {
        int curDis = r - l + 1;

```

```

        int minDis = ans[1] - ans[0] - 1;

        if (curDis < minDis) {
            ans[0] = l - 1;
            ans[1] = r + 1;
        }
    }
}

if (ans[2] == 0) return "0";

return ans[0] + " " + ans[1] + ":" + ans[2];
}
}

```

83.垃圾短信识别

题目描述

大众对垃圾短信深恶痛绝，希望能对垃圾短信发送者进行识别，为此，很多软件增加了垃圾短信的识别机制。

经分析，发现正常用户的短信通常具备交互性，而垃圾短信往往都是大量单向的短信，按照如下规则进行垃圾短信识别：

本题中，发送者A符合以下条件之一的，则认为A是垃圾短信发送者：

- A发送短信的接收者中，没有发过短信给A的人数 $L > 5$ ；
- A发送的短信数 - A接收的短信数 $M > 10$ ；
- 如果存在X，A发送给X的短信数 - A接收到X的短信数 $N > 5$ ；

输入描述

第一行是条目数，接下来几行是具体的条目，每个条目，是一对ID，第一个数字是发送者ID，后面的数字是接收者ID，中间空格隔开，所有的ID都为[无符号整型](#)，ID最大值为100；

同一个条目中，两个ID不会相同（即不会自己给自己发消息）

最后一行为指定的ID

输出描述

输出该ID是否为垃圾短信发送者，并且按序列输出 L M 的值（由于 N 值不唯一，不需要输出）；输出均为字符串。

用例

输入	15 1 2 1 3 1 4 1 5 1 6 1 7 1 8 1 9 1 10 1 11 1 12 1 13 1 14 14 1 1 15 1
输出	true 13 13
说明	true 表示1是垃圾短信发送者，两个数字，代表发送者1对应的L和M值。true 13 13中间以一个空格分割。注意true是字符串输出。

输入	15 1 2 1 3 1 4 1 5 1 6 1 7 1 8 1 9 1 10 1 11 1 12 1 13 1 14 14 1 1 15 2
输出	false 0 -1
说明	无

题目解析

感觉是纯[逻辑题](#)，解析请看代码注释。

Java算法源码

```
import java.util.*;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[][] arr = new int[n][2];
        for (int i = 0; i < n; i++) {
            arr[i][0] = sc.nextInt();
            arr[i][1] = sc.nextInt();
        }

        int id = sc.nextInt();

        System.out.println(getResult(id, arr));
    }

    public static String getResult(int tid, int[][] arr) {
        // send记录 tid发送短信给“哪些人”
        LinkedList<Integer> send = new LinkedList<>();
        // receive记录 “哪些人”发送短信给tid
        LinkedList<Integer> receive = new LinkedList<>();

        // sendCount记录 tid发送了“几次”（对象属性值）短信给某个人（对象属性）
        HashMap<Integer, Integer> sendCount = new HashMap<>();
        // receiveCount记录 某人（对象属性）发送了“几次”（对象属性值）短信给tid
        HashMap<Integer, Integer> receiveCount = new HashMap<>();
```



```

for (int[] ele : arr) {
    int sid = ele[0];
    int rid = ele[1];

    if (sid == tid) {
        send.addLast(rid);
        sendCount.put(rid, sendCount.getOrDefault(rid, 0) + 1);
    }

    if (rid == tid) {
        receive.addLast(sid);
        receiveCount.put(sid, receiveCount.getOrDefault(sid, 0) + 1);
    }
}

HashSet<Integer> sendSet = new HashSet<>(send);
HashSet<Integer> receiveSet = new HashSet<>(receive);

// connect记录和tid有交互的id
List<Integer> connect =
send.stream().filter(receiveSet::contains).collect(Collectors.toList());

int l = sendSet.size() - connect.size();
int m = send.size() - receive.size();

boolean isSpammers = l > 5 || m > 10;

// 如果已经通过l和m确定了tid是垃圾短信发送者，那就不需要再确认n了，否则还是需要确认n
if (!isSpammers) {
    for (Integer id : connect) {
        if (sendCount.getOrDefault(id, 0) - receiveCount.getOrDefault(id, 0) >
5) {
            // 一旦发现x,则可以判断，则确定tid是垃圾短信发送者，可提前结束循环
            isSpammers = true;
            break;
        }
    }
}

return isSpammers + " " + l + " " + m;
}
}

```

84.信号发射和接收

题目描述

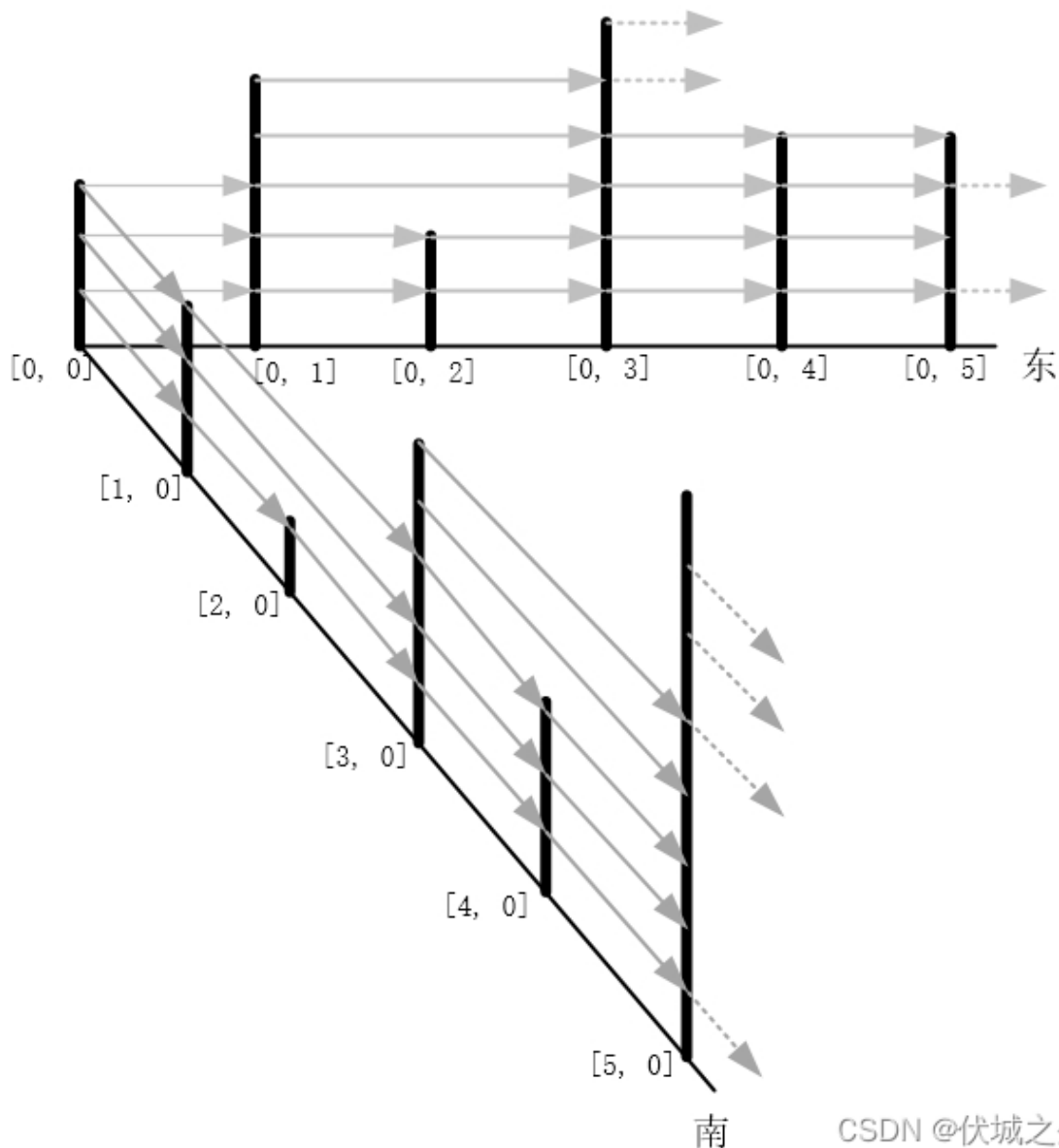
有一个二维的天线矩阵，每根天线可以向其他天线发射信号，也能接收其他天线的信号，为了简化起见，我们约定每根天线只能向东和向南发射信号，换言之，每根天线只能接收东向或南向的信号。

每根天线有自己的高度`anth`，每根天线的高度存储在一个二维数组中，各个天线的位置用`[r, c]`表示，`r`代表天线的行位置（从0开始编号），`c`代表天线的列位置（从0开始编号）。

在某一方向（东向或南向），某根天线可以收到多根其他天线的信号（也可能收不到任何其他天线的信号），对任一天线X和天线Y，天线X能接收到天线Y的条件是：

1. 天线X在天线Y的东边或南边
2. 天线X和天线Y之间的其他天线的高度都低于天线X和天线Y，或天线X和天线Y之间无其他天线，即无遮挡。

如下图示意：



CSDN @伏城之外

在天线矩阵的第0行上：

- 天线 $[0, 0]$ 接收不到任何其他天线的信号，
- 天线 $[0, 1]$ 可以接收到天线 $[0, 0]$ 的信号，
- 天线 $[0, 2]$ 可以接收到天线 $[0, 1]$ 的信号，
- 天线 $[0, 3]$ 可以接收到天线 $[0, 1]$ 和天线 $[0, 2]$ 的信号，
- 天线 $[0, 4]$ 可以接收到天线 $[0, 3]$ 的信号，
- 天线 $[0, 5]$ 可以接收到天线 $[0, 4]$ 的信号；

在天线的第0列上：

- 天线 $[0, 0]$ 接收不到任何其他天线的信号，
- 天线 $[1, 0]$ 可以接收到天线 $[0, 0]$ 的信号，

- 天线[2, 0]可以接收到天线[1, 0]的信号,
- 天线[3, 0]可以接收到天线[1, 0]和天线[2, 0]的信号,
- 天线[4, 0]可以接收到天线[3, 0]的信号,
- 天线[5, 0]可以接收到天线[3, 0]和天线[4, 0]的信号;

给一个m行n列的矩阵（二维数组），矩阵存储各根天线的高度，求出每根天线可以接收到多少根其他天线的信号，结果输出到m行n列的矩阵（二维矩阵）中。

输入描述

输入为1个m行n列的矩阵（二维矩阵）anth[m][n]，矩阵存储各根天线的高度，高度值anth[r][c]为大于0的整数。

第一行为输入矩阵的行数和列数，如：

m n

第二行为输入矩阵的元素值，按行输入，如：

anth[0][0] anth[0][1] ... anth[0][n-1] anth[1][0] anth[1][1] ... anth[1][n-1] ... anth[m-1][0] ... anth[m-1][n-1]

输出描述

输出一个m行n列的矩阵（二维数组）ret[m][n]，矩阵存储每根天线能收到多少根其他天线的信号，根数为ret[r][c]。

第一行为输出矩阵的行数和列数，如：

m n

第二行为输出矩阵的元素值，按行输出，如：

ret[0][0] ret[0][1] ... ret[0][n-1] ret[1][0] ret[1][1] ... ret[1][n-1] ... ret[m-1][0] ... ret[m-1][n-1]

备注

- $1 \leq m \leq 500$
- $1 \leq n \leq 500$
- $0 < anth[r][c] < 10^5$

用例

输入	1 6 2 4 1 5 3 3
输出	1 6 0 1 1 2 1 1
说明	输入为1行6列的天线矩阵的高度值[2 4 1 5 3 3]输出为1行6列的结果矩阵[0 1 1 2 1 1]

输入	2 6 2 5 4 3 2 8 9 7 5 10 10 3
输出	2 6 0 1 1 1 1 4 1 2 2 4 2 2
说明	输入为2行6列的天线矩阵高度值[2 5 4 3 2 8][9 7 5 10 10 3]输出为2行6列的结果矩阵[0 1 1 1 1 4][1 2 2 4 2 2]

题目解析

首先，本题我们需要从输入的一维数组中解析出二维天线矩阵，JS的实现略微麻烦，具体逻辑请看源码。

下面我们以用例1为例子，来解析本题，如下图是用例1的二维天线矩阵anth

	2	4	1	5	3	3	

我们从anth[0][0]开始发射，本题说了，天线信号发射只能向东或者向南发射，即如下图所示

	2	→ 4	1	5	3	3	
	↓						

由于用例1只有一层，因此只需要考虑向东发射信号。

而东边的天线是否能收到信号，也有前提条件，即“发射天线”与“接收天线”之间的天线的高度都低于“发射天线”与“接收天线”，或者“发射天线”与“接收天线”之间没有其他天线

因此，上面用例1中anth[0][0]可以发射到anth[0][1]，因为它们之间没有其他天线

	2	4	1	5	3	3	

但是anth[0][0]不能发射到anth[0][2]，因为它们之间的天线大于等于了它们

	2	4	1	5	3	3	

其实这一步，不需要走到 $\text{anth}[0][2]$ ，因为 $\text{anth}[0][1] \geq \text{anth}[0][0]$ ，因此 $\text{anth}[0][0]$ 必然会被 $\text{anth}[0][1]$ 遮挡，导致无法继续向东发射。

因此，对于 $\text{anth}[0][0]$ 作为发射点的所有情况已经讨论完了，它只有一个接收点，那就是 $\text{anth}[0][1]$ 。

接下来继续讨论 $\text{anth}[0][1]$ 作为发射点

	2	4	1	5	3	3
				CSDN @伏城之外		

首先，相邻的 $\text{anth}[0][2]$ 肯定能接收到信号。

并且由于 $\text{anth}[0][2]$ 小于 $\text{anth}[0][1]$ ，因此无法完全将 $\text{anth}[0][1]$ 发射的信号遮挡，

	2	4	1	5	3	3
				CSDN @伏城之外		

因此 $\text{anth}[0][3]$ 可以接收到 $\text{anth}[0][1]$ 的信号？

注意：这里我打了一个问号，因为题目要求，如果“发射天线”和“接收天线”之间有其他天线，那么其他天线的高度必须低于“发射天线”和“接收天线”。

上面打问号的原因是：我们只判断了中间天线 $\text{anth}[0][2] < \text{anth}[0][1]$ 发射天线，并没有判断中间天线 $\text{anth}[0][2]$ 也小于 $\text{anth}[0][3]$ 接收天线。

- 而，这里中间天线 $\text{anth}[0][2]$ 确实是小于 $\text{anth}[0][3]$ 接收天线的，因此 $\text{anth}[0][3]$ 可以接收到 $\text{anth}[0][1]$ 的信号。

如果，我们采用双重for，外层遍历发射天线，内层遍历接收天线，则还需一个for遍历求得发射天线和接收天线之间的：所有中间天线中最最高的高度 h ，如果这个高度 h 大于等于发射天线，或者接收天线，则发射天线和接收天线之间无法进行通信。

这其实已经是三重for了，再加上每个天线都会接收来自东向和南向这两个方向的信号，因此需要进行两次三重for。

这里的优化，我们可以利用单调递减栈。

首先定义一个单调递减栈 stack ，然后开始遍历天线 $\text{anth}[i][j]$ （比如先处理东向，即按行从左到右遍历）：

- 1、如果 stack 为空，则直接将天线 $\text{anth}[i][j]$ 加入 stack
- 2、如果 stack 不为空，则获取栈顶天线 top


```

        ret[i][j] += 1;
        stack.removeLast(); // 维护严格递减栈
    }
    // 此情况必然是: anth[i][j] < stack.at(-1), 那么此时anth[i][j]可以接收栈顶天
线的信号,
    // 比如6 5 2 3, 最后一个3可以接收到前面5的信号, 但是无法继续接收更前面6的信号, 因
此这里anth[i][j]结束处理
    else {
        ret[i][j] += 1;
    }
}

stack.add(anth[i][j]);
}
}

StringJoiner sj = new StringJoiner(" ");

// 再处理东向发射信号,和上面同理
for (int i = 0; i < m; i++) {
    LinkedList<Integer> stack = new LinkedList<>();
    for (int j = 0; j < n; j++) {
        // 如果栈顶天线比anth[i][j], 则anth[i][j]必然能接收到栈顶天线的信号, 并且还能继续接
收栈顶前面一个天线的信号(递减栈, 栈顶前面天线高度必然大于栈顶天线高度)
        while (stack.size() > 0 && anth[i][j] > stack.getLast()) {
            ret[i][j] += 1;
            stack.removeLast();
        }

        // 走到此步, 如果stack还有值, 那么由于是递减栈, 因此此时栈顶天线高度必然
        if (stack.size() > 0) {
            // 如果栈顶天线高度 == anth[i][j], 那么此时anth[i][j]可以接收栈顶天线的信号,
            // 比如5 3 2 3, 最后一个3可以接收到前面等高3的信号, 但是无法继续接收前面5的信号,
因此这里anth[i][j]结束处理
            if (anth[i][j] == stack.getLast()) {
                ret[i][j] += 1;
                stack.removeLast(); // 维护严格递减栈
            }
            // 此情况必然是: anth[i][j] < stack.at(-1), 那么此时anth[i][j]可以接收栈顶天
线的信号,
            // 比如6 5 2 3, 最后一个3可以接收到前面5的信号, 但是无法继续接收更前面6的信号, 因
此这里anth[i][j]结束处理
            else {
                ret[i][j] += 1;
            }
        }

        stack.add(anth[i][j]);
        sj.add(ret[i][j] + "");
    }
}

return m + " " + n + "\n" + sj.toString();
}
}

```

提取公共代码，优化代码结构

```
import java.util.LinkedList;
import java.util.Scanner;
import java.util.StringJoiner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
        int n = sc.nextInt();

        int[][] anth = new int[m][n];

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                anth[i][j] = sc.nextInt();
            }
        }

        System.out.println(getResult(anth, m, n));
    }

    public static String getResult(int[][] anth, int m, int n) {
        int[][] ret = new int[m][n];

        // 先处理南向发射信号
        for (int j = 0; j < n; j++) {
            LinkedList<Integer> stack = new LinkedList<>();
            for (int i = 0; i < m; i++) {
                common(stack, anth, ret, i, j);
            }
        }

        StringJoiner sj = new StringJoiner(" ");

        // 再处理东向发射信号,和上面同理
        for (int i = 0; i < m; i++) {
            LinkedList<Integer> stack = new LinkedList<>();
            for (int j = 0; j < n; j++) {
                common(stack, anth, ret, i, j);
                sj.add(ret[i][j] + "");
            }
        }

        return m + " " + n + "\n" + sj.toString();
    }

    public static void common(LinkedList<Integer> stack, int[][] anth, int[][]
ret, int i, int j) {
        // 如果栈顶天线比anth[i][j],则anth[i][j]必然能接收到栈顶天线的信号,并且还能继续接收栈
        // 顶前面一个天线的信号(递减栈,栈顶前面天线高度必然大于栈顶天线高度)
        while (stack.size() > 0 && anth[i][j] > stack.getLast()) {
            ret[i][j] += 1;
        }
    }
}
```



```

        stack.removeLast();
    }

    // 走到此步，如果stack还有值，那么由于是递减栈，因此此时栈顶天线高度必然
    if (stack.size() > 0) {
        // 如果栈顶天线高度 == anth[i][j]，那么此时anth[i][j]可以接收栈顶天线的信号，
        // 比如5 3 2 3，最后一个3可以接收到前面等高3的信号，但是无法继续接收前面5的信号，因此这
        // 里anth[i][j]结束处理
        if (anth[i][j] == stack.getLast()) {
            ret[i][j] += 1;
            stack.removeLast(); // 维护严格递减栈
        }
        // 此情况必然是：anth[i][j] < stack.at(-1)，那么此时anth[i][j]可以接收栈顶天线的信
        // 号，
        // 比如6 5 2 3，最后一个3可以接收到前面5的信号，但是无法继续接收更前面6的信号，因此这里
        // anth[i][j]结束处理
        else {
            ret[i][j] += 1;
        }
    }

    stack.add(anth[i][j]);
}
}

```

85.静态扫描

题目描述

静态扫描可以快速识别源代码的缺陷，静态扫描的结果以扫描报告作为输出：

- 1、文件扫描的成本和文件大小相关，如果文件大小为N，则扫描成本为N个[金币](#)
- 2、扫描报告的缓存成本和文件大小无关，每缓存一个报告需要M个金币
- 3、扫描报告缓存后，后继再碰到该文件则不需要扫描成本，直接获取缓存结果

给出源代码文件标识序列和文件大小序列，求解采用合理的缓存策略，最少需要的金币数。

输入描述

第一行为缓存一个报告金币数M， $L \leq M \leq 100$

第二行为文件标识序列：F1,F2,F3,....,Fn。

第三行为文件大小序列：S1,S2,S3,....,Sn。

备注：

- $1 \leq N \leq 10000$
- $1 \leq F_i \leq 1000$
- $1 \leq S_i \leq 10$

输出描述

采用合理的缓存策略，需要的最少金币数

用例

输入	5 1 2 2 1 2 3 4 1 1 1 1 1 1 1
输出	7
说明	文件大小相同，扫描成本均为1个金币。缓存任意文件均不合算，因而最少成本为7金币。

输入	5 2 2 2 2 2 5 2 2 2 3 3 3 3 1 3 3 3
输出	9
说明	无

题目解析

简单的贪心思维[逻辑题](#)，解析请看代码注释。

Java算法源码

```
import java.util.Arrays;
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = Integer.parseInt(sc.nextLine());
        Integer[] f =
            Arrays.stream(sc.nextLine().split("
")).map(Integer::parseInt).toArray(Integer[]::new);
        Integer[] s =
            Arrays.stream(sc.nextLine().split("
")).map(Integer::parseInt).toArray(Integer[]::new);

        System.out.println(getResult(m, f, s));
    }

    public static int getResult(int m, Integer[] f, Integer[] s) {
        // count用于保存每个文件出现的次数
        HashMap<Integer, Integer> count = new HashMap<>();
        // size用于保存文件的大小，即扫描成本
        HashMap<Integer, Integer> size = new HashMap<>();

        for (int i = 0; i < f.length; i++) {
            // k是文件标识
            Integer k = f[i];
```

```

        count.put(k, count.getDefault(k, 0) + 1);
        size.putIfAbsent(k, s[i]);
    }

    int ans = 0;
    for (Integer k : count.keySet()) {
        // 选择每次都重新扫描的成本 和 扫描一次+缓存的成本 中最小的
        ans += Math.min(count.get(k) * size.get(k), size.get(k) + m);
    }

    return ans;
}
}

```

86.过滤组合字符串

题目描述

数字0、1、2、3、4、5、6、7、8、9分别关联 a~z 26个英文字母。

- 0 关联 "a","b","c"
- 1 关联 "d","e","f"
- 2 关联 "g","h","i"
- 3 关联 "j","k","l"
- 4 关联 "m","n","o"
- 5 关联 "p","q","r"
- 6 关联 "s","t"
- 7 关联 "u","v"
- 8 关联 "w","x"
- 9 关联 "y","z"

例如7关联"u","v", 8关联"x","w", 输入一个字符串例如"78", 和一个屏蔽字符串"ux", 那么"78"可以组成多个字符串例如: "ux", "uw", "vx", "vw", 过滤这些完全包含屏蔽字符串的每一个字符的字符串, 然后输出剩下的字符串。

输入描述

无

输出描述

无

用例

输入	78 ux
输出	uw vx vw
说明	ux完全包含屏蔽字符串ux，因此剔除

题目解析

本题可以使用[深度优先搜索DFS](#)。

本题类似于[LeetCode - 17 电话号码的字母组合](#) [伏城之外的博客-CSDN博客](#)

另外本题中，判断一个字符串A完全包含另一个字符串B全部字符，的方法请看

[华为OD机试 - 密室逃生游戏](#) [伏城之外的博客-CSDN博客](#)

Java算法源码

```
import java.util.*;

public class Main {
    static String[] map = {"abc", "def", "ghi", "jkl", "mno", "pqr", "st", "uv",
        "wx", "yz"};

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Integer[] arr =

Arrays.stream(sc.next().split("")).map(Integer::parseInt).toArray(Integer[]::new);
        String filter = sc.next();

        System.out.println(getResult(arr, filter));
    }

    public static String getResult(Integer[] arr, String filter) {
        String[] newArr = Arrays.stream(arr).map(val ->
map[val]).toArray(String[]::new);

        char[] cArr = filter.toCharArray();
        Arrays.sort(cArr);
        filter = new String(cArr);

        ArrayList<String> res = new ArrayList<>();
        dfs(newArr, 0, new LinkedList<>(), res, filter);

        StringJoiner sj = new StringJoiner(" ", "", "");
        for (String str : res) {
            sj.add(str);
        }
    }
}
```

```

    }
    return sj.toString();
}

public static void dfs(
    String[] arr, int index, LinkedList<Character> path, ArrayList<String>
res, String filter) {
    if (index == arr.length) {
        // 过滤这些完全包含屏蔽字符串的每一个字符的字符串
        if (!include(path, filter)) {
            StringBuilder sb = new StringBuilder();
            for (Character c : path) {
                sb.append(c);
            }
            res.add(sb.toString());
        }
        return;
    }

    for (int i = 0; i < arr[index].length(); i++) {
        path.addLast(arr[index].charAt(i));
        dfs(arr, index + 1, path, res, filter);
        path.removeLast();
    }
}

public static boolean include(LinkedList<Character> path, String filter) {
    StringBuilder sb = new StringBuilder();
    path.stream().sorted().forEach(sb::append);
    String src = sb.toString();

    if (filter.length() > src.length()) return false;

    int i = 0;
    int j = 0;

    while (i < src.length() && j < filter.length()) {
        if (src.charAt(i) == filter.charAt(j)) j++;
        i++;
    }

    return j == filter.length();
}
}

```

87.Excel单元格数值统计

题目描述

Excel工作表中对选定区域的数值进行统计的功能非常实用。

仿照Excel的这个功能，请对给定表格中选区域中的单元格进行求和统计，并输出统计结果。

为简化计算，假设当前输入中每个单元格内容仅为数字或公式两种。

如果为数字，则是一个非负整数，形如3、77

如果为公式，则固定以=开头，且仅包含下面三种情况：

- 等于某单元格的值，例如=B12
- 两个单元格的双目运算（仅为+或-），形如=C1-C2、C3+B2
- 单元格和数字的双目运算（仅为+或-），形如=B1+1、100-B2

注意：

1. 公式内容都是合法的，例如不存在，=C+1、=C1-C2+B3,=5、=3+5
2. 不存在循环引用，例如A1=B1+C1、C1=A1+B2
3. 内容中不存在空格、括号

输入描述

第一行两个整数rows cols，表示给定表格区域的行数和列数， $1 \leq rows \leq 20$ ， $1 \leq cols \leq 26$ 。

接下来rows行，每行cols个以空格分隔的字符串，表示给定表格values的单元格内容。

最后一行输入的字符串，表示给定的选中区域，形如A1:C2。

输出描述

一个整数，表示给定选中区域各单元格中数字的累加总和，范围-2,147,483,648 ~ 2,147,483,647

备注

表格的行号1~20，列号A~Z，例如单元格B3对应values[2][1]。

用例

输入	1 3 1 =A1+C1 3 A1:C1					
输出	8					
说明						
		A	B	C	D	E
	1	1	(=A1+C1)= 4	3		
	2					
	3					

输入	5 3 10 12 =C5 15 5 6 7 8 =3+C2 6 =B2-A1 =C2 7 5 3 B2:C4
输出	29
说明	无

题目解析

本题逻辑不难，但是实现起来比较麻烦。

我的解题思路如下：

首先，要搞清楚Excel表格坐标和matrix输入矩阵的索引的对应关系，比如上面用例中，输入的matrix矩阵为：`[["1", "=A1+C1", "3"]]`

其中“1”值，对应矩阵 `matrix[0][0]`，而对应的Excel表格坐标是A1，其中A代表列号，1代表行号。

因此，我们容易得到Excel表格坐标和matrix输入矩阵的索引的对应关系：

- 列对应关系：`String('A').charCodeAt() - 65 = 0`（PS：'A'的ASCII码值为65）
- 行对应关系：`1 - 1 = 0`

解下来，我们需要弄清楚，如何将Excel坐标，如A1，B2，C3中的列号和行号解析出来，因为只有解析出来，才能方便处理，之后才能对应到matrix的索引。

这里我们使用了正则表达式的捕获组，正则：`/^([A-Z])(\d+)$/`



```
> const regExp = /^([A-Z])(\d+)$/
< undefined

> regExp.exec('A1')
< ▶ (3) ['A1', 'A', '1', index: 0, input: 'A1', groups: undefined]

> regExp.exec('C3')
< ▶ (3) ['C3', 'C', '3', index: 0, input: 'C3', groups: undefined]

> |
```

CSDN @伏城之外

接下来，我们就可以实现根据Excel坐标，获取到matrix矩阵元素的逻辑了，我们定义一个方法`getCell`，入参Excel坐标，然后通过上面的正则解析出来对应列号、行号，然后再根据Excel列号、行号转化求得matrix矩阵的行索引、列索引，进而求得matrix矩阵对应索引的值。

此时，取得的值有两类：

- 1、非公式的值，比如1
- 2、公式，以=开头

对于非公式的值，直接将其转为数值后返回；

对于公式，又分为三种情况：

- `=A1+B1`，即Excel坐标之间的运算
- `=A1-2`，即Excel坐标和数值之间的运算
- `=A1`，即Excel坐标

我们可以通过`getCell`方法获取到Excel坐标对应的值，然后再来运算

Java算法源码

```
import java.util.Scanner;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Main {
    static Pattern p = Pattern.compile("^([A-Z])(\\d+)$");

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int rows = sc.nextInt();
        int cols = sc.nextInt();

        String[][] matrix = new String[rows][cols];
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                matrix[i][j] = sc.next();
            }
        }

        String[] range = sc.next().split(":");

        System.out.println(getResult(matrix, rows, cols, range[0], range[1]));
    }

    /**
     * @param matrix 给定表格区域
     * @param rows 给定表格区域的行数
     * @param cols 给定表格区域的列数
     * @param start 选中区域的左上角位置
     * @param end 选中区域的右下角位置
     * @return 求和选中区域所有数的和
     */
    public static int getResult(String[][] matrix, int rows, int cols, String
start, String end) {
        // 正则p于分解出Excel单元格位置坐标（形式如A1, B2, C3）的列和行，注意字母是列号，数字是行
        号

        Matcher m1 = p.matcher(start);
        Matcher m2 = p.matcher(end);

        if (m1.find() && m2.find()) {
            // 选中区域左上角坐标的解析
            int col_start = m1.group(1).charAt(0);
            int row_start = Integer.parseInt(m1.group(2));

            // 选中区域右下角坐标的解析
            int col_end = m2.group(1).charAt(0);
            int row_end = Integer.parseInt(m2.group(2));

            // ans保存选中区域所有数的和
            int ans = 0;
            // 从左上角坐标遍历到右下角坐标
            for (int j = col_start; j <= col_end; j++) {
```



```

        for (int i = row_start; i <= row_end; i++) {
            char c = (char) j;
            ans += getCell(c + "" + i, matrix);
        }
    }
    return ans;
}

// 异常情况，应该不会走到这步
return 0;
}

/**
 * @param pos 指定Excel表格坐标，pos形式如A1, B2, C3
 * @param matrix Excel给定表格区域
 * @return 指定坐标pos的单元格内的值
 */
public static int getCell(String pos, String[][] matrix) {
    Matcher m = p.matcher(pos);
    if (m.find()) {
        // 题目说列号取值A~Z，起始列A对应的码值65，A列等价于matrix矩阵的第0列
        int col = m.group(1).charAt(0) - 65;
        // 起始行1，等价于matrix矩阵的第0行
        int row = Integer.parseInt(m.group(2)) - 1;

        String cell = matrix[row][col];
        // 如果单元格内容以=开头，则为公式
        if (cell.startsWith("=")) {
            // 公式有三种情况
            // 等于某单元格的值，例如=B12
            // 两个单元格的双目运算（仅为+或-），形如=C1-C2、C3+B2
            String[] combine = cell.split("[\\=\\+\\-]");

            String cell1 = combine[1];

            // 对于 =A1 这种情况，cell2没有值
            String cell2 = null;
            if (combine.length > 2) {
                cell2 = combine[2];
            }

            int cell1_val;
            if (cell1.matches("^-?\\d+$")) {
                // 如果cell解析出来是值，则直接使用
                cell1_val = Integer.parseInt(cell1);
            } else {
                // 如果cell解析出来不是值，那就是Excel坐标
                cell1_val = getCell(cell1, matrix);
            }

            // 同上
            int cell2_val;
            if (cell2 != null) {
                if (cell2.matches("^-?\\d+$")) {
                    cell2_val = Integer.parseInt(cell2);
                }
            }
        }
    }
}

```

```

        } else {
            cell2_val = getCell(cell2, matrix);
        }
    } else {
        cell2_val = 0;
    }

    // 如果cell1和cell2是相加
    if (cell.contains("+")) {
        matrix[row][col] = cell1_val + cell2_val + "";
    }
    // 如果cell1和cell2是相减
    else if (cell.contains("-")) {
        matrix[row][col] = cell1_val - cell2_val + "";
    }
    // 如果没有运算，那就只可能是单值，直接使用
    else {
        matrix[row][col] = cell1_val + "";
    }
}

// 如果单元格内容以=开头，则以上逻辑会将单元格内容更新为数值
// 如果单元格内容不以=开头，则为可以直接使用的数值
return Integer.parseInt(matrix[row][col]);
}

// 异常情况，应该不会走到这步
return 0;
}
}

```

88.统一限载货物数最小值

题目描述

火车站附近的货物中转站负责将到站货物运往仓库，小明在中转站负责调度2K辆中转车（K辆干货中转车，K辆湿货中转车）。

货物由不同供货商从各地发来，各地的货物是依次进站，然后小明按照卸货顺序依次装货到中转车，一个供货商的货只能装到一辆车上，不能拆装，但是一辆车可以装多家供货商的货；

中转车的限载货物量由小明统一指定，在完成货物中转的前提下，请问中转车的统一限载货物数最小值为多少。

输入描述

- 第一行 length 表示供货商数量 $1 \leq \text{length} \leq 10^4$
- 第二行 goods 表示供货数数组 $1 \leq \text{goods}[i] \leq 10^4$
- 第三行 types表示对应货物类型，types[i]等于0或者1，其中0代表干货，1代表湿货
- 第四行 k表示单类中转车数量 $1 \leq k \leq \text{goods.length}$

输出描述

运行结果输出一个整数，表示中转车统一限载货物数

备注

中转车最多跑一趟仓库

用例

输入	4 3 2 6 3 0 1 1 0 2
输出	6
说明	货物1和货物4为干货，由2辆干货中转车中转，每辆车运输一个货物，限载为3货物2和货物3为湿货，由2辆湿货中转车中转，每辆车运输一个货物，限载为6这样中转车统一限载货物数可以设置为6（干货车和湿货车限载最大值），是最小的取值

输入	4 3 2 6 8 0 1 1 1 1
输出	16
说明	货物1为干货，由1辆干货中转车中转，限载为3货物2、货物3、货物4为湿货，由1辆湿货中转车中转，限载为16这样中转车统一限载货物数可以设置为16（干货车和湿货车限载最大值），是最小的取值

题目解析

本题其实就是[LeetCode - 1723 完成所有工作的最短时间](#) 伏城之外的博客-CSDN博客

的变种题。

解析大家可以看上面链接博客。

Java算法源码

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] goods = new int[n];
```

```

    for (int i = 0; i < n; i++) {
        goods[i] = sc.nextInt();
    }

    int[] types = new int[n];
    for (int i = 0; i < n; i++) {
        types[i] = sc.nextInt();
    }

    int k = sc.nextInt();

    System.out.println(getResult(n, goods, types, k));
}

/**
 * @param n 供货商数量
 * @param goods 供货数数组
 * @param types 表示对应货物类型，types[i]等于0或者1，其中0代表干货，1代表湿货
 * @param k 表示单类中转车数量
 * @return 中转车的统一限载货物数最小值为多少
 */
public static int getResult(int n, int[] goods, int[] types, int k) {
    ArrayList<Integer> dry = new ArrayList<>();
    ArrayList<Integer> wet = new ArrayList<>();

    for (int i = 0; i < n; i++) {
        if (types[i] == 0) {
            dry.add(goods[i]);
        } else {
            wet.add(goods[i]);
        }
    }

    return Math.max(getMinMaxWeight(dry, k), getMinMaxWeight(wet, k));
}

public static int getMinMaxWeight(ArrayList<Integer> weights, int k) {
    int n = weights.size();

    if (n <= k) {
        return weights.stream().max((a, b) -> a - b).orElse(0);
    }

    weights.sort((a, b) -> b - a);

    int min = weights.get(0);
    int max = weights.stream().reduce(Integer::sum).get();

    while (min < max) {
        int mid = (min + max) >> 1;

        int[] buckets = new int[k];
        if (check(0, weights, buckets, mid)) {
            max = mid;
        } else {

```

```

        min = mid + 1;
    }
}

return min;
}

public static boolean check(int index, ArrayList<Integer> weights, int[]
buckets, int limit) {
    if (index == weights.size()) {
        return true;
    }

    int select = weights.get(index);
    for (int i = 0; i < buckets.length; i++) {
        if (buckets[i] + select <= limit) {
            buckets[i] += select;
            if (check(index + 1, weights, buckets, limit)) return true;
            buckets[i] -= select;
            if (buckets[i] == 0) return false;
        }
    }

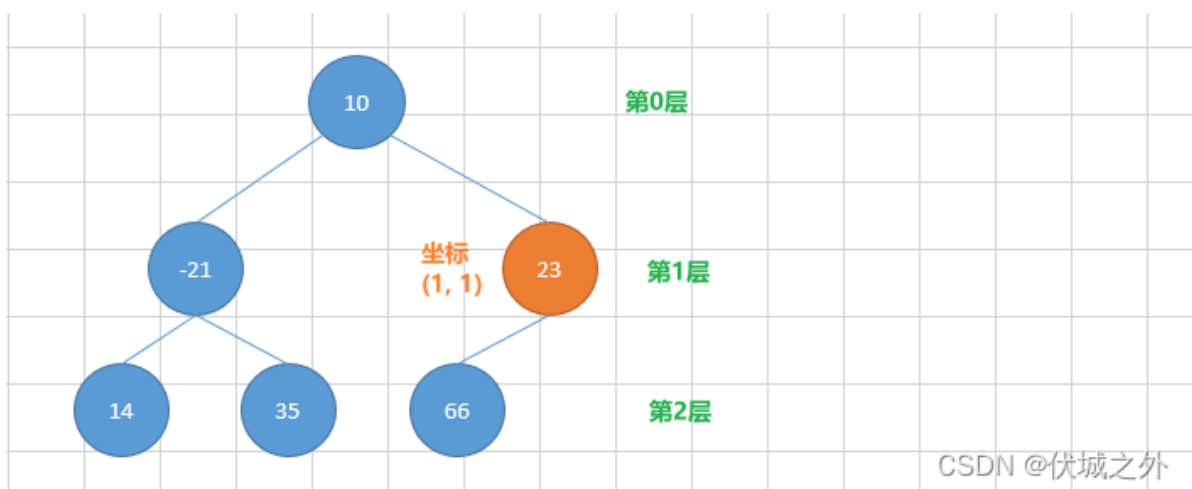
    return false;
}
}

```

89.查找树中元素

题目描述

已知**树形结构**的所有节点信息，现要求根据输入坐标 (x,y) 找到该节点保存的内容值，其中x表示节点所在的层数，根节点位于第0层，根节点的子节点位于第1层，依次类推；y表示节点在该层内的相对偏移，从左至右，第一个节点偏移0，第二个节点偏移1，依次类推；



举例：上图中，假定圆圈内的数字表示节点保存的**内容值**，则根据坐标(1,1)查到的内容值是23。

输入描述

每个节点以一维数组（int[]）表示，所有节点信息构成二维数组（int[][]），二维数组的0位置存放根节点；
表示单节点的一维数组中，0位置保存**内容值**，后续位置保存子节点在二维数组中的**索引位置**。
对于上图中：

- 根节点的可以表示为{10, 1, 2},
- 树的整体表示为{{10,1,2},{-21,3,4},{23,5},{14},{35},{66}}

查询条件以长度为2的一维数组表示，上图查询坐标为(1,1)时表示为{1,1}

使用Java标准IO键盘输入进行录入时，

1. 先录入节点数量
2. 然后逐行录入节点
3. 最后录入查询的位置

对于上述示例为：

```
6
10 1 2
-21 3 4
23 5
14
35
66
1 1
```

输出描述

查询到内容时，输出{内容值}，查询不到时输出{}
上图中根据坐标(1,1)查询输出{23}，根据坐标(1,2)查询输出{}

用例

输入	6 10 1 2 -21 3 4 23 5 14 35 66 1 1
输出	{23}
说明	无

2023.1.16新增用例说明，之前代码只是根据用例1写的，误以为题目中说的树是二叉树，因此代码存在问题，后面发现了原题更多的用例说明，意识到本题的树是多叉树，但是本题代码中多叉树的处理逻辑和二叉树相同，只需要微调代码即可

输入	14 0 1 2 3 4 -11 5 6 7 8 113 9 10 11 24 12 35 66 13 77 88 99 101 102 103 25 104 2 5
输出	{102}
说明	无

输入	14 0 1 2 3 4 -11 5 6 7 8 113 9 10 11 24 12 35 66 13 77 88 99 101 102 103 25 104 3 2
输出	{}
说明	无

输入	1 1000 0 0
输出	{1000}
说明	无

题目解析

这题输入描述比较晦涩难懂，但我还是猜出来一点：

每个节点以一维数组（int[]）表示，所有节点信息构成二维数组（int[][]），二维数组的0位置存放根节点；

这句话的意思就是：用例的第二行输入到倒数第二行输入可以构成一个二维数组，二维数组的元素是一维数组，如下图

```
[
  [10,1,2],
  [-21,3,4],
  [23,5],
  [14],
  [35],
  [66]
]
```

假设二维数组是matrix的话，则每一个matrix[i]对应树的一个节点，其中matrix[0]（如用例[10,1,2]）代表的是树的根节点。

表示单节点的一维数组中，0位置保存**内容值**，后续位置保存子节点在二维数组中的**索引位置**。

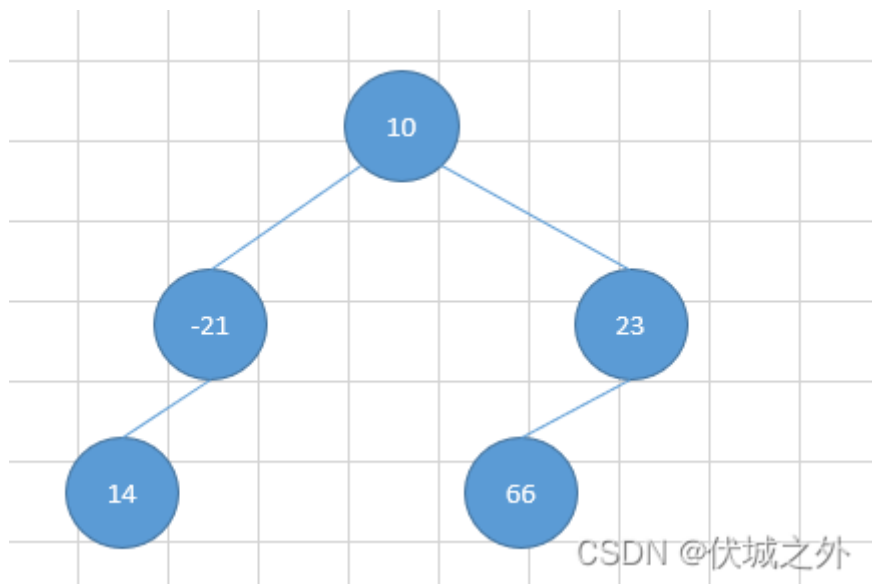
上面这句话的意思是，matrix[i]（是一个一维数组），我们用arr = matrix[i]，则arr[0]表示节点的内容值，arr[1]、arr[2]表示的是该节点的子节点在matrix中的索引位置。

比如[10,1,2]表示内容值为10的节点，有两个子节点，分别是matrix[1]和matrix[2]，即[-21,3,4]，[23,5]。

由于本题是二叉树，因此一个节点最多有两个子节点，最少没有子节点。

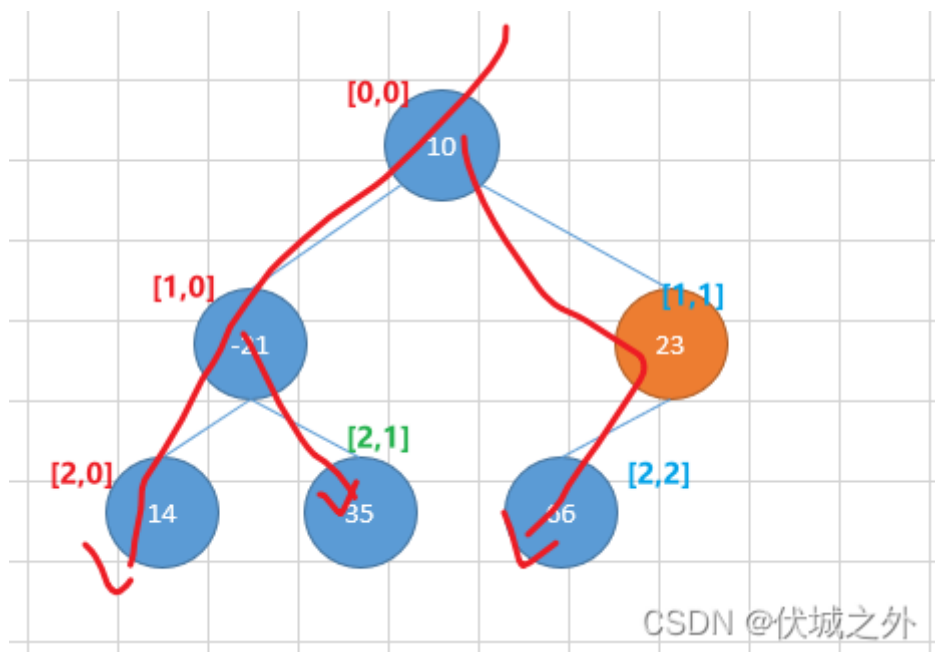
以上是关于输入描述的解读。

本题并没有说二叉树是[完全二叉树](#)，因此二叉树的节点非常有可能像：



因此无法直接利用完全二叉树特性。

但是也不难，本题就是就是一个简单的深度优先搜索题



本题的意思其实就是让我们再构造一个二维数组matrix2，这个二维数组形式如上图所示，具体内容如下

```

[
  [10],
  [-21, 23],
  [14,35,66]
]
  
```

因此，我们很容易想到利用深度优先搜索，从根节点开始递归，dfs(root, 0)，dfs参数含义是：当前节点root，得到的节点值将放置于二维数组matrix2的第0行。

从根节点还可以得到其左右子节点的索引，因此我们可以继续递归其左右子节点 dfs(left, 1)、dfs(right,1)，只是此时的得到的子节点将放于matrix2的第1行。

由此，我们就可以将节点值放到正确的行位置上了。

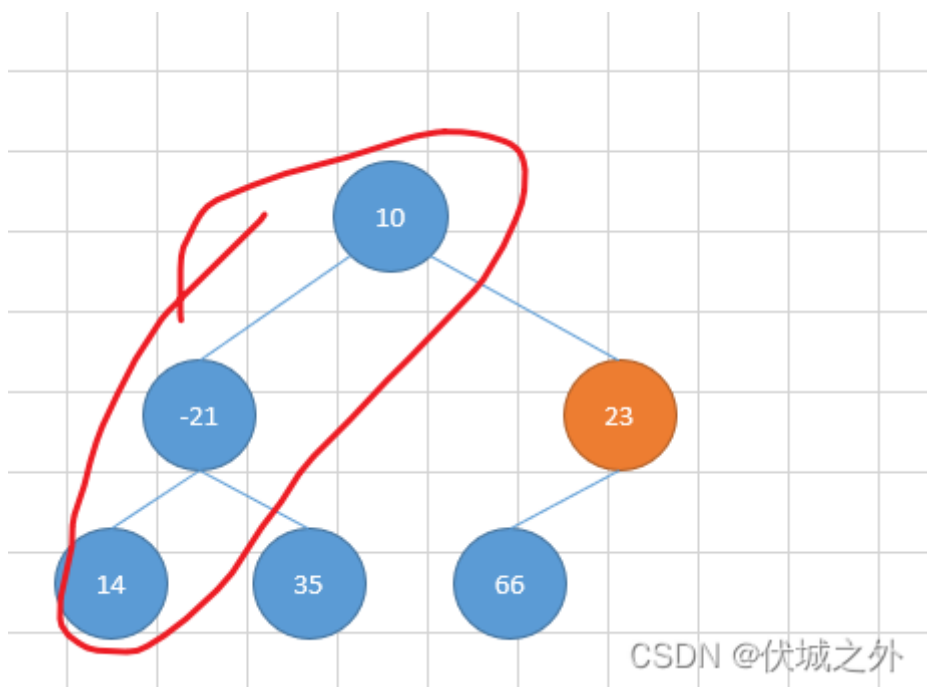
那么列位置该如何确认呢？

很好解决，matrix2的每一行都是一个数组，由于我们是优先左子树遍历，因此，总是每一行最左边的节点被优先遍历出来，我们只需要将遍历出来的节点值push压入对应行的数组中即可。比如：

matrix2一开始是空数组[]，然后遍历得到根节点，压入第0行的数组元素中，但是第0行没有数组元素，因此就构造一个[val]压入，同理处理根节点的左子节点，然后继续处理左子节点的左子节点，因此第一个深搜回溯时，得到

```
[
  [10],
  [-21],
  [14]
]
```

即确认了每一行数组的第一个元素，刚好是



然后回溯过程将其余节点值依次加入每一行数组中，最终得到

```
[
  [10],
  [-21, 23],
  [14, 35, 66]
]
```

此时，找任意位置的节点值都是so easy

2023.03.17 补充一个用例

本题中最后一行输入的目标节点坐标位置可能是负数，尼玛，我服了,比如下面用例

```
6
10 1 2
-21 3 4
23 5
14
35
```

如果坐标位置为负数，则会出现数组越界的问题，加了这个就100%了

Java算法源码

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    static ArrayList<Integer[]> nodes;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(sc.nextLine());

        nodes = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            Integer[] node =
                Arrays.stream(sc.nextLine().split("
")).map(Integer::parseInt).toArray(Integer[]::new);
            nodes.add(node);
        }

        int tx = sc.nextInt();
        int ty = sc.nextInt();

        System.out.println(getResult(nodes, tx, ty));
    }

    public static String getResult(ArrayList<Integer[]> nodes, int tx, int ty) {
        // 2023.03.17, 尼玛，谁能想到还有tx,ty小于0的用例，题目描述一点没说
        if (tx < 0 || ty < 0) return "{}";

        ArrayList<ArrayList<Integer>> matrix = new ArrayList<>();
        dfs(matrix, nodes.get(0), 0);

        if (tx < matrix.size() && ty < matrix.get(tx).size()) {
            return "{" + matrix.get(tx).get(ty) + "}";
        } else {
            return "{}";
        }
    }

    public static void dfs(ArrayList<ArrayList<Integer>> matrix, Integer[] node,
        Integer level) {
        if (node == null) return;

        int val = node[0];

        if (level < matrix.size()) {
            matrix.get(level).add(val);
        }
    }
}
```

```

    } else {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(val);
        matrix.add(list);
    }

    // 将二叉树处理逻辑，改成多叉树
    //     if (node.length > 1) {
    //         int left = node[1];
    //         dfs(matrix, nodes.get(left), level + 1);
    //     }
    //
    //     if (node.length > 2) {
    //         int right = node[2];
    //         dfs(matrix, nodes.get(right), level + 1);
    //     }

    for (int i = 1; i < node.length; i++) {
        int child = node[i];
        dfs(matrix, nodes.get(child), level + 1);
    }
}
}

```

90.去除多余空格

题目描述

去除文本多余空格，但不去除配对单引号之间的多余空格。给出关键词的起始和结束下标，去除多余空格后刷新关键词的起始和结束下标。

条件约束：

- 1, 不考虑关键词起始和结束位置为空格的场景；
- 2, 单词的开始和结束下标保证涵盖一个完整的单词，即一个坐标对开始和结束下标之间不会有多余的空格；
- 3, 如果有单引号，则用例保证单引号成对出现；
- 4, 关键词可能会重复；
- 5, 文本字符长度length取值范围：[0, 100000]；

输入描述

输入为两行字符串：

第一行：待去除多余空格的文本，用例保证如果有单引号，则单引号成对出现，且单引号可能有多对。

第二行：关键词的开始和结束坐标，关键词间以逗号区分，关键词内的开始和结束位置以单空格区分。

输出描述

输出为两行字符串：

第一行：去除多余空格后的文本

第二行：去除多余空格后的关键词的坐标开始和结束位置，为数组方式输出。

用例

输入	Life is painting a picture, not doing 'a sum'. 8 15,20 26,43 45
输出	Life is painting a picture, not doing 'a sum'. [8, 15][19, 25][42, 44]
说明	a和picture中间多余的空格进行删除

输入	Life is painting a picture, not doin 'a sum'. 8 15,19 25,42 44
输出	Life is painting a picture, not doing 'a sum'. [8, 15][19, 25][42, 44]
说明	a和sum之间有多余的空格，但是因为有成对单引号，不去除多余空格

题目解析

用例1图示

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
L	i	f	e		i	s		p	a	i	n	t	i	n	g		a		p	i	c	t	u	r	e	,		n	o	t		d	o	i	n	g		'	a		s	u	m		.		
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	
L	i	f	e		i	s		p	a	i	n	t	i	n	g		a		p	i	c	t	u	r	e	,		n	o	t		d	o	i	n	g		'	a		s	u	m		.		

CSDN @ 伏城之外

用例2图示

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	
L	i	f	e		i	s		p	a	i	n	t	i	n	g		a		p	i	c	t	u	r	e	,		n	o	t		d	o	i	n	g		'	a		s	u	m		.	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46
L	i	f	e		i	s		p	a	i	n	t	i	n	g		a		p	i	c	t	u	r	e	,		n	o	t		d	o	i	n	g		'	a		s	u	m		.	

CSDN 0伏城之外

我的解题思路如下：

定义一个数组needDel，记录需要被删除的空格的位置，定义一个变量quotaStart，记录是否已有未闭合的单引号，默认为false，即没有。

遍历输入字符串的每一个字符，

首先看被遍历的字符c是不是" "空格

- 如果不是，则继续后面逻辑
- 如果是，则看当前遍历字符的前一个字符是不是也是" "，如果不是则继续后面逻辑，如果是，则看quotaStart是否为false，若不是，则继续后面逻辑，若是，则说明空格c就是需要删除的空格，我们将其索引位置记录到needDel中

然后看遍历的字符c是不是单引号，若是，则将quotaStart取反（表示单引号开闭）

当扫描完后，我们就可以做两件事：

1、将输入字符串转为数组，根据needDel中记录的索引，来删除对应空格（即替换对应数组元素为""空串），然后将数组join后打印

2、双重for，外层遍历needDel获得每一个要删除的空格的索引，内存遍历题目第一行输入的多组开始、结束位置arr，如果要删除的空格处于开始位置之前，则对应开始，结束位置都要减去1。

我们需要注意的是：我们不能基于原始的arr处理，应该对arr进行[深拷贝](#)后，对拷贝数组进行处理，因为如果基于原始arr进行处理，则会造成needDel中要删除的索引，和arr中记录的索引不同步。

比如 needDel = [18,19], arr = [[8,15], [20,26], [43,45]]

如果我们直接在arr上进行操作，则

首先needDel遍历出18，然后对比每一个arr范围的开始位置，发现[20,26], [43,45]在18后面，因此当18位置的空格删除后，这两个范围都要前移，因此arr变为了[[8,15], [19,25], [42,44]]。

之后needDel遍历出19，然后对比每一个arr范围的开始位置，发现和[19,25]产生了冲突，因为19坐标位置既是要删除的空格，又是一个关键词的起始位置，造成这种冲突的原因是此时needDel 19 和 arr范围[19,25] 已经不同步了。

因此我们要对，arr进行深拷贝后（数组元素是数组，需要深拷贝，可以使用最简单的JSON方法实现深拷贝），然后needDel和原始arr进行对比操作，具体范围移动操作在arr的拷贝对象上处理。

但是本题的文本字符长度length取值范围：[0, 100000];

这个有点大啊，会不会爆内存呢？有点悬。

Java算法源码

克隆版（85%通过率）

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        Integer[][] ranges =
            Arrays.stream(sc.nextLine().split(","))
                .map(s -> Arrays.stream(s.split(" "))
                    .map(Integer::parseInt)
                    .toArray(Integer[]::new))
                .toArray(Integer[][]::new);

        getResult(str, ranges);
    }
}
```

```

    public static void getResult(String str, Integer[][] ranges) {
        boolean quotaStart = false;
        ArrayList<Integer> needDel = new ArrayList<>();

        for (int i = 0; i < str.length(); i++) {
            if (i > 0 && ' ' == str.charAt(i) && ' ' == str.charAt(i - 1) &&
!quotaStart) {
                needDel.add(i);
            }

            if ('\'' == str.charAt(i)) {
                quotaStart = !quotaStart;
            }
        }

        char[] cArr = str.toCharArray();
        Integer[][] ans =
Arrays.stream(ranges).map(Integer[]::clone).toArray(Integer[][]::new);

        for (Integer del : needDel) {
            cArr[del] = '\u0000';
            for (int i = 0; i < ranges.length; i++) {
                int start = ranges[i][0];
                if (del < start) {
                    ans[i][0]--;
                    ans[i][1]--;
                }
            }
        }

        System.out.println(new String(cArr).replace("\u0000", ""));

        StringBuilder sb = new StringBuilder();
        for (Integer[] an : ans) {
            sb.append(Arrays.toString(an));
        }
        System.out.println(sb);
    }
}

```

倒序版

```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        Integer[][] ranges =

```

```

        Arrays.stream(sc.nextLine().split(","))
            .map(s -> Arrays.stream(s.split("
")).map(Integer::parseInt).toArray(Integer[]::new))
            .toArray(Integer[][]::new);

        getResult(str, ranges);
    }

    public static void getResult(String str, Integer[][] ranges) {
        boolean quotaStart = false;
        ArrayList<Integer> needDel = new ArrayList<>();

        String[] sArr = str.split("");

        for (int i = 0; i < sArr.length; i++) {
            if (i > 0 && " ".equals(sArr[i]) && " ".equals(sArr[i - 1]) &&
!quotaStart) {
                needDel.add(i);
            }

            if ("".equals(sArr[i])) {
                quotaStart = !quotaStart;
            }
        }

        for (int j = needDel.size() - 1; j >= 0; j--) {
            int del = needDel.get(j);
            sArr[del] = "";
            for (int i = 0; i < ranges.length; i++) {
                int start = ranges[i][0];
                if (del < start) {
                    ranges[i][0]--;
                    ranges[i][1]--;
                }
            }
        }

        StringBuilder sb = new StringBuilder();
        for (Integer[] an : ranges) {
            sb.append(Arrays.toString(an));
        }
        System.out.println(sb);
    }
}

```

91.机房布局

题目描述

小明正在规划一个大型[数据中心](#)机房，为了使得机柜上的机器都能正常满负荷工作，需要确保在每个机柜边上至少要有一个电箱。

为了简化题目，假设这个机房是一整排，M表示机柜，I表示间隔，请你返回这整排机柜，至少需要多少个电箱。如果无解请返回 -1 。

输入描述

无

输出描述

无

用例

输入	MIIM
输出	2
说明	无

逻辑分析解法（最优解法）

题目解析

本题其实只要朝一个方向优先放电箱即可，但是通常我们习惯从左向右遍历，因此这里优先将电箱放在机柜的右边。

为什么要将电箱优先放在右边呢？比如下面这个例子：

***/* M */* M**

优先将机柜放在红色I位置，则最终只需要一个电箱。如果将机柜放绿色I位置，则最终需要两个电箱。

再比如下面这个例子

M */* M I

由于我们是从左往右遍历，因此第一个机柜M，优先将电箱放在它的右边。

但是，如果机柜的右侧放不了电箱呢？比如

I M M I

此时，我们需要看机柜的左侧能不能放，如果能放，则放，不然对应机柜就没有配置电箱了。

如果机柜的左右两侧都放不了电箱呢？

M M I

此时应该直接返回-1。

另外，还有一个重要的点就是：

如果机柜的右侧可以放电箱，比如第 i 个位置是机柜，第 $i + 1$ 个位置是间隔，则我们可以将电箱放到第 $i + 1$ 个位置上，此时：第 $i + 2$ 个位置是什么还需要关注吗？

比如：

M I * M * I

上面红色的M还需关注放不放电箱吗？答案是不需要，因为他必然有一个电箱了。

此时我们可以直接跳到 $i + 3$ 位置重新开始上面的判断。

Java算法源码

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.next();

        System.out.println(getResult(str));
    }

    public static int getResult(String str) {
        int n = str.length();

        // ans记录用了几个电箱
        int ans = 0;

        for (int i = 0; i < n; i++) {
            // 如果当前是机柜
            if (str.charAt(i) == 'M') {
                // 则优先将电箱放到该机柜的右边，如果机柜右边有间隔I的话
                if (i + 1 < n && str.charAt(i + 1) == 'I') {
                    ans++;
                    // 如果放成功了，即当前第 i 个位置是机柜，第i+1个位置是 电箱，那么第i+2个位置无论是机柜还是间隔都无所谓，下次我们直接从第i+3位置开始判断
                    i += 2; // 这里 i += 2结合下轮for循环的i++，就是i += 3
                }
                // 如果当前第i位置的机柜右边无法放入电箱，则只能将电箱放到其左边第i-1位置，但是第i-1位置必须是间隔I
                else if (i - 1 >= 0 && str.charAt(i - 1) == 'I') {
                    ans++;
                }
                // 如果当前机柜左右都无法放入电箱，则返回-1
                else {
                    ans = -1;
                    break;
                }
            }
        }
    }
}
```

```
        return ans;
    }
}
```

92.识图谱新词挖掘

题目描述

小华负责公司[知识图谱](#)产品，现在要通过新词挖掘完善知识图谱。

新词挖掘：给出一个待挖掘问题内容字符串Content和一个词的字符串word，找到content中所有word的新词。

新词：使用词word的字符排列形成的字符串。

请帮小华实现新词挖掘，返回发现的新词的数量。

输入描述

第一行输入为待挖掘的文本内容content;

第二行输入为词word;

输出描述

在content中找到的所有word的新词的数量。

备注

- $0 \leq \text{content的长度} \leq 10000000$
- $1 \leq \text{word的长度} \leq 2000$

用例

输入	qweebaewqd qwe
输出	2
说明	起始索引等于0的子串是“qwe”，它是word的新词。起始索引等于6的子串是“ewq”，它是word的新词。
输入	abab ab

输入	abab ab
输出	3
说明	起始索引等于0的子串是“ab”，它是word的新词。起始索引等于1的子串是“ba”，它是word的新词。起始索引等于2的子串是“ab”，它是word的新词。

题目解析

本题最简单的解法就是利用定长（word长度）[滑动窗口](#)来选择 content 中的子串，只要选中的子串和 word 二者，在字典序排序后是一样的，则可以记为一个挖掘到的新词。代码如下：

```
/* JavaScript Node ACM模式 控制台输入获取 */
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

const lines = [];
rl.on("line", (line) => {
  lines.push(line);

  if (lines.length === 2) {
    const content = lines[0];
    const word = lines[1];

    console.log(getResult(content, word));
    lines.length = 0;
  }
});

function getResult(content, word) {
  if (content.length < word.length) {
    return 0;
  }

  const sorted_word = [...word].sort().join("");

  let ans = 0;
  let maxI = content.length - word.length;
  for (let i = 0; i <= maxI; i++) {
    const sorted_substr = [...content.slice(i, i + word.length)]
      .sort()
      .join("");

    if (sorted_substr === sorted_word) ans++;
  }

  return ans;
}
```

但是上面算法的时间复杂度在： $O(n*m)$ ，其中 n 是content长度， m 是word长度，也就是说，最多的会有 $10000000 * 2000$ 次循环，有人可能会疑问时间复杂度中 m 哪来的？那是content截取出来的子串字典序排序的时间复杂度 m 。

因此上面的算法极其容易超时。

那么上面算法是否有优化的可能呢？

我的优化思路是，利用word和content截取子串的字母数量 比较 来代替 字典序排序后比较。

因为字典序排序存在一个较大的性能浪费，比如

```
qweebaewqd
xyz
```

我们可以明显发现，content中不存在word新词，因为任意位置长度3的滑动窗口中的第一个字母都不属于word，因此仅从滑动窗口内第一个字母就可以判断出结果，而不是需要截取子串字典序排序后再比较得出结果。

但是，通过字母数量比较也有缺陷，那就是，如果只有滑动窗口内部各字母数量和word的字母数量严格一致，才能算挖掘到了新词。

我的实现思路是：

遍历滑动窗口内部的字母，如 c ，如果该字母 c 是word中的，则先看统计数量 $count[c]$ 是不是已经为0了，如果已经为0了，则说明当前滑动窗口遍历到的字母 c 是超标部分，因此当前滑动窗口不可用。下一个滑动窗口应该保证字母 c 不超标，即下一个滑动窗口的左边界应该在当前滑动窗口中字母 c 第一次出现位置的右边。

当然，如果遍历完滑动窗口内部所有字母，也没有出现超标现象，则说明该滑动窗口就是要挖掘的新词。

上面算法逻辑，其实我不太确定是否有多大的性能提升，但是相较于字典序排序应该有较多提升。我把上面算法逻辑称为跳跃式，即滑动窗口的左边界不是单纯一格一格往右滑的，而是可以多格滑动的。

为了以防万一，我这里还想了一个传统滑动窗口的解法，那就是一格一格移动，但是也是通过统计滑动窗口内部字母数量来判断是否是word新词，但是后一个滑动窗口可以通过差异化思想，快速基于前一个滑动窗口得出自身内部字母数量。这块逻辑可以参考：

[华为OD机试 - 完美走位伏城之外的博客-CSDN博客完美走位华为](#)

中求解最小覆盖子串的过程。

Java算法源码

Java这里只实现一个传统式的，因为感觉跳跃式的性能提升局限性太大。

```
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

String content = sc.next();
String word = sc.next();

System.out.println(getResult(content, word));
}

public static int getResult(String content, String word) {
    // 如果content长度小于word, 则直接返回0
    if (content.length() < word.length()) {
        return 0;
    }

    // ans保存题解
    int ans = 0;

    // total记录word字母总数
    int total = word.length();

    // count统计word中各字母数量
    HashMap<Character, Integer> count = new HashMap<>();
    for (int i = 0; i < word.length(); i++) {
        Character c = word.charAt(i);
        count.put(c, count.getOrDefault(c, 0) + 1);
    }

    // 初始滑动窗口内部字母遍历
    for (int i = 0; i < word.length(); i++) {
        Character c = content.charAt(i);
        if (count.containsKey(c)) {
            if (count.get(c) > 0) {
                total--;
            }
            count.put(c, count.get(c) - 1);
        }
    }

    if (total == 0) ans++;

    // 滑动窗口左指针的移动范围为 0~maxI
    int maxI = content.length() - word.length();
    for (int i = 1; i <= maxI; i++) {
        Character remove_c = content.charAt(i - 1);
        Character add_c = content.charAt(i + word.length() - 1);

        if (count.containsKey(remove_c)) {
            if (count.get(remove_c) >= 0) {
                total++;
            }
            count.put(remove_c, count.get(remove_c) + 1);
        }

        if (count.containsKey(add_c)) {
            if (count.get(add_c) > 0) {
                total--;
            }
        }
    }
}

```

```
    }  
    count.put(add_c, count.get(add_c) - 1);  
}  
  
if (total == 0) ans++;  
}  
return ans;  
}  
}
```

93.探索地块建立

题目描述

给一块 nm 的地块，相当于 nm 的二维数组，每个元素的值表示这个小地块的发电量；
求在这块地上建立正方形的边长为 c 的发电站，发电量满足目标电量 k 的地块数量。

输入描述

第一行为四个按空格分隔的正整数，分别表示 n, m, c, k

后面 n 行整数，表示每个地块的发电量

输出描述

输出满足条件的地块数量

用例

输入	2 5 2 6 1 3 4 5 8 2 3 6 7 1
输出	4
说明	无

题目解析

本题最优解题思路是使用：二维矩阵前缀

关于二维矩阵前缀和，请看[算法设计 - 前缀和 & 差分数列](#) [伏城之外的博客-CSDN博客](#)

Java算法源码

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();
        int c = sc.nextInt();
        int k = sc.nextInt();

        int[][] matrix = new int[n][m];

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        System.out.println(getResult(matrix, n, m, c, k));
    }

    /**
     * @param matrix n*m的地块
     * @param n 地块行数
     * @param m 地块列数
     * @param c 正方形的发电站边长为c
     * @param k 目标电量k
     * @return 可以建设几个发电站
     */
    public static int getResult(int[][] matrix, int n, int m, int c, int k) {
        int[][] preSum = new int[n + 1][m + 1];

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                preSum[i][j] =
                    preSum[i - 1][j] + preSum[i][j - 1] - preSum[i - 1][j - 1] +
                    matrix[i - 1][j - 1];
            }
        }

        int ans = 0;

        for (int i = c; i <= n; i++) {
            for (int j = c; j <= m; j++) {
                int square = preSum[i][j] - (preSum[i - c][j] + preSum[i][j - c]) +
                    preSum[i - c][j - c];
                if (square >= k) ans++;
            }
        }

        return ans;
    }
}
```

```
}
```

94.快递投放问题

题目描述

有N个快递站点用字符串标识，某些站点之间有道路连接。

每个站点有一些包裹要运输，每个站点间的包裹不重复，路上有检查站会导致部分货物无法通行，计算哪些货物无法正常投递？

输入描述

- 第一行输入M N，M个包裹N个道路信息.
- $0 \leq M, N \leq 100$,
- 检查站禁止通行的包裹如果有多个以空格分开

输出描述

- 输出不能送达的包裹，如：package2 package4,
- 如果所有包裹都可以送达则输出：none,
- 输出结果按照升序排列。

用例

输入	4 2 package1 A C package2 A C package3 B C package4 A C A B package1 A C package2
输出	package2
说明	无

题目解析

这题。。。。我能说我读都费劲嘛。。。

这题意思我只能猜一猜，因为我无法肯定自己的理解是对的。

用例第一行

- 输入的4代表有4个包裹，分别是package1、package2、package3、package4
- 输入的2代表：有2个：两站点之间无法通行的包裹，比如A B package1 代表A、B站点之间无法通行包裹package1。

用例第2~5行（对应第一行输入的4），对应4个包裹，以及它们要从哪个站点发往哪个站点。比如 package1 A C，表示package1要从A发往C站点。

用例第6~7行（对应第二行输入的2），表示两站点之间无法通行的包裹。

因此，按照上面理解：

A-B无法通行package1，但是package1要从A发往C，因此不受影响。

A-C无法通行package2，而package2刚好要从A发往C，因此受到影响，无法发送。

因此最后，只有package2无法发送。

Java算法源码

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Integer[] tmp =
            Arrays.stream(sc.nextLine().split("
")).map(Integer::parseInt).toArray(Integer[]::new);
        int m = tmp[0];
        int n = tmp[1];

        String[][] want = new String[m][];
        for (int i = 0; i < m; i++) {
            want[i] = sc.nextLine().split(" ");
        }

        String[][] cant = new String[n][];
        for (int i = 0; i < n; i++) {
            cant[i] = sc.nextLine().split(" ");
        }

        System.out.println(getResult(want, cant));
    }

    /**
     * @param want 二维数组，元素是 [包裹， 起始站点， 目的站点]
     * @param cant 二维数组，元素是 [起始站点， 目的站点， ...无法通行的包裹列表]
     * @return 无法通行的包裹
     */
    public static String getResult(String[][] want, String[][] cant) {
        HashMap<String, HashSet<String>> wantMap = new HashMap<>();
        HashMap<String, HashSet<String>> cantMap = new HashMap<>();

        for (String[] arr : want) {
            String pkg = arr[0];
            String path = arr[1] + "-" + arr[2];
            wantMap.putIfAbsent(path, new HashSet<>());
            wantMap.get(path).add(pkg);
        }
    }
}
```

```

for (String[] arr : cant) {
    String path = arr[0] + "-" + arr[1];
    String[] pkgs = Arrays.copyOfRange(arr, 2, arr.length);
    cantMap.putIfAbsent(path, new HashSet<>());
    cantMap.get(path).addAll(Arrays.asList(pkgs));
}

ArrayList<String> ans = new ArrayList<>();

for (String path : wantMap.keySet()) {
    HashSet<String> wantPKG = wantMap.get(path);
    HashSet<String> cantPKG = cantMap.get(path);

    if (cantPKG == null) continue;

    for (String pkg : wantPKG) {
        if (cantPKG.contains(pkg)) {
            ans.add(pkg);
        }
    }
}

if (ans.size() == 0) return "none";

ans.sort((a, b) -> Integer.parseInt(a.substring(7)) -
Integer.parseInt(b.substring(7)));

StringJoiner sj = new StringJoiner(" ");
for (String an : ans) {
    sj.add(an);
}
return sj.toString();
}
}

```

95.数字加减游戏

题目描述

小明在玩一个数字加减游戏，只使用加法或者减法，将一个数字s变成数字t。
 每个回合，小明可以用当前的数字加上或减去一个数字。
 现在有两种数字可以用来加减，分别为a,b(a!=b)，其中b没有使用次数限制。
 请问小明最少可以用多少次a，才能将数字s变成数字t。
 题目保证数字s一定能变成数字t。

输入描述

输入的唯一一行包含四个正整数s,t,a,b(1<=s,t,a,b<=105)，并且a!=b。

输出描述

输出的唯一一行包含一个整数，表示最少需要使用多少次a才能将数字s变成数字t。

用例

输入	1 10 5 2
输出	1
说明	初始值1加一次a变成6，然后加两次b变成10，因此a的使用次数为1

输入	11 33 4 10
输出	2
说明	11减两次a变成3，然后加三次b变成33，因此a的使用次数为2次

题目解析

本题有点像数学问题

比如用例1其实就是求解：

满足 $5 * x + 2 * y = 9$ 的所有解中绝对值最小的x

比如用例2其实就是求解：

满足 $4 * x + 10 * y = 22$ 的所有解中绝对值最小的x

因此，我们可以让x从0开始尝试，然后尝试1，-1，然后尝试2，-2，直到找到一个x能够让（比如用例1） $(9 - 5 * x) / 2$ 为一个整数。

由于本题 $1 \leq s, t, a, b \leq 105$ ，数量级较小，因此上面逻辑可行。

Java算法源码

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[] arr = new int[4];
        for (int i = 0; i < 4; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.println(getResult(arr[0], arr[1], arr[2], arr[3]));
    }
}
```

```

    }

    public static int getResult(int s, int t, int a, int b) {
        int x = 0;
        int diff = t - s;
        while (true) {
            if ((diff - a * x) % b == 0 || (diff + a * x) % b == 0) {
                return Math.abs(x);
            }
            x++;
        }
    }
}

```

96. 几何平均值最大子数组

题目描述

从一个长度为N的正数数组numbers中找出长度至少为L且几何平均值最大子数组，并输出其位置和大小。（K个数的几何平均值为K个数的乘积的K次方根）

若有多个子数组的几何平均值均为最大值，则输出长度最小的子数组。

若有多个长度相同的子数组的几何平均值均为最大值，则输出最前面的子数组。

输入描述

第一行输入为N、L

- N表示numbers的大小 ($1 \leq N \leq 100000$)
- L表示子数组的最小长度 ($1 \leq L \leq N$)

之后N行表示numbers中的N个数，每个一行 ($10^{-9} \leq \text{numbers}[i] \leq 10^9$)

输出描述

输出子数组的位置（从0开始计数）和大小，中间用一个空格隔开。

备注

用例保证除几何平均值为最大值的子数组外，其他子数组的几何平均值至少比最大值小 10^{-10} 倍

用例

输入	3 2 2 2 3
----	-----------

输入	3 2 2 2 3
输出	1 2
说明	长度至少为2的子数组共三个，分别是{2,2}、{2,3}、{2,2,3}，其中{2,3}的几何平均值最大，故输出其位置1和长度2

输入	10 2 0.2 0.1 0.2 0.2 0.2 0.1 0.2 0.2 0.2 0.2
输出	2 2
说明	有多个长度至少为2的子数组的几何平均值为0.2，其中长度最短的为2，也有多个，长度为2且几何平均值为0.2的子数组最前面的那个为从第二个数开始的两个0.2组成的子数组

题目解析

本题其实就是 [LeetCode - 644 子数组最大平均数 II 伏城之外的博客-CSDN博客](#)

的变种题。但是本题更难。

建议大家先把leetcode 644 这题做会了，再来看本题。

本题和leetcode 644的区别在于，leetcode 644求解的长度大于等于k的 最大算术平均值 的连续子序列，而本题求解的是 长度大于等于k的 最大几何平均值 的连续子序列。

一个数组的nums = [a1, a2, a3, ..., aN]的

- 算术平均值 = $(a1 + a2 + a3 + \dots + aN) / N$
- 几何平均值 = $N \sqrt[N]{a1 * a2 * a3 * \dots * aN}$

因此，在求解 长度大于等于k 的子序列时，我们不能在沿用leetcode 644的解法，leetcode 644解法如下

首先，求出 0~i 子序列的和 sum

然后，求出 0~0 到 0~i-k 中所有子序列的最小和 min_pre_sum

最后，sum - min_pre_sum >= 0的话，说明midVal可能取小了

本题需要换成除法

首先，求出 0~i 子序列的所有元素乘积 fact

然后，求出 0~0 到 0~i-k 中所有子序列的最小乘积 min_pre_fact

最后，fact / min_pre_fact >= 1的话，说明midVal可能取小了

原理如下：有一个数组 $\text{nums} = [a_1, a_2, a_3, \dots, a_N]$ ，假设其几何平均值为 avg ，则有等式如下：

$$N \sqrt[N]{(a_1 * a_2 * a_3 * \dots * a_N)} == \text{avg}$$

再转换一下，如下：

$$a_1 * a_2 * a_3 * \dots * a_N == \text{avg}^N$$

再转换一下，如下：

$$(a_1 / \text{avg}) * (a_2 / \text{avg}) * (a_3 / \text{avg}) * \dots * (a_N / \text{avg}) == 1$$

如果 avg 取大了，则 $(a_1 / \text{avg}) * (a_2 / \text{avg}) * (a_3 / \text{avg}) * \dots * (a_N / \text{avg}) < 1$

如果 avg 取小了，则 $(a_1 / \text{avg}) * (a_2 / \text{avg}) * (a_3 / \text{avg}) * \dots * (a_N / \text{avg}) > 1$

另外，本题需要输出最大几何平均值对应的子数组的起始位置和长度，这个很简单，只需要记录每次被挖去的最小平均值子数组的结尾索引即可，根据结尾索引 min_pre_fact_end ，即可得出最大平均值数组的起始索引和长度，计算公式如下

```
if (fact / min_pre_fact >= 1) {  
    ans[0] = min_pre_fact_end + 1;  
    ans[1] = i - min_pre_fact_end;  
    return true;  
}
```

CSDN @伏城之外

Java算法源码

```
import java.util.Scanner;  
  
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        int n = sc.nextInt();  
        int l = sc.nextInt();  
  
        double[] numbers = new double[n];  
        for (int i = 0; i < n; i++) {  
            numbers[i] = sc.nextDouble();  
        }  
  
        System.out.println(getResult(n, l, numbers));  
    }  
  
    public static String getResult(int n, int l, double[] numbers) {  
        double minAvg = Integer.MAX_VALUE;  
        double maxAvg = Integer.MIN_VALUE;  
        for (double num : numbers) {  
            minAvg = Math.min(num, minAvg);  
            maxAvg = Math.max(num, maxAvg);  
        }  
  
        double diff = maxAvg / Math.pow(10, 10);
```

```

int[] ans = new int[2];
while (maxAvg - minAvg >= diff) {
    double midAvg = (minAvg + maxAvg) / 2;

    if (check(n, l, numbers, midAvg, ans)) {
        minAvg = midAvg;
    } else {
        maxAvg = midAvg;
    }
}

return ans[0] + " " + ans[1];
}

public static boolean check(int n, int l, double[] numbers, double avg, int[]
ans) {
    double fact = 1;

    for (int i = 0; i < l; i++) {
        fact *= numbers[i] / avg;
    }

    if (fact >= 1) {
        ans[0] = 0;
        ans[1] = l;
        return true;
    }

    double pre_fact = 1;
    double min_pre_fact = Integer.MAX_VALUE;
    int min_pre_fact_end = 0;

    for (int i = l; i < n; i++) {
        fact *= numbers[i] / avg;
        pre_fact *= numbers[i - l] / avg;

        if (pre_fact < min_pre_fact) {
            min_pre_fact = pre_fact;
            min_pre_fact_end = i - l;
        }

        if (fact / min_pre_fact >= 1) {
            ans[0] = min_pre_fact_end + 1;
            ans[1] = i - min_pre_fact_end;
            return true;
        }
    }

    return false;
}
}

```

97. 微服务的集成测试

题目描述

现在有n个容器服务，服务的启动可能有一定的依赖性（有些服务启动没有依赖），其次服务自身启动加载会消耗一些时间。

给你一个 $n \times n$ 的二维矩阵useTime，其中

- useTime[i][i]=10 表示服务i自身启动加载需要消耗10s
- useTime[i][j] = 1 表示服务i启动依赖服务j启动完成
- useTime[i][k]=0 表示服务i启动不依赖服务k

其实 $0 \leq i, j, k < n$ 。

服务之间启动没有[循环依赖](#)（不会出现环），若想对任意一个服务i进行集成测试（服务i自身也需要加载），求最少需要等待多少时间。

输入描述

第一行输入服务总量 n，
之后的 n 行表示服务启动的依赖关系以及自身启动加载耗时
最后输入 k 表示计算需要等待多少时间后可以对服务 k 进行[集成测试](#)
其中 $1 \leq k \leq n, 1 \leq n \leq 100$

输出描述

最少需要等待多少时间(s)后可以对服务 k 进行集成测试

用例

输入	3 5 0 0 1 5 0 0 1 5 3
输出	15
说明	服务3启动依赖服务2，服务2启动依赖服务1，由于服务1，2，3自身加载需要消耗5s，所以5+5+5=15，需要等待15s后可以对服务3进行集成测试

输入	3 5 0 0 1 1 0 1 1 0 1 1 2
输出	26
说明	服务2启动依赖服务1和服务3，服务3启动需要依赖服务1，服务1，2，3自身加载需要消耗5s，10s，11s，所以5+10+11=26s，需要等待26s后可以对服务2进行集成测试。

输入	420000300114011154
输出	12
说明	服务3启动依赖服务1和服务2，服务4启动需要依赖服务1，2，3，服务1，2，3自身加载需要消耗2s,3s,4s,5s，所以3+4+5=12s（因为服务1和服务2可以同时启动），要等待12s后可以对服务4进行集成测试。

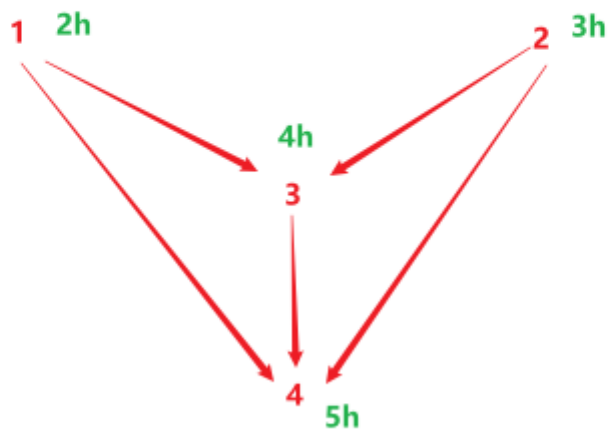
输入	510000020001130011040001155
输出	11
说明	服务3启动依赖服务1和服务2，服务4启动需要依赖服务1，2，服务5启动需要依赖服务3，5，服务1，2，3，4，5自身加载需要消耗1s,2s,3s,4s,5s，所以2+4+5=11s（因为服务1和服务2可以同时启动，服务3和服务4可以同时启动），要等待11s后可以对服务5进行集成测试。

题目解析

本题看上去很像[拓扑排序](#)，但是拓扑排序并不是解决本题的最佳方案。

本题最佳解题思路是，利用递归，求解要求的服务点的启动时间，具体思路如下：

比如用例3图示如下：



CSDN @伏城之外

现在要求解服务4的启动时间，其实就是：

服务4的启动时间 = max(服务1启动时间， 服务2启动时间， 服务3启动时间) + 本身启动时间

其中，服务1，服务2没有前置服务，因此他们的启动时间就是本身启动时间。

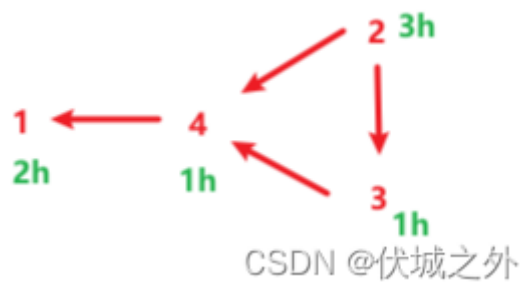
而，服务3有前置服务，因此服务3的启动时间 = max(服务1启动时间， 服务2启动时间) + 本身启动时间

因此，服务3的启动时间 = max(2, 3) + 4 = 7

进而，服务4的启动时间 = max(2, 3, 7) + 5 = 12

这里提供一个测试用例：

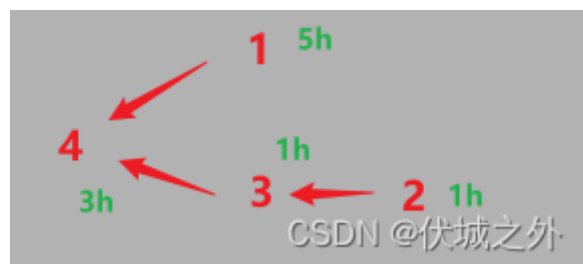
```
4
2 0 0 1
0 3 0 0
0 1 1 0
0 1 1 1
1
```



输出应该是7

再提高一个测试用例：

```
4
5 0 0 0
0 1 0 0
0 1 1 0
1 0 1 3
4
```



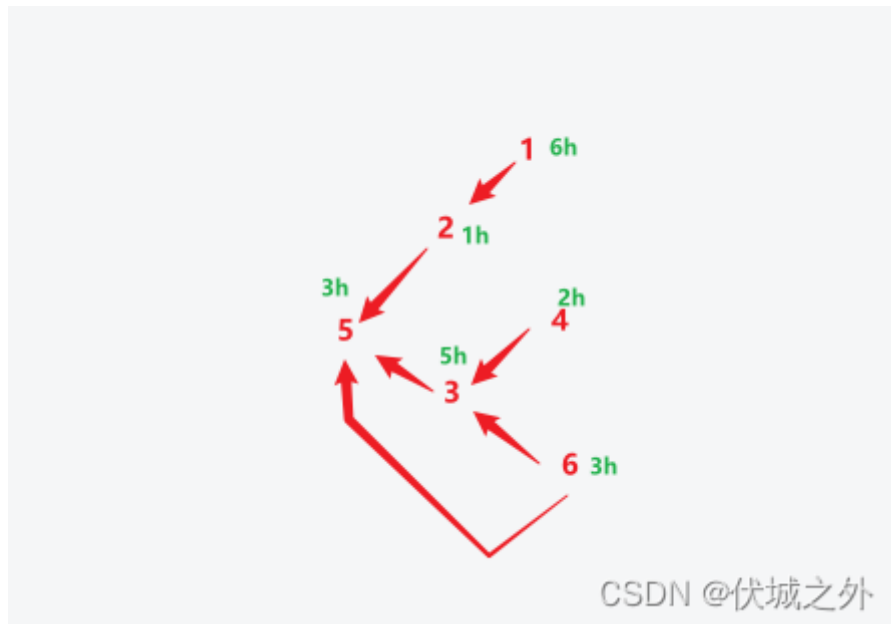
输出应该是8

再补充一个测试用例：

```
6
6 0 0 0 0 0
1 1 0 0 0 0
0 0 5 1 0 1
0 0 0 2 0 0
0 1 1 0 3 1
0 0 0 0 0 3
5
```

输出应该是11

图示如下



依赖关系如下：

- 5 依赖于 2, 3服务启动完成；
- 2 依赖于 1 服务启动完成；
- 1 不依赖于其他服务
- 3 依赖于 4, 6服务启动完成；
- 4 不依赖于其他服务
- 6 不依赖于其他服务

我们假设当前时刻为0，然后1，4，6同时开始启动

0时刻：1，4，6启动中

1时刻：1，4，6启动中

2时刻：1，6启动中，4启动完成

3时刻：1 启动中，6启动完成，因此该时刻3服务可以启动了

4时刻：1，3启动中

5时刻：1，3启动中

6时刻：3启动中，1启动完成，因此该时刻2服务可以启动了

7时刻：3启动中，2启动完成

8时刻：3启动完成，因此该时刻5服务可以启动了

9时刻：5启动中

10时刻：5启动中

11时刻：5启动完成

Java算法源码

以下代码经网友反馈通过率75%，原因是超时（也有可能是超出内存限制），经过思考，超时（超内存）原因应该是下面代码中对于pre的统计，理论上，我们不需要统计pre信息，新的算法请看第二个源码

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[][] matrix = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        int k = sc.nextInt();

        System.out.println(getResult(matrix, n, k));
    }

    public static int getResult(int[][] matrix, int n, int k) {
        // pre用于保存每个点的前驱点
        HashMap<Integer, ArrayList<Integer>> pre = new HashMap<>();

        // 开始统计
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (i == j) continue;
                pre.putIfAbsent(i, new ArrayList<>());

                // useTime[i][j] = 1表示服务i启动依赖服务j启动完成，即j的后继点是i；i的前驱点是j
                if (matrix[i][j] == 1) {
                    pre.get(i).add(j);
                }
            }
        }

        return dfs(k - 1, pre, matrix);
    }

    public static int dfs(int k, HashMap<Integer, ArrayList<Integer>> pre, int[][] matrix) {
        if (pre.get(k).size() == 0) {
            return matrix[k][k];
        }

        int maxPreTime = 0;
```

```

        for (Integer p : pre.get(k)) {
            maxPreTime = Math.max(maxPreTime, dfs(p, pre, matrix));
        }

        return matrix[k][k] + maxPreTime;
    }
}

```

改进版本代码

```

import java.util.Arrays;
import java.util.Scanner;

public class Main {
    static int[] cache;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        cache = new int[n];
        Arrays.fill(cache, -1);

        int[][] matrix = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        int k = sc.nextInt();

        System.out.println(getResult(matrix, n, k));
    }

    public static int getResult(int[][] matrix, int n, int k) {
        return dfs(k - 1, matrix);
    }

    public static int dfs(int k, int[][] matrix) {
        // cache用于记录每个服务所需要时间，避免多个子服务依赖同一个父服务时，对父服务启动时间的重复递归求解
        if (cache[k] != -1) return cache[k];

        int[] preK = matrix[k];

        int maxPreTime = 0;
        for (int i = 0; i < preK.length; i++) {
            if (i != k && preK[i] != 0) {
                maxPreTime = Math.max(maxPreTime, dfs(i, matrix));
            }
        }

        cache[k] = matrix[k][k] + maxPreTime;
    }
}

```

```
        return cache[k];  
    }  
}
```

98. 组装新的数组

题目描述

给你一个整数M和数组N，N中的元素为连续整数，要求根据N中的元素组装成新的数组R，组装规则：

1. R中元素总和加起来等于M
2. R中的元素可以从N中重复选取
3. R中的元素最多只能有1个不在N中，且比N中的数字都要小（不能为负数）

输入描述

第一行输入是连续数组N，采用空格分隔

第二行输入数字M

输出描述

输出的是组装办法数量，int类型

备注

- $1 \leq M \leq 30$
- $1 \leq N.length \leq 1000$

用例

输入	2 5
输出	1
说明	只有1种组装办法，就是[2,2,1]

输入	2 3 5
输出	2
说明	一共两种组装办法，分别是[2,2,1]，[2,3]

题目解析

我们可以换个说法来看本题，

在数组N任意选取多个元素，且同一个元素可以重复选取，只要最终选取的所有元素之和等于m，或者：小于m但是差值不超过N.min()。

现在是不是感觉本题简单多了。

我们可以使用[回溯算法](#)来在N中选取多个元素（同一个元素可以重复选取），这个逻辑其实就是求可重复元素组合情况，每得到一个组合就看其和sum是否等于m，或者m - sum 是否已经小于 N.min，若是，则该组合是符合要求的，count++，否则不符合要求，继续找。当sum > m时，则说明往后的组合已经无法符合要求，此时需要进行回溯了。

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Integer[] arr =
            Arrays.stream(sc.nextLine().split("
")).map(Integer::parseInt).toArray(Integer[]::new);

        int m = Integer.parseInt(sc.nextLine());

        System.out.println(getResult(arr, m));
    }

    public static int getResult(Integer[] arr, int m) {
        Integer[] newArr = Arrays.stream(arr).filter(val -> val <=
m).toArray(Integer[]::new);

        int min = newArr[0];

        return dfs(newArr, 0, 0, min, m, 0);
    }

    public static int dfs(Integer[] arr, int index, int sum, int min, int m, int
count) {
        if (sum > m) {
            return count;
        }

        if (sum == m || (m - sum < min && m - sum > 0)) {
            return count + 1;
        }

        for (int i = index; i < arr.length; i++) {
            count = dfs(arr, i, sum + arr[i], min, m, count);
        }
    }
}
```

```
        return count;
    }
}
```

题目描述

公司某部门软件教导团正在组织新员工每日打卡学习活动，他们开展这项学习活动已经一个月了，所以想统计下这个月优秀的打卡员工。每个员工会对应一个id，每天的打卡记录记录当天打卡员工的id集合，一共30天。

请你实现代码帮助统计出打卡次数top5的员工。加入打卡次数相同，将较早参与打卡的员工排在前面，如果开始参与打卡的时间还是一样，将id较小的员工排在前面。

注：不考虑并列的情况，按规则返回前5名员工的id即可，如果当月打卡的员工少于5个，按规则排序返回所有有打卡记录的员工id。

输入描述

第一行输入为新员工数量N，表示新员工编号id为0到N-1， N的范围为[1,100]

第二行输入为30个整数，表示每天打卡的员工数量，每天至少有1名员工打卡。

之后30行为每天打卡的员工id集合，id不会重复。

输出描述

按顺序输出打卡top5员工的id，用空格隔开。

用例

输入	1144111111111111111111111111111122017100161010101010 10610710
输出	100176
说明	员工编号范围为0~10，id为10的员工连续打卡30天，排第一，id为0,1,6,7的员工打卡都是两天，id为0,1,7的员工在第一天就打卡，比id为6的员工早，排在前面，0,1,7按id升序排列，所以输出[10,0,1,7,6]

[illegible]


```

}

public static String getResult(int[][] dayIds) {
    HashMap<Integer, Integer[]> employees = new HashMap<>();

    for (int i = 0; i < dayIds.length; i++) {
        int[] ids = dayIds[i];

        for (int id : ids) {
            if (employees.containsKey(id)) {
                employees.get(id)[0]++;
            } else {
                // 加入数组含义是：该id员工的 [打卡次数, 第一天打卡日期]
                employees.put(id, new Integer[] {1, i});
            }
        }
    }

    ArrayList<Integer[]> list = new ArrayList<>();
    for (Integer id : employees.keySet()) {
        Integer[] employee = employees.get(id);
        int count = employee[0];
        int firstDay = employee[1];
        list.add(new Integer[] {id, count, firstDay});
    }

    list.sort(
        (a, b) ->
            a[1].equals(b[1]) ? (a[2].equals(b[2]) ? a[0] - b[0] : a[2] - b[2]) :
            b[1] - a[1]);

    StringJoiner sj = new StringJoiner(" ");
    // 不考虑并列的情况，按规则返回前5名员工的id即可，如果当月打卡的员工少于5个，按规则排序返回所有
    有打卡记录的员工id
    for (int i = 0; i < Math.min(5, list.size()); i++) {
        sj.add(list.get(i)[0] + "");
    }
    return sj.toString();
}

```

100.单词倒序

题目描述

输入单行英文句子，里面包含英文字母，空格以及.,?三种标点符号，请将句子内每个单词进行倒序，并输出倒序后的语句。

输入描述

输入字符串S，S的长度 $1 \leq N \leq 100$

输出描述

输出倒序后的字符串

备注

标点符号左右的空格 ≥ 0 ，单词间空格 > 0

用例

输入	yM eman si boB.
输出	My name is Bob.
说明	无

输入	woh era uoy ? I ma enif.
输出	how are you ? I am fine.
说明	无

题目解析

从用例可以看出，单词的倒序并不难，将字符串单词转为字符数组后，reverse一下就行了。但是单词中如果有标点符号的话，则标点符号的位置不能改变，比如enif. 倒序后为 fine. 其中. 的位置在倒序前后是一样的。

我的解题思路如下，从左到右遍历每一个字符，如果字符是 , . ? 或者空格，则看成一个分界符，将分界符之间的单词片段进行倒序。

Java算法源码

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        System.out.println(getResult(str));
    }
}
```

```

public static String getResult(String str) {
    ArrayList<Integer> idxs = new ArrayList<>();

    idxs.add(-1);
    for (int i = 0; i < str.length(); i++) {
        if (",.?" .indexOf(str.charAt(i)) != -1) {
            idxs.add(i);
        }
    }
    idxs.add(str.length());

    char[] chars = str.toCharArray();

    for (int i = 0; i < idxs.size() - 1; i++) {
        int l = idxs.get(i) + 1;
        int r = idxs.get(i + 1) - 1;

        while (l < r) {
            char tmp = chars[l];
            chars[l] = chars[r];
            chars[r] = tmp;
            l++;
            r--;
        }
    }

    StringBuilder sb = new StringBuilder();
    for (char c : chars) {
        sb.append(c);
    }
    return sb.toString();
}
}

```

101.区间交叠问题

题目描述

给定坐标轴上的一组线段，线段的起点和终点均为整数并且长度不小于1，请你从中找到最少数量的线段，这些线段可以覆盖柱所有线段。

输入描述

第一行输入为所有线段的数量，不超过10000，后面每行表示一条线段，格式为"x,y"，x和y分别表示起点和终点，取值范围是 $[-10^5, 10^5]$ 。

输出描述

最少线段数量，为正整数

用例

输入	3 1,4 2,5 3,6
输出	2
说明	无

题目解析

用例1图示如下



可以发现，只要选择[1,4]和[3,6]就可以覆盖住所有给定线段。

我的解题思路如下：

首先，将所有区间ranges按照开始位置升序。

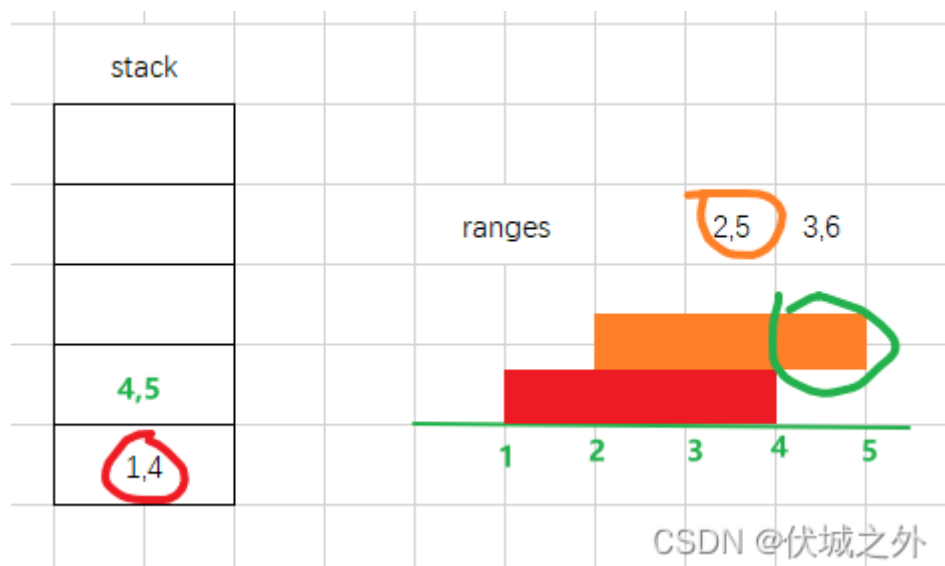
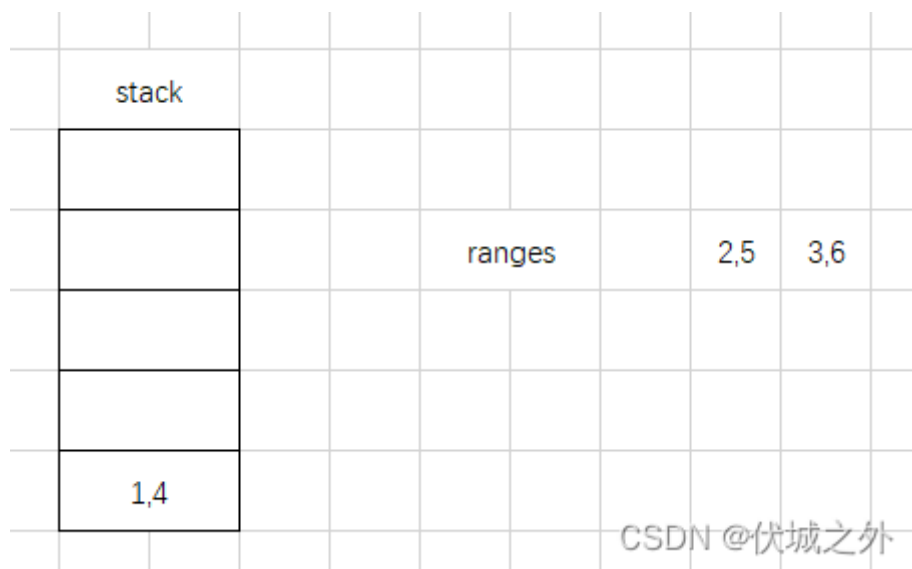
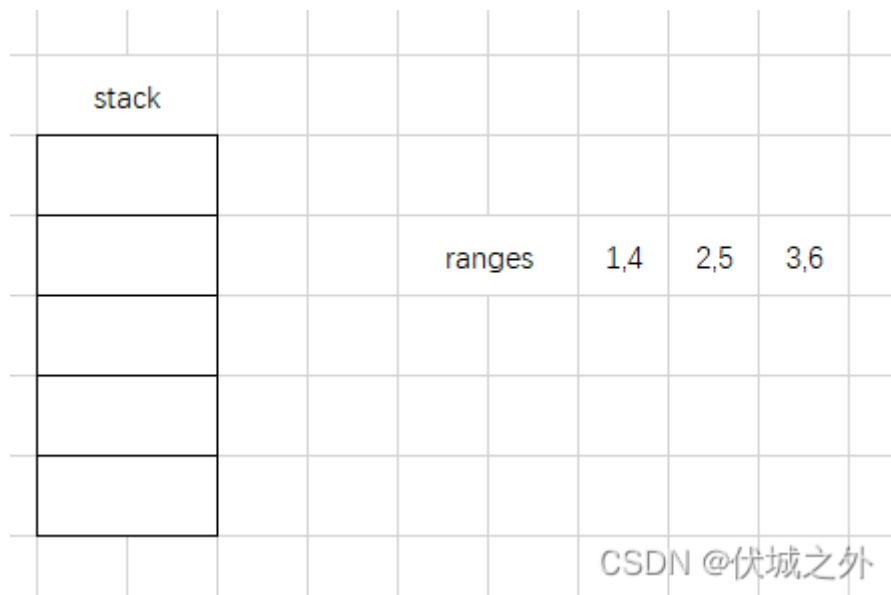
然后，创建一个辅助的栈stack，初始时将排序后的第一个区间压入栈中。

然后，遍历出1~ranges.length范围内的每一个区间ranges[i]，将遍历到ranges[i]和stack栈顶区间对比：

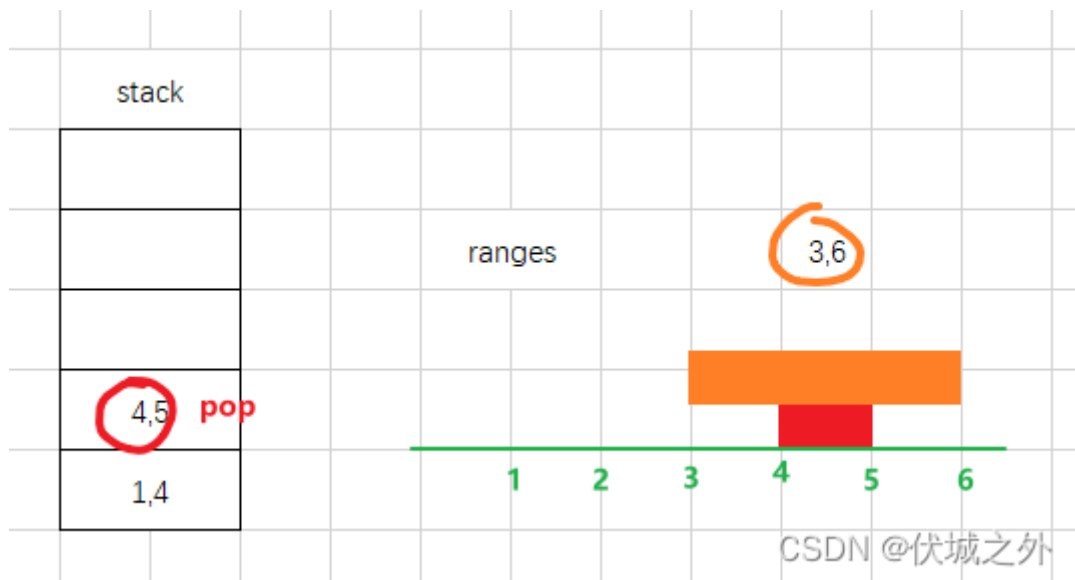
- 如果stack栈顶区间可以包含ranges[i]区间，则range[i]不压入栈顶
- 如果stack栈顶区间被ranges[i]区间包含，则弹出stack栈顶元素，继续比较ranges[i]和stack新的栈顶元素
- 如果stack栈顶区间和ranges[i]无法互相包含，只有部分交集，则将ranges[i]区间去除交集部分后，剩余部分区间压入stack
- 如果stack栈顶区间和ranges[i]区间没有交集，那么直接将ranges[i]压入栈顶

这样的话，最终stack中有多少个区间，就代表至少需要多少个区间才能覆盖所有线段。

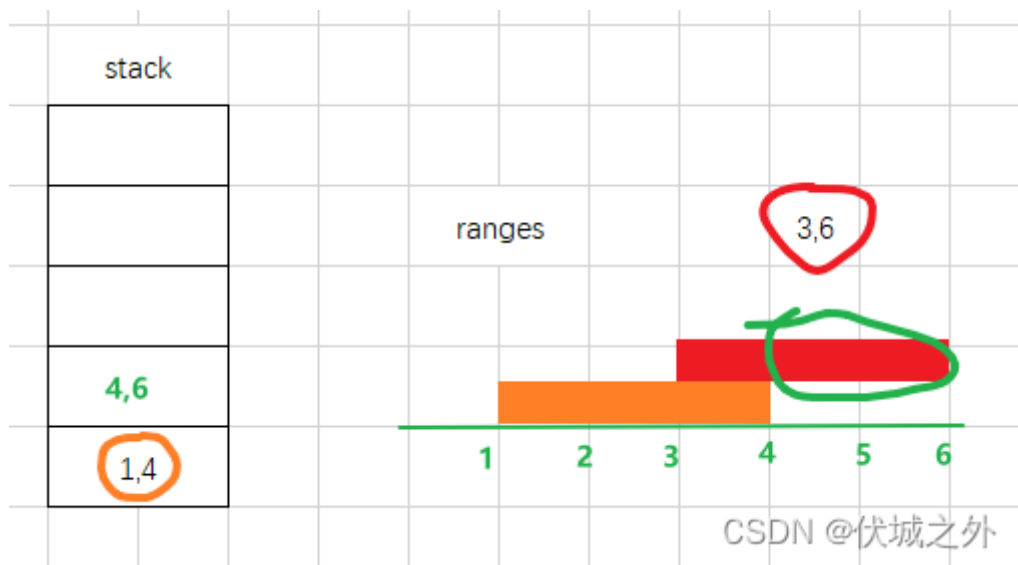
比如，用例1的运行流程如下：



2,5 和 1,4 存在重叠区间，我们只保留2,5区间的非重叠部分4,5

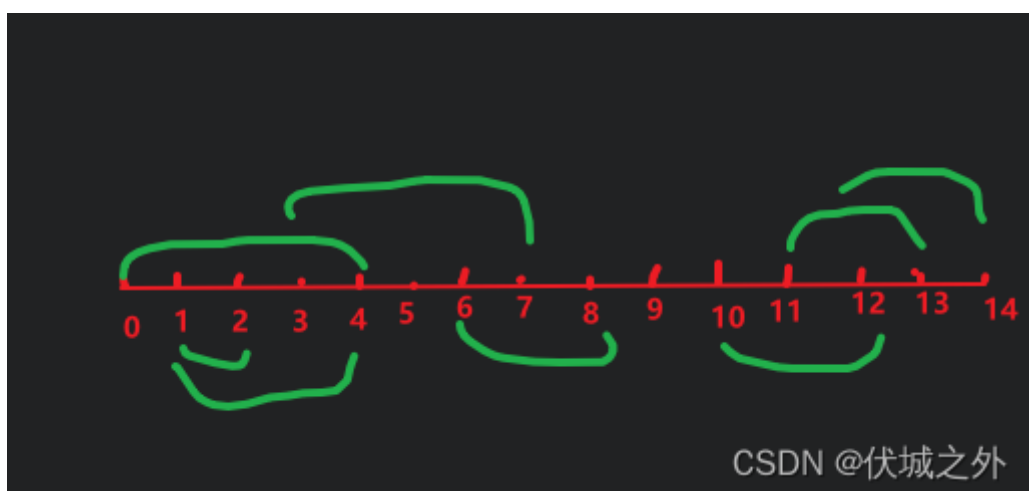


比较4,5区间和3,6区间，发现3,6完全涵盖2,5,因此2,5区间不再需要，可以从stack中弹栈删掉，即原始的2,5区间被删除了。



继续比较1,4和3,6区间，发现无法互相涵盖，因此都需要保留，但是3,6有部分区间和1,4重叠，因此只保留3,6不重叠部分4,6。

最终只需要两个区间，对应1,4、3,6，即可涵盖所有线段



自测用例：

8
0,4
1,2
1,4
3,7
6,8
10,12
11,13
12,14

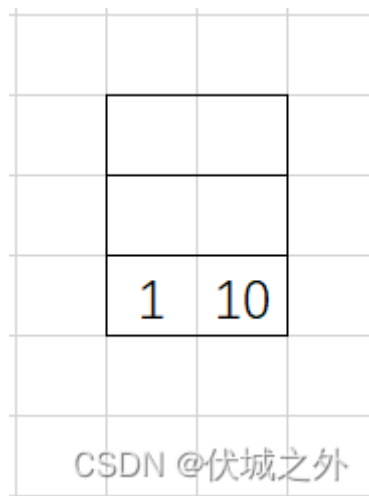
输出5，即至少需要上面标红的五个区间才能覆盖所有线段。

2023.01.27

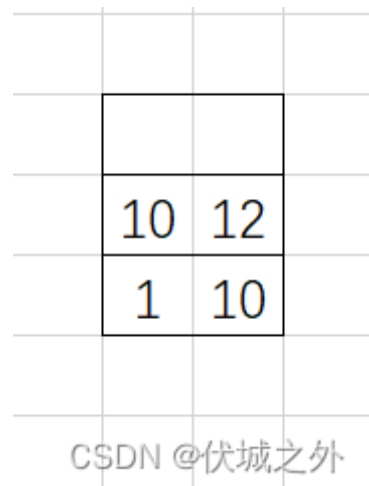
根据网友指正，上面逻辑缺失一个场景，比如：

3
1,10
5,12
8,11

按找前面逻辑，首先对所有区间按开始位置升序，然后将1,10入栈



然后尝试将5,12入栈，发现和栈顶区间有交集，因此去除交集部分后，5,12变为10,12，入栈



然后尝试将8,11入栈，但是此时出现一个尴尬的情况，那就是栈顶区间10,12不能完全包含8,11，因此8,11区间还需要和栈顶前一个区间1,10继续比较，这就背离了我们一开始将所有区间按开始位置升序的初衷了。。。

而导致这个问题的根本原因是，栈顶区间10,12是被裁剪过的，因此导致它的起始位置落后了，即可能无法包含住升序后下一个区间的起始位置了，但是转念一想，先入栈的区间的起始位置肯定是要小于等于后入栈的区间的，因此栈顶区间被裁剪，说明栈顶区间和前一个区间必然是严密结合的，因此8,11的起始位置超出了栈顶区间，其实还是会被栈顶前一个区间包含进去。因此这里8,11不入栈。

Java算法源码

```
import java.util.Arrays;
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(sc.nextLine());

        Integer[][] ranges = new Integer[n][];
        for (int i = 0; i < n; i++) {
            ranges[i] =

Arrays.stream(sc.nextLine().split(",")).map(Integer::parseInt).toArray(Integer[
]::new);
        }

        System.out.println(getResult(ranges));
    }

    public static int getResult(Integer[][] ranges) {
        Arrays.sort(ranges, (a, b) -> a[0] - b[0]);

        LinkedList<Integer[]> stack = new LinkedList<>();
        stack.add(ranges[0]);

        for (int i = 1; i < ranges.length; i++) {
            Integer[] range = ranges[i];

            while (true) {
                if (stack.size() == 0) {
                    stack.add(range);
                    break;
                }

                Integer[] top = stack.getLast();
                int s0 = top[0];
                int e0 = top[1];

                int s1 = range[0];
                int e1 = range[1];

                if (s1 <= s0) {
```

```

        if (e1 <= s0) {
            break;
        } else if (e1 < e0) {
            break;
        } else {
            stack.removeLast();
        }
    } else if (s1 < e0) {
        if (e1 <= e0) {
            break;
        } else {
            stack.add(new Integer[] {e0, e1});
            break;
        }
    } else {
        stack.add(range);
        break;
    }
}
}

return stack.size();
}
}

```

102.九宫格

题目描述

[九宫格](#)是一款广为流传的游戏，起源于河图洛书。

游戏规则是：1到9九个数字放在3×3的格子中，要求每行、每列以及两个对角线上的三数之和都等于15。在金瓶名著《射雕英雄传》中黄蓉曾给九宫格的一种解法，口诀：戴九恩一，左三右七，二四有肩，八六为足，五居中央。解法如图所示。

现在有一种新的玩法，给九个不同的数字，将这九个数字放在3×3的格子中，要求每行、每列以及两个对角线上的三数之积相等（三阶积[幻方](#)）。其中一个三阶幻方如图：

解释：每行、每列以及两个对角线上的三数之积相等，都为216。请设计一种算法，将给定的九个数字重新排列后，使其满足三阶积幻方的要求。

排列后的九个数字中：第1-3个数字为方格的第一行，第4-6个数字为方格的第二行，第7-9个数字为方格的第三行。

输入描述

九个不同的数字，每个数字之间用空格分开。

$0 < \text{数字} < 10^7$ 。0<排列后满足要求的每行、每列以及两个对角线上的三数之积 $< 2^{31}-1$ 。

输出描述

九个数字所有满足要求的排列，每个数字之间用空格分开。每行输出一个满足要求的排列。
要求输出的排列升序排序，即：对于排列A (A1.A2.A3...A9)和排列B(B1,B2,B3...B9) ， 从排列的第1个数字开始，遇到 $A_i < B_i$ ，则排列A<排列B （ $1 \leq i \leq 9$ ）。

说明：用例保证至少有一种排列组合满足条件。

用例

输入	75 36 10 4 30 225 90 25 12
输出	10 36 75 225 30 4 12 25 90 10 225 12 36 30 25 75 4 90 12 25 90 225 30 4 10 36 75 12 225 10 25 30 36 90 4 75 75 4 90 36 30 25 10 225 12 75 36 10 4 30 225 90 25 12 90 4 75 25 30 36 12 225 10 90 25 12 4 30 225 75 36 10
说明	无

题目解析

简单的全排列问题。

关于全排列的入门，可以看[组合与排列的区别，回溯算法求解的时候，有何不同？ | LeetCode: 46.全排列哔哩哔哩bilibili](#)

练习题可以做leetcode上：
[LeetCode - 46 全排列 伏城之外的博客-CSDN博客](#)

基于回溯算法的全排列求解是一种暴力解法，即枚举出全部排列情况，因此对大数量级而言，我们应该慎用，但是本题，已经明确指出了求解9个数字的全排列，因此排列情况共有9!个，即362880个，数量级还好，因此可以使用暴力求解。

另外，求出符合要求的排列，还需要对各排列进行排序，排序是按各个数字大小来比较的，大家可以看下代码中关于排序的实现。

Java算法源码

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Integer[] arr =
```

```

        Arrays.stream(sc.nextLine().split("
")).map(Integer::parseInt).toArray(Integer[]::new);
        getResult(arr);
    }

    public static void getResult(Integer[] arr) {
        boolean[] used = new boolean[arr.length];
        LinkedList<Integer> path = new LinkedList<>();
        ArrayList<Integer[]> res = new ArrayList<>();

        dfs(arr, used, path, res);

        res.sort(
            (a, b) -> {
                for (int i = 0; i < 9; i++) {
                    if (!Objects.equals(a[i], b[i])) return a[i] - b[i];
                }
                return 0;
            }
        );

        for (Integer[] re : res) {
            StringJoiner sj = new StringJoiner(" ");
            for (Integer i : re) {
                sj.add(i + "");
            }
            System.out.println(sj);
        }
    }

    public static void dfs(
        Integer[] arr, boolean[] used, LinkedList<Integer> path,
        ArrayList<Integer[]> res) {
        if (path.size() == arr.length) {
            if (check(path)) {
                Integer[] a = path.toArray(new Integer[0]);
                res.add(a);
            }
            return;
        }

        for (int i = 0; i < arr.length; i++) {
            if (!used[i]) {
                path.add(arr[i]);
                used[i] = true;
                dfs(arr, used, path, res);
                used[i] = false;
                path.removeLast();
            }
        }
    }

    public static boolean check(LinkedList<Integer> path) {
        Integer[] a = path.toArray(new Integer[0]);

        int r1 = a[0] * a[1] * a[2];
    }

```

```
int r2 = a[3] * a[4] * a[5];
if (r1 != r2) return false;

int r3 = a[6] * a[7] * a[8];
if (r1 != r3) return false;

int c1 = a[0] * a[3] * a[6];
if (r1 != c1) return false;

int c2 = a[1] * a[4] * a[7];
if (r1 != c2) return false;

int c3 = a[2] * a[5] * a[8];
if (r1 != c3) return false;

int s1 = a[0] * a[4] * a[8];
if (r1 != s1) return false;

int s2 = a[2] * a[4] * a[6];
if (r1 != s2) return false;

return true;
}
}
```

103.找数字、找等值元素

题目描述

给一个二维数组nums，对于每一个元素nums[i]，找出距离最近的且值相等的元素，输出横纵坐标差值的绝对值之和，如果没有等值元素，则输出-1。

输入描述

输入第一行为二维数组的行
输入第二行为二维数组的列
输入的数字以空格隔开。

输出描述

数组形式返回所有坐标值。

用例

输入	3 5 0 3 5 4 2 2 5 7 8 3 2 5 4 2 4
输出	[[-1, 4, 2, 3, 3], [1, 1, -1, -1, 4], [1, 1, 2, 3, 2]]
说明	无

题目解析

我的解题思路如下：

首先遍历输入的矩阵，将相同数字的位置整理到一起。

然后再遍历一次输入的矩阵，找到和遍历元素相同的其他数字（通过上一步统计结果快速找到），求距离，并保留最小的距离。

上面逻辑的[算法复杂度](#)为 $O(nmk)$ ，其中 n 是输入矩阵行， m 是输入矩阵列， k 是每组相同数字的个数，可能会达到 $O(N^3)$ 的时间复杂度。

有一个优化点就是，遍历元素A找其他相同数字B时计算的距离可以缓存，这样的话，当遍历元素B时找其他相同数字A时，就可以从缓存中读取已经计算好的距离了，而不是重新计算。但是这样会浪费较多空间。

实际机试时，可以看用例数量级来决定是否要做上面这个优化。如果数量级很小，则无需上面优化。如果数量级较大，则有优化的必要。

Java算法源码

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        int[][] matrix = new int[n][m];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        System.out.println(getResult(matrix, n, m));
    }
}
```

```

public static String getResult(int[][] matrix, int n, int m) {
    HashMap<Integer, ArrayList<Integer[]>> nums = new HashMap<>();

    // 统计输入矩阵中，相同数字的位置
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            Integer num = matrix[i][j];
            Integer[] arr = {i, j};
            nums.putIfAbsent(num, new ArrayList<>());
            nums.get(num).add(arr);
        }
    }

    // 遍历矩阵每一个元素，和其他相同数字的位置求距离，，取最小距离
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            int num = matrix[i][j];

            int minDis = Integer.MAX_VALUE;
            for (Integer[] pos : nums.get(num)) {
                int i1 = pos[0];
                int j1 = pos[1];

                if (i1 != i || j1 != j) {
                    int dis = Math.abs(i1 - i) + Math.abs(j1 - j);
                    minDis = Math.min(minDis, dis);
                }
            }

            matrix[i][j] = minDis == Integer.MAX_VALUE ? -1 : minDis;
        }
    }

    return
    Arrays.toString(Arrays.stream(matrix).map(Arrays::toString).toArray(String[]::new));
}

```

104.模拟商场优惠打折

题目描述

模拟商场优惠打折，有三种[优惠券](#)可以用，满减券、打折券和无门槛券。

满减券：满100减10，满200减20，满300减30，满400减40，以此类推不限制使用；

打折券：固定折扣92折，且打折之后[向下取整](#)，每次购物只能用1次；

无门槛券：一张券减5元，没有使用限制。

每个人结账使用优惠券时有以下限制：

每人每次只能用两种优惠券，并且同一种优惠券必须一次用完，不能跟别的穿插使用（比如用一张满减，再用一张打折，再用一张满减，这种顺序不行）。

求不同使用顺序下每个人用完券之后得到的最低价格和对应使用优惠券的总数；如果两种顺序得到的价格一样低，就取使用优惠券数量较少的那个。

输入描述

第一行三个数字m,n,k，分别表示每个人可以使用的满减券、打折券和无门槛券的数量；

第二行一个数字x, 表示有几个人购物；

后面x行数字，依次表示是这几个人打折之前的商品总价。

输出描述

输出每个人使用券之后的最低价格和对应使用优惠券的数量

用例

输入	3 2 5 3 100 200 400
输出	65 6 135 8 275 8
说明	输入：第一行三个数字m,n,k，分别表示每个人可以使用的满减券、打折券和无门槛券的数量。输出：第一个人使用 1 张满减券和5张无门槛券价格最低。（100-10=90, 90-55=65）第二个人使用 3 张满减券和5张无门槛券价格最低。（200-20-10-10=160, 160 - 55 = 135）第三个人使用 3 张满减券和5张无门槛券价格最低。（400-40-30-30=300, 300 - 5*5=275）

题目解析

本题的解题思路如下，首先实现满减，打折，无门槛的逻辑：

- 满减逻辑，只要总价price大于等于100，且还有满减券，则不停 $price -= \text{Math.floor}(price / 100) * 10$ ；直到总价price小于100，或者满减券用完。
- 打折逻辑，按照题目意思，打折券只能使用一次，因此无论打折券有多少张，都只能使用一次，因此只要打折券数量大于等于1，那么 $price = \text{Math.floor}(price * 0.92)$ ；
- 无门槛逻辑，只要总价price大于0，且还有无门槛券，则不停 $price -= 5$ ；直到price小于等于0，或者无门槛券用完。

接下来就是求上面三种逻辑的任选2个的排列：

假设满减是M，打折是N，无门槛是K，则有排列如下：

- MN、NM
- MK、KM
- NK、KN

注意，券的使用对顺序敏感。

因此，求出以上排列后，对每个人的总价使用六种方式减价，只保留减价最多，用券最少的那个。

根据网友iygvh提供的优化思路：

对于无门槛券的使用，无门槛券总是在最后使用才会最优。

对于满减来说，无门槛肯定是最后使用最优惠，

对于92折来说，

- 先用无门槛后打折 $(x-5y)0.92 = x0.92 - 50.92y$
- 先打折后用无门槛 $x*0.92 - 5y$

对比可以看出，先92折，再无门槛最优惠，因此确实可以直接排除KM和KN的情况，即先无门槛的情况。

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
        int n = sc.nextInt();
        int k = sc.nextInt();
        int x = sc.nextInt();

        int[] arr = new int[x];
        for (int i = 0; i < x; i++) {
            arr[i] = sc.nextInt();
        }

        getResult(arr, m, n, k);
    }

    public static void getResult(int[] arr, int m, int n, int k) {
        for (int i = 0; i < arr.length; i++) {
            Integer[][] ans = new Integer[4][2]; // 4的含义对应4种使用券的方式：
            // MN, NM, MK, NK, 2的含义对应每种方式下：剩余总价，剩余券数量
            int price = arr[i];

            int[] resM = M(price, m); // 先满减
            int[] resN = N(price, n); // 先打折

            // MN
            int[] resMN_N = N(resM[0], n); // 满减后打折
            ans[0] =
                new Integer[] {
                    resMN_N[0], m + n - resM[1] - resMN_N[1]
                }; // resMN_N[0]是“满减后打折”的剩余总价， m + n - resM[1] - resMN_N[1]
            // 是 该种用券方式的：总券数 m+n, 剩余券数
            // resM[1] + resMN_N[1], 因此使用掉的券数： m+n - (resM[1] + resMN_N[1])
        }
    }
}
```

```

// NM
int[] resNM_M = M(resN[0], m); // 打折后满减
ans[1] = new Integer[] {resNM_M[0], n + m - resN[1] - resNM_M[1]};

// MK
int[] resMK_K = K(resM[0], k); // 满减后无门槛
ans[2] = new Integer[] {resMK_K[0], m + k - resM[1] - resMK_K[1]};

// NK
int[] resNK_K = K(resN[0], k); // 打折后无门槛
ans[3] = new Integer[] {resNK_K[0], n + k - resN[1] - resNK_K[1]};

Arrays.sort(
    ans,
    (a, b) ->
        a[0].equals(b[0])
            ? a[1] - b[1]
            : a[0] - b[0]); // 对ans进行排序，排序规则是：优先按剩余总价升序，如果
// 剩余总价相同，则再按“使用掉的券数量”升序
System.out.println(ans[0][0] + " " + ans[0][1]);
}
}

/**
 * @param price 总价
 * @param m 满减券数量
 * @return 总价满减后结果，对应数组含义是 [用券后剩余总价, 剩余满减券数量]
 */
public static int[] M(int price, int m) {
    while (price >= 100 && m > 0) {
        price -= price / 100 * 10; // 假设price=340, 那么可以优惠 340/100 * 10 = 30元
        m--;
    }
    return new int[] {price, m};
}

/**
 * @param price 总价
 * @param n 打折券数量
 * @return 总价打折后结果，对应数组含义是 [用券后剩余总价, 剩余打折券数量]
 */
public static int[] N(int price, int n) {
    if (n >= 1) {
        price = (int) Math.floor((price * 0.92));
        n--;
    }
    return new int[] {price, n};
}

/**
 * @param price 总价
 * @param k 无门槛券数量
 * @return 无门槛券用后结果，对应数组含义是 [用券后剩余总价, 剩余无门槛券数量]
 */

```

```
public static int[] K(int price, int k) {
    while (price > 0 && k > 0) {
        price -= 5;
        price = Math.max(price, 0); // 感谢m0_71826536提供的思路，当无门槛券过多时，是有可能导致优惠后总价低于0的情况的，此时我们应该避免总价小于0的情况
        k--;
    }
    return new int[] {price, k};
}
```

105.总最快检测效率

题目描述

在系统、网络均正常的情况下组织核酸采样员和志愿者对人群进行[核酸检测](#)筛查。

每名采样员的效率不同，采样效率为N人/小时。

由于外界变化，采样员的效率会以M人/小时为粒度发生变化，M为采样效率浮动粒度， $M=N10\%$ ，输入保证N10%的结果为整数。

采样员效率浮动规则：采样员需要一名志愿者协助组织才能发挥正常效率，在此基础上，每增加一名志愿者，效率提升1M，最多提升3M；如果没有志愿者协助组织，效率下降2M。

怎么安排速度最快？求**总最快检测效率**（总检查效率为各采样人员效率值相加）。

输入描述

第一行：第一个值，采样员人数，取值范围[1, 100]；第二个值，志愿者人数，取值范围[1, 500]；
第二行：各采样员基准效率值（单位人/小时），取值范围[60, 600]，保证序列中每项值计算10%为整数。

输出描述

第一行：总最快检测效率（单位人/小时）

用例

输入	2 2 200 200
输出	400
说明	输入需要保证采样员基准效率值序列的每个值*10%为整数。

题目解析

用例意思是：

有两个采样员，两个志愿者。

两个采样员的正常效率都是200，但是要给每个采样员配一个志愿者才能发挥正常效率。

现在刚好采样员和志愿者是一比一，因此可以发挥出总效率是： $200 + 200 = 400$ 。

如果，我们给一个采样员配两个志愿者，那么该采样员发挥的效率是： $200 + 200 * 10\% = 220$ 。

但是另一名采样员就没有志愿者了，因此发挥不了正常效率， $200 - 200 * 20\% = 160$ ，此时总效率是 $220 + 160 = 380$ 。

因此，总最快效率是400。

需要注意的是，用例中采样员的正常效率只是凑巧相同，很有可能出现一个采样员的正常效率极高，一个采样员的正常效率极低的情况。

我的解题思路如下：

首先分两种情况：

- 1、志愿者数量少于采样员
- 2、志愿者数量不少于采样员

对于情况1，我们应该将不多的志愿者优先分配给高效率的采样员，默认一比一分配。

接下来，我们应该考虑，剥夺低效率的采样员的志愿者 给 高效率的采样员，只要 高效率采样员增加的10%的效率 可以大于 低效率采样员减少的20%的效率。

其中还要考虑，高效率的采样员最多可以追加3个志愿者，即最多增加30%的效率。如果最高效率的采样员已经提升30%效率，则第二高效率的采样员称为最高优先级，继续上面剥夺逻辑。

对于情况2，我们应该先按一比一的方式，给每个采样员分配一个志愿者。

然后，如果还多出志愿者的话，则优先分配给高效率的采样员，同样需要注意每个采样员最追加3个志愿者。

当多出的志愿者分配完后，我们需要考虑剥夺低效率的采样员的志愿者 给 高效率的采样员，只要 高效率采样员增加的10%的效率 可以大于 低效率采样员减少的20%的效率。逻辑同情况1。

根据ant_shi网友的指正，上面解析对于志愿者超出采样员数量四倍时的情况考虑不够全面：

另外，对于情况2而言，如果采样员：志愿者 的比例，超过了1：4，那么超出4倍采样员范围的志愿者将没有效率提升作用，因此有效志愿者数量最多是四倍的采样员数量。

贪心思维解法

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int x = sc.nextInt();
        int y = sc.nextInt();

        Integer[] arrX = new Integer[x];
        for (int i = 0; i < x; i++) {
            arrX[i] = sc.nextInt();
        }

        System.out.println(getResult(arrX, x, y));
    }

    public static int getResult(Integer[] arr, int x, int y) {
        // 按照正常效率降序
        Arrays.sort(arr, (a, b) -> b - a);

        int max = 0;
        int count = 0;
        int i;
        int j;

        // 如果志愿者少于采样员，则优先将志愿者分配给正常效率高的采样员
        if (y < x) {
            // 0~y-1范围内高效率的采样员优先获得一个志愿者，因此保持正常效率，而y~x-1范围内的低效率
            // 采样员则没有志愿者，效率下降20%
            for (int k = 0; k < x; k++) {
                max += k < y ? arr[k] : arr[k] * 0.8;
            }

            i = 0;
            j = y - 1;
        }
        // 如果志愿者 不少于 采样员，那么默认情况下每个采样员都分配一个志愿者
        else {
            // 如果志愿者人数超过采样员四倍，则多出来的志愿者就没有作用了
            if (y >= 4 * x) {
                y = 4 * x;
            }

            // 每个采样员都默认发挥正常效率
            for (Integer val : arr) {
                max += val;
            }

            // surplus记录每个采样员分配一个志愿者后，还多出来的志愿者
            int surplus = y - x;
        }
    }
}
```

```

i = 0;
j = x - 1;

// 优先将多出来的志愿者分配给高效率的采样员
while (surplus > 0) {
    max += arr[i] * 0.1;
    surplus--;
    if (++count == 3) {
        count = 0;
        i++;
    }
}

// 志愿者分配完后，则继续考虑剥夺低效率采样员的志愿者给高效率的采样员
while (i < j) {
    // 如果最高效率的采样员上升10%的效率 是否大于 最低效率的采样员下降20%的效率，那么就值得剥夺
    if (arr[i] * 0.1 > arr[j] * 0.2) {
        max += arr[i] * 0.1 - arr[j] * 0.2;

        // 由于一个采样员最多只能提升30%，即除了一个基础志愿者外，最多再配3个志愿者，多配了也没用
        if (++count == 3) {
            count = 0;
            i++;
        }
        j--;
    } else {
        break;
    }
}

return max;
}
}

```

优先队列解法

本题最优思路是采用优先队列解法。

即将所有采样员加入（大顶堆）优先队列中，优先级是：该采样员新增一名志愿者，所能提升的效率。

初始时，加入优先队列的采样员都不搭配采样员，此时新增一名志愿者，每个采样员都能提升20%，但是由于基准效率base不同，因此实际每个采样员提升的效率值为base * 0.2。

之后，我们取出堆顶最大提升效率的采样员，为其新增一名志愿者：

- 如果其已有志愿者数量为0，则新增一名志愿者后，恢复为base效率
- 如果其已有志愿者数量≤3，则新增一名志愿者后，其总效率 += base * 0.1
- 如果其已有志愿者数量 > 3，即至少为4，则再新增一名志愿者，不会带来效率提升，即新增效率为0

当我们更新完取出的采样员的志愿者数量和总效率后，将其重新压入优先队列，进行排序。

最后，当志愿者用完了，或者所有采样员都安排了4名志愿者了，则结束上面操作。

接下来就是取出优先队列的所有采样员，并累加他们的总效率作为题解。

Java算法源码

```
import java.util.PriorityQueue;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int x = sc.nextInt();
        int y = sc.nextInt();

        Integer[] arrX = new Integer[x];
        for (int i = 0; i < x; i++) {
            arrX[i] = sc.nextInt();
        }

        System.out.println(getResult(arrX, x, y));
    }

    /**
     * @param arr 各采样员基准效率值（单位人/小时） [60, 600]
     * @param x 采样员人数 [1, 100]
     * @param y 志愿者人数 [1, 500]
     * @return 总最快检测效率（单位人/小时）
     */
    public static int getResult(Integer[] arr, int x, int y) {
        // 大顶堆优先队列，堆顶元素总是：新增一个志愿者后，提升效率最多的采样员
        PriorityQueue<Sampler> pq = new PriorityQueue<>((a, b) -> (int) (getAdd(b) - getAdd(a)));

        // 将所有采样员加入优先队列，初始时采样员都不搭配志愿者
        for (int base : arr) {
            pq.offer(new Sampler(0, base));
        }

        // 只要还有志愿者，就将其分配给采样员
        while (y > 0) {
            // 如果堆顶采样员已有四个志愿者，那么该采样员能提升的效率为0，说明此时提升0的效率就是所有采样员中能提升的最大效率，即说明所有采样员都安排到了四个采样员，因此再增加志愿者也不会带来效率提升
            if (pq.isEmpty() || pq.peek().volunteer == 4) break;

            // 如果上一步不成立，则取出堆顶采样员
            Sampler s = pq.poll();

            // 为其新增一个志愿者，并提升相应效率
            s.total += getAdd(s);
            s.volunteer += 1;

            // 重新压入队列
        }
    }
}
```

```

        pq.offer(s);
        y--;
    }

    double ans = 0;
    while (!pq.isEmpty()) ans += pq.poll().total;
    return (int) ans;
}

// 采样员s增加一名志愿者能提升的效率
public static double getAdd(Sampler s) {
    // 如果当前采样员没有志愿者，则新增一名志愿者可以提升base * 20%的效率
    if (s.volunteer == 0) return s.base * 0.2;
    // 如果当前采样员搭配的志愿者数量小于等于3个，则说明再新增一个志愿者，可以提升base * 10%
    // 的效率
    else if (s.volunteer <= 3) return s.base * 0.1;
    // 如果当前采样员已有4个志愿者，则再新增一个志愿者，不能提升效率，即提升效率为0
    else return 0;
}

// 采样员类
class Sampler {
    int volunteer = 0; // 该采样员搭配的志愿者人数
    double base = 0; // 该采样员的基准效率
    double total = 0; // 该采样员的搭配志愿者后的总效率

    public Sampler(int volunteer, double base) {
        this.volunteer = volunteer;
        this.base = base;
        this.total = base * 0.8; // 初始时采样员没有搭配志愿者，则效率只有base*0.8
    }
}

```

106.二进制差异数

题目描述

对于任意两个正整数A和B，定义它们之间的**差异值**和**相似值**：

差异值：A、B转换成二进制后，对于二进制的每一位，对应位置的bit值不相同则为1，否则为0；

相似值：A、B转换成二进制后，对于二进制的每一位，对应位置的bit值都为1则为1，否则为0；

现在有n个正整数A₀到A_(n-1)，问有多少(i, j) (0 ≤ i < j < n)，A_i和A_j的差异值大于相似值。

假设A=5，B=3；则A的二进制表示101；B的二进制表示011；

则A与B的差异值二进制为110；相似值二进制为001；

A与B的差异值十进制等于6，相似值十进制等于1，满足条件。

输入描述

一个n接下来n个正整数

数据范围： $1 \leq n \leq 10^5$, $1 \leq A[i] < 2^{30}$

输出描述

满足差异值大于相似值的对数

用例

输入	4 4 3 5 2
输出	4
说明	满足条件的分别是(0,1)(0,3)(1,2)(2,3)，共4对。

题目解析

题目描述中，

A，B差异值其实就是A和B二进制的[按位异或](#)运算，即 $A \oplus B$ 。

A，B相似值其实就是A和B二进制的按位与运算，即 $A \& B$ 。

因此，如果本题使用暴力法求解，则逻辑非常简单，如下

```
// 入参arr是用例第二行输入对应的数组
function getResult(arr) {
  const ans = [];
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      const diff = arr[i] ^ arr[j];
      const simi = arr[i] & arr[j];
      if (diff > simi) {
        ans.push([i, j]);
      }
    }
  }
  console.log(ans);
  return ans.length;
}
```

但是 $1 \leq n \leq 10^5$ ，而上面[算法复杂度](#)达到了 $O(n^2)$ ，因此会超时。

对于 $1 \leq n \leq 10^5$ ，[算法时间复杂度](#)应该至少控制在 $O(n)$ 。

因此，我们需要发现规律，而不是暴力求解。

本题的规律其实很容易发现，那就是看A，B的最高位1是否处于相同位，如果相同，比如

A: 1010

B: 1100

那么差异值就是0110，相似值就是1000，可以发现，A,B最高位的1，在按位异或运算下被换成0，在按位与的运算下，变成了1，因此这种情况下，相似值必然大于差异值，不符合要求。

如果A，B的最高位1不处于相同位，比如

A: 1010

B: 0110

那么差异值就是1100，相似值就是0010，可以发现，A，B的最高位不同，因此按位异或运算下被换成了1，而按位与运算下变成了0，因此这种情况下，差异值必然大于相似值，符合要求。

有了以上规律，我们可以统计出，每个数的最高位1处于哪一位，最高位1所处位数相同的数之间无法组合，最高位1所处位数不同的数之间可以组合。

这里我们可以借助，Number.prototype.toString(radix)方法来求出每一个数的二进制形式字符串，由于该方法不会补足前导0，比如Number(6).toString(2)的结果为：110，而不是0000 0110这种带前导0的形式。

因此我们可以直接将Number(6).toString(2)结果110的字符串长度当成其最高位1所处的位数。

按此逻辑，我们就可以将最高位1位数相同的数统计到一起了，然后双重for求统计数之间的乘积之和即可。

另外，我们需要注意的是：0和1 这两个二进制数的长度都是1，但是“0”却没有最高位1，因此要和“1”区别开来。

Java算法源码

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.println(getResult(arr));
    }

    public static int getResult(int[] arr) {
```

```

// highBit是一个数组，由于题目说给定的A[i]取值小于2^30，因此我们可以定义一个30长度的数组，其元素含义是，比如highBit[i]代表最高位1处于第i位的数的个数
// int[] highBit = new int[30];
int[] highBit = new int[60]; // 经网友反馈，这里长度改为60可得100%通过率

for (int a : arr) {
    String bin = Integer.toBinaryString(a);
    int len = bin.length(); // 数的二进制形式字符串（无前导0）的长度，就是该数二进制最高位1所处位数

    // 将“0”和“1”区别开来
    if ("0".equals(bin)) {
        highBit[0]++;
    } else {
        highBit[len]++;
    }
}

int ans = 0;
for (int i = 0; i < highBit.length; i++) {
    for (int j = i + 1; j < highBit.length; j++) {
        ans += highBit[i] * highBit[j];
    }
}

return ans;
}
}

```

107.查找单入口空闲区域

题目描述

给定一个 $m \times n$ 的矩阵，由若干字符 'X' 和 'O' 构成，'X' 表示该处已被占据，'O' 表示该处空闲，请找到最大的单入口空闲区域。

解释：

空闲区域是由连通的'O'组成的区域，位于边界的'O'可以构成入口，

单入口空闲区域即有且只有一个位于边界的'O'作为入口的由连通的'O'组成的区域。

如果两个元素在水平或垂直方向相邻，则称它们是“连通”的。

输入描述

第一行输入为两个数字，第一个数字为行数 m ，第二个数字为列数 n ，两个数字以空格分隔， $1 \leq m, n \leq 200$ 。

剩余各行为矩阵各行元素，元素为 'X' 或 'O'，各元素间以空格分隔。

输出描述

若有唯一符合要求的最大单入口空闲区域，输出三个数字

- 第一个数字为入口行坐标（0~m-1）
- 第二个数字为入口列坐标（0~n-1）
- 第三个数字为区域大小

三个数字以空格分隔；

若有多个符合要求，则输出区域大小最大的，若多个符合要求的单入口区域的区域大小相同，则此时只需要输出区域大小，不需要输出口坐标。

若没有，输出NULL。

用例

输入	44XXXXXOOXXOOXXOXX
输出	3 1 5
说明	存在最大单入口区域，入口坐标(3,1)，区域大小5

输入	45XXXXXOOOOXXOOOXXOXXO
输出	3 4 1
说明	存在最大单入口区域，入口坐标（3,4），区域大小1

输入	54XXXXXOOOXOOOXOOXXXXX
输出	NULL
说明	不存在最大单入口区域

输入	54XXXXXOOOXXXXXOOOXXXX
输出	3
说明	存在两个大小为3的最大单入口区域，两个入口坐标分别为(1,3)、(3,3)

题目解析

本题可以使用[深度优先搜索](#)来解题。

首先，我们可以遍历矩阵元素，当遍历到“O”时，已该“O”的坐标位置开始向其上、下、左、右方向开始深度优先搜索，每搜索到一个“O”，则该空闲区域数量+1，如果搜索到的“O”的坐标位置处于矩阵第一列，或最后一列，或者第一行，或者最后一行，那么该“O”位置就是空闲区域的入口位置，我们将其缓存到out数组中。

当所有深度优先搜索的分支都搜索完了，则判断out统计的入口数量，

1. 如果只有1个，则该空闲区域是符合题意得单入口空闲区域，输出入口坐标位置，以及空闲区域数量。
2. 如果有多个，则该区域不符合要求

另外，我们还需要定义一个check集合来缓存，已经被递归过的"O"位置，避免重复的深度优先搜索。

Java算法源码

```
import java.util.*;

public class Main {
    static int n;
    static int m;
    static String[][] matrix;
    static int[][] offset = {{0, -1}, {0, 1}, {-1, 0}, {1, 0}};
    static HashSet<String> checked = new HashSet<>();

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        n = sc.nextInt();
        m = sc.nextInt();

        matrix = new String[n][m];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                matrix[i][j] = sc.next();
            }
        }

        System.out.println(getResult(matrix, n, m));
    }

    public static String getResult(String[][] matrix, int n, int m) {
        ArrayList<Integer[]> ans = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if ("O".equals(matrix[i][j]) && !checked.contains(i + "-" + j))
                {
                    ArrayList<Integer[]> enter = new ArrayList<>();
                    int count = dfs(i, j, 0, enter);
                    if (enter.size() == 1) {
                        Integer[] pos = enter.get(0);
                        Integer[] an = {pos[0], pos[1], count};
                        ans.add(an);
                    }
                }
            }
        }

        if (ans.size() == 0) return "NULL";
        ans.sort((a, b) -> b[2] - a[2]);

        if (ans.size() == 1 || ans.get(0)[2] > ans.get(1)[2]) {
            StringJoiner sj = new StringJoiner(" ", "", "");
        }
    }
}
```

```

        for (Integer ele : ans.get(0)) {
            sj.add(ele + "");
        }
        return sj.toString();
    } else {
        return ans.get(0)[2] + "";
    }
}

}

public static int dfs(int i, int j, int count, ArrayList<Integer[]> enter) {
    String pos = i + "-" + j;

    if (i < 0 || i >= n || j < 0 || j >= m || "X".equals(matrix[i][j]) ||
checked.contains(pos)) {
        return count;
    }

    checked.add(pos);

    if (i == 0 || i == n - 1 || j == 0 || j == m - 1) enter.add(new
Integer[]{i, j});

    count++;

    for (int k = 0; k < offset.length; k++) {
        int offsetX = offset[k][0];
        int offsetY = offset[k][1];

        int newI = i + offsetX;
        int newJ = j + offsetY;
        count = dfs(newI, newJ, count, enter);
    }

    return count;
}
}

```

108.计算快递主站点

题目描述

快递业务范围有N个站点，A站点与B站点可以中转快递，则认为A-B站可达，如果A-B可达，B-C可达，则A-C可达。

现在给N个站点编号0、1、...n-1，用s[i][j]表示i-j是否可达，s[i][j]=1表示i-j可达，s[i][j]=0表示i-j不可达。

现用二维数组给定N个站点的可达关系，请计算至少选择从几个主站点出发，才能可达所有站点（覆盖所有站点业务）。

说明：s[i][j]与s[j][i]取值相同。

输入描述

第一行输入为N (1 < N < 10000) , N表示站点个数。
之后N行表示站点之间的可达关系, 第i行第j个数值表示编号为i和j之间是否可达。

输出描述

输出站点个数, 表示至少需要多少个主站点。

用例

输入	4 1 1 1 1 1 1 1 0 1 1 1 0 1 0 0 1
输出	1
说明	选择0号站点作为主站点, 0站点可达其他所有站点, 所以至少选择1个站点作为主站才能覆盖所有站点业务。

输入	4 1 1 0 0 1 1 0 0 0 0 1 0 0 0 0 1
输出	3
说明	选择0号站点可以覆盖0、1站点, 选择2号站点可以覆盖2号站点, 选择3号站点可以覆盖3号站点, 所以至少选择3个站点作为主站才能覆盖所有站点业务

题目解析

本题其实就是求解图中[连通分量](#)个数。可以使用并查集求解。

Java算法源码

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[][] matrix = new int[n][n];

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }
    }
}
```

```

    }

    System.out.println(getResult(matrix, n));
}

public static int getResult(int[][] matrix, int n) {
    UnionFindSet ufs = new UnionFindSet(n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (matrix[i][j] == 1) {
                ufs.union(i, j);
            }
        }
    }

    return ufs.count;
}

}

class UnionFindSet {
    int[] fa;
    int count;

    public UnionFindSet(int n) {
        this.count = n;
        this.fa = new int[n];
        for (int i = 0; i < n; i++) this.fa[i] = i;
    }

    public int find(int x) {
        if (x != this.fa[x]) {
            return (this.fa[x] = this.find(this.fa[x]));
        }
        return x;
    }

    public void union(int x, int y) {
        int x_fa = this.find(x);
        int y_fa = this.find(y);

        if (x_fa != y_fa) {
            this.fa[y_fa] = x_fa;
            this.count--;
        }
    }
}
}

```

109. 快递业务站

题目描述

快递业务范围有 N 个站点，A 站点与 B 站点可以中转快递，则认为 A-B 站可达，如果 A-B 可达，B-C 可达，则 A-C 可达。

现在给 N 个站点编号 0、1、...n-1，用 $s[i][j]$ 表示 i-j 是否可达， $s[i][j] = 1$ 表示 i-j 可达， $s[i][j] = 0$ 表示 i-j 不可达。

现用二维数组给定 N 个站点的可达关系，请计算至少选择从几个主站点出发，才能可达所有站点（覆盖所有站点业务）。

说明： $s[i][j]$ 与 $s[j][i]$ 取值相同。

输入描述

第一行输入为 N，N 表示站点个数。 $1 < N < 10000$

之后 N 行表示站点之间的可达关系，第 i 行第 j 个数值表示编号为 i 和 j 之间是否可达。

输出描述

输出站点个数，表示至少需要多少个主站点。

用例

输入	4 1 1 1 1 1 1 1 0 1 1 1 0 1 0 0 1
输出	1
说明	选择 0 号站点作为主站点，0 站点可达其他所有站点，所以至少选择 1 个站点作为主站才能覆盖所有站点业务。

输入	4 1 1 0 0 1 1 0 0 0 0 1 0 0 0 0 1
输出	3
说明	选择 0 号站点可以覆盖 0、1 站点，选择 2 号站点可以覆盖 2 号站点，选择 3 号站点可以覆盖 3 号站点，所以至少选择 3 个站点作为主站才能覆盖所有站点业务。

题目解析

本题其实就是求解[连通分量](#)的个数，可以用并查集求解。

Java算法源码

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[][] matrix = new int[n][n];

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        System.out.println(getResult(matrix, n));
    }

    public static int getResult(int[][] matrix, int n) {
        UnionFindSet ufs = new UnionFindSet(n);

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == 1) {
                    ufs.union(i, j);
                }
            }
        }

        return ufs.count;
    }
}

class UnionFindSet {
    int[] fa;
    int count;

    public UnionFindSet(int n) {
        this.count = n;
        this.fa = new int[n];
        for (int i = 0; i < n; i++) this.fa[i] = i;
    }

    public int find(int x) {
        if (x != this.fa[x]) {
            return (this.fa[x] = this.find(this.fa[x]));
        }
        return x;
    }

    public void union(int x, int y) {
        int x_fa = this.find(x);
```

```
int y_fa = this.find(y);

if (x_fa != y_fa) {
    this.fa[y_fa] = x_fa;
    this.count--;
}
}
}
```

110.最差产品奖

题目描述

A公司准备对他下面的N个产品评选最差奖，
评选的方式是首先对每个产品进行评分，然后根据评分区间计算相邻几个产品中最差的产品。
评选的标准是依次找到从当前产品开始前M个产品中最差的产品，请给出最差产品的评分序列。

输入描述

第一行，数字M，表示评分区间的长度，取值范围是 $0 < M < 10000$
第二行，产品的评分序列，比如[12,3,8,6,5]，产品数量N范围是 $-10000 < N < 10000$

输出描述

评分区间内最差产品的评分序列

用例

输入	3 12,3,8,6,5
输出	3,3,5
说明	12,3,8 最差的是3,3,8,6 最差的是3,8,6,5 最差的是5

题目解析

本题其实就是求解长度为m的[滑动窗口](#)移动过程中的内部最小值。

	12	3	8	6	5
	12	3	8	6	5
	12	3	8	6	5

因此最简单的解法是

```
function getResult(arr, m) {
    const ans = [];

    let j = m - 1;
    while (j < arr.length) {
        ans.push(Math.min.apply(null, arr.slice(j - m + 1, j + 1)));
        j++;
    }

    return ans.join();
}
```

由于滑动窗口的长度固定为 m ，因此依靠单指针就能确定滑动窗口，如上源码中 j 指针就是指向滑动窗口的尾巴，根据滑动窗口长度 m ，就可以计算出滑动窗口的头部在 $j - m + 1$ 的位置，因此我们只需要通过`apply`方法借用`Math.min`来求解`arr`数组的 $j - m + 1 \sim j$ 范围内的最小值即可。

但是上面算法可能存在重复的求解，比如下图所示：

	12	3	8	6	5
	12	3	8	6	5

滑动窗口右移一个，失去了12，新增了6

根据这两个变量，我们可以得出如下结论：

0~2滑动窗口内部的最小值3，没有失去

1~3滑动窗口只是在0~2滑动窗口的基础上失去了12，新增6，并没有失去3，且新增的6比3大，因此1~3的滑动窗口内部最小值还是3。

按照这种思路我们就可以避免对1~3滑动窗口内部重新求解最小值。做到一定的优化。

暴力解法（可100%通过）

Java算法源码

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Scanner;
import java.util.StringJoiner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
        Integer[] arr =

Arrays.stream(sc.next().split(",")).map(Integer::parseInt).toArray(Integer[]::new);

        System.out.println(getResult(arr, m));
    }

    public static String getResult(Integer[] arr, int m) {
        ArrayList<Integer> ans = new ArrayList<>();

        for (int i = 0; i <= arr.length - m; i++) {
            int min = Integer.MAX_VALUE;
            for (int j = i; j < i + m; j++) min = Math.min(min, arr[j]);
            ans.add(min);
        }

        StringJoiner sj = new StringJoiner(",");
        for (Integer an : ans) {
            sj.add(an + "");
        }
        return sj.toString();
    }
}
```

滑窗解法

Java算法源码

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Scanner;
import java.util.StringJoiner;

public class Main2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
```

```

Integer[] arr =

Arrays.stream(sc.next().split(",")).map(Integer::parseInt).toArray(Integer[]::new);

System.out.println(getResult(arr, m));
}

public static String getResult(Integer[] arr, int m) {
    int min = Integer.MAX_VALUE;
    for (int i = 0; i < m; i++) {
        min = Math.min(min, arr[i]);
    }

    ArrayList<Integer> ans = new ArrayList<>();
    ans.add(min);

    int j = m;
    while (j < arr.length) {
        if (arr[j - m] > min) {
            min = Math.min(min, arr[j]);
        } else {
            if (arr[j] <= min) {
                min = arr[j];
            } else {
                min = arr[j - m + 1];
                for (int i = j - m + 2; i <= j; i++) {
                    min = Math.min(min, arr[i]);
                }
            }
        }
        ans.add(min);
        j++;
    }

    StringJoiner sj = new StringJoiner(",");

    for (Integer an : ans) {
        sj.add(an + "");
    }

    return sj.toString();
}
}

```

111.打印机队列

题目描述

有5台打印机打印文件，每台打印机有自己的待打印队列。

因为打印的文件内容有轻重缓急之分，所以队列中的文件有1~10不同的优先级，其中**数字越大优先级越高**。

打印机会从自己的待打印队列中选择**优先级最高**的文件来打印。

如果存在两个优先级一样的文件，则选择**最早进入队列**的那个文件。

现在请你来模拟这5台打印机的打印过程。

输入描述

每个输入包含1个测试用例，

每个测试用例第一行给出发生事件的数量N ($0 < N < 1000$)。

接下来有 N 行，分别表示发生的事件。共有如下两种事件：

1. “IN P NUM”，表示有一个拥有优先级 NUM 的文件放到了打印机 P 的待打印队列中。 ($0 < P \leq 5$, $0 < NUM \leq 10$);
2. “OUT P”，表示打印机 P 进行了一次文件打印，同时该文件从待打印队列中取出。 ($0 < P \leq 5$)。

输出描述

- 对于每个测试用例，每次“OUT P”事件，请在一行中**输出文件的编号**。
- 如果此时没有文件可以打印，请输出“NULL”。
- 文件的编号定义为“IN P NUM”事件发生第 x 次，此处待打印文件的编号为x。编号从1开始。

用例

输入	7 IN 1 1 IN 1 2 IN 1 3 IN 2 1 OUT 1 OUT 2 OUT 2
输出	3 4 NULL
说明	无

输入	5 IN 1 1 IN 1 3 IN 1 1 IN 1 3 OUT 1
输出	2
说明	无

题目解析

本题可以基于[优先队列](#)实现打印机总是打印优先级最高的文件。

优先队列，如果想简单一点的话，则可以基于[有序数组](#)实现，但是有序数组是整体有序，每次有新任务入队，都需要 $O(n)$ 时间复杂度维持。

优先队列最好是基于堆结构实现，所谓堆结构，即一颗[完全二叉树](#)。本题是优先级数值越大，优先级越高，因此我们可以使用大顶堆。

关于基于堆结构实现优先队列，可以参考

[LeetCode - 1705 吃苹果的最大数目](#) [伏城之外的博客-CSDN博客](#)

Java算法源码

Java已经有优先队列实现类PriorityQueue，因此可以直接使用它。

```
import java.util.HashMap;
import java.util.PriorityQueue;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(sc.nextLine());
        String[][] tasks = new String[n][];

        for (int i = 0; i < n; i++) {
            String[] s = sc.nextLine().split(" ");
            tasks[i] = s;
        }

        getResult(tasks);
    }

    public static void getResult(String[][] tasks) {
        // print中存放每台打印机的等待队列
        HashMap<String, PriorityQueue<int[]>> print = new HashMap<>();

        // 文件的编号定义为"IN P NUM"事件发生第 x 次，此处待打印文件的编号为x。编号从1开始。
        int x = 1;
        for (int i = 0; i < tasks.length; i++) {
            String[] task = tasks[i];
            // IN,OUT都有type和printId
            String type = task[0];
            String printId = task[1];

            if ("IN".equals(type)) {
                // IN还有priority
                String priority = task[2];
                // arr是打印任务
                int[] arr = {x, Integer.parseInt(priority), i}; // i代表先来后到的顺序
                // 为打印机printId设置打印优先级，打印任务的priority越大，优先级越高
                print.putIfAbsent(
                    printId,
                    new PriorityQueue<>() {
                        (a, b) ->
                            a[1] != b[1] ? b[1] - a[1] : a[2] - b[2]); // 优先按
priority, 如果priority相同，按先来后到i
                // 将打印任务加入对应打印机
                print.get(printId).offer(arr);
                x++;
            } else {
                // 打印机等待队列中取出优先级最高的打印任务arr
                if (!print.containsKey(printId) || print.get(printId).isEmpty()) {
                    // 如果此时没有文件可以打印，请输出"NULL"。
                    System.out.println("NULL");
                }
            }
        }
    }
}
```



```
        } else {
            int[] arr = print.get(printId).poll();
            if (arr != null) System.out.println(arr[0]); // arr[0]是x
            else System.out.println("NULL");
        }
    }
}
```

112.最长的密码

题目描述

小王在进行游戏大闯关，有一个关卡需要输入一个密码才能通过，密码获得的条件如下：

在一个密码本中，每一页都有一个由26个小写字母组成的若干位密码，每一页的密码不同，需要从这个密码本中寻找这样一个最长的密码，

从它的末尾开始依次去掉一位得到的新密码也在密码本中存在。

请输出符合要求的密码，如果有多个符合要求的密码，则返回**字典序最大**的密码。

若没有符合要求的密码，则返回**空字符串**。

输入描述

密码本由一个[字符串数组](#)组成，不同元素之间使用空格隔开，每一个元素代表密码本每一页的密码。

输出描述

一个字符串

用例

输入	h he hel hell hello
输出	hello
说明	无

输入	b ereddred bw bww bwwl bwwlm bwwln
输出	bwwln
说明	无

题目解析

本题和[华为机试 - 真正的密码 伏城之外的博客-CSDN博客](#)

含义相同，解法类似。

本题存在一个疑问，那就是单字符算不算一个有效密码，比如用例

a b c d e f

本题用例没有相关例子，按照题目意思：

从它的末尾开始依次去掉一位得到的新密码也在密码本中存在

单字符删除末尾一位后就是空串，但是密码本中没有空串，因此是否可以认为不算有效密码呢？

但是也不能确定，因此这里我提供两种写法，一种不支持单字符密码，一种支持单字符密码

Java算法源码

不支持单字符为有效密码

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String[] arr = sc.nextLine().split(" ");
        System.out.println(getResult(arr));
    }

    public static String getResult(String[] arr) {
        Arrays.sort(arr, (a, b) -> a.length() != b.length() ? a.length() -
        b.length() : a.compareTo(b));

        LinkedList<String> link = new LinkedList<>(Arrays.asList(arr));
        HashSet<String> set = new HashSet<>(link);

        while (link.size() > 0) {
            String str = link.removeLast();
            int end = str.length() - 1;

            while (set.contains(str.substring(0, end))) {
                if (end == 1) {
                    return str;
                } else {
                    end--;
                }
            }
        }

        return "";
    }
}
```

```
}  
}
```

支持单字符为有效密码

```
import java.util.Arrays;  
import java.util.HashSet;  
import java.util.LinkedList;  
import java.util.Scanner;  
  
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        String[] arr = sc.nextLine().split(" ");  
        System.out.println(getResult(arr));  
    }  
  
    public static String getResult(String[] arr) {  
        Arrays.sort(arr, (a, b) -> a.length() != b.length() ? a.length() -  
b.length() : a.compareTo(b));  
  
        LinkedList<String> link = new LinkedList<>(Arrays.asList(arr));  
        HashSet<String> set = new HashSet<>(link);  
  
        String ans = "";  
  
        while (link.size() > 0) {  
            String str = link.removeLast();  
            int end = str.length() - 1;  
  
            if(end == 0 && "".equals(ans)) {  
                ans = str;  
                continue;  
            }  
  
            while (set.contains(str.substring(0, end))) {  
                if (end == 1) {  
                    return str;  
                } else {  
                    end--;  
                }  
            }  
        }  
  
        return ans;  
    }  
}
```

113.对称美学

题目描述

对称就是最大的美学，现有一道关于对称字符串的美学。已知：

- 第1个字符串：R
- 第2个字符串：BR
- 第3个字符串：RBBR
- 第4个字符串：BRRBRBBR
- 第5个字符串：RBBRBRRBBRRBRBBR

相信你已经发现规律了，没错！就是第 i 个字符串 = 第 i - 1 号字符串取反 + 第 i - 1 号字符串；

取反（R->B, B->R）；

现在告诉你n和k，让你求得第n个字符串的第k个字符是多少。（k的编号从0开始）

输入描述

第一行输入一个T，表示有T组用例；

解析来输入T行，每行输入两个数字，表示n， k

- $1 \leq T \leq 100$;
- $1 \leq n \leq 64$;
- $0 \leq k < 2^{(n-1)}$;

输出描述

输出T行表示答案；

输出 "blue" 表示字符是B；

输出 "red" 表示字符是R。

备注：输出字符串区分大小写，请注意输出小写字符串，不带双引号。

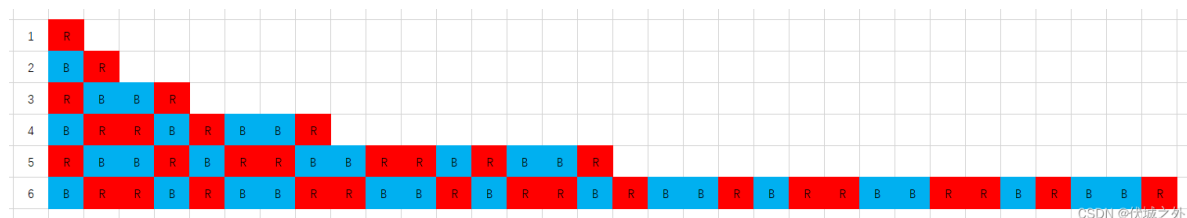
用例

输入	5 1 0 2 1 3 2 4 6 5 8
输出	red red blue blue blue
说明	第 1 个字符串：R -> 第 0 个字符为R 第 2 个字符串：BR -> 第 1 个字符为R 第 3 个字符串：RBBR -> 第 2 个字符为B 第 4 个字符串：BRRBRBBR -> 第 6 个字符为B 第 5 个字符串：RBBRBRRBBRRBRBBR -> 第 8 个字符为B

输入	1 64 73709551616
----	------------------

输入	1 64 73709551616
输出	red
说明	无

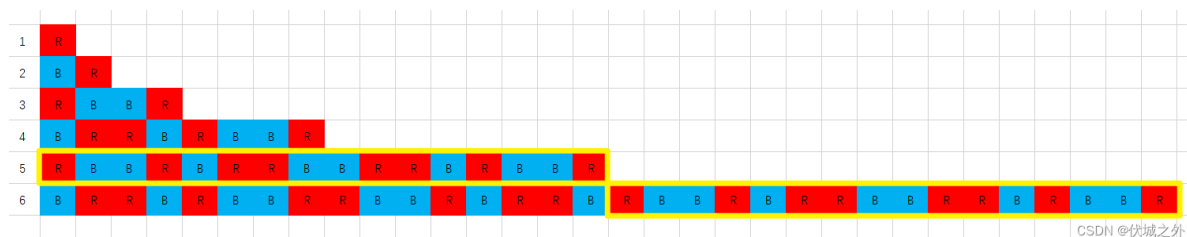
题目解析



上图所示，是第1~6个字符串，可以发现第6个已经很长了，那么本题的最多要求到第64个字符串，那么有多长呢？答： 2^{63} ，即 $2^{(n-1)}$ 。这个长度如果用字符串来存储的话，肯定爆内存，因此任何需要缓存字符串的动作都是禁止的。

我们只能找规律，来通过规律推导出第n个字符串的第k个字符。

那么规律是啥呢？



如上图黄框所示，我们可以发现，

第6个字符串的后半部分，和第5个字符串完全相同；

同理，

第5个字符串的后半部分，和第4个字符串完全相同；

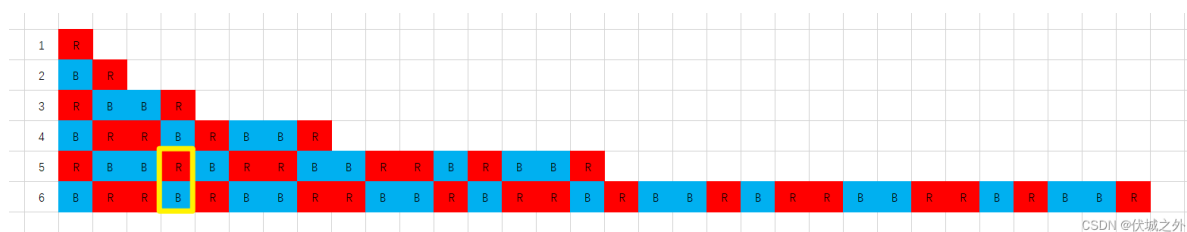
第4个字符串的后半部分，和第3个字符串完全相同；

第3个字符串的后半部分，和第2个字符串完全相同；

第2个字符串的后半部分，和第1个字符串完全相同；

因此，如果我们要找到的k位于第n个字符串的后半部分，假设为 $\text{get}(n, k)$ ，那么其等价于 $\text{get}(n-1, k - 2^{(n-2)})$ ，按此逻辑递归，就可以一直找到第2个字符串或第1个字符串，而这两个字符串很好确认，分别是“R”，“BR”。

那么如果我们要找到第k个字符串，位于第n个字符串的前半部分呢？



可以发现，其实 $\text{get}(n, k)$ ，如果 $k \leq 2^{(n-2)}$ ，则相当于 $\text{get}(n-1, k)$ 的颜色取反。

Java算法源码

需要注意的是，本题中

- $1 \leq n \leq 64$;
- $0 \leq k < 2^{(n-1)}$;

也就是说 k 最大值可以取到 $2^{63} - 1$ ，即9223372036854775807，而这个数刚好是Java的long类型最大值，因此不会造成整型溢出。但是我们代码中所有的数都必须使用long类型装载。

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int t = sc.nextInt();

        long[][] arr = new long[t][2];
        for (int i = 0; i < t; i++) {
            arr[i][0] = sc.nextLong();
            arr[i][1] = sc.nextLong();
        }

        getResult(arr);
    }

    public static void getResult(long[][] arr) {
        for (long[] nk : arr) {
            System.out.println(getNK(nk[0], nk[1]));
        }
    }

    public static String getNK(long n, long k) {
        if (n == 1) {
            return "red";
        }

        if (n == 2) {
            if (k == 0) return "blue";
            else return "red";
        }

        long half = 1L << (n - 2);

        if (k >= half) {
            return getNK(n - 1, k - half);
        } else {
            return "red".equals(getNK(n - 1, k)) ? "blue" : "red";
        }
    }
}
```

114.字符串解密

题目描述

给定两个字符串string1和string2。
string1是一个被加扰的字符串。

string1由小写英文字母（'a'~'z'）和数字字符（'0'~'9'）组成，而加扰字符串由'0'~'9'、'a'~'f'组成。

string1里面可能包含0个或多个加扰子串，剩下可能有0个或多个有效子串，这些有效子串被加扰子串隔开。

string2是一个参考字符串，仅由小写英文字母（'a'~'z'）组成。

你需要在string1字符串里找到一个有效子串，这个有效子串要同时满足下面两个条件：

- （1）这个有效子串里不同字母的数量不超过且最接近于string2里不同字母的数量，即小于或等于string2里不同字母的数量的同时且最大。
- （2）这个有效子串是满足条件（1）里的所有子串（如果有多个月的话）里字典序最大的一个。

如果没有找到合适条件的子串的话，请输出"Not Found"

输入描述

input_string1
input_string2

输出描述

output_string

用例

输入	123admyffc79pt ssyy
输出	pt
说明	将输入字符串1里的加扰子串“123ad”、“ffc79”去除后得到有效子串序列: "my"、"pt", 其中"my"里不同字母的数量为2（有'm'和'y'两个不同字母），“pt”里不同字母的数量为2（有'p'和't'两个不同字母）；输入字符串2里不同字母的数量为2（有's'和'y'两个不同字母）。可得到最终输出结果为“pt”，其不同字母的数量最接近与“ssyy”里不同字母的数量的同时字典序最大。
输入	123admyffc79ptaagghi2222smeersst88mnrt ssyyfgh

输入	123admyffc79ptaagghi2222smeersst88mnrt ssyyfgh
输出	mnrt
说明	将输入字符串1里的加扰子串“123ad”、“ffc79”、“aa”、“2222”、“ee”、“88”去除后得到有效子串序列：“my”、“pt”、“gghi”、“sm”、“rsst”、“mnrt”；输入字符串2里不同字母的数量为5（有's'、'y'、'f'、'g'、'h'5个不同字母）。可得到最终输出结果为“mnrt”，其不同字母的数量（为4）最接近于“ssyyfgh”里不同字母的数量，其他有效子串不同字母的数量都小于“mnrt”。

输入	abcmnq rt
输出	Not Found
说明	将输入字符串1里的加扰子串“abc”去除后得到有效子串序列：“mnq”；输入字符串2里不同字母的数量为2（有'r'、't'两个不同的字母）。可得到最终的输出结果为“Not Found”，没有符合要求的子串，因有效子串里的不同字母的数量（为3），大于输入字符串2里的不同字母的数量。

题目解析

这题出奇的简单，但是划在200分档，难道是我审题有误？

我的解题思路如下：

首先将str1按照加扰字符串来分割，这里我使用正则表达式来作为分隔符，正则为/[0-9a-f]+/，因此用例1中str1会被分割，产生一个数组valids = ["", "my", "pt"]

```
> const str = '123admyffc79pt'
< undefined
> const reg = /[0-9a-f]+/
< undefined
> str.split(reg)
< ▶ (3) ['', 'my', 'pt'] CSDN @伏城之外
>
```

然后需要求出str2的不同字母数量，这里可以使用new Set来去重，然后取size，就是str2的不同字母数量。

同样地，也可以对valids中数组元素使用上面逻辑求不同字母数量，然后将valids数组中每个元素不同字母数量超过str2的filter掉，这样valids中剩余的元素都是不同字母数量不超过str2的。

接着，对valids中元素先按照不同字母数量降序，然后按照字典序降序，取valids[0]作为题解。

如果valids没有元素，则返回Not Found

Java算法源码

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str1 = sc.next();
        String str2 = sc.next();

        System.out.println(getResult(str1, str2));
    }

    public static String getResult(String str1, String str2) {
        String reg = "[0-9a-f]+";

        String[] valids = str1.split(reg);

        int count = getDistinctCount(str2);

        String[] ans = Arrays.stream(valids).filter(valid -> !"".equals(valid)
&& getDistinctCount(valid) <= count).toArray(String[]::new);

        if(ans.length == 0) return "Not Found";

        Arrays.sort(ans, (a,b)->{
            int c1 = getDistinctCount(a);
            int c2 = getDistinctCount(b);
            return c1 != c2 ? c2 - c1 : b.compareTo(a);
        });

        return ans[0];
    }

    public static int getDistinctCount(String str) {
        HashSet<Character> set = new HashSet<>();
        for (char c : str.toCharArray()) {
            set.add(c);
        }
        return set.size();
    }
}
```

115.最短木板长度

题目描述

小明有 n 块木板，第 i ($1 \leq i \leq n$) 块木板长度为 a_i 。

小明买了一块长度为 m 的木料，这块木料可以切割成任意块，拼接到已有的木板上，用来加长木板。

小明想让最短的模板尽量长。请问小明加长木板后，最短木板的长度可以为多少？

输入描述

输入的第一行包含两个正整数， $n (1 \leq n \leq 10^3)$, $m (1 \leq m \leq 10^6)$, n 表示木板数， m 表示木板长度。
输入的第二行包含 n 个正整数， $a_1, a_2, \dots, a_n (1 \leq a_i \leq 10^6)$ 。

输出描述

输出的唯一一行包含一个正整数，表示加长木板后，最短木板的长度最大可以为多少？

用例

输入	5 3 4 5 3 5 5
输出	5
说明	给第1块木板长度增加1，给第3块木板长度增加2后，这5块木板长度变为[5,5,5,5,5]，最短的木板的长度最大为5。

输入	5 2 4 5 3 5 5
输出	4
说明	给第3块木板长度增加1后，这5块木板长度变为[4,5,4,5,5]，剩余木料的长度为1。此时剩余木料无论给哪块木板加长，最短木料的长度都为4。

题目解析

本题的题意是比较明确的，我的解题思路如下：

要想让最短的木板尽可能长，那么我们就需要不停地递进式补足最短板，比如用例输入有5个板：4 5 3 5 5，可用材料 $m=3$

最短的板长度是3，只有一个，那么我们就将他补足到4，此时消耗了一单位长度的材料， $m=2$

这样的话，只剩下两种长度的板4，5，

且4长度有两个，5长度有三个，最短板是长度4.

接下来我们应该尽量将最短板4长度的板补足到5长度，而刚好剩余材料 $m=2$ ，可以将所有4长度的板补足到5长度，此时所有板都是5长度，且材料耗尽。

我们还需要考虑一种特殊情况，那就是 m 还有值，但是只剩下一一种长度的板，此时我们应该平分材料到每一个板，

假设只剩一种长度的板有count个，则平均分的话，每个板能分得 m / count 长度，这个值有可能是小数，我们举个例子：

5个一样长度x的板， $m = 13$ ，则 $13 / 5 = 2...3$ ，因此最短板长度就是 $x+2$ ，

再比如

5个一样长度x的板， $m = 15$ ，则 $13 / 5 = 3$ ，因此最短板长度就是 $x+3$ 。

本题有点[贪心](#)思维，即优先分配m的长度给最短板。

关于算法的时间复杂度，由于板子数量最多1000个，因此统计相同长度板子数量的时间复杂度很低。

然后m的长度达到了 10^6 ，这就比较大了，但是我们通过不断递进式累计最短板数量，最终不会受到m长度的影响，只会受到板子数量的影响。

因此下面整体复杂度是 $O(N)$ ，且N取值最多是1000。

贪心思维解法

Java算法源码

```
import java.util.HashMap;
import java.util.PriorityQueue;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        int[] a = new int[n];
        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
        }

        System.out.println(getResult(m, a));
    }

    public static int getResult(int m, int[] a) {
        // 统计每种长度板的数量，记录到woods中，key是板长度，val是板数量
        HashMap<Integer, Integer> woods = new HashMap<>();
        for (Integer ai : a) {
            if (woods.containsKey(ai)) {
                Integer val = woods.get(ai);
                woods.put(ai, ++val);
            } else {
                woods.put(ai, 1);
            }
        }
    }
}
```

```

// 将统计到的板，按板长度排优先级，长度越短优先级越高，这里使用优先队列来实现优先级
PriorityQueue<Integer[]> pq = new PriorityQueue<>((b, c) -> b[0] - c[0]);
for (Integer wood : woods.keySet()) {
    pq.offer(new Integer[] {wood, woods.get(wood)});
}

// 只要还有剩余的m长度，就将他补到最短板上
while (m > 0) {
    // 如果只有一种板长度，那么就尝试将m平均分配到各个板上
    if (pq.size() == 1) {
        Integer[] info = pq.poll();
        int len = info[0];
        int count = info[1];
        return len + m / count;
    }

    // 如果有多种板长度
    // min1是最短板
    Integer[] min1 = pq.poll();
    // min2是第二最短板
    Integer[] min2 = pq.peek();

    // diff是最短板和第二最短板的差距
    int diff = min2[0] - min1[0];
    // 将所有最短板补足到第二短板的长度，所需要总长度total
    int total = diff * min1[1];

    // 如果m的长度不够补足所有最短板，那么说明此时最短板的长度就是题解
    if (total > m) {
        return min1[0] + m / min1[1];
    }

    // 如果m的长度刚好可以补足所有最短板，那么说明最短板可以全部升级到第二短板，且刚好用完m，
    因此第二短板的长度就是题解
    else if (total == m) {
        return min2[0];
    }

    // 如果m的长度足够长，能补足所有最短板到第二短板，还能有剩余，则将最短的数量加到第二短板的
    数量上，继续下轮循环
    else {
        m -= total;
        min2[1] += min1[1];
    }
}

return pq.peek()[0];
}
}

```

动态规划解法

Java算法源码

```
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        int[] a = new int[n];
        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
        }

        System.out.println(getResult(n, m, a));
    }

    /**
     * @param n 木板数量
     * @param m 可用的木料长度
     * @param arr 木板长度数组
     * @return 最短木板长度
     */
    public static int getResult(int n, int m, int[] arr) {
        // 木板长度升序
        Arrays.sort(arr);

        // dp用于保存将所有arr[i-1]长度的木板补足到arr[i]长度，所需要的木料长度
        int dp = 0;
        for (int i = 1; i < n; i++) {
            // 比如将arr[0]长度的木板补足到arr[1]所需的木料长度为arr[1] - arr[0]
            // 比如将arr[1]长度的木板补足到arr[2]所需的木料长度为(arr[2] - arr[1]) *
            // 2，注意这里*2的原因是arr[0]已经被补足到arr[1]长度了，因此arr[1]长度此时有2个
            int need = (arr[i] - arr[i - 1]) * i;

            // 如果m可用木料不满足上面，则将剩余可用m-dp的材料均分给i根木料，返回此时的最短木料
            if (dp + need > m) {
                return (m - dp) / i + arr[i - 1];
            }

            dp += need;
        }

        // 如果将所有木板都补足到最长木板的长度了，还有剩余木料可用，则均分
        return (m - dp) / n + arr[n - 1];
    }
}
```